Universitatea Națională de Știință și Tehnologie

POLITEHNICA București

Facultatea de Inginerie Industrială și Robotică

PROIECT

DE

DIPLOMĂ

Autor,

Absolvent: VASILE Giulia-Georgiana

Conducător Științific,

Ș.l.Dr.ing. Vlad GHEORGHIȚĂ

Universitatea Națională de Știință și Tehnologie POLITEHNICA *București*

Facultatea de Inginerie Industrială şi Robotică

☎ (021) 4029520, (021) 4029302/ Fax: 0213107753

http://www.fiir.upb.ro

Departamentul Tehnologia Construcțiilor de Mașini

Studii universitare de Licenţă

Domeniul: Inginerie Industrială

Programul de studii: Informatică Aplicată în Inginerie Industrială

TEMA
PROIECTULUI DE DIPLOMĂ

PROIECTAREA UNUI ALGORITM ȘI REALIZAREA UNEI APLICAȚII INFORMATICE PENTRU GESTIONAREA DE MATERII PRIME A UNUI DEPOZIT

Autor,

Absolvent: VASILE Giulia-Georgiana

Proiectul de diplomă răspunde temei

Director de departament,

Prof.dr.ing. Nicolae IONESCU

Conducător științific,

Ș.l.dr.ing. Vlad GHEORGHIȚĂ

2026

Aprobat,
Decan

Domnule Decan,

Subsemnata **VASILE T. Giulia-Georgiana** [1],

absolventă a **Universității Naționale de Știință și Tehnologie POLITEHNICA București, Facultatea de Inginerie Industrială și Robotică**,

domeniul **Inginerie industrială**,

programul de studii **Informatică aplicată în inginerie industrială**,

promoţia **2025**, vă rog să-mi aprobaţi înscrierea la examenul de diplomă,
în sesiunea **FEBRUARIE**, anul **2026**.

Am mai susţinut examenul de diplomă în sesiunea _____, anul _____.

Am întocmit proiectul de diplomă cu titlul [2]: **PROIECTAREA UNUI ALGORITM ȘI REALIZAREA UNEI APLICAȚII INFORMATICE PENTRU GESTIONAREA DE MATERII PRIME A UNUI DEPOZIT**, sub

îndrumarea ştiinţifică a **Sl. Vlad GHEORGHITA**.

(gradul didactic, numele și prenumele conducătorului științific al proiectului)

Data: **19.02.2026** Semnătura: _____

Viza conducătorului ştiinţific al proiectului de diplomă: _____

Media generală de promovare a anilor de studii / ECTS: **8,10 / 260**

Avizat,
Secretar şef facultate

Domnului Decan al Facultatii de **Inginerie Industriala si Robotica**

---

1) Numele, initiala/initialele prenumelui/prenumelor tatalui/mamei si prenumele, cu majuscule.
2) Denumirea proiectului de diploma, cu majuscule.

Cuprins

1. INTRODUCERE – PREZENTARE GENERALĂ 4
1.1 Introducere 4
1.2 Prezentarea proiectului 4
1.3 Beneficiile informatizării procesului de gestionare a materiilor prime 4
1.4 Stadiul actual 5
1.4.1 InvenTree 5
1.4.2 Odoo 9
2. PRINCIPII ȘI METODE UTILIZATE PENTRU REZOLVAREA TEMEI DE PROIECT 12
2.1 Tehnologii utilizate 12
2.1.1 JavaScript – Limbaj de programare 12
2.1.2 Node.js – Mediu de execuție pentru JavaScript pe partea de server 12
2.1.3 Express – Framework backend 12
2.1.4 PostgreSQL – Sistem de gestiune a bazelor de date 13
2.1.5 pg – Driver PostgreSQL pentru Node.js 13
2.1.6 HTML – Hyper Text Markup Language 13
2.1.7 CSS – Cascading Style Sheets 13
2.1.8 Bootstrap – Framework pentru frontend 14
2.1.9 React – Bibliotecă frontend 14
2.1.10 Vite – Instrument în dezvoltarea frontend 14
2.1.11 Axios – Bibliotecă pentru cereri HTTP 15
2.1.12 SheetJS (xlsx) – Bibliotecă pentru manipularea fișierelor Excel 15
2.1.13 express-session – Middleware pentru gestionarea sesiunilor 15
2.1.14 bcrypt – Bibliotecă pentru criptarea parolelor 15
2.1.15 CORS – Middleware pentru gestionarea cererilor Cross-Origin 15
2.1.16 Redux Toolkit – Bibliotecă pentru gestionarea stării globale 16
2.1.17 React Router – Bibliotecă pentru rutare în aplicații React 16
2.1.18 React Hot Toast – Bibliotecă pentru notificări în aplicații React 16
2.2 Structura 16
2.2.1 Interfața cu operatorul 17
2.2.2 Interfața cu administratorul 17
2.3 Logica aplicației 18
3. PROIECTAREA DETALIATĂ A APLICAȚIEI INFORMATICE 19
3.1 Crearea unui proiect backend și configurarea aplicației 19
3.2 Crearea bazei de date PostgreSQL și conectarea la server 21
3.3 Realizarea modelelor pentru baza de date din PostgreSQL 24
3.3.1 Fișierul „userModel.js” 24

3.3.2 Fișierul „productsModel.js” 27
3.3.3 Fișierul „transactionsModel.js” 29
3.4 Arhitectura și funcționarea backend-ului aplicației 37
3.4.1 Structura componentelor backend 37
3.4.2 Logarea, înregistrarea și autentificarea utilizatorilor 37
3.4.3 Implementarea controllerelor și a rutelor HTTP ale aplicației 39
3.4.3.1 Controller-ul de autentificare din „authController.js” 40
3.4.3.2 Controller-ul pentru produse din „productsController.js” 42
3.4.3.3 Controller-ul pentru tranzații din „transactionsController.js” 45
3.4.3.4 Controller-ul administrării utilizatorilor din „usersController.js” 49
3.4.4 Autorizarea și securizarea rutelor 51
3.5 Arhitectura și funcționarea frontend-ului aplicației 52
3.5.1 Structura aplicației frontend 52
3.5.2 Implementarea rutării în aplicația React 53
3.5.3 Gestionarea stării globale cu Redux[20] Toolkit[20] 54
3.5.4 Comunicarea cu backend-ul prin Axios 55
3.5.5 Implementarea funcționalității de export în format Excel 56
3.5.6 Componenta Bara de navigare 58
3.5.7 Implementarea componentei de autentificare și înregistrare 60
3.5.8 Implementarea modulului de gestionare a produselor 63
3.5.9 Implementarea componentei care gestionează tranzacțiile 65
3.5.10 Implementarea componentei care gestionează utilizatorii 67
4. FUNCȚIONALITĂȚII APLICAȚIEI 68
4.1 Testarea backend-ului 68
4.2 Testarea frontend-ului 70
5. INTERFAȚA CU UTILIZATORUL 71
5.1 Interfața utilizatorului cu rolul de operator 73
5.2 Interfața utilizatorului cu rolul de administrator 76
6. PROIECTAREA ȘI FABRICAREA REPERULUI „ARBORE PINION” DIN CADRUL PRODUSULUI „POMPĂ CU ROȚI DINȚATE” 82
6.1 Date generale 82
6.2 Date constructive - funcționale 82
6.2.1 Schița ansamblului 82
6.2.2 Caracteristici constructive prescrise produsului 84
6.2.3 Caracteristici de material 86
6.2.4 Masa produsului 86
6.2.5 Tehnologicitatea produsului 87
6.2.6 Interdependența toleranțelor dimensionale 90
6.3 Stabilirea etapelor de prelucrare 92

6.3.1 Procese tehnologice 92
6.3.2 Structura preliminară a proceselor și sistemelor de producție 95
6.3.3 Calcul adaos prelucrare 99
6.3.4 Calcul dimensiuni intermediare 99
6.4 Structura proceselor tehnologice cu sculele în poziție de lucru 101
6.5 Realizarea piesei prin fabricare aditivă folosind programul Z-suite 114
7. CONCLUZII ȘI DEZVOLTĂRI ULTERIOARE 117
7.1 Concluzii 117
7.2 Dezvoltări ulterioare 117
8. BIBLIOGRAFIE 118

1. INTRODUCERE – PREZENTARE GENERALĂ

1.1 Introducere

Managementul materiei prime are o importanță deosebită în operațiunile multor companii, fie că se referă la eficientizarea lanțurilor de producție sau la gestionarea întregului proces de aprovizionare. Totuși, administrarea manuală a acestor activități nu constituie, de cele mai multe ori, soluția optimă în ceea ce privește monitorizarea erorilor, stocurilor și a pierderilor de timp.

În prezent, opțiunea digitalizării proceselor de lucru dintr-un depozit a devenit o necesitate, fiind esențială îndeplinirii standardelor actuale de organizare și protecție a datelor. Adaptarea instrumentelor la spațiul electronic oferă o evidență precisă, limitează dezvoltarea erorilor și permit un control sporit asupra activităților efectuate.

Principiul de a extinde utilizarea acestor mijloace stă la baza realizării unui proiect menit să fluidizeze, atât procurarea de bunuri necesare, cât și coordonarea resurselor umane.

1.2 Prezentarea proiectului

Lucrarea urmărește realizarea unei aplicații informatice care să ofere sprijin în gestionarea stocurilor și vizualizarea clară a produselor, tranzacțiilor și utilizatorilor, păstrând totodată o interfață prietenoasă și eficientă. Platforma integrează un sistem de autentificare bazat pe mai multe roluri, având două categorii de utilizatori: operatori și administratori.

Operatorul, având un rol limitat, are posibilitatea de a examina produsele înregistrate în depozit, urmărind detaliile acestora, dar și calitatea de introducere a noilor tranzacții. În cazul editării mișcărilor de stoc, operatorul are dreptul doar de a modifica descrierea, fără a fi posibilă ștergerea sau a schimbarea lor semnificativă.

Administratorul, pe de altă parte, are un control complet asupra aplicației, fiindu-i atribuită producerea sau prefacerea oricărui element, procedeu sau client al platformei. Administrarea utilizatorilor este vizibilă exclusiv celor care au acest rol, subliniind distincția clară dintre cele două niveluri de acces.

1.3 Beneficiile informatizării procesului de gestionare a materiilor prime

Metoda clasică de management a materiilor prime constituie un efort semnificativ în ceea ce privește urmărirea inventarului și actualizarea tranzacțiilor, existând posibilitatea expunerii lor la erori și lipsă de ordine. Datorită integrării fluxurilor de lucru într-un mediu digital, ne putem bucura de o multitudine de beneficii însemnate, care ne permit să monitorizăm eficient și organizat fiecare proces în parte. Indiferent dacă ne referim la actualizarea constantă a tuturor produselor, notelor de intrare și respectiv ieșire sau la personalul înregistrat în sistem, performanța se resimte atât în materie de timp cât și în materie de securitate și integritate a datelor.

Prin urmare, dirijarea complexă a procedurilor fundamentale oricărui depozit de materie primă, răspunde unei nevoi concrete din câmpul economic, dar și a cerinței dezvoltărilor viitoare.

1.4 Stadiul actual

În prezent există o varietate de aplicații web care permit îndeplinirea numeroaselor nevoi specifice, ce includ urmărirea loturilor, vizualizarea activităților, gestionarea mai multor depozite și generarea de rapoarte detaliate. Câteva exemple relevante sunt prezentate în cele ce urmează.

1.4.1 InvenTree

InvenTree[1] este o platformă de planificare și control a stocurilor cu o multitudine de caracteristici diferite. Concentrată atât pe furnizori și producători, cât și pe vânzările către clienți, reușind astfel să urmărească tot ce ține de lanțul de aprovizionare. Ceea ce este unic în această interfață este reprezentat de integrarea API-urilor care facilitează legătura cu alte aplicații externe.

Pagina de start include și o variantă demo concepută pentru evidențierea modului în care sistemul de roluri și permisiuni funcționează. Nivelurile diferite de acces, conferă o experiență unică datorită perspectivei variate a fiecărui punct de autorizare în parte, facilitând înțelegerea avantajelor unui sistem flexibil în administrarea unui depozit digitalizat.

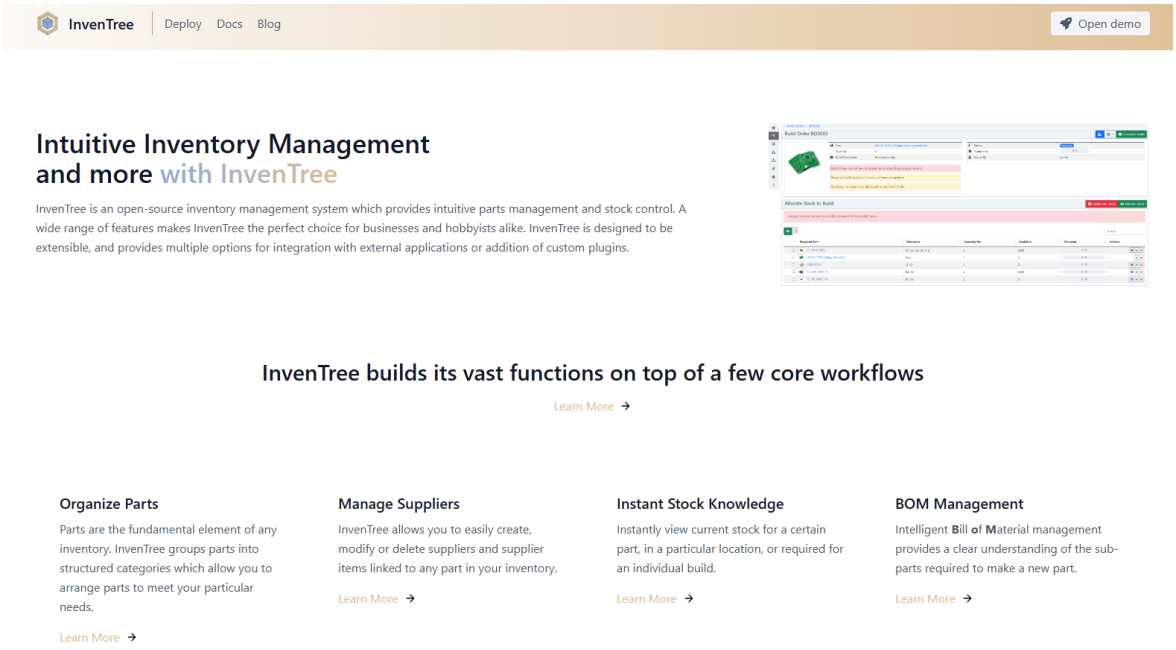


Fig.1.1 Pagina de start InvenTree[1]

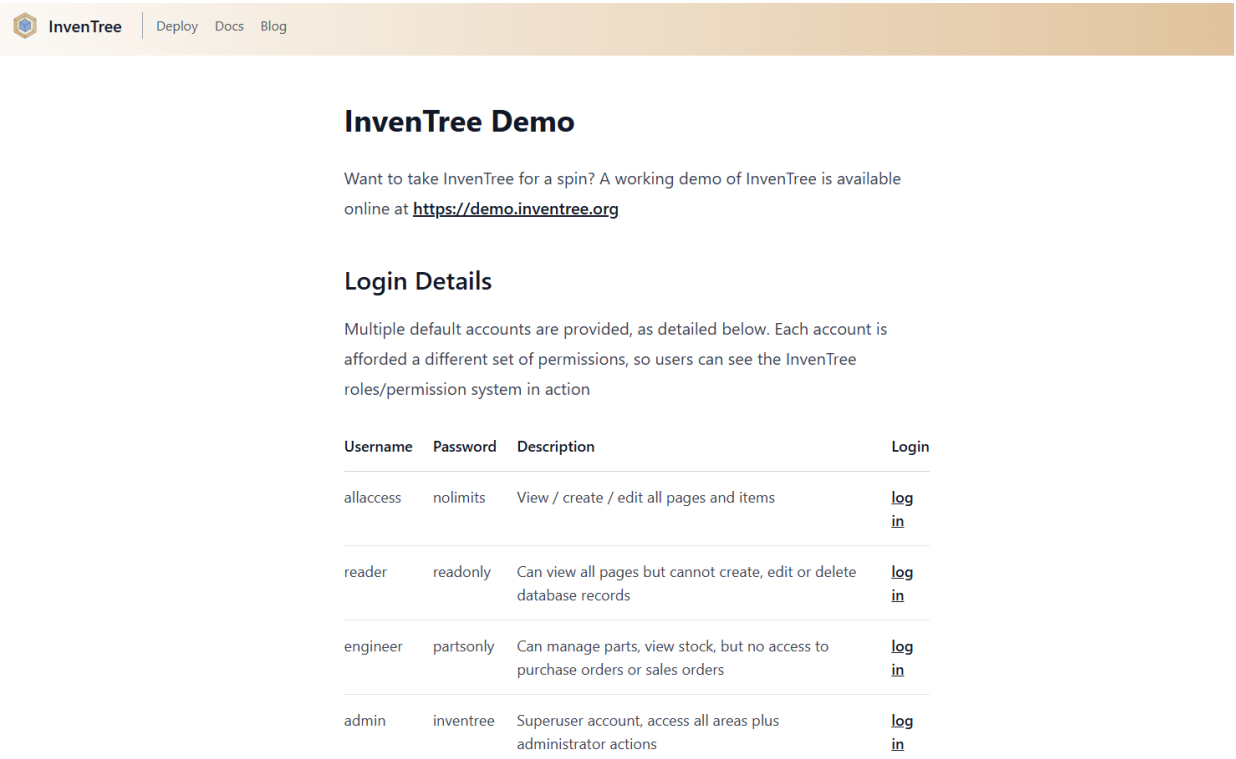


Fig.1.2 Pagina demo InvenTree[1]

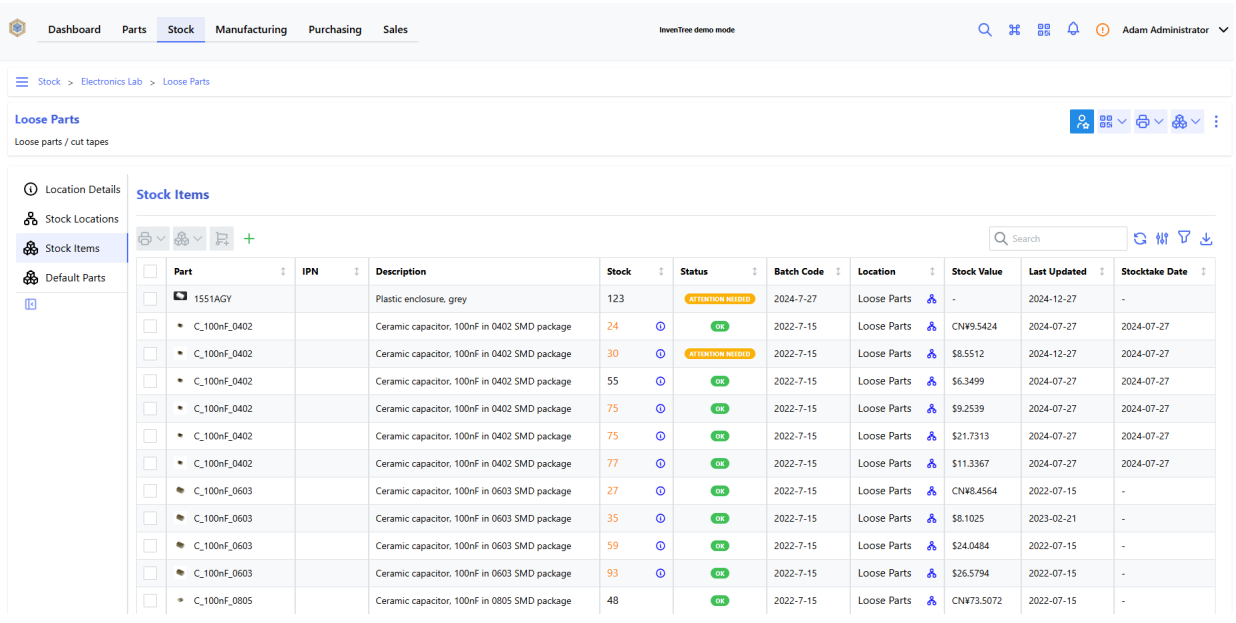


Fig.1.3 Vizualizarea stocului în InvenTree[1]

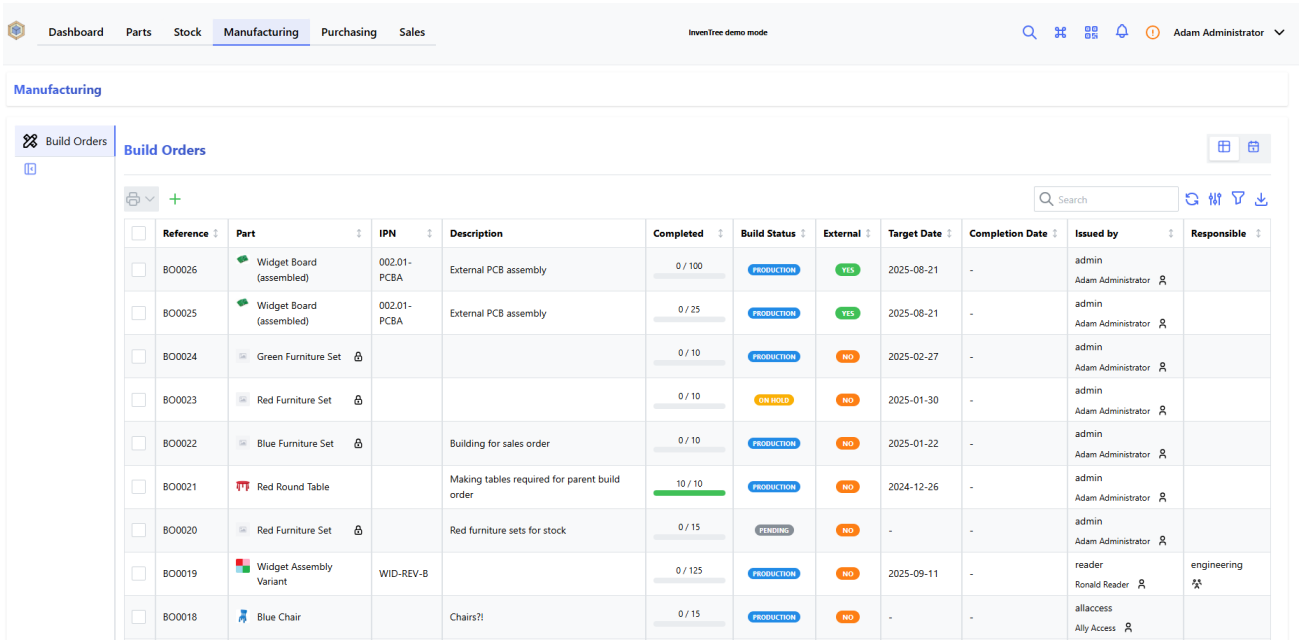


Fig.1.4 Vizualizarea producției în InvenTree[1]

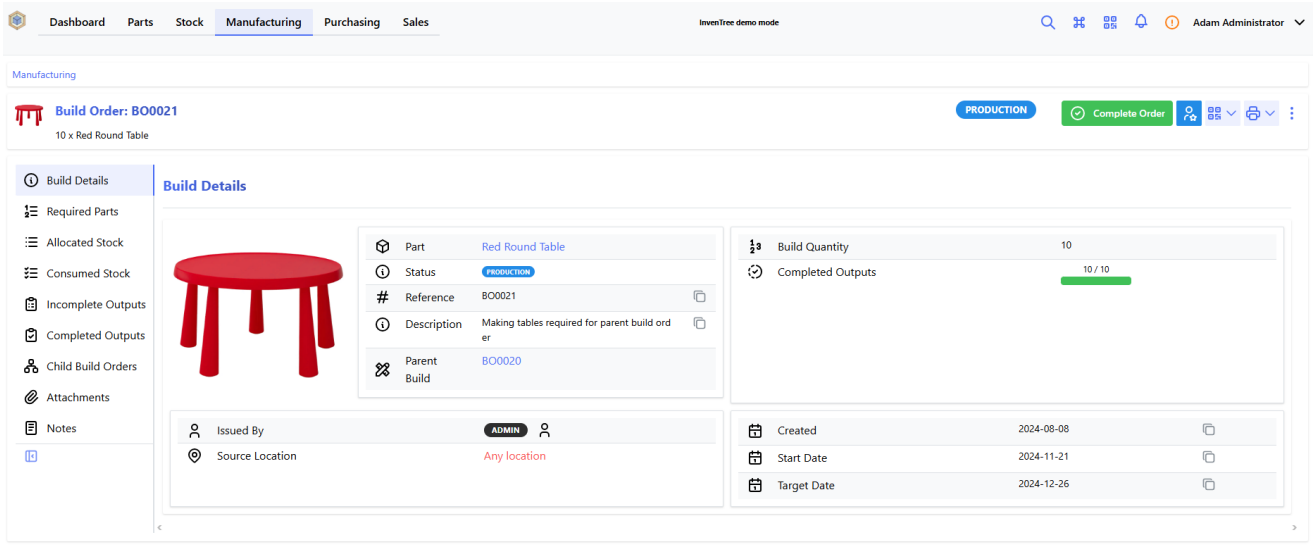


Fig.1.5 Vizualizarea ordinelor de asamblare în InvenTree[1]

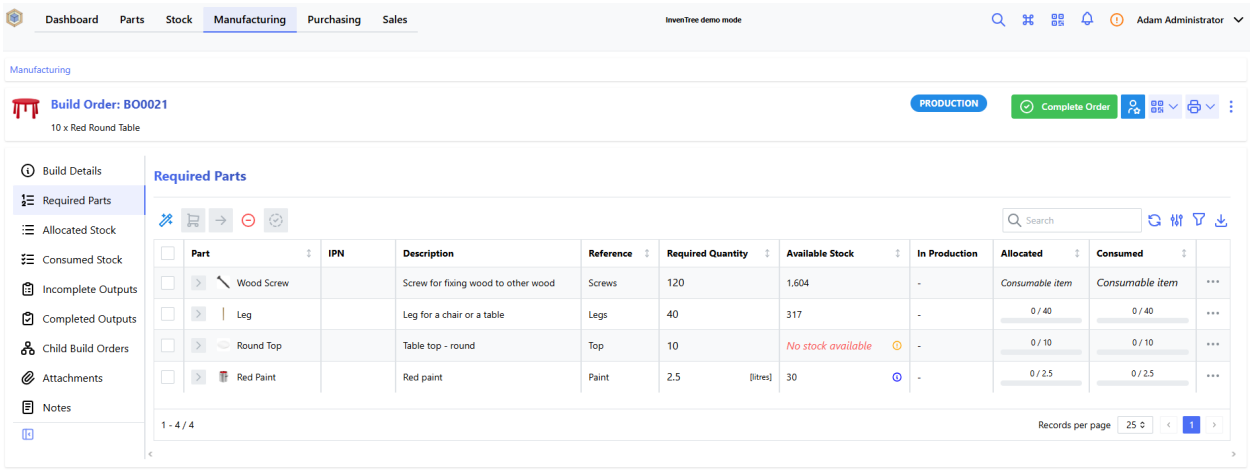


Fig.1.6 Vizualizarea componentelor necesare asamblării în InvenTree[1]

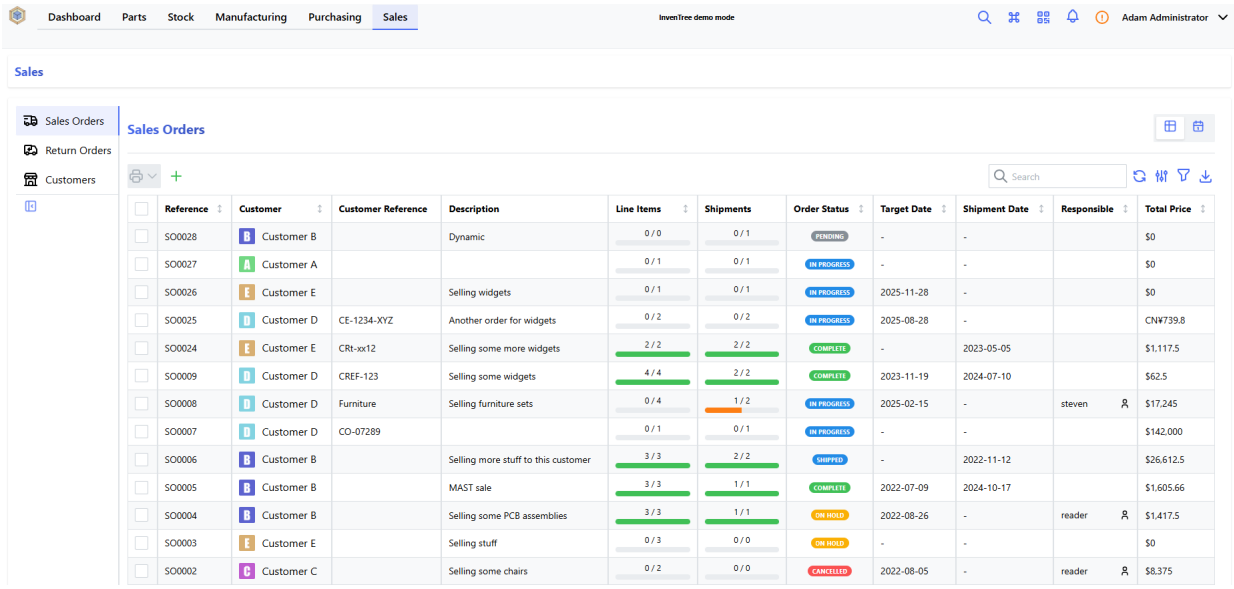


Fig.1.7 Vizualizarea clienților în InvenTree[1]

1.4.2 Odoo

Odoo[2] este o tehnologie ce permite utilizatorilor să își monitorizeze activitatea fără a folosi programe separate pentru contabilitate, vânzări, gestionare de produse sau resurse umane, toate acestea aflându-se într-un singur loc și folosind aceeași bază de date. Cu un design intuitiv, utilizatorul își poate alege doar programele de care are nevoie, iar în funcție de ele poate asigura un mediu de lucru simplu și eficient.

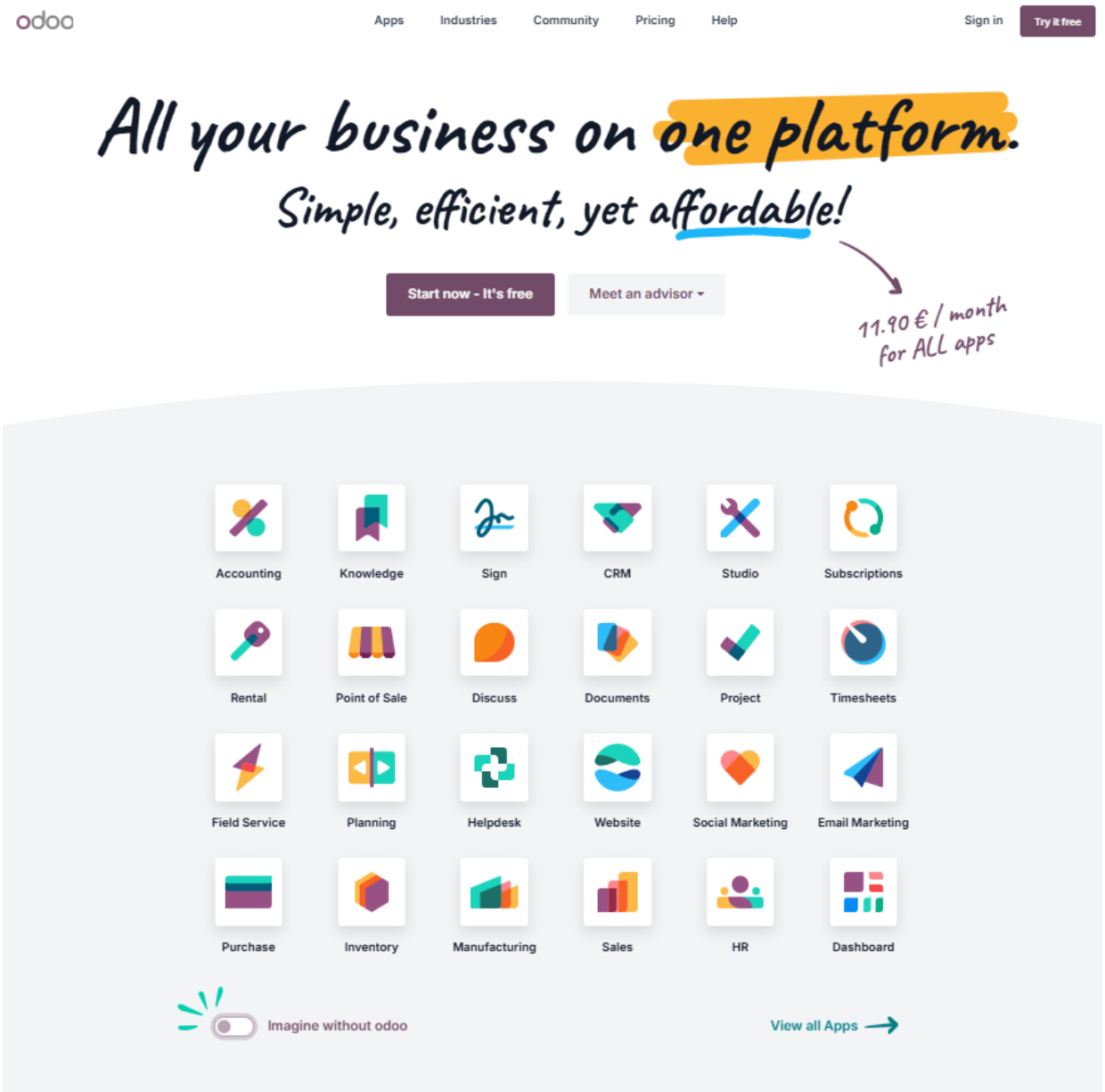


Fig.1.8 Pagina de start Odoo[2]

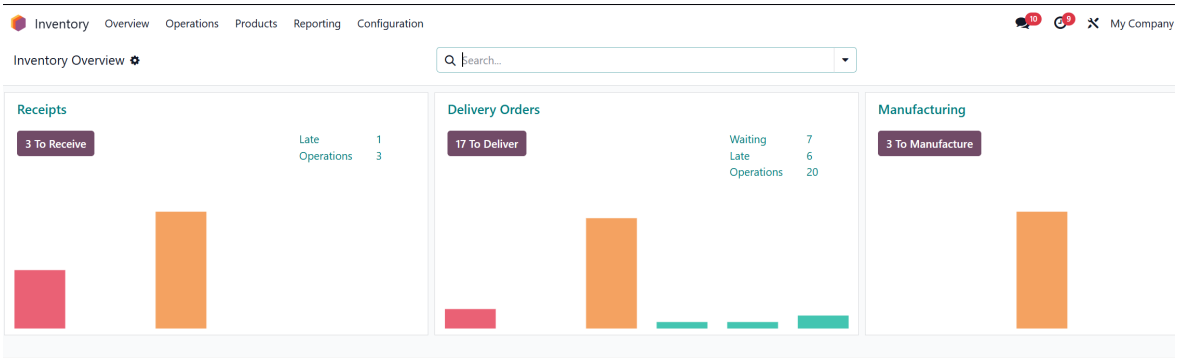


Fig.1.9 Prezentarea generală a inventarului în Odoo[2]

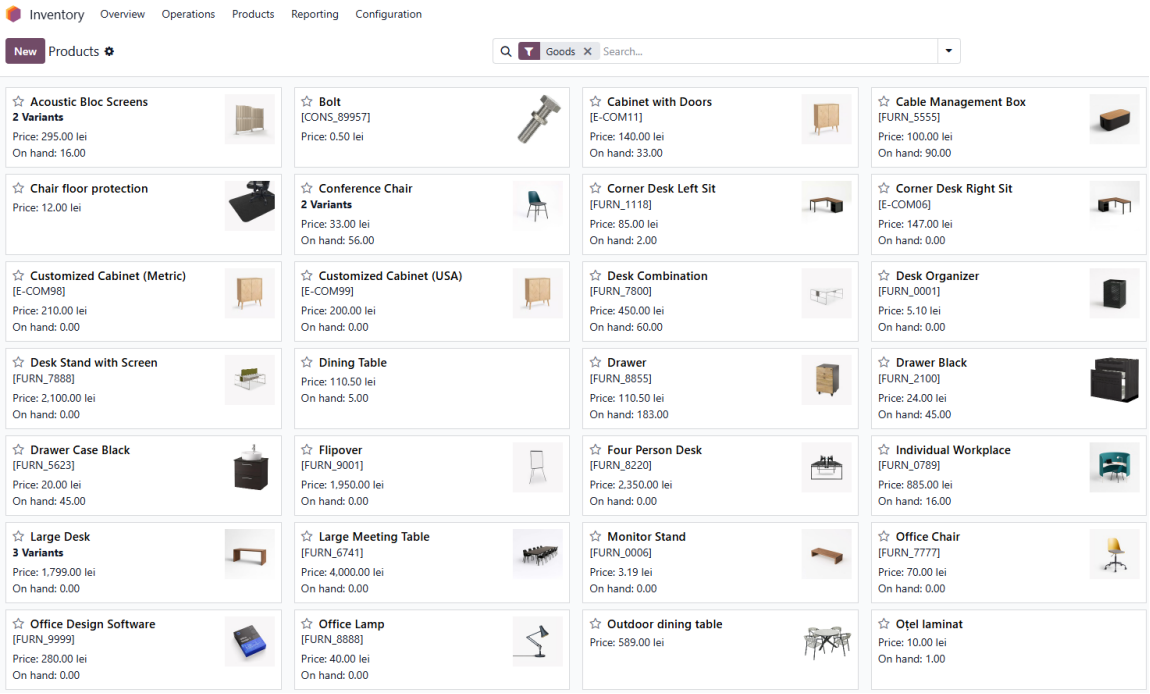


Fig.1.10 Afișarea produselor în Odoo[2]

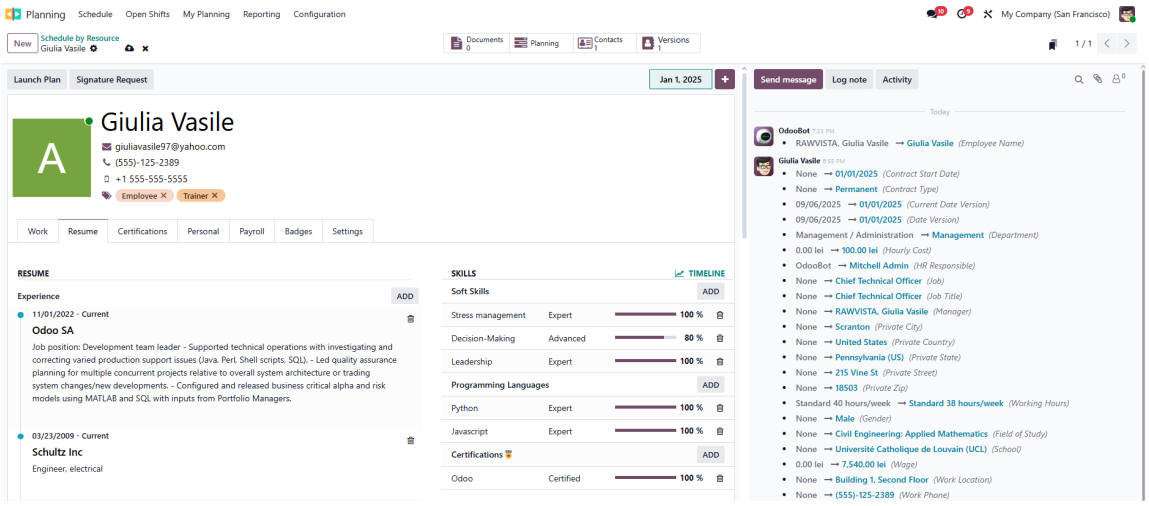


Fig.1.11 Afișarea datelor personalului în Odoo[2]

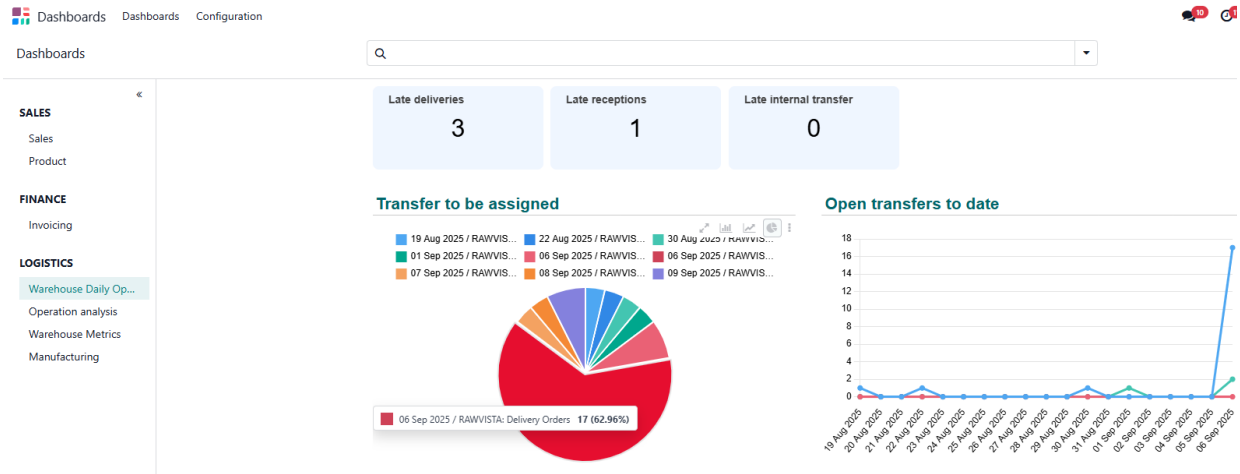


Fig.1.12 Dashboards în Odoo[2]

2. PRINCIPII ȘI METODE UTILIZATE PENTRU REZOLVAREA TEMEI DE PROIECT

2.1 Tehnologii utilizate

2.1.1 JavaScript – Limbaj de programare

JavaScript[3] este un limbaj de programare, interpretat, utilizat special pentru dezvoltarea aplicațiilor web. Fiind un limbaj ușor de învățat și flexibil, are o dinamică care permite implementarea unor funcții interactive în paginile noastre web.

Programarea bazată pe evenimente permite manipularea DOM-urilor (Modelul de obiecte al documentului), oferind dezvoltatorilor capacitatea de a modifica în timp real structura și stilul unei pagini. Facilitatea în realizarea aplicațiilor complexe este evidențiată de numeroase biblioteci și framework-uri care permit folosirea JavaScript[3] atât pe partea de backend, cât și pe cea a clientului.

În plus, JavaScript[3] este unul dintre cele mai folosite limbaje de programare, fiind esențial progresului digital. Totodată dispune de actualizări în mod regulat, ajutând astfel la îmbunătățirea funcționalității și performanței.

2.1.2 Node.js – Mediu de execuție pentru JavaScript pe partea de server

Node.js[4] este un mediu de execuție pentru JavaScript[3] care rulează pe motorul JavaScript V8, în afara browser-ului, ceea ce îi permite să fie foarte performant în dezvoltarea aplicațiilor server-side.

Cu un model asincron orientat pe evenimente, reușește să administreze mii de conexiuni simultane cu un singur server, optimizând astfel utilizarea resurselor. De asemenea, Node.js[4] include unul dintre cele mai mari registre de pachete și module reutilizabile care permit extinderea accelerată a oricărui proces în dezvoltare.

În prezent, Node.js[4] este una dintre cele mai folosite tehnologii pentru realizarea oricărei platforme moderne, justificarea rămânând flexibilitatea și comunitatea care prioritizează performanța.

2.1.3 Express – Framework backend

Express[5] este cel mai utilizat framework bazat pe Node.js[4] și servește drept bibliotecă principală pentru diverse alte framework-uri populare de Node.js[4]. Conceput pentru o gestionare flexibilă a rutelor și cererilor HTTP, unde arhitectura sa modulară simplifică procesul de creare a serverelor.

De asemenea, introducerea de „middleware” tratează cererile în orice etapă a procesului de manevrare a acestora, ele fiind extinse rapid cu funcționalități precum autentificare, înregistrare, validarea datelor sau tratarea erorilor.

Succesul de care dă dovadă Express[5] este reprezentat de reducerea timpului necesar implementării soluțiilor complexe, atât pentru dezvoltatorii începători, cât și pe cei avansați. Așadar, Express[5] rămâne una dintre cele mai utilizate soluții backend, construind baza multor aplicații web moderne.

2.1.4 PostgreSQL – Sistem de gestiune a bazelor de date

PostgreSQL[6] este un sistem important de baze de date relaționale, open source, care utilizează și extinde limbajul SQL oferind astfel siguranță în rezolvarea celor mai complicate sarcini de lucru. Reputația solidă este câștigată prin arhitectura sa fiabilă și integritatea datelor care oferă în mod constant soluții performante și inovatoare.

Unul dintre principalele avantaje ale PostgreSQL[6] constă în compatibilitatea cu tranzacțiile ACID (Atomicitate, Consistență, Izolare, Durabilitate) definite de utilizator. Adăugarea de funcții personalizate, integritatea datelor și mecanismele eficiente de optimizare a interogărilor, justifică preferința dezvoltatorilor și a marilor companii de a alege această tehnologie.

Datorită arhitecturii robuste, a comunității open-source și a extensibilității de care dă dovadă, PostgreSQL[6] constituie alegerea frecventă în diverse proiecte indiferent de cât de mare sau mic este setul de date.

2.1.5 pg – Driver PostgreSQL pentru Node.js

pg[7] este un modul oficial Node.js[4] care permite conectarea cu baza de date din PostgreSQL[6]. Acesta include mecanisme de administrare a erorilor datorită suportului pentru interogări parametrizate, devenind astfel standardul principal de conectare dintre PostgreSQL[6] și Node.js[4].

2.1.6 HTML – Hyper Text Markup Language

HTML (Hyper Text Markup Language)[8], [9] este un limbaj marker standard pentru crearea paginilor Web. Prin intermediul acestuia, se pot crea și organiza elemente de tip text, titlu, paragrafe, imagini, link, listă, tabel cu ajutorul etichetelor de tip tags, proces care se numește marcare. Etichetele au nume sugestiv și sunt încadrate între simbolurile ”<” ”>”. Acestea nu sunt case-sensitive, deci pot fi scrise cu litere mici sau majuscule, fără a fi afectată funcționalitatea. Etichetele sunt poziționate la începutul și sfârșitul unei secțiuni, cum ar fi <title> Title </title>.

Secțiunea „head” a unui document HTML[8],[9] nu apare pe site-ul propriu-zis atunci când pagina este încărcată, conținând informații precum: titlul (<title>), link-uri către CSS[10], sau alte elemente utilizate pentru stilizarea paginii web și date despre documentul HTML[8],[9].

Secțiunea „body” a unui document HTML[8],[9] include elemente de tip heading (<heading>), paragraf (<p>), liste (<li>), formulare (<form>), care formează conținutul principal al paginii web. Acestea pot fi așezate în pagină cu ajutorul containerelor de tiv , elemente block non-semantice care se ocupă cu organizarea informațiilor din pagina în secțiuni separate.

2.1.7 CSS – Cascading Style Sheets

CSS (Cascading Style Sheets)[10] oferă posibilitatea de înfrumusețare a paginilor web, oferind control asupra modului în care acestea sunt prezentate către utilizator. În general, paginile web sunt scrise în limbaje de marcare precum HTML[8],[9] sau XML care apar în browser ca niște documente text, fără o structură, culori sau stiluri de afișare specifice.

CSS[10] este un limbaj bazat pe reguli, ceea ce înseamnă că dezvoltatorul poate crea reguli specificând grupuri de stiluri care vor fi aplicate către anumite elemente ale paginii Web.

Acestea sunt definite prin intermediul unui sistem de tipul proprietate-valoare. Se pot aplica mai multe proprietăți în cadrul unui element, care însumate, vor rezulta în stilul de afișare dorit.

2.1.8 Bootstrap – Framework pentru frontend

Bootstrap[11] este un toolkit pentru frontend robust și extensibil care permite personalizarea ușoară și rapidă a interfețelor Web folosind SASS (Syntactically Awesome Style Sheets), un sistem de așezare în pagină bazat pe o grilă de plugin-uri JavaScript[3] ce pot fi incluse cu ușurință în proiecte pentru a oferi un aspect plăcut și pentru a genera elemente interactive în cadrul paginii Web, într-un timp scurt.

Printre clasele și elementele disponibile, se numără următoarele: butoane, formulare, elemente de tip carusel, dropdown-uri, bară de navigare, card-uri, modale, alerte și elemente ce țin de așezarea în pagină, reprezentate de sistemul grilă, împărțit în linii și coloane.

Bootstrap[11] poate fi instalat prin intermediul npm (Node Package Manager), folosind comenzile ”npm install Bootstrap@5.3.3”.

2.1.9 React – Bibliotecă frontend

React[12],[13] este o bibliotecă JavaScript[3] pentru randarea interfețelor utilizatorilor (UI) care este construită din unități mici, cum ar fi butoane, text și imagini. Indiferent de aplicația la care dezvoltatul lucrează, React[12],[13] îi permite combinarea lor în componente reutilizabile.

Documentația extinsă și biblioteca de resurse contribuie la evoluția și popularitatea sa continuă deoarece poate fi integrat cu alte pachete sau framework-uri pentru gestionarea și dezvoltarea de platforme moderne.

De asemenea, React și ReactDOM sunt atât de des utilizate împreună pentru că au roluri complementare care creează și optimizează sarcinile proiectelor. Abordarea fiecărui framework JavaScript[3] oferă o modalitate de a scrie interfețe într-un mod mai declarativ, descrierea lor fiind posibilă în DOM care face ca acest lucru să fie posibil.

2.1.10 Vite – Instrument în dezvoltarea frontend

Vite[14] este un instrument de dezvoltare mai rapidă și mai eficientă pentru proiectele web moderne. A fost dezvoltat de Evan You, cunoscut și ca autor al framework-ului Vue.js, și s-a răspândit rapid datorită simplității și performanței sale.

Principalul avantaj al Vite[14] este folosirea modulelor ESM (ECMAScript), pe care browser-ul le poate încărca rapid, unde transformă și servește codul sursă așa cum este solicitat. Codul din spatele importurilor dinamice condiționate este procesat numai dacă este utilizat efectiv pe ecranul curent.

Datorită comunității în creștere și a documentației accesibile, Vite[14] a devenit una dintre cele mai populare soluții moderne pentru construirea interfețelor web, fiind totodată și foarte flexibil deoarece permite integrarea cu mai multe framework-uri populare precum React[12],[13].

2.1.11 Axios – Bibliotecă pentru cereri HTTP

Axios[15] este o bibliotică JavaScript[3] open-source populară, utilizată pentru a face cereri HTTP din browsere web sau medii Node.js[4]. Este o modalitate de a simplifica trimiterea cererilor asincrone către endpoint-uri REST, gestionarea răspunsurilor și efectuarea diverselor sarcini legate de rețea.

Construit pe o bază de XMLHttpRequest din JavaScript[3] și a API-ului fetch, Axios[15] oferă o API mai ușor de utilizat, cu funcții precum transformarea automată a datelor JSON, gestionarea erorilor și suport pentru browserele mai vechi.

Datorită sintaxei simplificate, consistenței datelor, compatibilității și a securității îmbunătățite, Axios[15] a devenit una dintre cele mai utilizate biblioteci pentru cereri HTTP în proiectele bazate pe utilizarea framework-urilor precum React[12],[13] sau a programelor realizate în JavaScript[3].

2.1.12 SheetJS (xlsx) – Bibliotecă pentru manipularea fișierelor Excel

SheetJS[16] este biblioteca JavaScript[3] care permite analizarea și scrierea în diverse formate de foi de calcul, inclusiv xlsx din Excel. Devine foarte utilă în proiectele unde datele trebuie să fie exportate într-un format tabelar.

De asemenea, în loc să fie nevoie să citești sau să scrii fiecare celulă, SheetJS[16] te ajută să mapezi foaia de calcul într-un obiect JavaScript[3]. Concentrarea pe transferarea datelor, dar și posibilitatea de a converti datele într-o structură JSON îi permite să fie potrivit pentru o gamă largă de aplicații.

2.1.13 express-session – Middleware pentru gestionarea sesiunilor

express-session[17] este un middleware utilizat în aplicațiile dezvoltate cu Express[5], care gestionează sesiunile utilizatorilor, stocând datele sale specifice pe server și urmărindu-le prin cookie-uri. Acest mecanism oferă opțiuni pentru configurarea duratei de viață a sesiunii, securizarea cookie-urilor și gestionarea modului în care sunt salvate datele, fiind adesea folosit pentru implementarea autentificării bazate pe sesiuni.

2.1.14 bcrypt – Bibliotecă pentru criptarea parolelor

bcrypt[18] este o bibliotecă utilizată pentru hașurarea parolelor în aplicațiile software, concepută pentru a proteja atacurile de tip bruce-force. Astfel, permite convertirea parolelor într-un „hash” ireversibil, care este stocat înapoi în baza de date. În plus, bcrypt[18] adaugă automat un „salt” unic fiecărei parole, astfel încât chiar dacă doi sau mai mulți utilizatori aleg aceeași parolă, hash-urile rezultate vor fi diferite.

2.1.15 CORS – Middleware pentru gestionarea cererilor Cross-Origin

CORS (Cross-Origin Resource Sharing)[19] reprezintă un mecanism de securitate implementat în browser, care controlează accesul la solicitarea de resurse dintr-un domeniu diferit de al său. Configurația middleware-ului este esențială în aplicațiile moderne în care frontend-ul și backend-ul rulează pe origini diferite, asigurând o comunicare securizată între acestea.

2.1.16 Redux Toolkit – Bibliotecă pentru gestionarea stării globale

Redux Toolkit[20] este pachetul oficial recomandat pentru utilizarea Redux în aplicațiile JavaScript[3] moderne, în special în cele dezvoltate cu React[12],[13]. Acesta simplifică configurarea magazinului global (store) și reduce complexitatea implementării funcțiilor de actualizare a stării și a acțiunilor.

Prin intermediul funcției „configureStore”, se poate crea un magazin centralizat pentru administrarea stării aplicației. De asemenea, Redux Toolkit[20] introduce conceptul de „slice”, care permite organizarea logică a stării în secțiuni distincte. Pentru gestionarea operațiilor asincrone, biblioteca oferă funcția „createAsyncThunk”, care facilitează apelurile către servicii externe și tratarea stărilor de tip „pending”, „fulfilled” și „rejected”.

2.1.17 React Router – Bibliotecă pentru rutare în aplicații React

React Router[21] este o bibliotecă utilizată pentru implementarea rutării în aplicații de tip Single Page Application (SPA) dezvoltate cu React[12],[13]. Aceasta permite navigarea între diferite componente fără reîncărcarea completă a paginii, oferind o experiență fluidă utilizatorului.

Biblioteca pune la dispoziție componente precum „BrowserRouter”, „Routes” și „Route”, care permit definirea căilor de acces și asocierea acestora cu anumite componente. De asemenea, React Router[21] oferă mecanisme pentru redirecționare și protejarea rutelor, facilitând controlul accesului în funcție de diferite condiții.

2.1.18 React Hot Toast – Bibliotecă pentru notificări în aplicații React

React Hot Toast[22] este o bibliotecă destinată afișării notificărilor temporare de tip „toast” în aplicațiile dezvoltate cu React[12],[13]. Aceste notificări sunt utilizate pentru a transmite utilizatorului informații rapide despre rezultatul unor acțiuni, precum succesul unei operații sau apariția unei erori.

Biblioteca oferă metode simple pentru afișarea notificărilor, precum „toast.success” și „toast.error”, permițând integrarea rapidă în interfață fără implementarea unor componente complexe suplimentare. Totodată, notificările afișate dispar automat după un anumit interval de timp, contribuind la îmbunătățirea experienței utilizatorului.

2.2 Structura

Aplicația este construită pentru a gestiona eficient materiile prime dintr-un depozit prin definirea produselor, urmărirea tranzacțiilor de intrare sau de ieșire și administrarea utilizatorilor pe roluri. Scopul principal al acesteia este de a reduce producerea de erori operaționale prin definirea clară a obiectivului fiecărui angajat.

Proiectul este bazat pe interacțiunea tuturor părților, administratorii fiind cei care vor adăuga, șterge sau edita produse, tranzacții sau personal. Aceștia se vor autentifica în platformă cu ajutorul email-ului și parolei, care după verificarea permisiunii rolului, îi vor permite utilizatorului să acceseze pagina principală. Rolul unui administrator este de a defini catalogul de produse și de utilizatori, de a supraveghea stocul și de a corecta situațiile excepționale, dacă este cazul.

Operatorul în schimb, este persoana care primește catalogul de produse, iar cu acesta pregătește mișcările de stoc, reprezentând practic tranzacțiile de intrare și de ieșire. Nu îi este permis să adauge produse noi, iar în ceea ce privește recepționarea de tranzacții acesta are câteva limitări, pagina utilizatorilor nefiind activă pentru acest rol.

Toate acțiunile aplicației sunt transparente, fiind mereu vizibilă persoana care a produs schimbări bazei de date a proiectului, iar claritatea interfeței impune reguli care previn scăderea stocului sub zero sau introducerea de date invalide.

2.2.1 Interfața cu operatorul

Interfața destinată operatorului este gândită ca fiind utilizată pentru treburi mai uzuale sau care nu necesită o responsabilitate foarte mare. Imediat după autentificare, operatorul ajunge pe pagina de start (Dashboard) unde îi este afișat numele și rolul.

Examinarea produselor se face dintr-un tabel care are detalii precum denumirea, SKU-ul, unitatea de măsură, stocul curent, stocul minim, raftul (locația fizică), prețul și autorul ultimei modificări. În bara de căutare poți să găsești produsele după nume, SKU, raft, autor și descriere, iar produsele care au un stoc sub nivelul stocului minim sunt evidențiate vizual pentru a permite utilizatorului să ia măsuri în privința aprovizionării. În meniul de acțiuni există secțiunea „Detalii produs” care permite vizualizarea tuturor componentelor produsului, inclusiv și poza acestuia.

Înregistrarea tranzacțiilor reprezintă scopul zilnic al operatorilor. Responsabilitatea lor este de a crea tranzacții de tip „IN” (intrare, recepție) sau „OUT” (ieșire/consum/livrare) pentru un produs, selectându-i tipul, cantitatea și descrierea (care este opțională). Sistemul verifică dacă tipul este IN/OUT, cantitatea să fie un număr întreg mai mare decât zero și dacă stocul este suficient, asta în condițiile tranzacțiilor de ieșire.

Pentru a fi păstrată diferența dintre cele două roluri, operatorul are posibilități limitate de editare asupra mișcărilor de stoc deja înregistrate, modificările majore rămân astfel responsabilitatea administratorului. Atât lista de produse, cât și istoricul tranzacțiilor pot fi exportate în format Excel, respectând orice filtru introdus în tabelul fiecăruia de valori, astfel orice înregistrare sau materie primă, să poată fi accesibilă oricui.

2.2.2 Interfața cu administratorul

Administratorul lucrează pe aceeași interfață ca operatorul, dar are la dispoziție pagini și acțiuni suplimentare acolo unde este nevoie de responsabilitate și decizie. Autentificare pentru ambele roluri rămâne aceeași, sesiunea fiind păstrată prin cookie-uri de sesiune. Paginile aferente acestui rol sunt pagina de start (Dashboard) cu numele și rolul său, paginile pentru produse și respectiv tranzacții și cea a utilizatorilor, special creată acestui rol.

Definirea catalogului de produse se realizează prin pagina corespunzătoare acestei acțiuni unde, administratorul poate crea, edita sau șterge articolele din stoc. Pentru fiecare produs trebuie completată denumirea, SKU-ul unic, unitatea de măsură, stocul și stocul minim, raftul locației fizice, descrierea, prețul și dacă se dorește și link-ul către o imagine care ajută la identificarea mai clară a acestuia.

Administratorul poate să creeze, editeze sau să șteargă orice tranzacție inclusiv schimbarea tipului acesteia, subliniind importanța unui mediu controlat asupra stocului. La salvarea sau editarea fiecărei mișcări de stoc, recalcularea materiilor prime realizându-se instant și corespunzător. Când ștergerea unui stoc sub zero are loc, operația este blocată pentru a preveni dezechilibrările.

Gestionarea utilizatorilor este atribuită doar acestui rol, dar este desfășurată cu aceeași logică și simplitate. Administratorul poate să creeze conturi noi unde trebuie introduse date precum nume, prenume, adresa de email, parola și rolul pentru cei care au nevoie de acces în aplicație, dar poate să editeze sau să elimine utilizatori dacă este cazul, fiind singura persoana care poate edita rolul altui client în aplicație.

Eficiența aplicației se bazează pe informațiile care sunt vizibile pentru fiecare dintre elementele din aceasta, fiind clar și rapid cine a făcut ultima modificare și ce produse sunt sub stocul minim. Exportul în formatul Excel permite păstrarea filtrelor active pe ecran și facilitează funcționarea optimă a platformei.

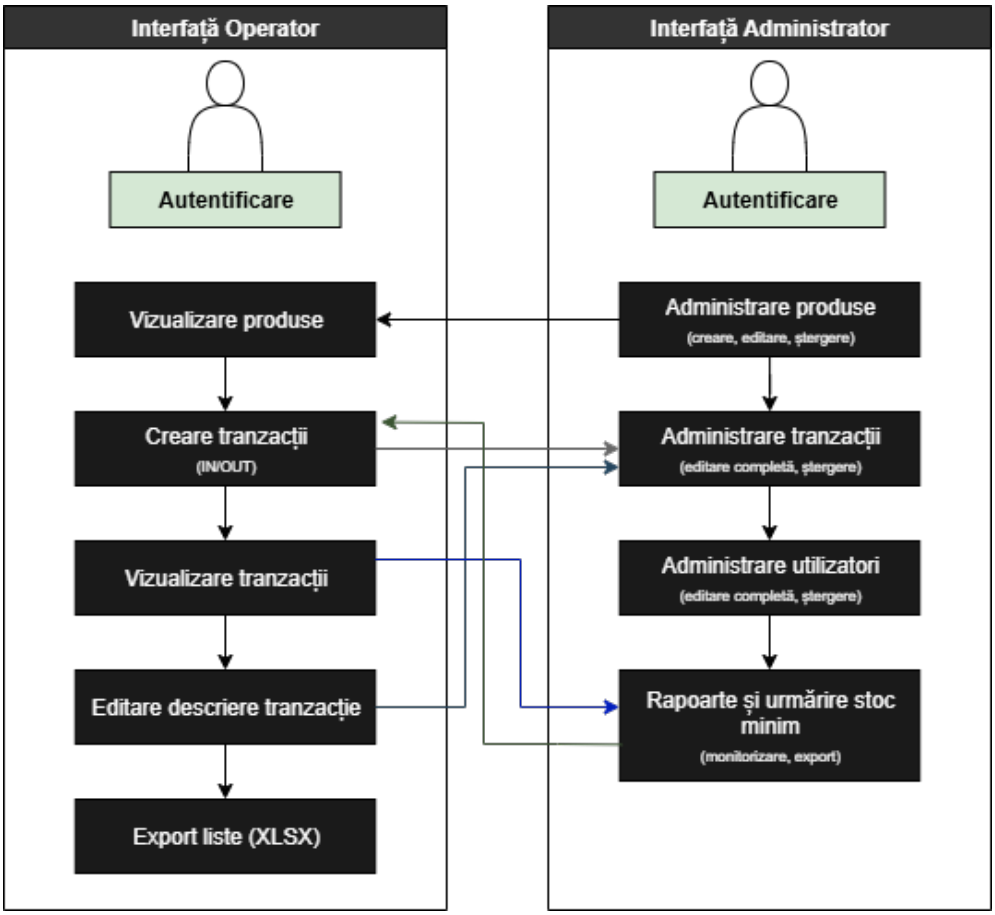


Fig.2.1 Modalitate de lucru a interfețelor

2.3 Logica aplicației

Aplicația se bazează pe cele două nivele de acces, Operator și Administrator, păstrând pentru fiecare o interfață simplă pentru gestiunea de materii prime dintr-un depozit. Acțiunile din cadrul său se realizează cu ajutorul paginilor și butoanelor aferente fiecărei nevoi ale utilizatorului. După autentificarea acestuia, în pagina principală (Dashboard) se observă numele și rolul său, iar sesiunea este păstrată cu ajutorul cookie-urilor, astfel încât reconectarea frecventă să nu fie necesară.

Principalul scop al aplicației este reprezentat de activitatea dintre produse și tranzacții. În lista de produse sunt oferite detalii precum denumirea, SKU-ul unic, unitatea de măsură, stocul curent, stocul minim, raftul (locația fizică), prețul, descrierea și autorul ultimei modificări. Detalierea oricărui produs este afișată printr-un buton „Detalii produs” unde se poate vizualiza și imaginea acestuia, dacă aceasta a fost introdusă la crearea articolului. Dacă se dorește exportarea unui anumit produs într-un tabel Excel, acest lucru este posibil cu ajutorul meniului de căutare care funcționează multi-câmp pentru numele, SKU-ul, raftul, autorul și descrierea produsului

Tranzacțiile reprezintă nucleul operării zilnice, iar principalul său utilizator este operatorul care selectează produsul, tipul operațiunii (IN/OUT – intrare/ieșire) și cantitatea. Dacă apar nereguli cum ar fi în cazul cantității unde trebuie introdus un număr întreg strict pozitiv sau stoc inexistent în cazul tranzacțiilor de ieșire, se vor afișa mesaje clare precum „Stoc insuficient” sau „Cantitate invalidă”. Pentru o bună funcționare a programului, fiecare tranzacție reține stocul (înainte și după fiecare operațiune), autorul și momentul exact al creării sau editării.

Gestionarea utilizatorilor este reprezentată de administrator care poate crea conturi cu nume, prenume, email, parolă și rol, totodată să modifice rolurile sau de să elimine accesul în aplicație. Supravegherea produselor care au intrat sub minimul acceptat îi revin tot administratorului care constituie responsabilitatea platformei de lucru, care trebuie să urmărească și sa cerceteze orice neregularitate care se arată pe tot parcursul desfășurării organizatorice a depozitului.



Fig.2.2 Diagrama flux a aplicației

3. PROIECTAREA DETALIATĂ A APLICAȚIEI INFORMATICE

3.1 Crearea unui proiect backend și configurarea aplicației

Pentru realizarea acestei aplicații, a fost necesară instalarea Node.js[4] și PostgreSQL[6]. Serviciul backend a fost construit cu Express[5] și folosește pachete precum pg[7] pentru accesul la baza de date, dotenv pentru variabilele ascunse, bcrypt[18] pentru hașurarea parolelor, express-session[17] pentru gestionarea sesiunii și CORS[19] pentru verificarea accesului dintre backend și frontend. Toate liniile de cod necesare pentru funcționarea platformei au fost scrise în Visual Studio Code, fiind un editor gratuit, foarte popular printre programatori.

Configurarea scheletului de bază al aplicației este definit de rularea comenzii „npm init -y” care creează un fișier „package.json” cu setări implicite, de adăugarea pachetelor necesare prin comanda „npm i express pg dotenv bcrypt express-session cors” și de folosirea „npm install -g nodemon” care repornește automat aplicația prin „nodemon index.js” de fiecare dată când detectează schimbări la codul sursă.

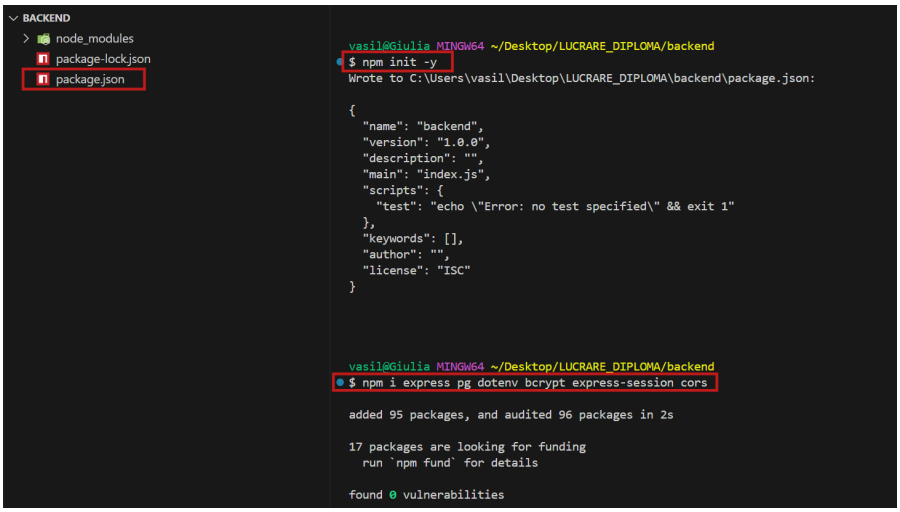


Fig.3.1 Instalarea pachetelor necesare

Conectarea la baza de date se realizează în interiorul folder-ului și fișierului „config/connectDB.js” unde, se citesc din „.env” variabilele ascunse necesare relației dintre baza de date și aplicație. După realizarea conexiunii, în consola VS Code apare mesajul „Conectat la BD!”

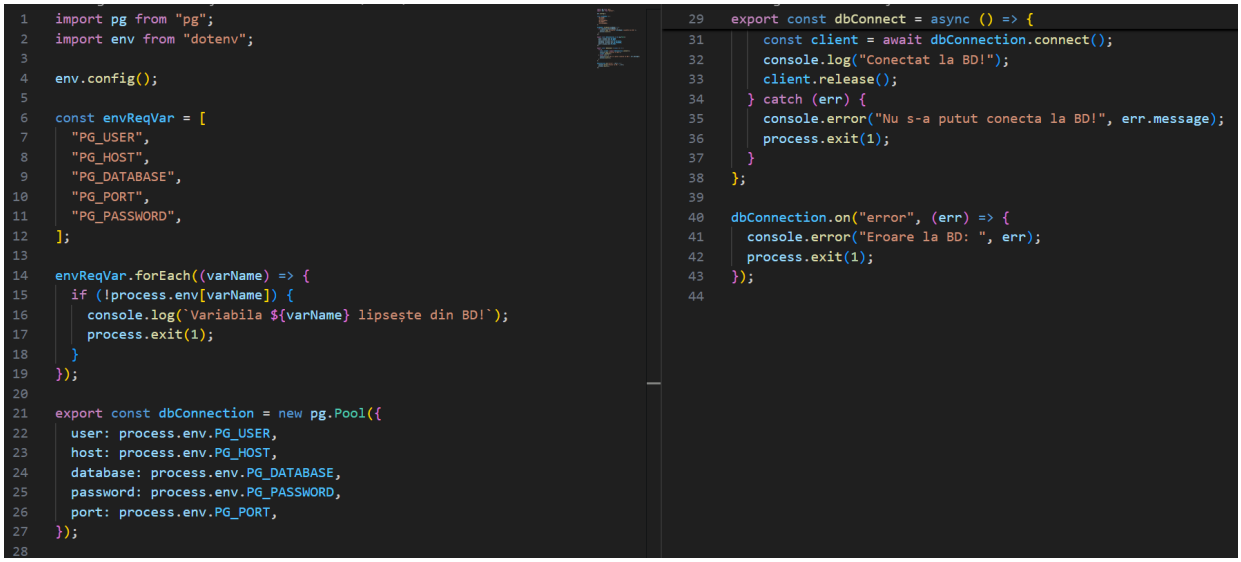


Fig.3.2 Codul necesar conectării cu PostgreSQL[6]

Backend-ul pornește în „index.js” prin încărcarea variabilelor ascunse din „.env”, pregătirea API-urilor cu middleware-ul „express.json()”, configurarea CORS[19] pe originea frontend-ului și prin permiterea cookie-urilor de sesiune între domenii în timpul autentificării.

Gestiunea sesiunii se face cu ajutorul pachetului express-session[17] unde cookie-ul cu numele „warehouse_session” are o durată de viață de 24 de ore și nu este re-salvat inutil. După confirmarea conexiunii, serverul montează rutele necesare, domeniile de funcționalitate fiind separate și ușor de testat. Pentru verificarea endpoint-ului se face o testare minimală unde acesta răspunde cu „API funcționează!”.

```
import express from "express";
import session from "express-session";
import env from "dotenv";
import cors from "cors";
import { dbConnect } from "./config/connectDB.js";
import authRouter from "./routes/authRoutes.js";
import prodRouter from "./routes/productRoutes.js";
import userRouter from "./routes/userRouter.js";
import trsRouter from "./routes/transactionsRoutes.js";

env.config();

const app = express();
const PORT = process.env.PORT;

app.use(express.json());
app.use(
  cors({
    origin: process.env.ORIGIN_SESS,
    credentials: true,
  })
);

app.use(
  session({
    name: "warehouse_session",
    secret: "ijuytfuijokmjnhygtef",
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      sameSite: "lax",
      maxAge: 24 * 60 * 60 * 1000,
    },
  })
);

dbConnect();

app.use("/api/auth", authRouter);
app.use("/api/products", prodRouter);
app.use("/api/users", userRouter);
app.use("/api/transactions", trsRouter);

app.get("/", (_req, res) => res.send("API functioneaza!"));

app.listen(PORT, () => {
  console.log(`Server-ul ruleaza pe portul ${PORT}!`);
});
```

Fig.3.3 Codul necesar conectării la baza de date din „index.js”

3.2 Crearea bazei de date PostgreSQL și conectarea la server

Baza de date a fost creată în programul PostgreSQL[6] și conține tabelele „users”, „products” și „transactions” acoperind astfel, personalul depozitului, catalogul de produse și istoricul mișcărilor de stoc. Conectarea aplicației la aceasta se realizează printr-un pool care permite reutilizarea datelor, iar verificarea este suportată de funcția „dbConnect()” care confirmă practic conexiunea.

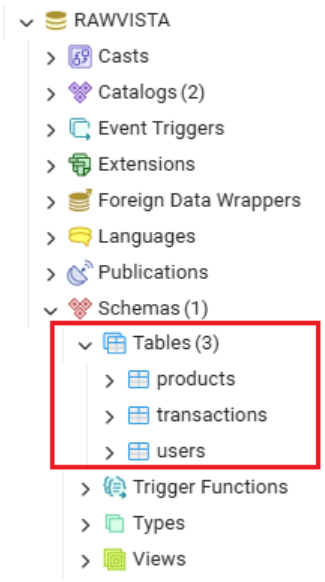


Fig.3.4 Afișarea tabelelor în „PostgreSQL[6]”

Tabelul „users” a fost conceput pentru administrarea utilizatorilor aplicației, cuprinzând diferite câmpuri de identificare (nume, prenume), autentificare (email, parolă) și autorizare (rol). În cadrul acestuia, email-ul unic utilizat la crearea sau accesarea contului, parola criptată și rolul, constituie împreună un element logic în ceea ce privește securitatea și diferențierea drepturilor în sistem.

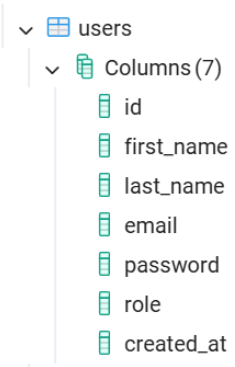


Fig.3.5 Structura tabelului „users”

Tabelul „products” a fost creat pentru gestionarea produselor și a stocurilor, având atribute precum: denumire, unitate, poziție pe raft (rack), descriere și link imagine. Un element important utilizat pentru prevenirea dublurilor produselor, folosit de asemenea și ca identificator, este câmpul „sku” care are o valoare unică. Pentru controlul stocurilor s-au definit coloanele „stock” și „stock_min”, folosite pentru semnalarea oricărei situații în care cantitatea poate scădea sub pragul minim acceptat. Tabelul conține de asemenea și câmpul „user_id”, prin care se permite vizualizarea utilizatorului care a creat produsul.

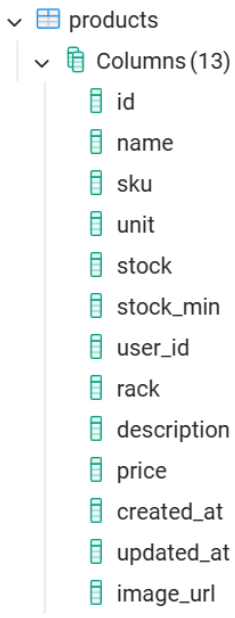


Fig.3.6 Structura tabelului „products”

Tabelul „transactions” a fost proiectat pentru evidențierea mișcărilor de stoc din depozit (intrări/ieșiri). Structura acestuia include câmpuri cu rol specific precum „delta”, care indică modificarea efectivă a stocului („+” pentru „IN”, „-” pentru „OUT”), dar și câmpuri care oferă trasabilitate prin salvarea stocul înainte și după operație, precum „before_stock” și „after_stock”. Aceste operații ajută la verificarea ulterioară a coerenței stocului fără a depinde exclusiv de valoarea curentă din tabel. În plus, acesta conține relații între tabelele „users” și „products” prin câmpurile „product_id” și „user_id”, asociind fiecare tranzacție cu utilizatorul și produsul aferent.

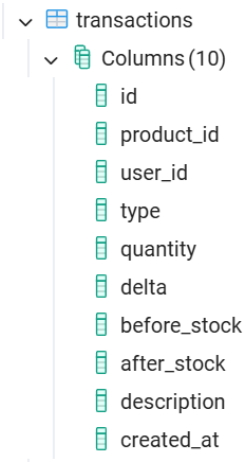


Fig.3.7 Structura tabelului „transactions”

Baza de date astfel definită asigură integritatea datelor, relațiile fiind testate prin înserarea de înregistrări care permit legături între utilizatori, produse și tranzacții.

3.3 Realizarea modelelor pentru baza de date din PostgreSQL

Accesul la baza de date din PostgreSQL[6] se regăsește în folder-ul „models/” care reprezintă centralizarea rutelor de acces și transformarea SQL-ului într-o interfață clară pe care rutele o folosesc. Practic, cu ajutorul modelelor create pentru fiecare tabel, codul devine mai lizibil și ușor de întreținut, protejând astfel regulile codului sursă.

Modelele au rolul de a oferi reutilizarea funcțiilor pentru manipularea datelor păstrând o logică de interogare atribuită fiecărei funcții în parte, asigurându-se astfel o separare clară între componentele aplicației și logica accesării datelor.

3.3.1 Fișierul „userModel.js”

În „userModel.js” sunt implementate funcțiile „createUser”, „getUserEmail”, „getUserId”, „getAllUsers”, „updateUser”, „deleteUser” care ajută la transferarea informațiilor din tabele pentru prelucrarea lor ulterioară în cadrul platformei.

Funcția „createUser” este utilizată pentru introducerea unui utilizator nou în tabelul „users”. Datele necesare creării unui cont reprezintă principalii parametrii care construiesc o interogare SQL de tip „INSERT”, rezultatul fiind returnat cu datele utilizatorului creat, dar fără ca parola acestuia să fie vizibilă pentru a evita posibilele expuneri de informații cu caracter personal.

Utilizarea acestei funcții este îndeplinită atât în procesul de înregistrare a utilizatorilor, cât și în administrarea utilizatorilor de către administrator.

```
export const createUser = async (
  first_name,
  last_name,
  email,
  password,
  role = "OPERATOR"
) => {
  const query = `
    INSERT INTO users (first_name, last_name, email, password, role)
    VALUES ($1,$2,$3,$4,$5)
    RETURNING id, first_name, last_name, email, role;
    `;
  const values = [first_name, last_name, email, password, role];
  const { rows } = await dbConnection.query(query, values);
  return rows[0];
};
```

Fig.3.8 Implementarea funcției „createUser”

Funcția „getUserEmail” are rolul de a căuta în baza de date un utilizator căruia i se atribuie o anumită adresă de email. Aceasta este folosită în special pentru a verifica existența unui utilizator înainte de înregistrarea sa în aplicație, precum și la procesul de autentificare. Evitarea duplicării adresei de email contribuie totodată în procesul de securitate al autentificării în cont.

Această funcție returnează primul rezultat găsit sau nedefinit (în cazul în care utilizatorul nu există) după ce se execută o interogare SQL de tip „SELECT”.

```
export const getUserEmail = async (email) => {
  const query = `SELECT * FROM users WHERE email =$1`;
  const { rows } = await dbConnection.query(query, [email]);
  return rows[0];
};
```

Fig.3.9 Implementarea funcției „getUserEmail”

Funcția „getUserId” este utilizată pentru a obține informații referitoare la un utilizator pe baza identificatorului său unic numit „id”. Diferența dintre această funcție și cea de căutare după email este că, la returnarea datelor, parola nu este expusă, afișarea fiind una stric necesară utilizării în aplicație. În acest fel, se poate verifica existența unui utilizator în cadrul operației administrative.

```
export const getUserId = async (id) => {
  const { rows } = await dbConnection.query(
    `SELECT id, first_name, last_name, email, role FROM users WHERE id=$1`,
    [id]
  );
  return rows[0];
};
```

Fig.3.10 Implementarea funcției „getUserId”

Funcția „getAllUsers” este responsabilă de prelucrarea tuturor utilizatorilor din baza de date, vizualizând astfel lista completă a celor existenți. Utilizatorul care are rolul de administrator poate să proceseze toate aceste informații esențiale atribuite fiecărui membru din cadrul aplicației.

La finalul interogării SQL, se returnează un set de rezultate care conține informații esențiale despre fiecare participant din cadrul depozitului.

```
export const getAllUsers = async () => {
  const query = `SELECT id, first_name, last_name, email, role FROM users`;
  const { rows } = await dbConnection.query(query);
  return rows;
};
```

Fig.3.11 Implementarea funcției „getAllUsers”

Funcția „updateUser” este folosită pentru modificarea datelor unui utilizator deja existent. Aceasta abordare oferă o flexibilitate in gestiunea datelor, evitând astfel o posibilă suprascriere inutilă a datelor existente.

```
export const updateUser = async (id, first_name, last_name, email, role) => {
  const query = `
    UPDATE USERS
    SET first_name = COALESCE($1, first_name),
    last_name = COALESCE($2, last_name),
    email = COALESCE($3, email),
    role = COALESCE($4, role)
    WHERE id = $5
    RETURNING id, first_name, last_name, email, role;
    `;

  const values = [first_name, last_name, email, role, id];
  const { rows } = await dbConnection.query(query, values);
  return rows[0];
};
```

Fig.3.12 Implementarea funcției „updateUser”

Conform Fig.3.12 „updateUser” este una dintre cele mai dezvoltate funcții din „usersModel.js”, rolul său fiind de a actualiza un utilizator deja existent în tabelul „users” și de a întoarce rândul parțial actualizat. Valoarea „COALESCE” este cea care aplică o modificare parțială pentru fiecare coloană unde, dacă datele primite sunt „NULL”, se păstrează valoarea veche, orice altă valoare trimisă devenind valoarea respectivă, coloana actualizându-se.

Funcția „deleteUser” este aplicată doar în cazul în care se dorește ștergerea unui utilizator din baza de date. Această interogare SQL de tip „DELETE” identifică utilizatorul pe baza id-ului unic și execută cerința de ștergere a acestuia, utilizarea funcției fiind exclusiv în zona de administrare a aplicației.

```
56  export const deleteUser = async (id) => {
57    const query = `DELETE FROM users WHERE id = $1 RETURNING id, email`;
58    const { rows } = await dbConnection.query(query, [id]);
59    return rows[0];
60  };
61
```

Fig.3.13 Implementarea funcției „deleteUser”

3.3.2 Fișierul „productsModel.js”

În „productsModel.js” sunt implementate funcțiile „createProduct”, „getIdProduct”, „getAllProducts”, „updateProduct”, „deleteProduct” care ajută la gestionarea câmpurilor principale ale produsului.

Funcția „createProduct” ajută la inserarea unui produs nou în tabelul de produse. În cadrul acesteia se regăsește o interogare SQL de tip „INSERT”, care adaugă valorile produsului creat în tabel.

Un aspect important al acestei funcții este că permite obținerea imediată a informațiilor despre utilizatorul care a introdus datele noului articol, printr-o subinterogare asupra tabelului „users”.

```
export const createProduct = async ({
  name,
  sku,
  unit = "buc",
  stock = 0,
  stock_min = 0,
  user_id = null,
  rack = null,
  description = null,
  price = 0,
  image_url = null,
}) => {
  const sql = `
    INSERT INTO products (name, sku, unit, stock, stock_min, user_id, rack, description, price, image_url)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)
    RETURNING id, name, sku, unit, stock, stock_min, user_id, rack, description, price, image_url,created_at, updated_at,
    (SELECT u.first_name || ' ' || u.last_name FROM users u WHERE u.id = products.user_id) AS created_by
  `;
  const values = [
    name,
    sku,
    unit,
    stock,
    stock_min,
    user_id,
    rack,
    description,
    price,
    image_url,
  ];
  const { rows } = await dbConnection.query(sql, values);
  return rows[0];
};
```

Fig.3.14 Implementarea funcției „createProduct”

Funcția „getIdProduct” are rolul de a recunoaște un produs pe baza identificatorului unic (id) unde, prin interogarea SQL de tip „SELECT”, se recunoaște existența sau nu a acestuia înainte de efectuarea operațiilor de modificare sau ștergere din tabel. Totodată, dacă articolul există, funcția va returna un singur rezultat, dacă acesta nu există, returnarea se va face sub formă de „undefined”.

```
export const getIdProduct = async (id) => {
  const query = `SELECT * FROM products WHERE id=$1`;
  const { rows } = await dbConnection.query(query, [id]);
  return rows[0];
};
```

Fig.3.15 Implementarea funcției „getIdProduct”

Funcția „getAllProducts” este utilizată pentru prelucrarea tuturor produselor din baza de date. Interogarea SQL folosește „LEFT JOIN” care combină datele din tabelele „products” și „users”, pentru a obține informații despre fiecare produs creat de un anumit utilizator. Rezultatele returnate la executarea funcției permit ordonarea descrescătoare după identificator, ceea ce facilitează afișarea celor mai recente produse din cadrul bazei de date.

```
export const getAllProducts = async () => {
  const sql = `
    SELECT
      p.id, p.name, p.sku, p.unit, p.stock, p.stock_min, p.user_id, p.rack,
      p.price, p.image_url, p.description, p.updated_at,
      (u.first_name || ' ' || u.last_name) AS created_by
    FROM products p
    LEFT JOIN users u ON u.id = p.user_id
    ORDER BY p.id DESC;
  `;
  const { rows } = await dbConnection.query(sql);
  return rows;
};
```

Fig.3.16 Implementarea funcției „getAllProducts”

Funcția „updateProduct” este principala modalitate de modificare a datelor unui produs existent. Actualizarea selectivă a câmpurilor este posibilă folosind „COALESCE”, ceea ce permite furnizarea valorilor modificate și păstrarea celor neschimbate.

În plus, câmpul „updated_at” ajută la actualizarea automată a momentului în care a fost înregistrată o modificare a unui articol, fiind returnată versiunea recentă a produsului și informațiile despre utilizatorul implicat.

```
export const updateProduct = async (id, data) => {
  const {
    name = null,
    sku = null,
    unit = null,
    stock = null,
    stock_min = null,
    user_id = null,
    rack = null,
    description = null,
    price = null,
    image_url = null,
  } = data;

  const sql = `
    UPDATE products
    SET
      name        = COALESCE($1, name),
      sku         = COALESCE($2, sku),
      unit        = COALESCE($3, unit),
      stock       = COALESCE($4, stock),
      stock_min   = COALESCE($5, stock_min),
      user_id     = COALESCE($6, user_id),
      rack        = COALESCE($7, rack),
      description = COALESCE($8, description),
      price       = COALESCE($9, price),
      image_url   = COALESCE($10, image_url),
      updated_at  = NOW()
    WHERE id = $11
    RETURNING
      id, name, sku, unit, stock, stock_min, user_id, rack, description, price, image_url,
      created_at, updated_at,
      (SELECT u.first_name || ' ' || u.last_name FROM users u WHERE u.id = products.user_id) AS created_by
  `;
```

Fig.3.17 Implementarea funcției „updateProduct” (partea 1)

```
  const values = [
    name,
    sku,
    unit,
    stock,
    stock_min,
    user_id,
    rack,
    description,
    price,
    image_url,
    id,
  ];
  const { rows } = await dbConnection.query(sql, values);
  return rows[0];
};
```

Fig.3.18 Implementarea funcției „updateProduct” (partea 2)

Funcția „deleteProduct” este responsabilă cu ștergerea unui produs din baza de date, executând o interogare SQL de tip „DELETE”. Cu ajutorul identificatorului produsului, se confirmă eliminarea acestuia din baza de date.

```
export const deleteProduct = async (id) => {
  const query = `DELETE FROM products WHERE id=$1 RETURNING id,name,sku`;
  const { rows } = await dbConnection.query(query, [id]);
  return rows[0];
};
```

Fig.3.19 Implementarea funcției „deleteProduct”

3.3.3 Fișierul „transactionsModel.js”

În „transactionsModel.js” fiecare mișcare de stoc păstrează o trasabilitate completă prin descrierea tipului și calcularea valorii „delta” care ne ajută să actualizăm sau nu stocul produsului. Produsul este blocat cu ajutorul funcției „SELECT ... FOR UPDATE” ceea ce ne permite verificarea „after_stock >= 0”, unde acesta este actualizat și inserat în tranzacție. La orice eșec al funcției, are loc „ROLLBACK” care practic anulează întreaga operație începută cu „BEGIN” astfel încât baza de date să revină la starea de dinainte de începerea operațiunilor, iar dacă aceasta este un succes, „END” confirmă modificările.

Funcția „createTransaction” are rolul de a înregistra o nouă tranzacție de tip „IN” sau „OUT”. Pentru evitarea modificărilor concurente, produsul este selectat cu „SELECT ... FOR UPDATE”, ceea ce blochează rândul corespunzător acestuia până la finalizarea tranzacției.

În cadrul acestei funcții se calculează o valoare notată cu „delta”, prin care se poate observa variația efectivă a stocului precum și valorile „before_stock” și „after_stock”. După ce este înregistrată actualizarea stocului, înserarea se face automat în tabelul de tranzacții, iar în cazul unei erori apărute, modificările sunt anulate prin comanda „ROLLBACK”.

```
export const createTransaction = async ({
  product_id,
  user_id = null,
  type,
  quantity,
  description = null,
}) => {
  const client = await dbConnection.connect();
  try {
    await client.query("BEGIN");

    const { rows: prodRows } = await client.query(
      "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
      [product_id]
    );
    if (!prodRows[0]) throw new Error("Produsul nu a fost găsit!");

    const before = prodRows[0].stock;
    const typeUp = String(type).toUpperCase();
    const qty = Number(quantity);
    const delta = typeUp === "IN" ? qty : -qty;
    const after = before + delta;
    if (after < 0) throw new Error("Stock-ul este insuficient!");

    await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
      after,
      product_id,
    ]);
```

Fig.3.20 Implementarea funcției „createTransaction” (partea 1)

```
    const { rows: ins } = await client.query(
      `INSERT INTO transactions
      (product_id, user_id, type, quantity, delta, before_stock, after_stock, description)
      VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
      RETURNING id`,
      [product_id, user_id, typeUp, qty, delta, before, after, description]
    );
    const newId = ins[0].id;

    const { rows: full } = await client.query(SELECT_FULL_BY_ID, [newId]);

    await client.query("END");
    return full[0];
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();
  }
};
```

Fig.3.21 Implementarea funcției „createTransaction” (partea 2)

Funcția „getAllTransactions” este utilizată pentru prelucrarea tuturor tranzacțiilor din baza de date. Cu ajutorul operațiunii „JOIN” din cadrul interogării SQL, se permite obținerea informațiilor despre tranzacționarea produsului și a utilizatorului care a realizat operația.

Rezultatele sunt afișate în ordine descrescătoare după data creării, facilitând afișarea celor mai recente tranzacții din cadrul aplicației.

```
export const getAllTransactions = async () => {
  const { rows } = await dbConnection.query(
    `${SELECT_FULL} ORDER BY t.created_at DESC`
  );
  return rows;
};
```

Fig.3.22 Implementarea funcției „getAllTransactions”

Funcția „getAllTransactionsID” permite obținerea și afișarea detaliilor complete ale unei tranzacții pe baza id-ului unic al acesteia.

```
export const getAllTransactionsID = async (id) => {
  const { rows } = await dbConnection.query(
    `SELECT
      t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
      t.before_stock, t.after_stock, t.description, t.created_at,
      p.name  AS product_name,
      p.sku   AS product_sku,
      p.rack  AS product_rack,
      (u.first_name || ' ' || u.last_name) AS created_by
    FROM transactions t
    LEFT JOIN products p ON p.id = t.product_id
    LEFT JOIN users u    ON u.id = t.user_id
    WHERE t.id = $1`,
    [id]
  );
  return rows[0];
};
```

Fig.3.23 Implementarea funcției „getAllTransactionsID”

Funcția „getAllProductTransactions” se folosește pentru gestionarea tuturor tranzacțiilor aferente unui anumit produs prin filtrarea sa după „product_id” și vizualizarea istoricului complet al acestuia.

```
export const getAllProductTransactions = async (product_id) => {
  const { rows } = await dbConnection.query(
    `SELECT
      t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
      t.before_stock, t.after_stock, t.description, t.created_at,
      p.name  AS product_name,
      p.sku   AS product_sku,
      p.rack  AS product_rack,
      (u.first_name || ' ' || u.last_name) AS created_by
    FROM transactions t
    LEFT JOIN products p ON p.id = t.product_id
    LEFT JOIN users u    ON u.id = t.user_id
    WHERE t.product_id = $1
    ORDER BY t.created_at DESC`,
    [product_id]
  );
  return rows;
};
```

Fig.3.24 Implementarea funcției „getAllProductTransaction

Funcția „getUserTransactions” ajută la returnarea tuturor tranzacțiilor efectuate de un anumit utilizator, datele fiind vizibile numai pentru utilizatorul care a realizat propriile înregistrări. Cu toate acestea, accesul la afișarea întregului istoric este posibil doar prin autentificarea într-un cont care are rolul de administrator.

```
export const getUserTransactions = async (user_id) => {
  const { rows } = await dbConnection.query(
    `${SELECT_FULL} WHERE t.user_id = $1 ORDER BY t.created_at DES
    [user_id]
  );
  return rows;
};
```

Fig.3.25 Implementarea funcției „getUserTransactions”

Funcția „updateTransactionDescription” modifică descrierea unei tranzacții existente, fără a permite afectarea stocului unui produs. În plus, funcția este destinată în special utilizatorilor care au rolul de operator și care pot aduce modificări minore în editarea tranzacțiilor.

```
export const updateTransactionDescription = async (id, description) => {
  const { rows: upd } = await dbConnection.query(
    `UPDATE transactions
      SET description = $1
      WHERE id = $2
      RETURNING id`,
    [description, id]
  );
  if (!upd[0]) return null;
  const { rows: full } = await dbConnection.query(SELECT_FULL_BY_ID, [id]);
  return full[0];
};
```

Fig.3.26 Implementarea funcției „updateTransactionDescription”

Funcția „updateTransactionFULL” este modalitatea prin care utilizatorul cu rolul de administrator poate modifica total o tranzacție, reprezentând una dintre cele mai complexe funcții din aplicație. Aceasta folosește o tranzacție SQL care îi permite să revină asupra informațiilor tranzacției inițiale și să modifice valoarea stocului unde, după aplicarea noilor valori și revizuirea datelor, recalculează stocul final.

Principalele comenzi prin care ajustarea datelor tranzacțiilor este posibilă sunt comenzile „BEGIN” și „ROLLBACK” care asigură prevenirea apariției datelor incorecte sau parțial actualizate din cadrul aplicației.

```
export const updateTransactionFULL = async (
  id,
  { product_id, type, quantity, description }
) => {
  const client = await dbConnection.connect();
  try {
    await client.query("BEGIN");

    const { rows: trsRows } = await client.query(
      `SELECT id, product_id AS old_product_id, type AS old_type,
              quantity AS old_quantity, delta AS old_delta
       FROM transactions
       WHERE id = $1
       FOR UPDATE`,
      [id]
    );
    if (!trsRows[0]) throw new Error("Tranzacția nu a fost găsită!");

    const { old_product_id, old_delta } = trsRows[0];

    const newType = String(type).toUpperCase();
    const qty = Number(quantity);
    if (!["IN", "OUT"].includes(newType))
      throw new Error("Tip de tranzacție invalid!");
    if (!Number.isInteger(qty) || qty <= 0)
      throw new Error("Cantitate invalidă!");

    const newDelta = newType === "IN" ? qty : -qty;
    const targetProduct = product_id ?? old_product_id;
```

Fig.3.27 Implementarea funcției „updateTransactionFULL” (partea 1)

```
    const { rows: oldProdRows } = await client.query(
      "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
      [old_product_id]
    );
    if (!oldProdRows[0]) throw new Error("Produsul nu a fost găsit!");
    const revertedStock = oldProdRows[0].stock - Number(old_delta);
    if (revertedStock < 0)
      throw new Error("Stocul nu este suficient pentru a putea fi modificat!");
    await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
      revertedStock,
      old_product_id,
    ]);

    let beforeStockValue, afterStockValue;
    if (targetProduct === old_product_id) {
      beforeStockValue = revertedStock;
      afterStockValue = beforeStockValue + newDelta;
      if (afterStockValue < 0) throw new Error("Stocul nu este suficient!");
      await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
        afterStockValue,
        targetProduct,
      ]);
    } else {
      const { rows: newProdRows } = await client.query(
        "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
        [targetProduct]
      );
      if (!newProdRows[0])
        throw new Error("Produsul selectat nu a fost găsit!");
      beforeStockValue = newProdRows[0].stock;
      afterStockValue = beforeStockValue + newDelta;
      if (afterStockValue < 0) throw new Error("Stock-ul este insuficient!");
      await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
        afterStockValue,
        targetProduct,
      ]);
    }
```

Fig.3.28 Implementarea funcției „updateTransactionFULL” (partea 2)

```
    await client.query(
      `UPDATE transactions
         SET product_id = $1,
             type       = $2,
             quantity   = $3,
             delta      = $4,
             before_stock = $5,
             after_stock  = $6,
             description  = COALESCE($7, description)
       WHERE id = $8`,
      [
        targetProduct,
        newType,
        qty,
        newDelta,
        beforeStockValue,
        afterStockValue,
        description ?? null,
        id,
      ]
    );

    const { rows: full } = await client.query(SELECT_FULL_BY_ID, [id]);

    await client.query("END");
    return full[0];
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();
  }
};
```

Fig.3.29 Implementarea funcției „updateTransactionFULL” (partea 3)

Funcția „deleteTransaction” are ca scop principal ștergerea unei tranzacții existente din baza de date. Totuși, valoarea stocului este ajustată prin inversarea valorii „delta” care permite păstrarea unei consistențe a inventarul chiar dacă valoarea sa a fost înlăturată.

```
export const deleteTransaction = async (id) => {
  const client = await dbConnection.connect();
  try {
    await client.query("BEGIN");

    const { rows: trsRows } = await client.query(
      "SELECT id, product_id, delta FROM transactions WHERE id = $1 FOR UPDATE",
      [id]
    );
    if (!trsRows[0]) throw new Error("Tranzacția nu a fost găsită!");

    const { product_id, delta } = trsRows[0];

    const { rows: prodRows } = await client.query(
      "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
      [product_id]
    );
    if (!prodRows[0]) throw new Error("Produsul nu a fost găsit!");

    const currentStock = prodRows[0].stock;
    const newStock = currentStock - Number(delta);
    if (newStock < 0)
      throw new Error("Nu este stock suficient pentru a termina operația!");

    await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
      newStock,
      product_id,
    ]);
```

Fig.3.30 Implementarea funcției „deleteTransaction” (partea 1)

```
    const { rows: del } = await client.query(
      "DELETE FROM transactions WHERE id = $1 RETURNING id, product_id",
      [id]
    );

    await client.query("END");
    return del[0];
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();
  }
};
```

Fig.3.31 Implementarea funcției „deleteTransaction” (partea 2)

În urma analizei fiecărei funcții din fișierele „usersModel.js”, „productsModel.js” și „transactionsModels.js”, se observă că definirea modelelor aplicației construiește structura de funcționare a tabelelor. Acestea sunt scrise în limbajul de programare JavaScript[3] și rulate cu ajutorul mediului de execuție Node.js[4] care, prin intermediul unei biblioteci specializate, realizează operații esențiale accesului la baza de date din PostgreSQL[6], precum crearea, citirea, actualizarea și ștergerea înregistrărilor din tabele.

Gestionarea operațiilor mai complexe de modificare totală sau de ștergere a tranzacțiilor, a fost posibilă cu ajutorul comenzilor „BEGIN”, „COMMIT” și „ROLLBACK” care mențin executarea logică a mai multor comenzi SQL.

Datorită funcțiilor JavaScript[3] care manipulează aceleași entități, evidențierea relațiilor dintre tabelele din baza de date reiese din Fig. 3.32, unde tabelul „transactions” face legătura între „users” și „products”. De asemenea, fiecare tranzacție corespunde unui singur produs și unui singur utilizator prin atributul „product_id” și, respectiv, „user_id”, ceea ce permite produsului sau utilizatorului să poată avea multiple tranzacții asociate, formând astfel relații One-to-Many.

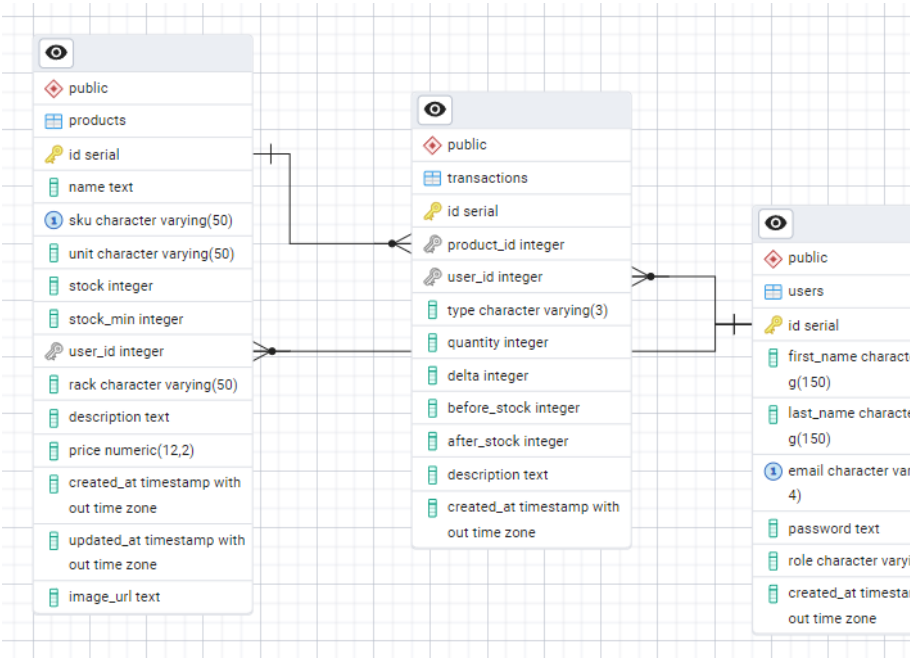


Fig.3.32 Diagrama relațională a bazei de date din PostgreSQL[6]

Tabelele din baza de date, prezentate în schema anterioară, sunt detaliate în cele ce urmează:

users: tabel realizat cu scopul înregistrării și autentificării în aplicație, dar și pentru gestionarea utilizatorilor din sistem.

Coloane:
ID = tip: număr întreg, autoincrementabil, cheie primară.
first_name = tip: text (varchar), maxim 150 de caractere, nu permite valori nule.
last_name = tip: text (varchar), maxim 150 de caractere, nu permite valori nule.
email = tip: text (varchar), maxim 254 caractere, nu permite valori nule, valoare unică.
password = tip: text, nu permite valori nule.
role = tip: text (varchar), maxim 50 de caractere, nu permite valori nule.
created_at = tip: timestamp fără fus orar, nu permite valori nule.

products: tabel realizat pentru stocarea informațiilor despre produsele din cadrul depozitului.

Coloane:
ID = tip: număr întreg, autoincrementabil, cheie primară.
name = tip: text, nu permite valori nule.
sku = tip: text (varchar), maxim 50 de caractere, nu permite valori nule, valoare unică.
unit = tip: text (varchar), maxim 50 de caractere.
stock = tip: număr întreg.
stock_min = tip: număr întreg.
user_id: tip: număr întreg, cheie străină, relație de tip Many-to-One cu tabelul „users”.
rack = tip: text (varchar), maxim 50 de caractere.
description = tip: text.
price = tip: numeric (12,2).
image_url = tip: text.
created_at = tip: timestamp fără fus orar.
updated_at = tip: timestamp fără fus orar.

transactions: tabel destinat înregistrării istoricului mișcărilor de stoc, având rolul de a monitoriza toate operațiile de intrare și de ieșire ale produselor.

Coloane:
ID = tip: număr întreg, autoincrementabil, cheie primară.
product_id = tip: număr întreg, cheie străină, relație de tip Many-to-One cu tabelul „products”.
user_id = tip: număr întreg, cheie străină, relație de tip Many-to-One cu tabelul „users”, permite valoare nulă.
type = tip: text (varchar), maxim 3 caractere, nu permite valori nule.
quantity = tip: număr întreg, nu permite valori nule.
delta = tip: număr întreg.
before_stock = tip: număr întreg.
after_stock = tip: număr întreg.
description = tip: text.
created_at = tip: timestamp fără fus orar, nu permite valori nule.

3.4 Arhitectura și funcționarea backend-ului aplicației

3.4.1 Structura componentelor backend

În structura backend-ului aplicației, sistemul este organizat în mai multe directoare, fiecare având un rol clar în procesarea cererilor HTTP. Conform Fig. 3.33, stratul de acces la baza de date este realizat cu ajutorul fișierelor următoare: Controller, acest document conține funcții care gestionează datele primite de la cererile HTTP, unde aplică validările necesare, accesează baza de date și construiește răspunsurile returnate clientului. Cu alte cuvinte, controller-ele conțin logica principală a aplicației, fiind punctul central unde rutele definite și stratul de acces la baza de date se coordonează. Models, un fișier care definește funcții JavaScript[3] și execută interogări SQL asupra bazei de date din PostgreSQL[6], fiind astfel modalitatea de acces la mecanismul de stocare. Routes, componenta responsabilă de definirea punctelor de acces ale aplicației. Astfel, fișierele stocate în aceasta declară rute pentru autentificare, utilizatori, produse sau tranzacții, utilizând metode HTTP precum „GET”, „POST”, „PUT”, „PATCH” și „DELETE”. În schimb, componentele Config și Middleware nu reprezintă o relație directă între componentele aplicației, de aceea poziționarea lor în diagramă are rolul de relaționare opțională. Cu toate acestea, componenta Config stabilește conexiunea la baza de date, asigurându-se ca informațiile sensibile sunt separate de logica de bază a aplicației, iar Middleware-ul este responsabil cu implementarea mecanismelor de acces și securitate, verificându-se astfel autentificarea utilizatorilor, rolurile și permisiunile pentru accesarea anumitor rute din cadrul platformei.

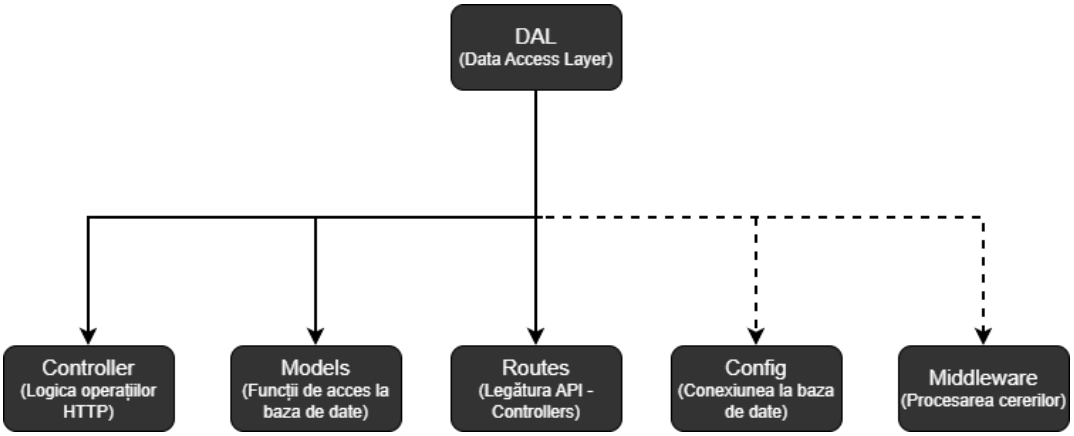


Fig.3.33 Arhitectura backend-ului aplicației Node.js[4]

3.4.2 Logarea, înregistrarea și autentificarea utilizatorilor

Mecanismele oferite de protocolul HTTP, mediul Node.js[4] și framework-ul Express[5] permit funcționalităților de înregistrare, autentificare și gestionare a sesiunilor utilizatorului să se grupeze în cadrul rutelor dedicate autentificării, acestea fiind regăsite în fișierul „authRoutes.js”.

Accesarea generală a serviciului de autentificare a fost realizată prin intermediul adresei locale „http://localhost:3000/api/auth”, portul utilizat fiind obținut prin citirea fișierului „.env” care permite modificarea acestuia fără a putea fi afectat codul sursă al aplicației.

```
const PORT = process.env.PORT;

app.listen(PORT, () => {
  console.log(`Server-ul ruleaza pe portul ${PORT}!`);
});
```

Fig.3.34 Preluarea portului din fișierul „.env” și pornirea serverului

Întregul proces de verificare a sesiunii, înregistrare, autentificare, delogare a fost realizat prin intermediul cererilor HTTP diferite: logarea și înregistrarea au fost realizate prin folosirea cererilor de tip „POST”, verificarea sesiunii cu ajutorul cererii „GET”, iar delogarea utilizând comanda „DELETE”.

```
import express from "express";
import { register, login, logout, me } from "../controllers/authController.js";

const authRouter = express.Router();

authRouter.post("/register", register);

authRouter.post("/login", login);

authRouter.get("/me", me);

authRouter.delete("/logout", logout);

export default authRouter;
```

Fig.3.35 Definirea rutelor de autentificare din fișierul „authRoutes.js”

Conform Fig.3.35, se poate observa că înregistrarea utilizatorilor folosește ruta „/register”, unde această cerere transmite datele utilizatorului către server înainte de a fi salvate în baza de date. Totuși, în cadrul înregistrării, parola este procesată folosind biblioteca bcrypt[18], care permite aplicarea algoritmului de criptare, asigurându-se astfel o stocare securizată a parolei în baza de date.

Autentificarea utilizatorului folosește ruta „/login” prin care serverul verifică existența sa și validează accesarea în sesiune. De asemenea, verificarea sesiunii prin accesarea rutei „/me” oferă datele persoanei conectate fără retrimiterea informațiilor necesare identificării contului.

Mecanismul care stă la baza menținerii stării de autentificare este oferit de middleware-ul express-session[17], care configurează cookie-ul de sesiune cu o durată de viață de 24 de ore și îl transmite automat între client și server. În momentul autentificării, cookie-ul denumit „warehouse_session” stochează informații esențiale despre utilizatorul conectat la server. Proprietățile de securitate pe care acesta le are sunt „httpOnly” și „sameSite”, care, în primă fază, împiedică codul JavaScript[3] rulat în browser să acceseze valoarea cookie-ului de sesiune atât pentru citire cât și pentru modificare, ca mai apoi să nu fie trimis în cereri care provin din alte site-uri, prevenindu-se utilizarea acestuia în contexte cu potențial malițios.

```
app.use(
  session({
    name: "warehouse_session",
    secret: "ijuytfuijokmjnhygtef",
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      sameSite: "lax",
      maxAge: 24 * 60 * 60 * 1000,
    },
  })
);
```

Fig.3.36 Configurarea cookie-ului de sesiune din „index.js”

Pentru transmiterea corectă a sesiunii între client și server, sunt activate opțiunile „credentials: true” și „withCredentials: true”, regăsite în configurația CORS[19] pe server, respectiv în configurația Axios[15] pe client, permițând transferul cookie-ului de sesiune în cererile HTTP.

```
app.use(express.json());
app.use(
  cors({
    origin: process.env.ORIGIN_SESS,
    credentials: true,
  })
);
```

Fig.3.37 Configurarea CORS[19] din „index.js”

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
  headers: { "Content-Type": "application/json" },
});
```

Fig.3.38 Configurarea Axios[15] din „Axios.js”

Delogarea utilizatorului are ca efect încheierea sesiunii curente și eliminarea cookie-ului asociat contului prin accesarea rutei „/logout”. În urma acestei operații, utilizatorului nu i se mai permite accesarea rutelor protejate din cadrul aplicației.

3.4.3 Implementarea controllerelor și a rutelor HTTP ale aplicației

Logica de procesare a cererilor HTTP se regăsește în fișierul „controllers”, care preia cererile din interfața frontend-ului și apelează funcțiile din documentul „models”, unde se înregistrează operațiile asupra bazei de date. Totodată, pe lângă mecanismele de securitate aplicate prin cadrul unui middleware de autentificare, anumite controllere includ măsuri de protecție sporite datorită verificărilor suplimentare ale rolului utilizatorului, prevenind orice urmă de acces neautorizat.

3.4.3.1 Controller-ul de autentificare din „authController.js”

Controller-ul de autentificare administrează numeroase funcții prin care este realizată înregistrarea, logarea, delogarea și identificarea utilizatorului conectat în aplicație. Implementarea acestora este îndeplinită prin obținerea datelor din modelul „userModel.js”, iar biblioteca bcrypt[18] verifică și criptează parolele asociate conturilor.

Funcția „register” primește informațiile asociate profilului utilizatorului prin „req.body”, unde în primă fază se validează existența datelor, iar ulterior se verifică dacă emailul utilizat există deja în sistem. Dacă înregistrarea emailului a fost deja realizată, serverul returnează un status care indică acest lucru printr-un mesaj corespunzător.

Pentru securizarea și criptarea parolei se utilizează „bcrypt.hash”, care, printr-un număr de iterații de salt, permite algoritmului să aplice funcția de hash de mai multe ori, ceea ce face imposibilă ghicirea rapidă a parolei. După procesul de criptare, datele sunt salvate în baza de date prin funcția „createUser”, iar înregistrarea utilizatorului este confirmată prin returnarea unui status pozitiv la finalul operației.

```
export const register = async (req, res) => {
  try {
    const { first_name, last_name, email, password, role } = req.body;

    if (!first_name || !last_name || !email || !password) {
      return res.status(400).json({ message: "Lipsesc câmpuri obligatorii!" });
    }

    const existingUser = await getUserEmail(email);
    if (existingUser) {
      return res
        .status(400)
        .json({ message: "Acest email a fost deja folosit!" });
    }

    const saltedPassword = await bcrypt.hash(password, saltRounds);

    const newUser = await createUser(
      first_name,
      last_name,
      email,
      saltedPassword,
      role,
    );

   return res.status(201).json({ message: "Înregistrare reușită!", user: newUser });
  } catch (err) {
    console.error("Eroare la înregistrare: ", err);
    return res.status(500).json({ message: "Eroare server!" });
  }
};
```

Fig.3.39 Implementarea funcției „register”

Funcția „login” permite autentificarea utilizatorului prin primirea emailului și parolei din cadrul bazei de date. În primă fază, se face o verificare a emailului, iar dacă acesta este atribuit unui cont se trece la verificarea parolei cu ajutorul „bcrypt.compare”. În cazul în care autentificarea a reușit, sesiunea activă din care face parte utilizatorul reține identificatorul și rolul său printr-un cookie de sesiune păstrat în browser.

```
export const login = async (req, res) => {
  try {
    const {email,password} = req.body
    const user = await getUserEmail(email);
    if (!user) {
      return res
        .status(400)
        .json({ message: "Email-ul sau parola sunt incorecte!" });
    }

    const comparePassword = await bcrypt.compare(password, user.password);
    if (!comparePassword) {
      return res
        .status(400)
        .json({ message: "Email-ul sau parola sunt incorecte!" });
    }

    req.session.user = { id: user.id, role: user.role };
    return res.status(200).json({
      message: "Autentificarea a reușit!",
      user: {
        id: user.id,
        first_name: user.first_name,
        last_name: user.last_name,
        email: user.email,
        role: user.role,
      },
    });
  } catch (err) {
    console.error("Eroare la autentificare: ", err);
    return res.status(500).json({ message: "Eroare server!" });
  }
};
```

Fig.3.40 Implementarea funcției „login”

Funcția „me” verifică dacă utilizatorul este sau nu autentificat în cont. Dacă sesiunea există, datele utilizatorului sunt returnate către frontend, iar dacă nu există, proprietatea „req.session.user” este nulă, returnându-se astfel un răspuns negativ.

```
export const me = async (req, res) => {
  try {
    if (!req.session.user) {
      return res.status(401).json({ message: "Întâi trebuie să te loghezi!" });
    }

    const user = await getUserId(req.session.user.id);
    if (!user) {
      return res.status(404).json({ message: "Utilizatorul nu a fost găsit!" });
    }

    return res.status(200).json(user);
  } catch (err) {
    console.error("Error la /me: ", err);
    return res.status(500).json({ message: "Eroare server!!!" });
  }
};
```

Fig.3.41 Implementarea funcției „me”

Funcția „logout” elimină sesiunea activă de pe server utilizând proprietatea „req.session.destroy”, iar cookie-ul „warehouse_session” este șters din browser utilizând „res.clearCookie”, ceea ce permite delogarea completă a utilizatorului.

```
export const logout = (req, res) => {
  req.session.destroy((err) => {
    if (err)
      return res.status(400).json({ message: "Nu s-a putut face logout!" });
    res.clearCookie("warehouse_session");
    return res.status(200).json({ message: "Ai fost delogat cu succes!" });
  });
};
```

Fig.3.42 Implementarea funcției „logout”

3.4.3.2 Controller-ul pentru produse din „productsController.js”

Controller-ul pentru produse permite gestionarea corectă a catalogului de materii prime cu ajutorul operațiilor de tip CRUD. Utilizarea metodelor din „productsModel.js” și restricționarea rutelor de accesare a datelor reprezintă principalele funcții ale acestuia.

Funcția „createProductController” ajută la crearea unui produs prin înserarea datelor din „req.body”. Înainte de stocarea lor în sistem este necesară verificarea rolului de administrator al utilizatorului pentru o protecție sporită, dar și a câmpurilor obligatorii necesare în finalizarea procesului de creare a produsului. În cazul în care există posibilitatea de „SKU” duplicat, codul „23505” confirmă o încălcare a constrângerii din baza de date PostgreSQL[6] și indică acest lucru prin mesajul „Acest SKU a fost deja folosit!”, totul fără a opri funcționarea serverului.

```
export const createProductController = async (req, res) => {
  try {
    const me = req.session?.user;
    if (!me)
      return res.status(401).json({ message: "Autentificare necesară." });
    if (me.role !== "ADMINISTRATOR") {
      return res
        .status(403)
        .json({ message: "Doar administratorul poate crea produse." });
    }

    const {
      name,
      sku,
      unit,
      stock,
      stock_min,
      rack,
      description,
      price,
      image_url,
    } = req.body;

    if (!name || !sku) {
      return res
        .status(400)
        .json({ message: "Numele și SKU sunt obligatorii." });
    }
```

Fig.3.43 Implementarea funcției „createProductController” (partea 1)

```
    const prod = await createProduct({
      name,
      sku,
      unit,
      stock,
      stock_min,
      user_id: me.id,
      rack,
      description,
      price,
      image_url,
    });

    return res.status(201).json(prod);
  } catch (error) {
    console.error("Eroare la crearea de produse!", error);
    if (error?.code === "23505") {
      return res
        .status(400)
        .json({ message: "Acest SKU a fost deja folosit!" });
    }
    return res.status(500).json({ message: "Eroare server!" });
  }
};
```

Fig.3.44 Implementarea funcției „createProductController” (partea 2)

Funcția „getAllProductsController” accesează toate produsele din „getAllProducts” pe care, mai apoi, le returnează către frontend într-un format JSON pentru o utilizare eficientă a datelor.

```
export const getAllProductsController = async (_req, res) => {
  try {
    const items = await getAllProducts();
    return res.json(items);
  } catch (err) {
    console.error("Eroare la afișarea produselor: ", err);
    return res.status(500).json({ message: "Eroare de server!" });
  }
};
```

Fig.3.45 Implementarea funcției „getAllProductsController”

Funcția „getIdProductController” preia identificatorul produsului din „req.params”, verifică existența acestuia în „getIdProduct”, iar apoi îl returnează în format JSON către frontend.

```
export const getIdProductController = async (req, res) => {
  try {
    const product = await getIdProduct(Number(req.params.id));
    if (!product)
      return res.status(404).json({ message: "Produsul nu există!" });
    return res.json(product);
  } catch (err) {
    console.error("Eroare la alegerea produsului dupa ID!", err);
    return res.status(500).json({ message: "Eroare de server!" });
  }
};
```

Fig.3.46 Implementarea funcției „getIdProductController”

Funcția „updateProductController” permite actualizarea unui produs care deja există în baza de date, verificând mai întâi dacă utilizatorul este conectat la sesiune și dacă acesta are rolul necesar editării informațiilor produsului dorit. De asemenea, după identificarea articolului stocat în baza de date, se realizează o verificare suplimentară care, în cazul în care rolul nu este cel de administrator, să se poată edita doar produsul pe care utilizatorul autentificat în sesiune l-a creat.

```
export const updateProductController = async (req, res) => {
  try {
    const me = req.session?.user;
    if (!me)
      return res.status(401).json({ message: "Autentificare necesară." });

    const id = Number(req.params.id);
    const existing = await getIdProduct(id);
    if (!existing)
      return res.status(404).json({ message: "Produsul nu exista!" });

    const isAdmin = me.role === "ADMINISTRATOR";
    const isOwner = me.id === existing.user_id;
    if (!isAdmin && !isOwner) {
      return res
        .status(403)
        .json({ message: "Nu ai voie să editezi acest produs." });
    }

    const payload = { ...req.body };
    if (!isAdmin) delete payload.user_id;

    const updated = await updateProduct(id, payload);
    return res.json(updated);
  } catch (err) {
    console.error("Eroare la editarea produsului!", err);
    if (err?.code === "23505")
      return res
        .status(400)
        .json({ message: "Acest SKU a fost deja folosit!" });
    return res.status(500).json({ message: "Eroare server!" });
  }
};
```

Fig.3.47 Implementarea funcției „updateProductController”

Funcția „deleteProductController” permite ștergerea unui produs din baza de date prin verificarea autentificării utilizatorului și a rolului acestuia, iar în cazul utilizatorilor care au rolul de operator, ștergerea este posibilă doar pentru produsele create de aceștia.

```
export const deleteProductController = async (req, res) => {
  try {
    const me = req.session?.user;
    if (!me)
      return res.status(401).json({ message: "Autentificare necesară." });

    const id = Number(req.params.id);
    const existing = await getIdProduct(id);
    if (!existing)
      return res.status(404).json({ message: "Produsul nu există!" });

    const isAdmin = me.role === "ADMINISTRATOR";
    const isOwner = me.id === existing.user_id;
    if (!isAdmin && !isOwner) {
      return res
        .status(403)
        .json({ message: "Nu ai voie să ștergi acest produs." });
    }

    const del = await deleteProduct(id);
    if (!del) return res.status(404).json({ message: "Produsul nu există!" });

    return res.json({
      message: "Produsul a fost șters cu succes!",
      product: del,
    });
  } catch (err) {
    console.error("Eroare la ștergerea produsului!", err);
    return res.status(500).json({ message: "Eroare la server!" });
  }
};
```

Fig.3.48 Implementarea funcției „deleteProductController”

3.4.3.3 Controller-ul pentru tranzații din „transactionsController.js”

Controller-ul pentru tranzacții gestionează toate operațiile de intrare și ieșire din stocul aplicației. Un element important din cadrul acestei funcții este că tranzacțiile nu afectează doar tabelul din care fac parte, ci modifică și stocul unui produs asociat procesului.

Funcția „createTransactionController” permite crearea de tranzacții noi și identifică utilizatorul asociat cu întregul proces. În primă fază, funcția extrage elementele necesare din „req.body” pentru a efectua verificări asupra datelor de intrare, precum și a tipului și cantității tranzacției. Totodată, orice eroare apărută în timpul realizării operației de mișcare a stocului din depozit este semnalată cu ajutorul funcției „mapTxError” care are rolul de a transforma erorile tehnice în mesaje HTTP ușor de înțeles pentru utilizator.

```
export const createTransactionController = async (req, res) => {
  try {
    const { product_id, type, quantity, description } = req.body;

    if (!product_id || !type || quantity == null) {
      return res.status(400).json({
        message: "Id-ul produsului, tipul și cantitatea sunt obigatorii!",
      });
    }

    const typeee = String(type).toUpperCase();
    const qty = Number(quantity);

    if (!["IN", "OUT"].includes(typeee)) {
      return res
        .status(400)
        .json({ message: "Tipul tranzacției trebuie să fie 'IN' sau 'OUT'!" });
    }
    if (!Number.isInteger(qty) || qty <= 0) {
      return res
        .status(400)
        .json({ message: "Cantitatea trebuie să fie un număr întreg > 0!" });
    }

    const user_id = req.session?.user?.id ?? null;

    const trs = await createTransaction({
      product_id: Number(product_id),
      user_id,
      type: typeee,
      quantity: qty,
      description: description || null,
    });

    return res.status(201).json(trs);
  } catch (err) {
    return mapTxError(err, res, "Eroare server!");
  }
};
```

Fig.3.49 Implementarea funcției „createTransactionController”

Funcția „getAllTransactionsController” reprezintă un filtru al rolurilor din sistem, unde administratorului îi este permis prin „getAllTransactions” să vizualizeze toate tranzacțiile înregistrate vreodată în cadrul aplicației, iar operatorului îi este permisă vizualizarea prin „getUserTransactions” a propriilor tranzacții.

```
export const getAllTransactionsController = async (req, res) => {
  try {
    const me = req.session?.user;
    const items =
      me?.role === "ADMINISTRATOR"
        ? await getAllTransactions()
        : await getUserTransactions(me?.id);
    return res.json(items);
  } catch (err) {
    console.error("Eroare: afișare tranzacții!", err);
    return res.status(500).json({ message: "Eroare de server!" });
  }
};
```

Fig.3.50 Implementarea funcției „getAllTransactionsController”

Funcția „getAllTransactionsIDController” permite vizualizarea unei tranzacții pe baza identificatorului propriu al acesteia, fiind astfel și o modalitate prin care se verifică drepturile de acces din sistem. În cazul în care utilizatorul autentificat dorește să vizualizeze o tranzacție care nu îi aparține, dar are rolul de operator, funcția îi va returna mesaje de eroare indicând acest lucru.

```
export const getAllTransactionsIDController = async (req, res) => {
  try {
    const me = req.session?.user;
    const trs = await getAllTransactionsID(Number(req.params.id));
    if (!trs) {
      return res.status(404).json({ message: "Această tranzacție nu există!" });
    }
    if (me?.role !== "ADMINISTRATOR" && trs.user_id !== me?.id) {
      return res
        .status(403)
        .json({ message: "Nu ai acces la această tranzacție." });
    }
    return res.json(trs);
  } catch (err) {
    console.error("Eroare: tranzacție/ID!", err);
    return res.status(500).json({ message: "Eroare la server!" });
  }
};
```

Fig.3.51 Implementarea funcției „getAllTransactionsIDController”

Funcția „getAllTransactionsProductController” reprezintă o modalitate prin care se pot afișa toate tranzacțiile asociate unui singur produs din cadrul aplicației.

```
export const getAllTransactionsProductController = async (req, res) => {
  try {
    const items = await getAllProductTransactions(
      Number(req.params.product_id)
    );
    return res.json(items);
  } catch (err) {
    console.error("Eroare: tranzacție/Produse!", err);
    return res.status(500).json({ message: "Eroare la server!" });
  }
};
```

Fig.3.52 Implementarea funcției „getAllTransactionsProductController”

Funcția „updateTransactionDescriptionController” permite modificarea descrierii unei tranzacții deja existente în sistem. Totodată, reprezintă o metodă prin care utilizatorii cu rolul de operator pot edita, adăuga sau corecta diferite informații referitoare la statusul unei mișcări de stoc, fără a fi necesară intervenția unui administrator.

```
export const updateTransactionDescriptionController = async (req, res) => {
  try {
    const { description } = req.body;
    if (description == undefined) {
      return res.status(400).json({
        message: "Descrierea este obligatorie pentru editarea tranzacției!",
      });
    }

    const updated = await updateTransactionDescription(
      Number(req.params.id),
      String(description)
    );
    if (!updated) {
      return res.status(404).json({ message: "Tranzacția nu există!" });
    }
    return res.json(updated);
  } catch (err) {
    return mapTxError(err, res, "Eroare server!");
  }
};
```

Fig.3.53 Implementarea funcției „updateTransactionDescriptionController”

Funcția „updateTransactionFullController” este modalitatea prin care administratorul poate să editeze și să actualizeze toate datele unei tranzacții din sistem. În timpul modificării, tipul mișcărilor de stoc este convertit în majuscule, iar inventarul asociat operației se ajustează automat, asigurând-se astfel o consistență a datelor.

```
export const updateTransactionFullController = async (req, res) => {
  try {
    const id = Number(req.params.id);
    const { product_id, type, quantity, description } = req.body;

    if (!type || quantity == null) {
      return res
        .status(400)
        .json({ message: "Tipul și cantitatea tranzacției sunt obligatorii!" });
    }

    const data = {
      product_id: product_id !== undefined ? Number(product_id) : undefined,
      type: String(type).toUpperCase(),
      quantity: Number(quantity),
      description: description ?? null,
    };

    const update = await updateTransactionFULL(id, data);
    return res.json(update);
  } catch (err) {
    return mapTxError(err, res, "Eroare la server!");
  }
};
```

Fig.3.54 Implementarea funcției „updateTransactionFullController”

Funcția „deleteTransactionController” permite ștergerea unei tranzacții din aplicație, recalculând stocul acesteia după terminarea operației.

```
export const deleteTransactionController = async (req, res) => {
  try {
    const deletee = await deleteTransaction(Number(req.params.id));
    if (!deletee) {
      return res.status(404).json({ message: "Tranzacția nu există!" });
    }
    return res.json({
      message: "Tranzacția a fost ștearsă, iar stocul a fost ajustat!",
      transaction: deletee,
    });
  } catch (err) {
    return mapTxError(err, res, "Eroare de server!");
  }
};
```

Fig.3.55 Implementarea funcției „deleteTransactionController”

3.4.3.4 Controller-ul administrării utilizatorilor din „usersController.js”

Controller-ul permite gestionarea operațiilor de creare, afișare, actualizare sau ștergere a utilizatorilor din aplicație. De asemenea, accesarea acestor funcționalități necesită rolul de administrator pentru a preveni posibilele modificări neautorizate ale informațiilor regăsite în sistem.

Funcția „createUserController” permite inserarea unui nou utilizator în platformă, preluând datele necesare și verificând dacă adresa introdusă la înregistrarea în cont există deja în baza de date. În situația în care nu a fost furnizat un rol la crearea contului, utilizatorul va fi salvat în mod automat cu rolul de operator.

```
export const createUserController = async (req, res) => {
  try {
    const {
      first_name,
      last_name,
      email,
      password,
      role = "OPERATOR",
    } = req.body;
    if (!first_name || !last_name || !email || !password) {
      return res.status(400).json({
        message: "Câmpurile cu nume, email și parola sunt obligatorii!",
      });
    }

    const exista = await getUserEmail(email);
    if (exista) {
      return res
        .status(400)
        .json({ message: "Acest email se află deja în baza noastră de date!" });
    }

    const saltedPassword = await bcrypt.hash(password, saltRounds);
    const user = await createUser(
      first_name,
      last_name,
      email,
      saltedPassword,
      role
    );

    return res.status(201).json(user);
  } catch (err) {
    console.log("Crearea noului utilizator a eșuat!", err);
    if (err.code === "23505")
      return res.status(400).json({ message: "Email-ul a fost deja folosit!" });
    return res.status(500).json({ message: "Eroare de server!" });
  }
};
```

Fig.3.56 Implementarea funcției „createUserController”

Funcția „getUserIdController” verifică, pe baza identificatorului propriu al utilizatorului, dacă acesta este sau nu înregistrat în platformă. De asemenea, funcția „getAllUsersController” permite listarea completă a conturilor pentru o gestionare mai ușoară de către administrator direct din interfața aplicației.

```
export const getUserIdController = async (req, res) => {
  try {
    const user = await getUserId(req.params.id);
    if (!user)
      return res
        .status(404)
        .json({ message: "Acest utilizator nu a fost găsit!" });
    return res.json(user);
  } catch (err) {
    console.error("Eroare la căutarea de user!", err);
    return res.status(500).json({ message: "Eroare server!" });
  }
};
```

Fig.3.57 Implementarea funcției „getUserIdController”

```
export const getAllUsersController = async (_req, res) => {
  try {
    const users = await getAllUsers();
    return res.json(users);
  } catch (err) {
    console.error("Afișarea utilizatorilor a eșuat!", err);
    return res.status(500).json({ message: "Eroare server!" });
  }
};
```

Fig.3.58 Implementarea funcției „getAllUsersController”

Funcția „updateUserController” facilitează procesul de actualizare a informațiilor conturilor existente, asigurând o înserare corectă a câmpurilor necesare pe parcursul procesului. În cazul în care un utilizator este inactiv, nu mai dorește să facă parte din sistem sau contul acestuia nu mai este necesar, se poate folosi funcția „deleteUserController”, care reprezintă modalitatea de eliminare în siguranță a datelor contului din baza de date.

```
export const updateUserController = async (req, res) => {
  try {
    const id = req.params.id;
    const { first_name, last_name, email } = req.body;
    const roleInput = req.body?.role;

    if (email) {
      const gasit = await getUserEmail(email);
      if (gasit && String(gasit.id) !== String(id)) {
        return res.status(400).json({
          message: "Acest email a fost deja folosit de un alt utilizator!",
        });
      }
    }

    let roleSet = null;
    if (roleInput !== undefined) {
      const ROLES_ALLW = ["ADMINISTRATOR", "OPERATOR"];
      const upper = String(roleInput).toUpperCase();
      if (!ROLES_ALLW.includes(upper)) {
        return res.status(400).json({
          message:
            "Rolul este invalid. Folosește 'ADMINISTRATOR' sau 'OPERATOR'.",
        });
      }
      roleSet = upper;
    }
```

Fig.3.59 Implementarea funcției „updateUserController” (partea 1)

```
    const updated = await updateUser(
      id,
      first_name ?? null,
      last_name ?? null,
      email ?? null,
      roleSet
    );
    if (!updated)
      return res
        .status(400)
        .json({ message: "Acest utilizator nu a fost găsit!" });
    return res.json(updated);
  } catch (err) {
    console.error("Eroare la update de profil al utilizatorului!", err);
    if (err.code === "23505")
      return res.status(400).json({ message: "Email-ul a fost deja folosit!" });
    return res.status(500).json({ message: "Eroare la server!" });
  }
};
```

Fig.3.60 Implementarea funcției „updateUserController” (partea 2)

```
export const deleteUserController = async (req, res) => {
  try {
    const deletee = await deleteUser(req.params.id);
    if (!deletee)
      return res
        .status(400)
        .json({ message: "Utilizatorul nu a fost găsit!" });
    return res.json({
      message: "Utilizatorul a fost șters cu succes!",
      user: deletee,
    });
  } catch (err) {
    console.error("Eroare la ștergerea utilizatorului!", err);
    return res.status(500).json({ message: "Eroare de server!" });
  }
};
```

Fig.3.61 Implementarea funcției „deleteUserController”

3.4.4 Autorizarea și securizarea rutelor

Resursele regăsite în server și securitatea aplicației utilizează mecanisme de autorizare a sesiunii, implementate cu ajutorul platformei software Node.js[4] și al framework-ului Express[5], prin intermediul fișierului „authMiddleware.js”. Acesta interceptează cererile de tip HTTP înainte de procesarea lor de către controllere, verificând utilizatorii și drepturile lor de acces.

Middleware-ul „isAuth” permite validarea unei sesiuni existente a unei cereri HTTP, analizând dacă contul autentificat conține informațiile stocate în obiectul de sesiune „req.session”. Dacă sesiunea curentă nu există sau a expirat, accesul este refuzat printr-un mesaj care indică lipsa posibilității de conectare pentru utilizarea elementului dorit.

Pentru protejarea și diferențierea autorizărilor bazate pe rolul utilizatorului, a fost necesară dezvoltarea unui middleware, denumit „reqRole”, care verifică corespondența dintre un cont autentificat și rolul asociat cu acesta. Operația de validare se realizează printr-o comparație dintre statutul profilului și lista cu roluri acceptate pe ruta respectivă. În cazul în care persoana care este conectată pe platformă nu deține permisiunile necesare pentru accesarea rutei solicitate, serverul va transmite un mesaj prin care se înțelege că accesul este interzis, ruta fiind restricționată. Astfel, se face o distincție clară dintre mecanismul de autentificare și cel de gestionare al permisiunilor indiferent de rolul prin care este accesată aplicația.

Gestionarea sesiunilor și a cookie-urilor se bazează pe modulul express-session[17], prezentat anterior în diferite funcții din cadrul menținerii autentificării (fig.3.36), prin care un cookie rămâne stocat în browser printr-o sesiune unică creată pentru a-i permite utilizatorului menținerea activă în sistem fără a fi necesară reautentificarea.

```
const fix = (rol) =>
  String(rol || "")
    .trim()
    .toUpperCase();

export const isAuth = (req, res, next) => {
  if (!req.session?.user) {
    return res
      .status(401)
      .json({ message: "Întai trebuie să te autentifici!" });
  }
  next();
};

export const reqRole = (...roles) => {
  const allow = roles.map(fix);
  return (req, res, next) => {
    const userRole = fix(req.session?.user?.role);

    const isAdmin = userRole === "ADMIN" && allow.includes("ADMINISTRATOR");
    if (!userRole || (!allow.includes(userRole) && !isAdmin)) {
      return res
        .status(403)
        .json({ message: "Rolul tău nu are permisiunea necesară!" });
    }
    next();
  };
};
```

Fig.3.62 Implementarea middleware-ului de autentificare „authMiddleware.js”

3.5 Arhitectura și funcționarea frontend-ului aplicației

3.5.1 Structura aplicației frontend

În procesul de realizare a frontend-ului a fost utilizată biblioteca React[12],[13], care permite folosirea componentelor într-o modalitate independentă, fiecare fișier având un rol bine definit în fluxul aplicației. Conform Fig.3.63, arhitectura software este împărțită în mai multe directoare principale. Componentele reprezentative pentru autentificare, administrare produse, tranzacții, utilizatori și a punctelor principale de acces îi revin directorului „pages”, care îi permite utilizatorului să coordoneze și să afișeze elementele vizuale cu care acesta interacționează. Directorul „components” ajută la evitarea duplicării codului prin reutilizarea unor componente, precum bara de navigare, ferestrele de dialog sau elemente de feedback vizual, aplicarea lor fiind posibilă în mai multe pagini din cadrul platformei. De asemenea, mecanismul prin care frontend-ul comunică cu backend-ul se regăsește în directorul „libs”, unde sunt definite funcții care ajută la realizarea cererilor HTTP, precum și configurarea Axios[15], care stabilește modul în care acestea sunt transmise către server. În paralel, directorul „features” reprezintă gestionarea principalelor informații legate de utilizatorul autentificat, dar și despre sesiunea activă, folosind Redux[20] Toolkit[20]. Totodată, fișierele „App.jsx” și „index.jsx” permit separarea clară între interfață, logica de funcționare și comunicarea cu serverul.

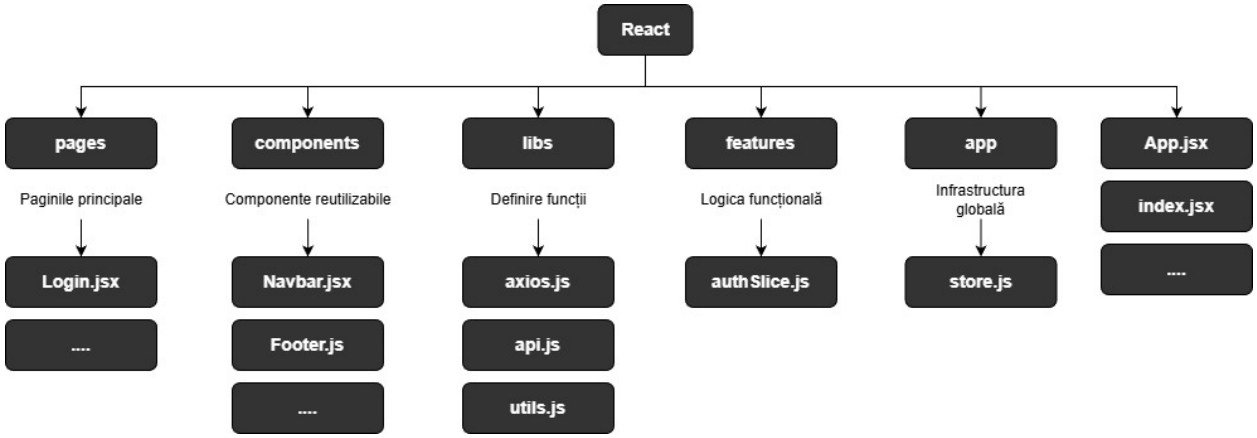


Fig.3.63 Arhitectura aplicației frontend

3.5.2 Implementarea rutării în aplicația React

Pentru implementarea rutării a fost folosită biblioteca React Router[21], care îi permite aplicației să fie construită ca SPA (Single Page Application), oferind posibilitatea navigării rapide între pagini și reprezentând modalitatea prin care se poate realiza o integrare ușoară cu rolurile și autentificarea din interiorul platformei.

În cadrul Fig.3.64, este prezentată principala metodă de rutare din fișierul „App.jsx”, unde componenta „BrowserRouter” conține întreaga aplicație, ceea ce permite gestionarea oricărei modificări aflate în bara de adrese și afișarea elementului corespunzător rutei accesate. Totodată, în interiorul fișierului se regăsește și componenta „Routes”, prin care sunt definite toate rutele aplicației. Astfel, fiecare rută este asociată cu o componentă React[12],[13] prin intermediul unui anumit URL, fiind posibilă crearea căilor publice de navigare care pot fi accesate fără o autentificare ulterioară în aplicație. Rutele publice care se regăsesc în această situație sunt următoarele: pagina de bun venit („/”), pagina de înregistrare („/register”) și pagina de autentificare („/login”).

```
export default function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <main className="section">
        <div className="container">
          <Routes>
            <Route path="/" element={<Welcome />} />
            <Route path="/register" element={<Register />} />
            <Route path="/login" element={<Login />} />
```

Fig.3.64 Definirea rutelor publice în „App.jsx”

Rutele protejate prin „RequireAuth” reprezintă funcționalitățile principale ale aplicației și sunt accesibile doar după realizarea autentificării. La introducerea comenzii, se verifică existența unui utilizator în centrul informațional al aplicației, iar dacă acesta nu este autentificat, componenta „Navigate” îl va redirecționa automat către pagina de logare. De asemenea, mecanismul care asigură gestionarea corectă a datelor este susținut de Redux Toolkit[20], facilitând întregul proces de autentificare.

```
<Route path="*" element={<Navigate to="/" replace />} />
<Route
  path="/products"
  element={
    <RequireAuth>
      <Products />
    </RequireAuth>
  }
/>
```

Fig.3.65 Mecanismului de redirecționare automată prin „Navigate”

Pentru cazul în care accesarea anumitor secțiuni nu trebuie să fie valabilă utilizatorilor cu rol de operator, a fost utilizată componenta „RequireRole” pentru a restricționa conectarea celor care nu se încadrează în lista rolurilor permise. Dacă utilizatorul nu are atribuit un rol corespunzător cererii, va fi redirecționat către o rută publică sau către o rută pentru care deține drepturi de acces.

```
<Route
  path="/users"
  element={
    <RequireAuth>
      <RequireRole allowed={["ADMINISTRATOR"]}>
        <Users />
      </RequireRole>
    </RequireAuth>
  }
/>
```

Fig.3.66 Protejarea rutelor utilizând „RequireAuth” și „RequireRole”

3.5.3 Gestionarea stării globale cu Redux Toolkit

Datele disponibile la nivel global sunt configurate în fișierul „store.js”, unde mecanismul central de gestionare al acestora este creat prin funcția „configureStore”, permițând logicii de actualizare a autentificării să asigure accesul uniform la informațiile despre cont în orice componentă a sistemului. În același timp, esența întregului proces de autorizare a contului se regăsește în fișierul „authSlice.js”, unde este definită o porțiune Redux[20] care permite administrarea datelor utilizatorului, dar și a unor acțiuni asincrone implementate prin „createAsyncThrunk” și destinate colaborării cu serviciile backend. De asemenea, în timpul tratării stărilor de „pending”, „fulfilled” și „rejected” se setează statusul de „loading” pentru așteptarea actualizării corecte a contului și pentru verificarea sau transmiterea erorilor care pot apărea în interfață, gestionarea acestor rezultate fiind realizată în secțiunea „extraReducers”.

```
export const store = configureStore({
  reducer: {
    auth: authReducer,
  },
});
```

Fig.3.67 Configurarea magazinului Redux[20] în „store.js”

```
const authSlice = createSlice({
  name: "auth",
  initialState,
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(RegisterUser.pending, (state) => {
        state.status = STATUS.loading;
        state.error = null;
      })
      .addCase(RegisterUser.fulfilled, (state, action) => {
        state.status = STATUS.success;
        state.user = action.payload;
      })
      .addCase(RegisterUser.rejected, (state, action) => {
        state.status = STATUS.error;
        state.error = action.error?.message;
      })
```

Fig.3.68 Gestionarea stărilor asincrone din „authSlice.js”

```
export const LoginUser = createAsyncThunk(
  "auth/login",
  async ({ email, password }) => {
    await apiLogin(email, password);
    return await apiMe();
  }
);
```

Fig.3.69 Implementarea operației asincrone de autentificare

În Fig.3.69 este prezentat un exemplu de operație prin care sunt transferate datele de autentificare către server cu ajutorul funcției „apiLogin”, obținând starea de autentificare a utilizatorului prin „apiMe”. La finalizarea acestor pași, starea gestionată prin cadrul constantei „store” este actualizată cu noul statut al utilizatorului în platformă.

Prin abordarea statusurilor de tip „pending”, „fulfilled” și „rejected”, se pot afișa mesaje relevante în funcție de fiecare rezultat al operațiilor asincrone datorită constantei „STATUS”. Această constantă este definită prin „Object.freeze” și stabilește valori standard pentru stările din sistem, cele trei etape fiind modificate astfel: în etapa de „pending”, statusul devine „loading”, indicând o procesare în curs a cererii; în „fulfilled”, statusul devine „success”, datele

```
export const STATUS = Object.freeze({
  idle: "Inactiv",
  loading: "Se incarca...",
  success: "Succes!",
  error: "Eroare",
});

const initialState = { user: null, status: STATUS.idle, error: null };
```

Fig.3.70 Configurarea statusurilor și a stării inițiale în modulul de autentificare

actualizându-se după primirea lor de la server prin obiectul „user”; iar în etapa „rejected”, statusul devine „error”, proprietatea „error” salvând astfel mesajul stării de eșec.

Un alt element important în procesul de gestionare al modulului de autentificare este „initialState”, care contribuie la evitarea afișării unor informații incorecte înainte de finalizarea cererilor către server. Structura constantei este formată din trei termeni, fiecare având un rol specific. Primul termen este „user: null” care clarifică faptul că nu există în prezent un alt utilizator autentificat, al doilea este „status: STATUS.idle” care păstrează starea neutră înaintea efectuării oricărei operații în sistem, iar al treilea este „error: null” care previne afișarea unor mesaje eronate la inițializarea procesului.

Pentru integrarea magazinului Redux[20] în aplicație se folosește componenta „Provider” din fișierul „index.jsx”, fiind o modalitate prin care se pot accesa ușor informațiile stocate în „store” din orice componentă a sistemului. Prin aceasta se realizează o legătură strânsă între interfața cu utilizatorul și nivelul de procesare a datelor.

```jsx
const root = createRoot(document.getElementById("root"));

root.render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

Fig.3.71 Integrarea store-ului Redux[20] prin componenta „Provider”

3.5.4 Comunicarea cu backend-ul prin Axios

Conform datelor prezentate anterior în Fig.3.38, configurarea Axios[15] este realizată în fișierul „libs/Axios.js”, unde aceasta definește URL-ul prin variabila „VITE_API_URL”, dar și opțiunea „withCredentials: true”, necesară transmiterii cookie-urilor de sesiune între client și server. În același fișier însă, se regăsește și funcția „api.interceptors.response.use” care procesează răspunsurile și definește modalitatea prin care erorile sunt tratate, construind astfel un mesaj pe baza codului HTTP sau pe baza informațiilor transmise de backend prin „resp.data.message” sau prin „resp.data.error”.

```jsx
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
  headers: { "Content-Type": "application/json" },
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    const resp = err?.response;
    const msg =
      resp?.data?.message ||
      resp?.data?.error ||
      (resp?.status === 404
        ? "Resursa nu a fost găsită."
        : resp?.status === 401
        ? "Nu ești autentificat."
        : resp?.status === 403
        ? "Nu ai permisiuni pentru acțiunea asta."
        : "Eroare de server.");

    err.message = msg;
    err.status = resp?.status;
    return Promise.reject(err);
  }
);

export default api;
```

Fig.3.72 Configurarea completă a clientului HTTP Axios[15] în aplicația frontend

După configurarea Axios[15], centralizarea apelurilor către server se realizează în fișierul „api.js” prin implementarea fiecărei funcții corespunzătoare unei rute. Folosirea operațiilor de tip autentificare, produse, tranzacții și utilizatori permite reutilizarea lor într-o manieră controlată, fără a fi necesară duplicarea codului. Totodată, aceste funcții corespund deja rutelor definite pe partea de backend, întorcând astfel răspunsul prin „response.data” și simplificând întregul proces de gestionare al erorilor și transmitere al informațiilor.

```
export const apiMe = () =>
  api.get("/api/auth/me").then((response) => response.data);
export const apiRegister = (data) =>
  api.post("/api/auth/register", data).then((response) => response.data);
export const apiLogin = (email, password) =>
  api
    .post("/api/auth/login", { email, password })
    .then((response) => response.data);
export const apiLogout = () =>
  api.delete("/api/auth/logout").then((response) => response.data);

export const apiAllProducts = () =>
  api.get("/api/products").then((response) => response.data);
export const apiProductsId = (id) =>
  api.get(`/api/products/${id}`).then((response) => response.data);
export const apiCreateProducts = (data) =>
  api.post("/api/products", data).then((response) => response.data);
export const apiUpdateProducts = (id, data) =>
  api.patch(`/api/products/${id}`, data).then((response) => response.data);
export const apiDeleteProducts = (id) =>
  api.delete(`/api/products/${id}`).then((response) => response.data);
```

Fig.3.73 Funcții pentru apelurile API de autentificare și gestionare a produselor

3.5.5 Implementarea funcționalității de export în format Excel

Pentru implementarea funcționalității de export în format Excel a fost necesară integrarea bibliotecii SheetJS[16] (XLSX), ceea ce permite generarea directă în browser a aplicațiilor de calcul tabelar. Datorită capacității sale de a procesa informațiile regăsite în tabelele din cadrul platformei, precum și flexibilității în afișarea structurii fișierului rezultat, biblioteca reprezintă modalitatea principală prin care este îmbunătățită experiența cu utilizatorul.

Realizarea întregii funcționalități a fost posibilă prin organizarea sa pe module, ceea ce îi permite logicii de export să fie reutilizată cu ușurință, facilitând astfel menținerea unei arhitecturi clare și ușor de extins. Astfel, biblioteca este importată prin „import * as XLSX” în fișierul „utils.js”, oferind posibilitatea accesării elementelor necesare pentru crearea unui registru de lucru (workbook) și a foilor de lucru (worksheets) din acesta. Primul pas pentru implementarea propriu-zisă a funcționalității este realizat prin funcția „exportXLSX(filename, rows, columns)”, care primește informații referitoare la datele ce urmează a fie exportate, dar și despre întreaga configurație a coloanelor. În plus, structura câmpurilor este definită sub forma unui array de obiecte ce include proprietăți precum „key”, „label” sau „format”, necesare în corelarea dintre termenii introduși în sistem și denumirile vizibile în Excel.

```
export function exportXLSX(filename, rows, columns) {

  const header = columns.map((c) => c.label);
  const data = rows.map((r) =>
    columns.map((c) => {
      const raw = get(r, c.key);
      const v = c.format ? c.format(raw, r) : raw;
      return v == null ? "" : v;
    })
  );
  const ws = XLSX.utils.aoa_to_sheet([header, ...data]);
  const wb = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(wb, ws, "Sheet1");
  const name = filename.endsWith(".xlsx") ? filename : `${filename}.xlsx`;
  XLSX.writeFile(wb, name);
}
```

Fig.3.73 Implementarea funcției „exportXLSX”

Așa cum se ilustrează în Fig.3.73, metoda „XLSX.utils.aoa_to_sheet” convertește tabelul creat în JavaScript[3] într-un obiect pe care Excel îl înțelege ca foaie de calcul , ca mai apoi să creeze registru de lucru nou prin „XLSX.utils.book_new” și o foaie de lucru prin „XLSX.utils.book_append_sheet” atașat de acesta. Generarea fișierului final se realizează prin „XLSX.writeFile”, care descarcă automat documentul de tip „.xlsx” în browser.

Pentru a fi posibilă integrarea în diferitele secțiuni ale aplicației, a fost creat un eveniment „onClick” care declanșează funcția de export la apăsarea butoanelor regăsite la nivelul platformei. Cu alte cuvinte, tranzacțiile, produsele și utilizatorii păstrează aceeași structură prin care se realizează extractul de date din sistem, chiar dacă se dorește filtrarea acestora. De exemplu, în cazul produselor, funcția este invocată cu parametrii „PRODUCT_XLSX_FILE”, „filtered” și „PRODUCT_COLUMNS”, unde variabila de filtrare reprezintă lista de valori obținută după procesul de selecție prin „filterByQuery”, proces care exportă exact datele afișate în interfață.

```
<button
  className="btn btn-outline-secondary btn-sm"
  onClick={() =>
    exportXLSX(PRODUCT_XLSX_FILE, filtered, PRODUCT_COLUMNS)
  }
>
  Export XLSX
</button>
```

Fig.3.74 Integrarea funcției „exportXLSX” prin intermediul unui buton

```
const PRODUCT_COLUMNS = [
  { key: "id", label: "#" },
  { key: "name", label: "Denumire" },
  { key: "sku", label: "SKU" },
  { key: "unit", label: "Unitate" },
  { key: "stock", label: "Stoc" },
  { key: "stock_min", label: "Stoc Min" },
  { key: "rack", label: "Raft" },
  { key: "price", label: "Preț (Lei)", format: (v) => Number(v).toFixed(2) },
  { key: "created_by", label: "Creat de" },
  {
    key: "updated_at",
    label: "Actualizat",
    format: (v) => (v ? new Date(v).toLocaleString() : ""),
  },
  { key: "description", label: "Descriere" },
];
const PRODUCT_XLSX_FILE = "produse.xlsx";

export default function Products() {
  const nav = useNavigate();
  const user = useSelector((s) => s.auth.user);
  const isAdmin = user?.role === "ADMINISTRATOR";
  const canManage = (p) => isAdmin || user?.id === p.user_id;

  const [rows, setRows] = useState([]);
  const [loading, setLoading] = useState(true);

  const [open, setOpen] = useState(false);
  const [sel, setSel] = useState(null);

  const [q, setQ] = useState("");
  const filtered = filterByQuery(rows, PRODUCT_SEARCH_FIELDS, q);
```

Fig.3.75 Definirea structurii coloanelor și a fișierului Excel în „Products.jsx”

3.5.6 Componenta Bara de navigare

Bara de navigare reprezintă modalitatea principală prin care utilizatorul poate interacționa în mod direct cu aplicația. De asemenea, permite o navigare sigură între pagini prin adaptarea interfeței la funcțiile de autentificare și rol. Definirea componentei fiind astfel regăsită în fișierul „Navnar.jsx”, unde este integrată la nivel global în „App.jsx”, ca mai apoi să fie afișată pe tot parcursul interacțiunii din platformă.

În prezentările anterioare din Fig.3.64, se putea observa implementarea componentei „ < Navbar />” în interiorul structurii „BrowserRouter”, poziționare care permite afișarea permanentă a meniului, indiferent de existența unei rute active. Totuși, componenta folosește mecanismele oferite de React[12],[13] și Redux[20] pentru identificarea oricărui utilizator conectat în aplicație, dar și a rolului pe care acesta îl deține. Având în vedere că, structura este definită în interiorul „Navbar.js”, accesul la informațiile despre utilizator se face prin mecanismul „useSelector”, unde se permite extragerea stării globale gestionate prin Redux Toolkit[20]. Prin intermediul acestei instrucțiuni, interfața va afișa diferitele adaptări ale barei de navigare, elemente disponibile atât pentru persoanele autentificate în cont, cât și pentru rolul acestora. În situația în care variabila „user” este „null”, aplicația va considera că nu există un cont activ în sistem și va afișa doar opțiunile de „Loghează-te” și „Înregistrează-te”. Dacă utilizatorul este autentificat, se vor afișa opțiuni suplimentare de tipul „Dashboard”, „Produse” și „Tranzacții”, iar dacă acesta are rolul de administrator, se va afișa și meniul „Utilizatori”.

```jsx
export default function Navbar() {
  const user = useSelector((s) => s.auth.user);
  const dispatch = useDispatch();

  return (
    <header className="nav app-navigation">
      <div className="container nav__row">
        <Link to="/" className="nav__brand">
          RAWVISTA
        </Link>

        {!user ? (
          <nav className="nav__menu">
            <Link to="/register" className="btn btn--primary">
              Înregistrează-te
            </Link>
            <Link to="/login" className="btn">
              Loghează-te
            </Link>
          </nav>
        ) : (
          <nav className="nav__menu" style={{ gap: 10 }}>
            <NavLink to="/dashboard" className="nav__link">
              Dashboard
            </NavLink>
            <NavLink to="/products" className="nav__link">
              Produse
            </NavLink>
            <NavLink to="/transactions" className="nav__link">
              Tranzacții
            </NavLink>
            {user.role === "ADMINISTRATOR" && (
              <NavLink to="/users" className="nav__link">
                Utilizatori
              </NavLink>
            )}
            <button className="btn" onClick={() => dispatch(LogOut())}>
              Logout
            </button>
          </nav>
        )}
      </div>
    </header>
  );
}
```

Fig.3.76 Implementarea componentei „Navbar” în „Navbar.jsx”

În Fig.3.76 sunt prezentate elemente precum „Link” sau „NavLink” importate din biblioteca react-router-dom[21], care mențin caracterul dinamic al aplicației de tip SPA (Single Page Application). Totodată, sunt menționate și clasele Bootstrap[11] precum „nav”, „nav__row”, „nav__brand” și „nav_link”, personalizând astfel aspectul întregului meniu prin setarea unor stiluri diferite ale elementelor barei de navigare în fișierul „index.css”.

Un element important al componentei este reprezentat de capacitatea sa de a se deconecta din sesiunea activă, fiind implementată apelând o acțiune Redux[20] asincronă din „authSlice.js”. În momentul în care utilizatorul dorește să se deconecteze, acesta apasă un buton de „Logout” prin care se declanșează o cerere HTTP către ruta „/api/auth/logout”, starea „user” fiind resetată ulterior înapoi la valoare „null”, indicând faptul ca niciun al utilizator nu mai este conectat în sesiune. Datorită acestui proces, se realizează o sincronizare cu sesiunea de pe server, meniul actualizându-se automat.

3.5.7 Implementarea componentei de autentificare și înregistrare

Autentificarea și înregistrarea reprezintă principala modalitate prin care se pot accesa funcționalitățile aplicației, fiind implementate cu ajutorul componentelor „Login.jsx” și „Register.jsx” și apoi integrate în „App.jsx”. Prin intermediul acestora se realizează atât validarea datelor introduse de către utilizator, cât și accesarea diferitelor rute disponibile la începerea sesiunii.

Componenta „Login.jsx” gestionează întregul proces de autentificare prin utilizarea diferitelor mecanisme de tipul React[12],[13] și Redux Toolkit[20], stocând astfel, prin intermediul funcției „useState”, variabilele „email” și „password”. În momentul în care au fost transmise datele necesare autentificării, se declanșează „onSubmit”, verificând câmpurile introduse de utilizator și prevenind transmiterea unor date incomplete către backend. De asemenea, este apelată și acțiunea „LoginUser”, definită în „authSlice.js”, prin intermediul „dispatch(LoginUser({ email: email.trim(), password }))”, care trimite o cerere către server pentru actualizarea stării globale în funcție de răspunsul primit de la acesta. Conform prezentării anterioare din Fig.3.68, se remarcă transmiterea, verificarea și finalizarea pașilor de actualizare a statutului utilizatorului prin modificarea automată a interfeței la care acesta este conectat. După realizarea autentificării, componenta „Login.jsx” permite redirecționarea utilizatorului către pagina dorită prin „useNavigate”, utilizând informația stocată in „useLocation”, ceea ce menține o coerență pe tot parcursul fluxului de navigare în sistem.

```jsx
import { useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import { LoginUser, STATUS } from "../features/authSlice";
import { useNavigate, useLocation } from "react-router-dom";
import { toast } from "react-hot-toast";

export default function Login() {
  const dispatch = useDispatch();
  const nav = useNavigate();
  const loc = useLocation();
  const from = loc.state?.from?.pathname || "/dashboard";
  const loading = useSelector((s) => s.auth.status) === STATUS.loading;

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [errors, setErrors] = useState({
    email: "",
    password: "",
    general: "",
  });

  const onSubmit = async (e) => {
    e.preventDefault();

    const next = { email: "", password: "", general: "" };
    if (!email.trim()) next.email = "Emailul este obligatoriu.";
    if (!password) next.password = "Parola este obligatorie.";

    if (next.email || next.password) {
      setErrors(next);
      toast.error("Te rugăm să completezi câmpurile.");
      return;
    }

    setErrors({ email: "", password: "", general: "" });

    try {
      await dispatch(LoginUser({ email: email.trim(), password })).unwrap();
      toast.success("Autentificat!");
      nav(from, { replace: true });
    } catch (err) {
      const msg =
        err?.message || err?.payload?.message || "Email sau parolă incorecte.";
      setErrors((p) => ({ ...p, general: msg }));
      toast.error(msg);
    }
  };
```

Fig.3.77 Implementarea funcției „Login” din „Login.jsx”(partea 1)

```jsx
return (
  <section>
    <h1 className="title">Logare</h1>

    <form className="form card" onSubmit={onSubmit} noValidate>
      {errors.general && (
        <div className="form__error" style={{ marginBottom: 10 }}>
          {errors.general}
        </div>
      )}

      <div className="form__group">
        <label className="label">Email</label>
        <input
          className={`input ${errors.email ? "input--invalid" : ""}`}
          type="text"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        {errors.email && <p className="form__error">{errors.email}</p>}
      </div>

      <div className="form__group">
        <label className="label">Parolă</label>
        <input
          type="password"
          className={`input ${errors.password ? "input--invalid" : ""}`}
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
        {errors.password && <p className="form__error">{errors.password}</p>}
      </div>

      <button className="btn btn--primary" disabled={loading}>
        {loading ? "Se încarcă…" : "Login"}
      </button>
    </form>
  </section>
);
}
```

Fig.3.78 Implementarea funcției „Login” din „Login.jsx”(partea 2)

Componenta „Register.jsx” păstrează un principiu similar cu cel de autentificare, însă în interiorul acesteia se află și procesul de creare a unui cont nou. Dezvoltarea câmpurilor „first_name”, „last_name”, „email”, „password” și „role” a fost realizată printr-un obiect „form”, care apelează acțiunea „RegisterUser” din „authSlice.js” pentru a permite conectarea automată în platformă a utilizatorului imediat după ce acesta a fost înregistrat. În Fig.3.79 se poate observa că în dreptul câmpului „role” este deja introdusă o valoare implicită „OPERATOR”, necesară în cazul în care la momentul creării contului, rolul nu a fost selectat.

Un element important al întregului mecanism este utilizarea sesiunilor gestionate de backend prin express-session[17]. Datorită configurării „withCredentials: true” în instanța Axios[15], cookie-ul de sesiune este transmis automat între client și server. Astfel, după autentificare, toate cererile ulterioare includ informațiile necesare identificării utilizatorului, fără a fi nevoie de token-uri suplimentare sau manipulări manuale la nivelul frontend-ului.

Protejarea paginilor interne ale aplicației este realizată prin componenta „RequireAuth”, care verifică existența obiectului „user” în starea globală. Dacă utilizatorul nu este autentificat, acesta este redirecționat automat către ruta „/login”. În mod similar, componenta „RequireRole” permite restricționarea accesului în funcție de rol, asigurând separarea clară între funcționalitățile disponibile administratorului și cele disponibile operatorului.

```jsx
import { useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import { RegisterUser, STATUS } from "../features/authSlice";
import { useNavigate } from "react-router-dom";
import { toast } from "react-hot-toast";

export default function Register() {
  const dispatch = useDispatch();
  const nav = useNavigate();
  const status = useSelector((state) => state.auth.status);
  const loading = status === STATUS.loading;

  const [form, setForm] = useState({
    first_name: "",
    last_name: "",
    email: "",
    password: "",
    role: "OPERATOR",
  });

  const onChange = (event) =>
    setForm((prevForm) => ({
      ...prevForm,
      [event.target.name]: event.target.value,
    }));

  const onSubmit = async (event) => {
    event.preventDefault();
    try {
      await dispatch(RegisterUser(form)).unwrap();
      toast.success("Contrul a fost creat! Te-ai logat automat.");
      nav("/dashboard");
    } catch (err) {
      toast.error(err?.message || "Eroare la inregistrare!");
    }
  };
```

Fig.3.79 Implementarea funcției „Register” din „Register.jsx”(partea 1)

```jsx
  return (
    <form onSubmit={onSubmit}>
      <div className="card p-4 shadow-sm">
        <h1 className="h4 mb-3">Înregistrare</h1>

        <div className="mb-3">
          <label className="form-label">Prenume</label>
          <input
            className="form-control"
            name="first_name"
            value={form.first_name}
            onChange={onChange}
          />
        </div>

        <div className="mb-3">
          <label className="form-label">Nume</label>
          <input
            className="form-control"
            name="last_name"
            value={form.last_name}
            onChange={onChange}
          />
        </div>
```

Fig.3.80 Implementarea funcției „Register” din „Register.jsx”(partea 2)

```jsx
return (
  <form onSubmit={onSubmit}>
    <div className="card p-4 shadow-sm">
      <h1 className="h4 mb-3">Înregistrare</h1>

      <div className="mb-3">
        <label className="form-label">Prenume</label>
        <input
          className="form-control"
          name="first_name"
          value={form.first_name}
          onChange={onChange}
        />
      </div>

      <div className="mb-3">
        <label className="form-label">Nume</label>
        <input
          className="form-control"
          name="last_name"
          value={form.last_name}
          onChange={onChange}
        />
      </div>
```

Fig.3.81 Implementarea funcției „Register” din „Register.jsx”(partea 3)

3.5.8 Implementarea modulului de gestionare a produselor

Modulul de gestionare a produselor reprezintă una dintre componentele centrale ale aplicației, fiind responsabil de afișarea, filtrarea, crearea, editarea și ștergerea produselor existente în baza de date. Implementarea acestui modul este realizată prin mai multe componente React[12],[13] interconectate, dintre care cea principală este „Products.jsx”, completată de „AddProducts.jsx”, „EditProducts.jsx” și „ProductDetailsModal.jsx”. Împreună, acestea formează un flux coerent de interacțiune între utilizator și sistem. Componenta „Products.jsx” este responsabilă de afișarea listei de produse într-un tabel dinamic. La încărcarea paginii, prin intermediul „useEffect”, este apelată funcția „apiAllProducts”, definită în fișierul „api.js”. Această funcție execută o cerere HTTP de tip GET către ruta „/api/products”, iar răspunsul primit este stocat în variabila locală „rows” prin „useState”. Astfel, datele sunt menținute la nivelul componentei și pot fi manipulate ulterior pentru filtrare sau actualizare locală.

```jsx
useEffect(() => {
  (async () => {
    try {
      const data = await apiAllProducts();
      setRows(data);
    } catch {
      toast.error("Nu se pot încărca produsele.");
    } finally {
      setLoading(false);
    }
  })();
}, []);
```

Fig.3.82 Încărcarea listei de produse din server în „Products.jsx”

În interiorul Fig.3.75, a fost implementată o funcționalitate de căutare locală prin utilizarea funcției „filterByQuery”, care permite filtrarea dinamică a produselor pe baza unui șir de caractere introdus într-un câmp de tip input. Variabila „filtered” reprezintă rezultatul acestei filtrări și este utilizată direct în generarea rândurilor tabelului. Astfel, utilizatorul poate vizualiza doar produsele relevante fără a genera cereri suplimentare către server. Un element important al acestei componente este gestionarea permisiunilor. Variabila „isAdmin” este determinată pe baza rolului utilizatorului autentificat, preluat din Redux[20] prin „useSelector”. În funcție de această valoare, sunt afișate sau ascunse butoanele pentru creare, editare și ștergere. În plus, funcția „canManage(p)” verifică dacă utilizatorul este administrator sau autorul produsului respectiv,

asigurând astfel un control suplimentar la nivelul interfeței. Totodată, crearea unui produs nou este realizată prin componenta „AddProducts.jsx”, unde datele formularului sunt gestionate prin „useState”. La trimiterea formularului, este apelată funcția „apiCreateProducts”, care transmite un obiect JSON către server printr-o cerere POST. Valorile numerice sunt convertite explicit prin „Number()”, iar câmpurile opționale sunt tratate astfel încât valorile goale să fie transformate în „null”. După crearea cu succes a produsului, utilizatorul este redirecționat către pagina principală a produselor. Editarea însă, este implementată în „EditProducts.jsx”. În această componentă, produsul este preluat inițial prin „apiProductsId(id)”, utilizând parametrul extras din URL prin „useParams”. După validarea drepturilor de modificare, datele sunt populate în formular și pot fi actualizate prin apelarea funcției „apiUpdateProducts”. Această funcție transmite modificările printr-o cerere HTTP de tip „PATCH”, actualizând doar câmpurile necesare.

```
export default function EditProduct() {
  const { id } = useParams();
  const nav = useNavigate();
  const user = useSelector((s) => s.auth.user);
  const [item, setItem] = useState(null);
  const [saving, setSaving] = useState(false);

  useEffect(() => {
    (async () => {
      try {
        const p = await apiProductsId(id);
        const isAdmin = user?.role === "ADMINISTRATOR";
        const isOwner = user?.id === p.user_id;
        if (!isAdmin && !isOwner) {
          toast.error("Nu ai voie să editezi acest produs.");
          nav("/products", { replace: true });
          return;
        }
        setItem({
          name: p.name || "",
          sku: p.sku || "",
          unit: p.unit || "buc",
          stock: p.stock ?? 0,
          stock_min: p.stock_min ?? 0,
          rack: p.rack || "",
          description: p.description || "",
          price: p.price ?? 0,
          image_url: p.image_url || "",
        });
      } catch (e) {
        toast.error("Produsul nu există.");
        nav("/products", { replace: true });
      }
    })();
  }, [id, nav, user]);
```

Fig.3.83 Funcția „apiProductsId” din „EditProducts.jsx”

```
const submit = async (e) => {
  e.preventDefault();
  setSaving(true);
  try {
    await apiUpdateProducts(id, {
      name: item.name.trim(),
      sku: item.sku.trim(),
      unit: item.unit || "buc",
      stock: Number(item.stock) || 0,
      stock_min: Number(item.stock_min) || 0,
      rack: item.rack?.trim() || null,
      description: item.description?.trim() || null,
      price: Number(item.price) || 0,
      image_url: item.image_url?.trim() || null,
    });
    toast.success("Produs actualizat.");
    nav("/products");
  } catch (err) {
    toast.error(err?.message || "Eroare la actualizare.");
  } finally {
    setSaving(false);
  }
};
```

Fig.3.84 Funcția „apiUpdateProducts” din „EditProducts.jsx”

Pentru vizualizarea detaliată a unui produs, este utilizată componenta „ProductDetailsModal.jsx”. Aceasta este afișată condiționat prin intermediul variabilei „open” și primește ca parametru obiectul „item”. Modalul afișează informații complete despre produs, inclusiv imaginea, prețul formatat cu două zecimale și data ultimei actualizări convertită într-un format lizibil prin „toLocaleString()”. Implementarea unui modal dedicat contribuie la păstrarea unei interfețe curate, evitând supraîncărcarea paginii principale. În cele din urmă, ștergerea unui produs este realizată prin apelarea funcției „apiDeleteProducts”, asociată unei cereri HTTP „DELETE”. Înainte de executarea operației, este afișată o fereastră de confirmare prin „confirm()”, prevenind ștergerea accidentală.

```
export default function ProductDetailsModal({ open, item, onClose }) {
  if (!open || !item) return null;

  const stop = (e) => e.stopPropagation();
  const hasDescription = (item.description ?? "").toString().trim().length > 0;
```

Fig.3.85 Afișarea detaliilor produsului în „ProductDetailsModal.jsx”

Structura tabelului este organizată astfel încât să evidențieze informațiile relevante, iar în cazul în care valoarea stocului este mai mică decât pragul minim (stock < stock_min), rândul respectiv este stilizat diferit, utilizând clase CSS[10] pentru a evidenția vizual situațiile critice. Această abordare oferă un mecanism rapid de identificare a produselor ce necesită reaprovizionare. Integrarea funcționalității de export în format Excel, implementată prin biblioteca SheetJS[16], completează acest modul.

```
<td
  className={
    p.stock < p.stock_min ? "text-danger fw-semibold" : ""
  }
>
  {p.stock}
</td>
```

Fig.3.85 Evidențierea produselor cu stocul sub pragul minim

3.5.9 Implementarea componentei care gestionează tranzacțiile

Componenta destinată gestionării tranzacțiilor reprezintă unul dintre cele mai importante module funcționale ale aplicației, întrucât reflectă în timp real modificările de stoc produse prin operații de tip „IN” și „OUT”. Implementarea acestei componente este realizată în fișierul „Transactions.jsx”, unde sunt integrate mecanismele de afișare, filtrare, creare, editare și ștergere a tranzacțiilor. La încărcarea paginii, datele sunt preluate de la server prin apelul funcției „apiAllTransactions()”, în cadrul unui „useEffect”. Rezultatul obținut este salvat în starea locală „rows”, iar înainte de afișare fiecare obiect este trecut prin funcția auxiliară „decorate”, care completează anumite câmpuri lipsă, precum „product_name” sau „created_by”. Această etapă contribuie la menținerea unei afișări coerente în interfață, chiar și în situațiile în care anumite informații nu sunt transmise explicit de backend.

```
const decorate = (t) => ({
  ...t,
  product_name: t.product_name ?? `#${t.product_id}`,
  created_by: t.created_by ?? `user#${t.user_id}`,
});
const filtered = filterByQuery(rows, TRX_SEARCH_FIELDS, q);

useEffect(() => {
  (async () => {
    try {
      const data = await apiAllTransactions();
      setRows(data.map(decorate));
    } catch {
      toast.error("Nu pot încărca tranzacțiile.");
    } finally {
      setLoading(false);
    }
  })();
}, []);
```

Fig.3.86 Încărcarea și decorarea listei de tranzacții în „Transactions.jsx”

Datele sunt afișate într-un tabel Bootstrap[11], cu evidențiere vizuală a tipului tranzacției prin utilizarea claselor „text-bg-success” pentru „IN” și „text-bg-danger” pentru „OUT”. Această diferențiere cromatică facilitează interpretarea rapidă a datelor și contribuie la o experiență vizuală intuitivă.

În partea superioară a paginii este implementat un mecanism de căutare locală, similar celui din componenta Produse. Variabila „q” stochează textul introdus de utilizator, iar lista filtrată este calculată prin apelul funcției „filterByQuery(rows, TRX_SEARCH_FIELDS, q)”. Astfel, utilizatorul poate căuta tranzacții după produs, SKU, tip, cantitate sau descriere, fără a genera apeluri suplimentare către server.

Un element esențial al componentei îl reprezintă gestionarea permisiunilor. Variabila „isAdmin” este calculată pe baza rolului utilizatorului, iar funcția „canManage(t)” determină dacă utilizatorul poate modifica sau șterge tranzacția respectivă. În interfață, butoanele „Editează” și „Șterge” sunt afișate doar dacă această condiție este îndeplinită.

```
const TRX_XLSX_FILE = "tranzactii.xlsx";

export default function Transactions() {
  const user = useSelector((s) => s.auth.user);
  const isAdmin = user?.role === "ADMINISTRATOR";
  const canManage = (t) => isAdmin || user?.id === t.user_id;

  const [rows, setRows] = useState([]);
  const [loading, setLoading] = useState(true);

  const [openDetails, setOpenDetails] = useState(false);
  const [detailRow, setDetailRow] = useState(null);

  const [openCreate, setOpenCreate] = useState(false);

  const [openEdit, setOpenEdit] = useState(false);
  const [editId, setEditId] = useState(null);


  const [q, setQ] = useState("");
  const decorate = (t) => ({
    ...t,
    product_name: t.product_name ?? `#${t.product_id}`,
    created_by: t.created_by ?? `user#${t.user_id}`,
  });
  const filtered = filterByQuery(rows, TRX_SEARCH_FIELDS, q);
```

Fig.3.87 Filtrarea tranzacțiilor, configurarea exportului și afișarea funcției „Transactions”

Crearea unei tranzacții este realizată prin intermediul componentei „CreateTransactionModal”, care este deschisă la apăsarea butonului „+ Creează tranzacție”. Formularul colectează informații despre produs, tip, cantitate și descriere, iar la validare trimite datele către server prin funcția „apiCreateTransaction”. După confirmarea operației, lista locală este actualizată prin inserarea noii tranzacții la începutul array-ului „rows”. Editarea tranzacției este realizată prin componenta „EditTransactionModal”, care permite două tipuri de modificări: administratorul poate actualiza toate câmpurile, în timp ce operatorul poate modifica doar descrierea. În cazul ștergerii însă, funcția „onDelete” solicită confirmarea utilizatorului și, în urma apelului „apiDeleteTransaction”, actualizează lista locală prin eliminarea elementului corespunzător.

3.5.10 Implementarea componentei care gestionează utilizatorii

Componenta destinată gestionării utilizatorilor este implementată în fișierul „Users.jsx” și este accesibilă exclusiv utilizatorilor cu rolul de „ADMINISTRATOR”, conform mecanismului de autorizare implementat la nivelul aplicației. Această restricționare este realizată atât prin componenta „RequireRole” din sistemul de rutare, cât și prin verificarea directă a rolului utilizatorului în interiorul componentei, prin variabila „isAdmin”.

La încărcarea paginii, lista utilizatorilor este preluată de la server prin apelul funcției „apiAllUsers()”, în cadrul unui „useEffect”. Rezultatul este salvat în starea locală „rows”, iar în cazul unei erori este afișat un mesaj corespunzător prin intermediul bibliotecii react-hot-toast[22]. Această abordare permite sincronizarea interfeței cu baza de date fără reîncărcarea manuală a paginii. La fel ca în cazul tranzacțiilor și produselor, utilizatorul poate filtra rezultatele după prenume, nume, email sau rol.

Crearea unui utilizator este realizată prin intermediul componentei „CreateUserModal”, deschisă la apăsarea butonului „+ Creează utilizator”. Formularul colectează informații precum prenume, nume, email, parolă și rol. La trimiterea datelor, funcția „apiCreateUser” transmite informațiile către server, iar în cazul unui răspuns favorabil, utilizatorul nou creat este adăugat în partea superioară a listei.

```jsx
export default function CreateUserModal({ open, onClose, onCreated }) {
  const [form, setForm] = useState({
    first_name: "",
    last_name: "",
    email: "",
    password: "",
    role: "OPERATOR",
  });
  const [saving, setSaving] = useState(false);

  if (!open) return null;

  const stop = (e) => e.stopPropagation();
  const change = (e) =>
    setForm((f) => ({ ...f, [e.target.name]: e.target.value }));

  const submit = async (e) => {
    e.preventDefault();
    if (!form.first_name.trim() || !form.last_name.trim() || !form.email.trim() || !form.password.trim()) {
      toast.error("Completează prenume, nume, email și parolă.");
      return;
    }
    setSaving(true);
    try {
      const created = await apiCreateUser({
        first_name: form.first_name.trim(),
        last_name: form.last_name.trim(),
        email: form.email.trim(),
        password: form.password,
        role: form.role,
      });
      toast.success("Utilizator creat.");
      onCreated?.(created);
      onClose();
    } catch (err) {
      toast.error(err?.message || "Crearea a eșuat.");
    } finally {
      setSaving(false);
    }
  };
```

Fig.3.88 Formularul de creare a utilizatorului în „CreateUserModal.jsx”

Un aspect important al acestei componente îl reprezintă controlul accesului. Înainte de afișarea tabelului, este verificat rolul utilizatorului prin condiția „if (!isAdmin)”, iar în cazul în care acesta nu are drepturi administrative, este afișat un mesaj de restricționare a accesului. Această verificare oferă un nivel suplimentar de protecție la nivel de interfață, completând mecanismul de autorizare implementat în backend.

```
if (!isAdmin) {
  return (
    <p className="text-danger">
      Doar administratorii pot vedea lista utilizatorilor.
    </p>
  );
}
```

Fig.3.89 Condiția „if (!isAdmin)” în „Users.jsx”

4. FUNCȚIONALITĂȚII APLICAȚIEI

Testarea aplicației a fost realizată în două etape distincte. În prima etapă a fost testat server-ul de backend dezvoltat în mediul Node.js[4], utilizând framework-ul Express[5] și conectat la baza de date PostgreSQL[6], unde a fost necesară verificarea funcționării corecte a tuturor rutelor HTTP implementate anterior, precum cele de autentificare, gestionare a produselor, tranzacțiilor și utilizatorilor. Această etapă a fost esențială pentru validarea logicii aplicației înainte de integrarea completă cu interfața frontend realizată în React[12],[13].

A doua etapă a constat în testarea aplicației frontend, prin verificarea apelurilor către server, a funcționării rutării, a gestionării stării globale prin Redux Toolkit[20], precum și a mecanismului de autentificare bazat pe sesiuni și cookie-uri. În această fază au fost analizate atât răspunsurile HTTP primite de la server, cât și comportamentul interfeței în diferite scenarii de utilizare.

4.1 Testarea backend-ului

Testarea server-ului de backend a fost realizată utilizând aplicația Postman, un instrument dedicat testării și simulării cererilor HTTP. Prin intermediul acestuia au fost create colecții de request-uri corespunzătoare fiecărei rute implementate în aplicație. Conform Fig.4.1, se prezintă partea stângă a folder-ului numit „RAWVISTA”, unde se află toate cererile către server-ul de backend, încadrate într-un director specific rutei. Fiecare dintre acestea are câte un nume specific și operația HTTP atribuită în urma apelului. Din figură se poate observa un exemplu de testare a rutei de înregistrare a unui utilizator, utilizând metoda HTTP POST și adresa „http://localhost:3000/api/auth/register”. În corpul cererii este transmis un obiect de tip JSON care conține câmpurile necesare creării contului, precum „first_name”, „last_name”, „email”, „password” și „role”.

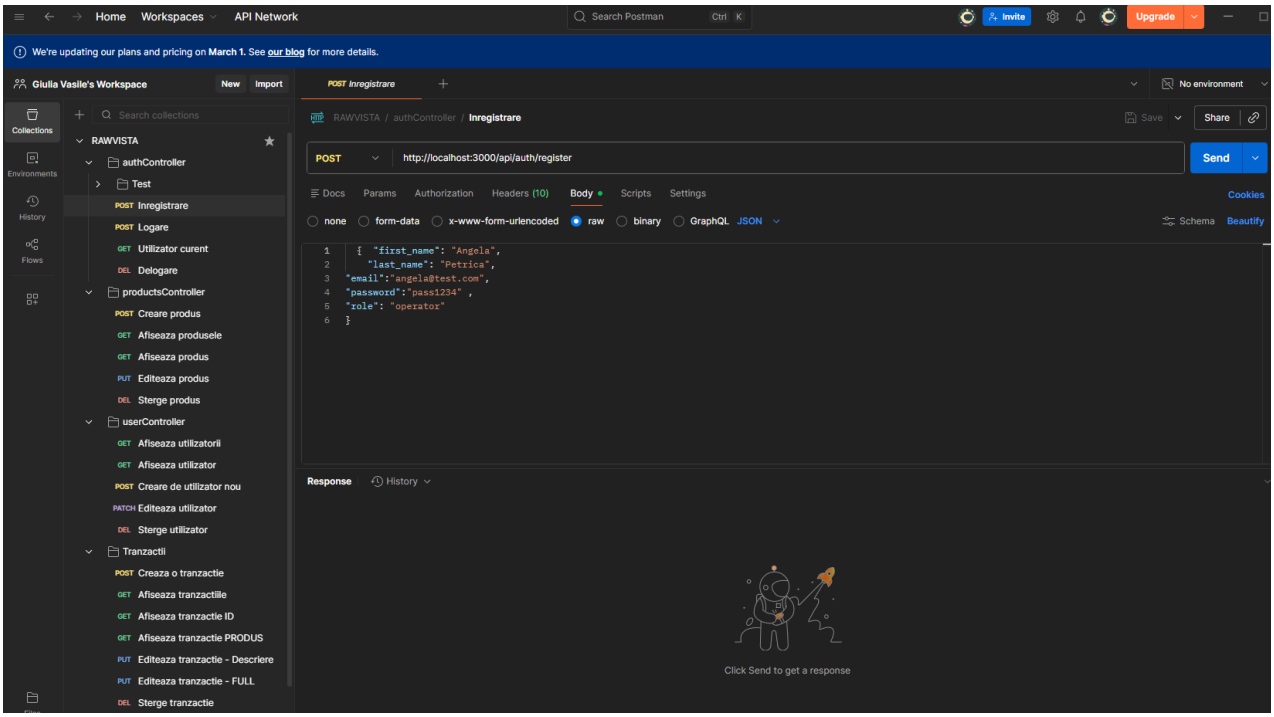


Fig.4.1 Aplicația „Postman”

În cazul în care se încearcă înregistrarea unui utilizator cu o adresă de email deja existentă în baza de date, server-ul returnează un cod de eroare (de exemplu 400 sau 409), împreună cu un mesaj explicit de tipul „Email deja utilizat”. Acest comportament confirmă existența validărilor implementate la nivelul modelului și controller-ului.



Fig.4.2 Exemplu de răspuns de eroare la înregistrare

În situația în care datele sunt valide și adresa de email nu există în sistem, server-ul returnează un cod de succes (201 Created sau 200 OK), confirmând inserarea corectă în tabela users din baza de date PostgreSQL[6].



Fig.4.3 Răspuns corect la operația de înregistrare

4.2 Testarea frontend-ului

Testarea aplicației frontend a fost realizată utilizând instrumentele integrate în browser, în special consola JavaScript[3] și tab-ul „Network” din DevTools. În timpul dezvoltării, au fost utilizate mesaje de tip „console.log()” pentru a verifica datele primite din apelurile API, precum și actualizarea corectă a stării globale prin Redux Toolkit[20]. Această metodă a permis identificarea rapidă a eventualelor discrepanțe între datele primite de la server și cele afișate în interfață.

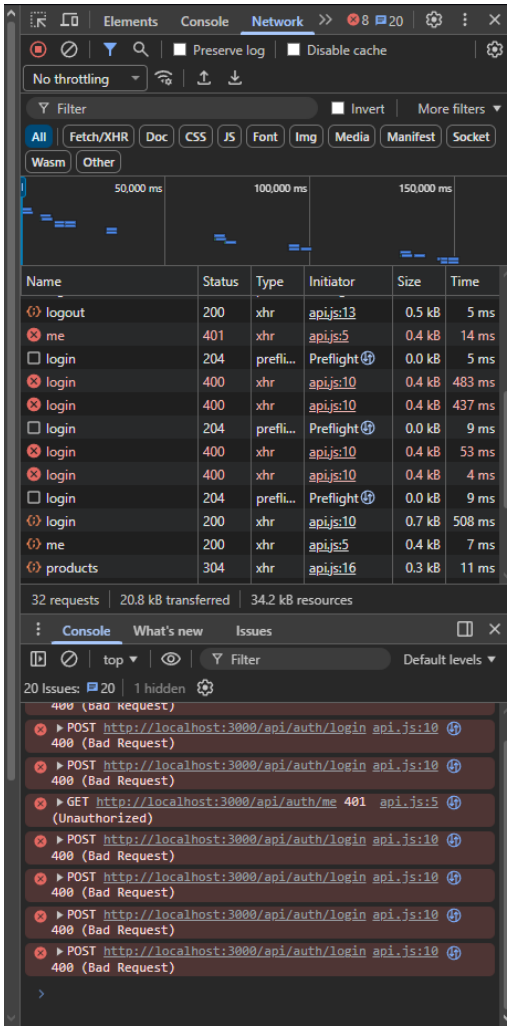


Fig.4.4 Tab-ul „Network” din browser

5. INTERFAȚA CU UTILIZATORUL

La prima accesare a platformei, utilizatorul este întâmpinat de o interfață intuitivă, care prezintă identitatea aplicației și scopul acesteia. Pagina de start oferă instrucțiuni clare de înregistrare sau autentificare, necesare pentru continuarea utilizării platformei și a funcționalităților diferite regăsite în aceasta. Astfel, în funcție de rolul atribuit, utilizatorului îi sunt afișate opțiuni personalizate în cadrul interfeței.



Fig.5.1 Pagina de start a aplicației „RAWVISTA”

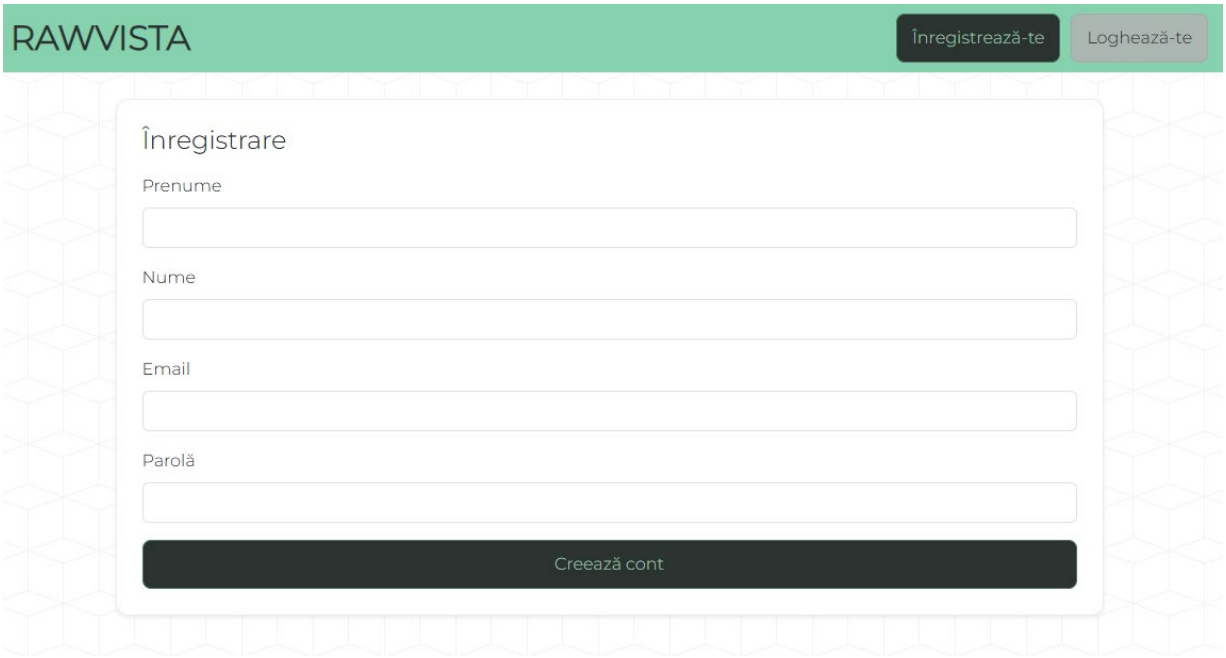


Fig.5.2 Pagina de înregistrare a aplicației „RAWVISTA”

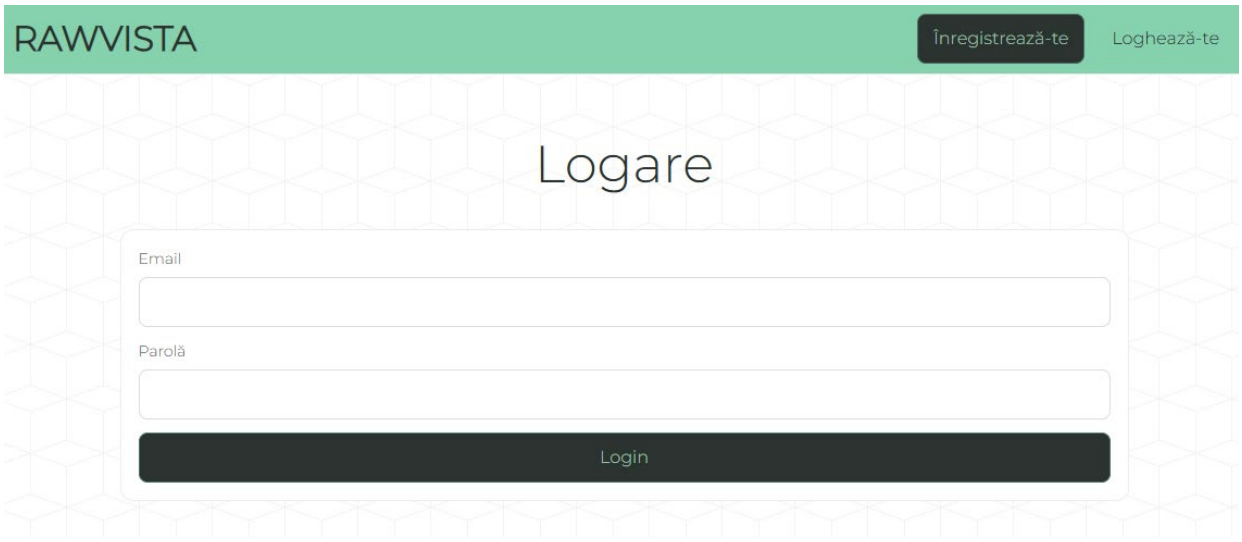


Fig.5.3 Pagina de autentificare a aplicației „RAWVISTA”

După finalizarea procesului de înregistrare, utilizatorul este autentificat automat în aplicație cu rolul de operator. Totodată, la redirecționarea sa către interfața corespunzătoare acestui rol, funcționalitățile devin disponibile conform permisiunilor definite în sistem.

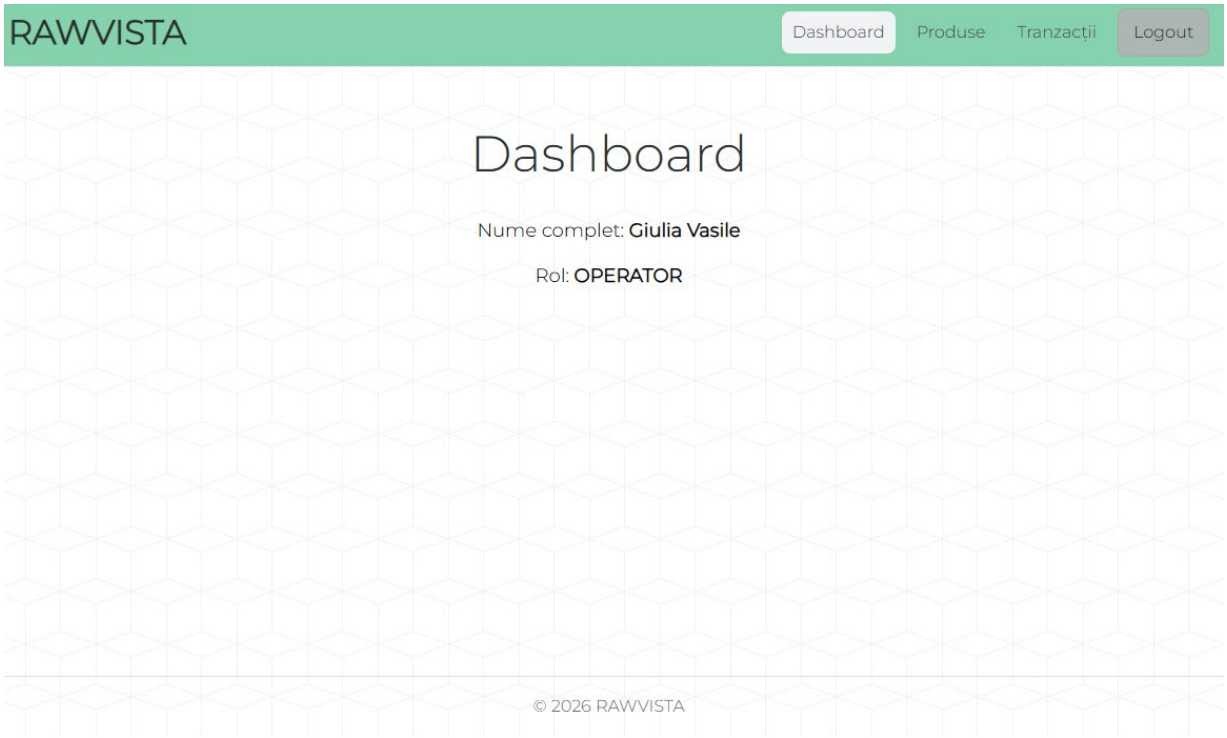


Fig.5.4 Interfața Dashboard pentru utilizatorul cu rol de „OPERATOR”

5.1 Interfața utilizatorului cu rolul de operator

Operatorul are un nivel de acces limitat, corespunzător responsabilităților sale. Acesta poate vizualiza produsele, poate înregistra tranzacții și poate consulta istoricul modificărilor, însă nu poate gestiona conturi de utilizator și nu poate șterge date din sistem. Interfața sa este simplificată, iar opțiunile indisponibile nu sunt afișate, evitând astfel confuzia și prevenind accesul neautorizat la funcționalități critice. Aceste responsabilități sunt ilustrate în figurile următoare.

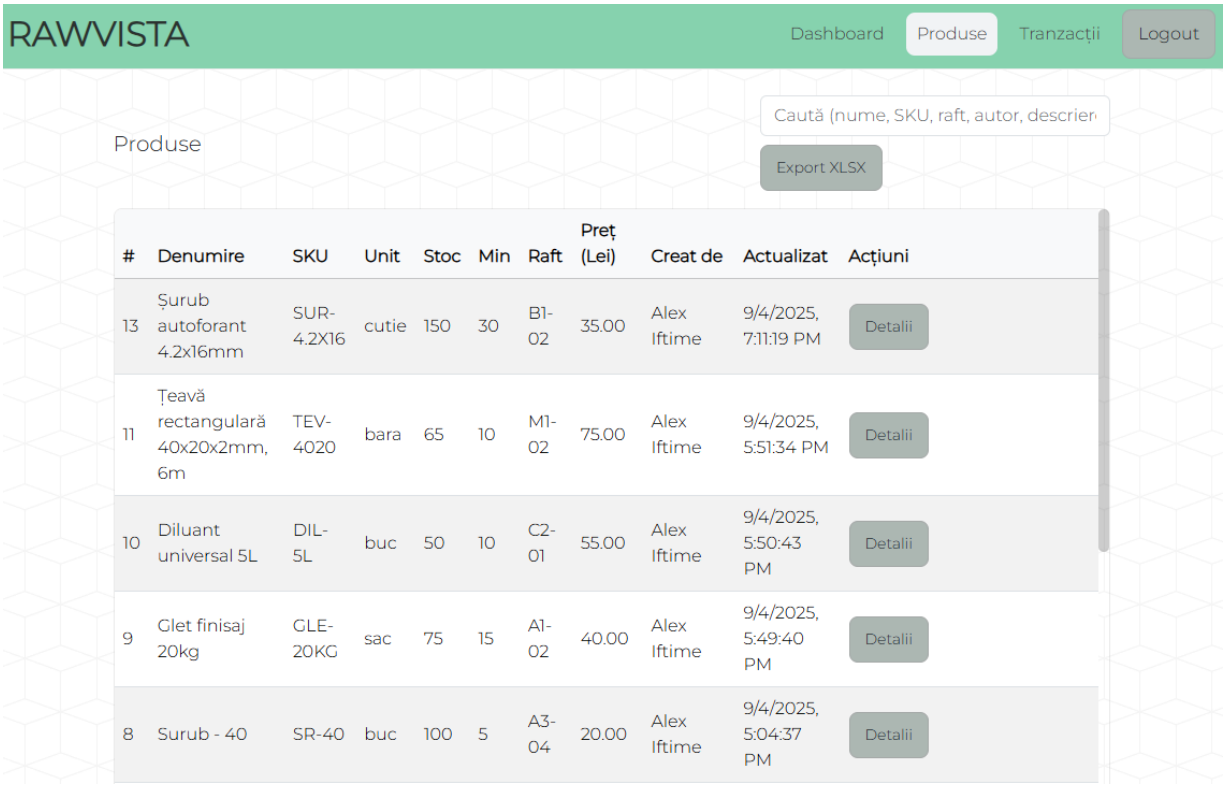


Fig.5.5 Interfața produselor pentru utilizatorul cu rol de „OPERATOR”

Conform Fig.5.5, utilizatorul poate vizualiza lista produselor existente în sistem, poate căuta și filtra produse după diferite criterii, poate exporta datele în format Excel și poate accesa detaliile fiecărui produs în parte.

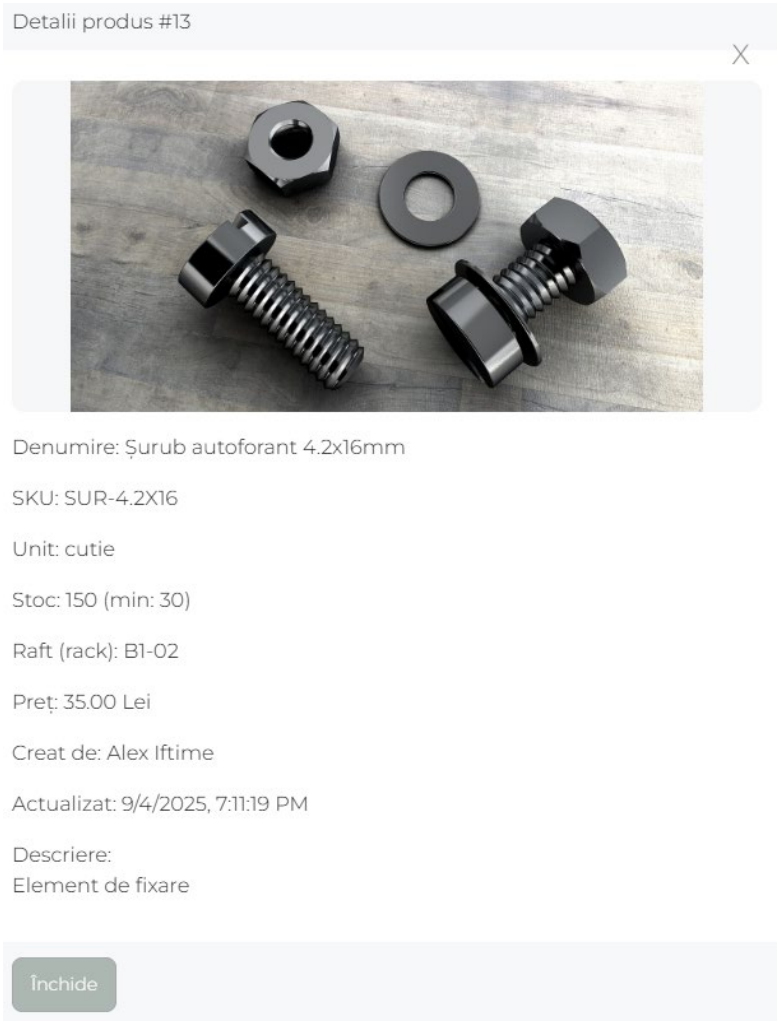


Fig.5.6 Acțiunea „Detalii” din cadrul platformei

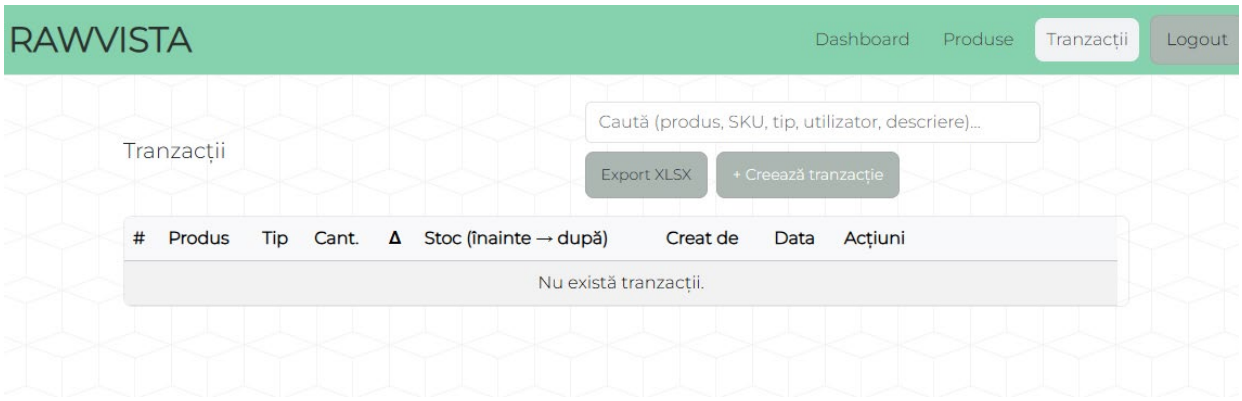


Fig.5.7 Interfața tranzacțiilor pentru utilizatorul cu rol de „OPERATOR” (partea 1)

În Fig.5.7, se poate observa că în dreptul tranzacțiilor, utilizatorul cu rol de operator vizualizează doar mișcările de stoc realizate de acesta, limitarea fiind implementată pentru a preveni modificările sau intervențiile accidentale asupra operațiilor efectuate de alte conturi.

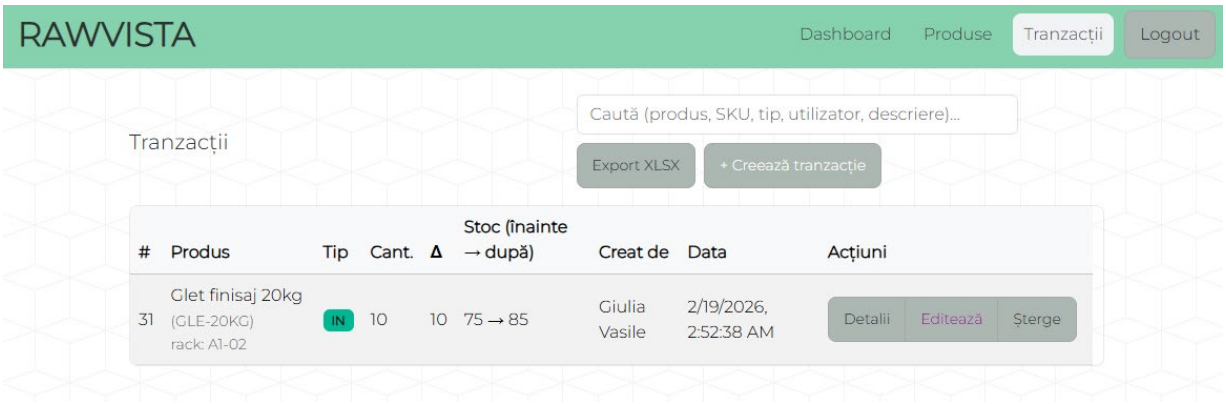


Fig.5.8 Interfața tranzacțiilor pentru utilizatorul cu rol de „OPERATOR” (partea 2)

După cum se observă în Fig. 5.8, este prezentată o tranzacție introdusă de operator, iar la utilizarea opțiunii de export, aceeași tranzacție va fi inclusă și la descărcarea în fișierul Excel.

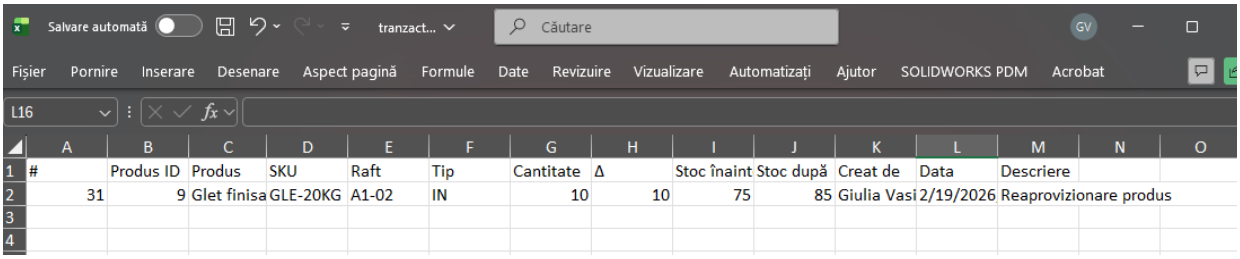


Fig.5.9 Fișierul Excel generat de operator prin exportul tranzacțiilor

În cazul utilizatorilor cu rol de operator, editarea unei tranzacții este limitată exclusiv la modificarea câmpului „Descriere”. Această restricție previne alterarea cantității, tipului sau a produsului asociat tranzacției, menținând astfel integritatea stocului și reducând riscul unor modificări eronate asupra datelor esențiale.

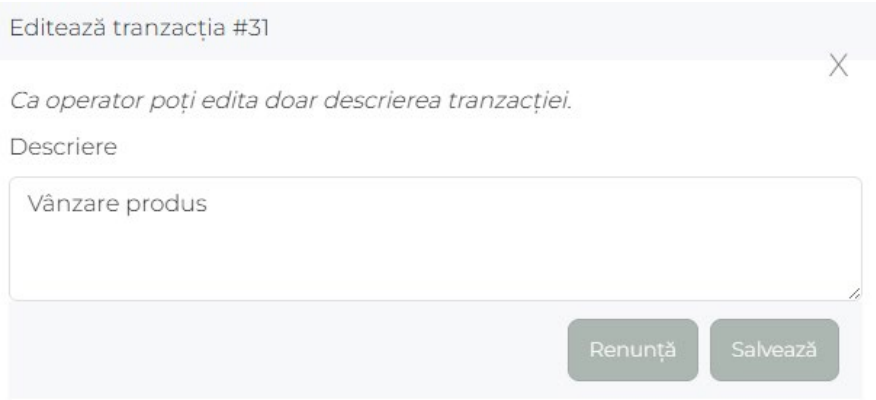


Fig.5.10 Meniul de editare al tranzacțiilor pentru „OPERATOR”

5.2 Interfața utilizatorului cu rolul de administrator

Administratorul beneficiază de un nivel extins de acces, corespunzător rolului său de coordonare și control al sistemului. Acesta poate gestiona integral produsele, tranzacțiile și conturile de utilizator, având posibilitatea de a crea, edita sau șterge înregistrări din aplicație. Interfața sa include opțiuni suplimentare față de cea a operatorului, permițând intervenții asupra tuturor datelor existente, în vederea menținerii coerenței și corectitudinii informațiilor din sistem.

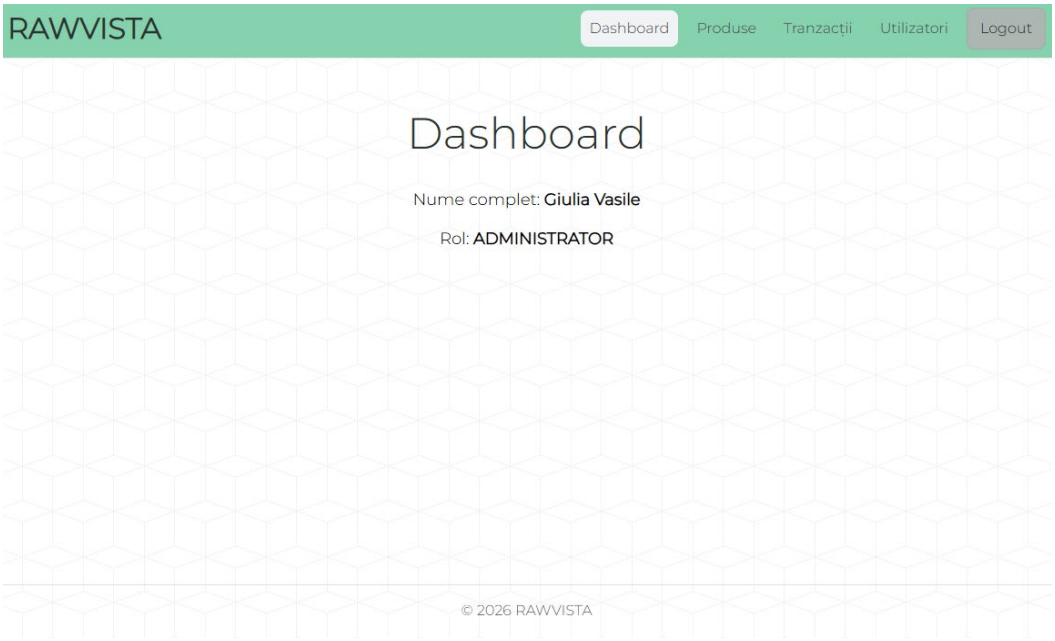


Fig.5.11 Interfața Dashboard pentru utilizatorul cu rol de „ADMINISTRATOR”

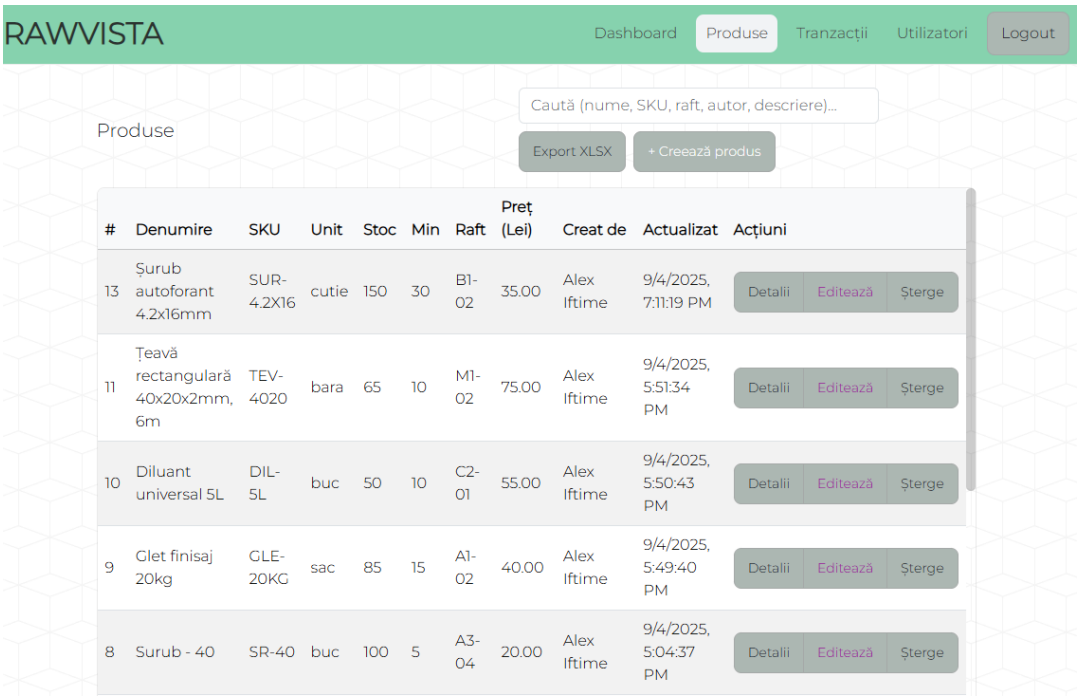


Fig.5.12 Interfața produselor pentru utilizatorul cu rol de „ADMINISTRATOR”

După cum se observă în Fig.5.12, utilizatorul cu rol de administrator are acces la butonul „Creează produs”, ceea ce îi permite adăugarea unor noi articole în sistem.

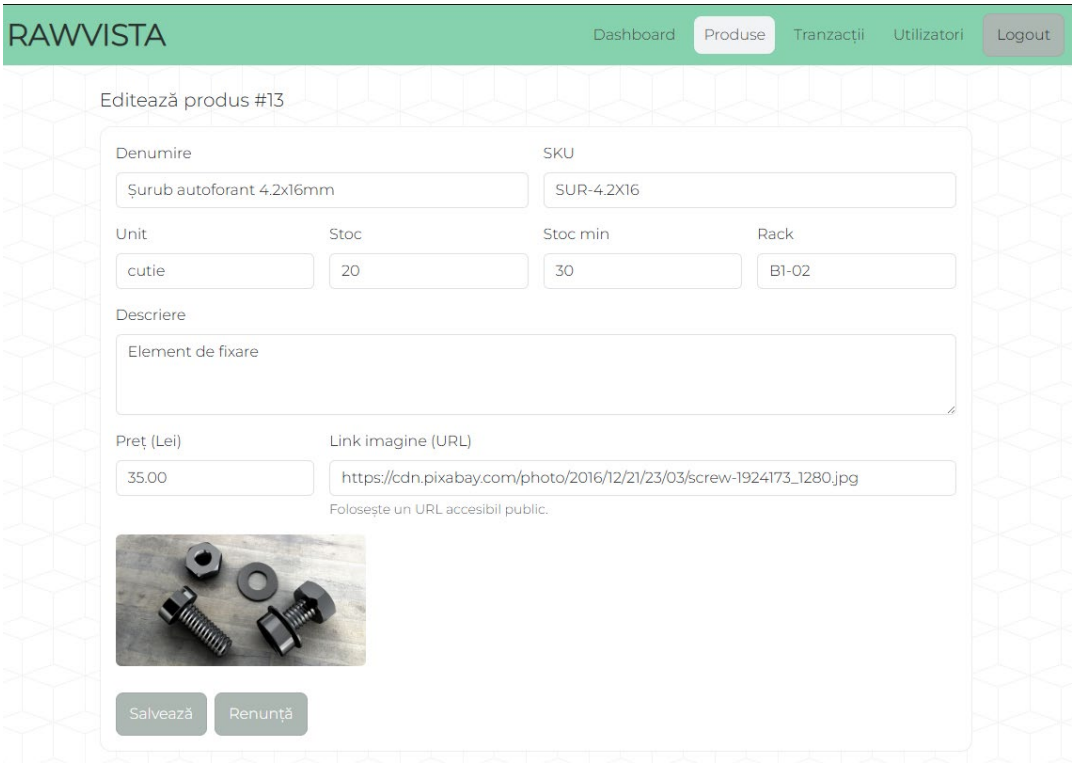


Fig.5.13 Meniul de editare al produselor pentru „ADMINISTRATOR”

Un alt avantaj al paginii de produse constă în faptul că sistemul evidențiază automat, printr-o culoare distinctă, produsele al căror stoc a scăzut sub valoarea minimă setată, permițând astfel utilizatorilor de orice tip să identifice rapid situațiile critice și să ia măsuri de reaprovizionare în timp util.

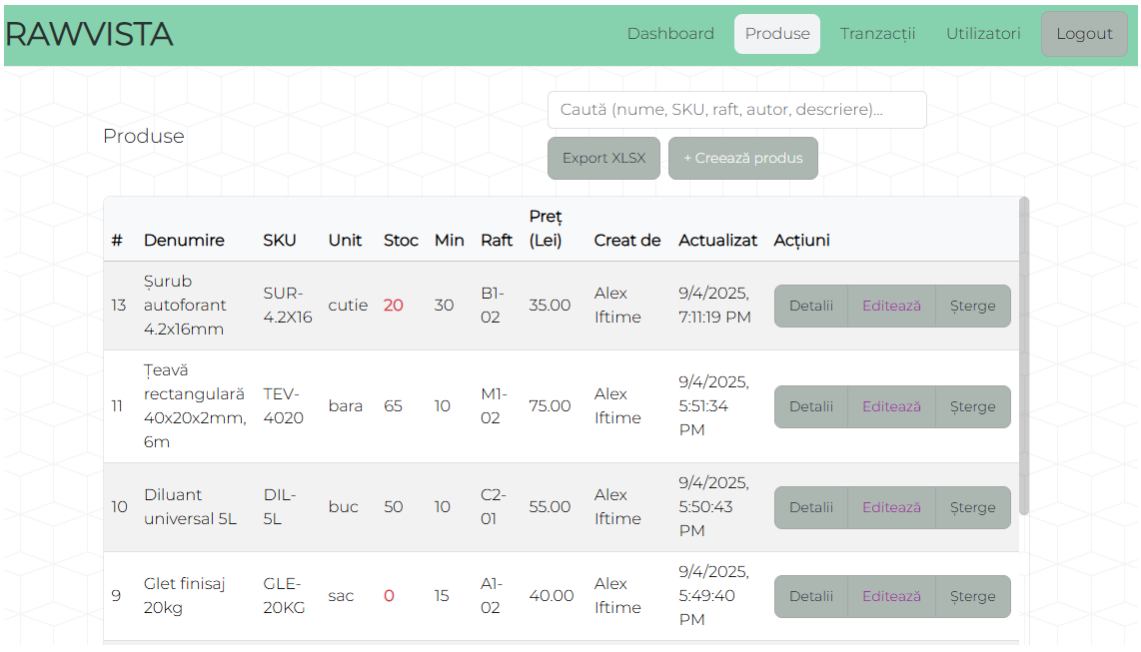


Fig.5.14 Marcarea stocului minim

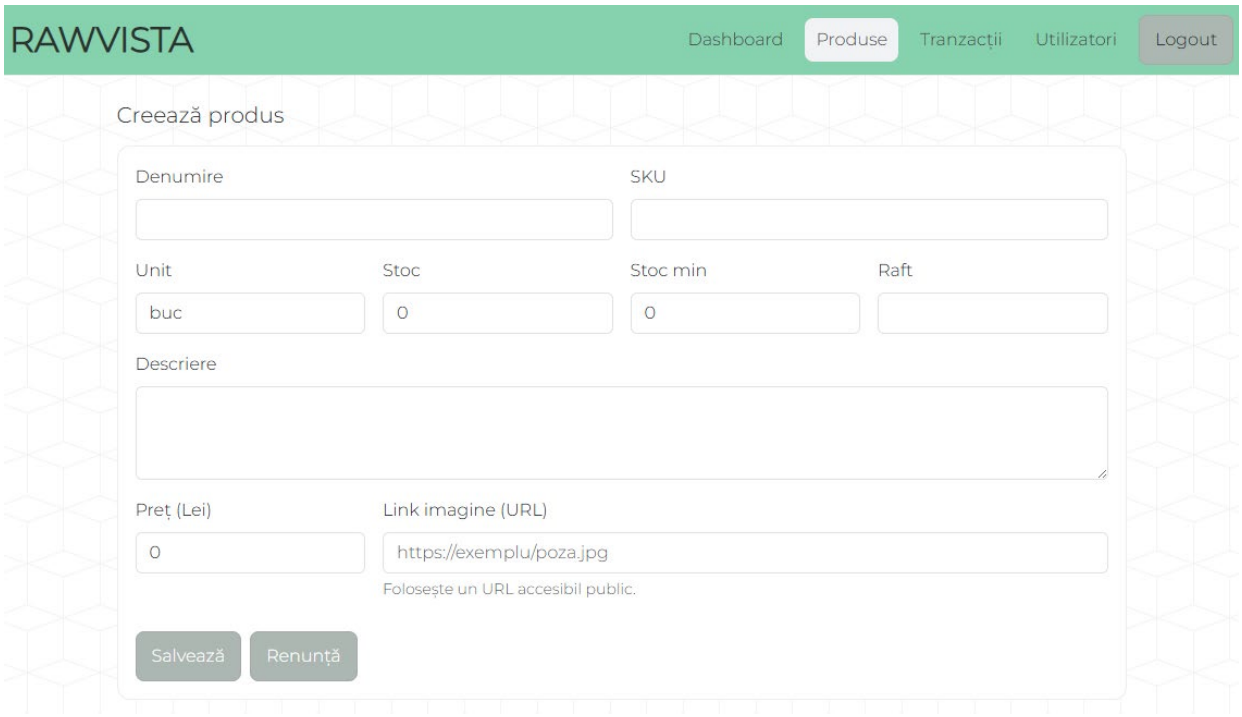


Fig.5.15 Interfața administratorului pentru crearea de produse

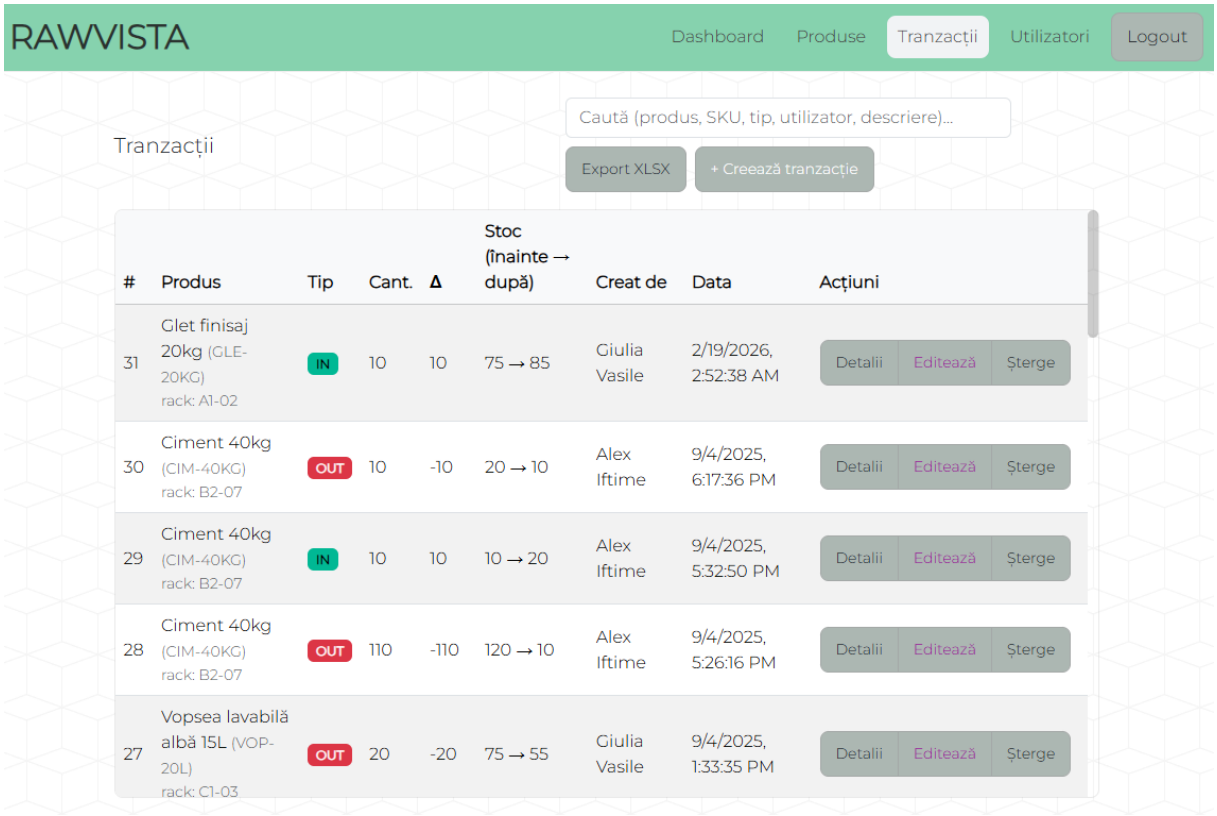


Fig.5.16 Interfața tranzacțiilor pentru utilizatorul cu rol de „ADMINISTRATOR”

În Fig. 5.16 se poate observa că tranzacțiile de tip „IN” și „OUT” sunt evidențiate prin culori diferite, facilitând identificarea rapidă a intrărilor și ieșirilor din stoc și contribuind la o interpretare vizuală mai clară a modificărilor efectuate asupra produselor.

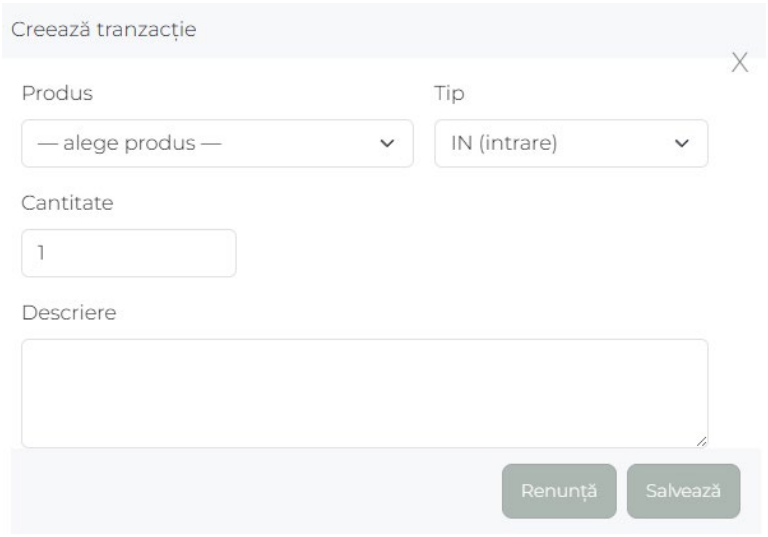


Fig.5.17 Meniul de creare a unei tranzacții noi

Un aspect important al aplicației este reprezentat de posibilitatea modificării tranzacțiilor existente. În momentul editării, sistemul actualizează automat stocul produsului asociat, recalculând diferența dintre valorile anterioare și cele noi. În cazul în care modificarea ar genera un stoc insuficient, operația este blocată și este afișat un mesaj de avertizare, asigurând astfel menținerea corectitudinii datelor și prevenirea erorilor în gestiunea resurselor.

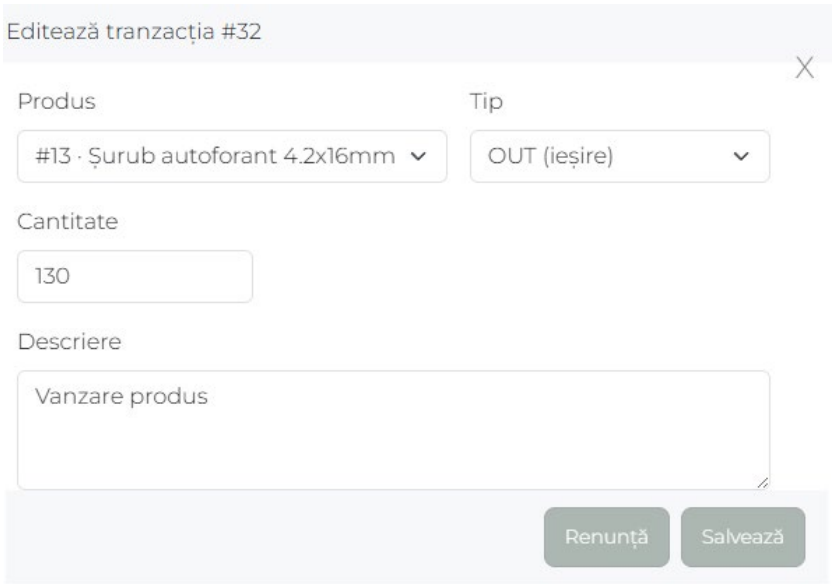
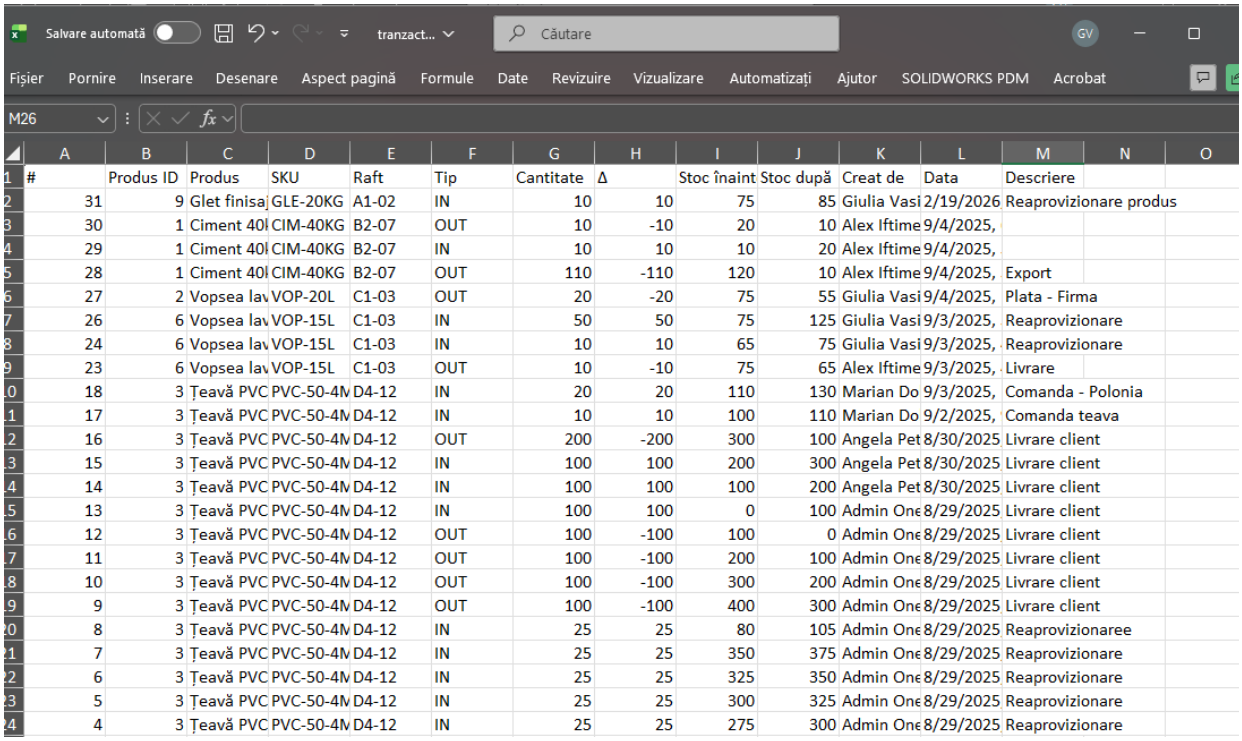


Fig.5.18 Meniul de editare al tranzacțiilor



#	Produs ID	Produs	SKU	Raft	Tip	Cantitate	Δ	Stoc înaint	Stoc după	Creat de	Data	Descriere
31	9	Glet finisa	GLE-20KG	A1-02	IN	10	10	75	85	Giulia Vasi	2/19/2026	Reaprovizionare produs
30	1	Ciment 40I	CIM-40KG	B2-07	OUT	10	-10	20	10	Alex Iftime	9/4/2025,	
29	1	Ciment 40I	CIM-40KG	B2-07	IN	10	10	10	20	Alex Iftime	9/4/2025,	
28	1	Ciment 40I	CIM-40KG	B2-07	OUT	110	-110	120	10	Alex Iftime	9/4/2025,	Export
27	2	Vopsea lav	VOP-20L	C1-03	OUT	20	-20	75	55	Giulia Vasi	9/4/2025,	Plata - Firma
26	6	Vopsea lav	VOP-15L	C1-03	IN	50	50	75	125	Giulia Vasi	9/3/2025,	Reaprovizionare
24	6	Vopsea lav	VOP-15L	C1-03	IN	10	10	65	75	Giulia Vasi	9/3/2025,	Reaprovizionare
23	6	Vopsea lav	VOP-15L	C1-03	OUT	10	-10	75	65	Alex Iftime	9/3/2025,	Livrare
18	3	Țeavă PVC	PVC-50-4M	D4-12	IN	20	20	110	130	Marian Do	9/3/2025,	Comanda - Polonia
17	3	Țeavă PVC	PVC-50-4M	D4-12	IN	10	10	100	110	Marian Do	9/2/2025,	Comanda teava
16	3	Țeavă PVC	PVC-50-4M	D4-12	OUT	200	-200	300	100	Angela Pet	8/30/2025	Livrare client
15	3	Țeavă PVC	PVC-50-4M	D4-12	IN	100	100	200	300	Angela Pet	8/30/2025	Livrare client
14	3	Țeavă PVC	PVC-50-4M	D4-12	IN	100	100	100	200	Angela Pet	8/30/2025	Livrare client
13	3	Țeavă PVC	PVC-50-4M	D4-12	IN	100	100	0	100	Admin One	8/29/2025	Livrare client
12	3	Țeavă PVC	PVC-50-4M	D4-12	OUT	100	-100	100	0	Admin One	8/29/2025	Livrare client
11	3	Țeavă PVC	PVC-50-4M	D4-12	OUT	100	-100	200	100	Admin One	8/29/2025	Livrare client
10	3	Țeavă PVC	PVC-50-4M	D4-12	OUT	100	-100	300	200	Admin One	8/29/2025	Livrare client
9	3	Țeavă PVC	PVC-50-4M	D4-12	OUT	100	-100	400	300	Admin One	8/29/2025	Livrare client
8	3	Țeavă PVC	PVC-50-4M	D4-12	IN	25	25	80	105	Admin One	8/29/2025	Reaprovizionaree
7	3	Țeavă PVC	PVC-50-4M	D4-12	IN	25	25	350	375	Admin One	8/29/2025	Reaprovizionare
6	3	Țeavă PVC	PVC-50-4M	D4-12	IN	25	25	325	350	Admin One	8/29/2025	Reaprovizionare
5	3	Țeavă PVC	PVC-50-4M	D4-12	IN	25	25	300	325	Admin One	8/29/2025	Reaprovizionare
4	3	Țeavă PVC	PVC-50-4M	D4-12	IN	25	25	275	300	Admin One	8/29/2025	Reaprovizionare

Fig.5.19 Fișierul Excel generat de administrator prin exportul tranzacțiilor

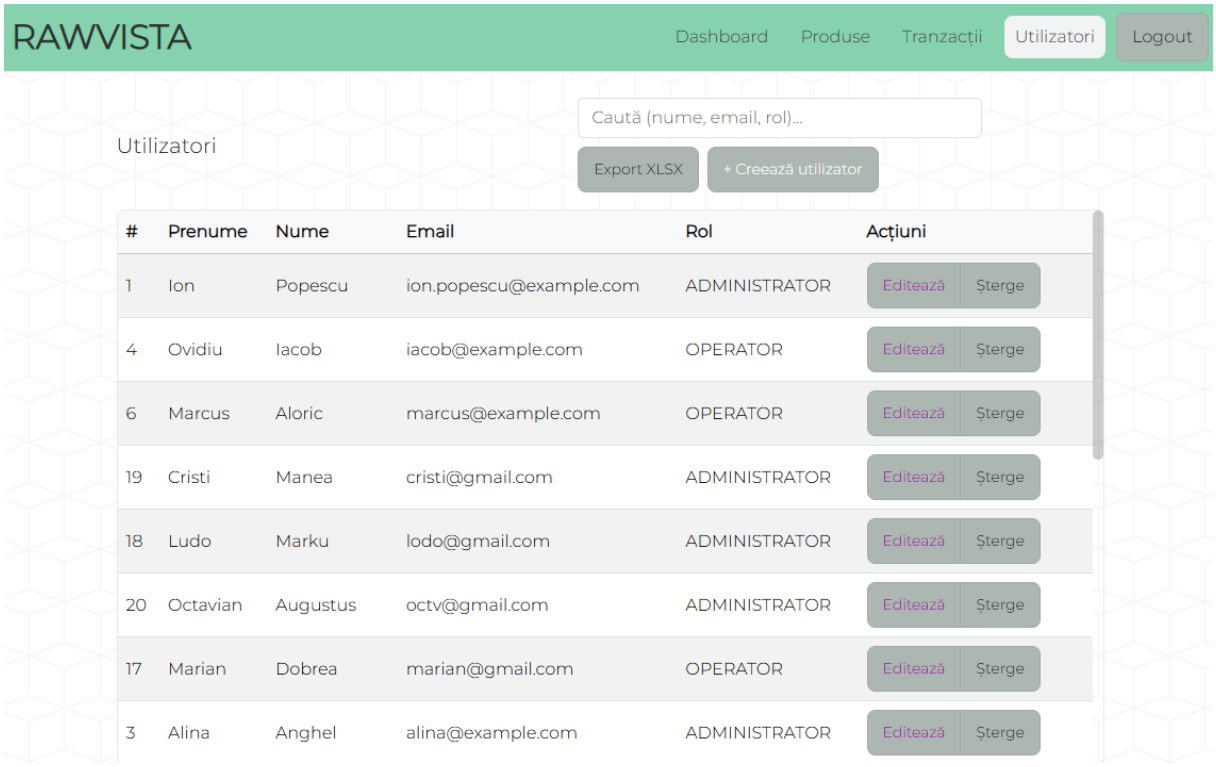


Fig.5.20 Interfața utilizatorilor pentru rolul de „ADMINISTRATOR”

Pagina „Utilizatori”, ilustrată în Fig.5.20, este accesibilă exclusiv utilizatorilor cu rol de administrator și permite vizualizarea, crearea, editarea și ștergerea conturilor din sistem, oferind astfel un control complet asupra gestionării drepturilor de acces și a structurii utilizatorilor aplicației.

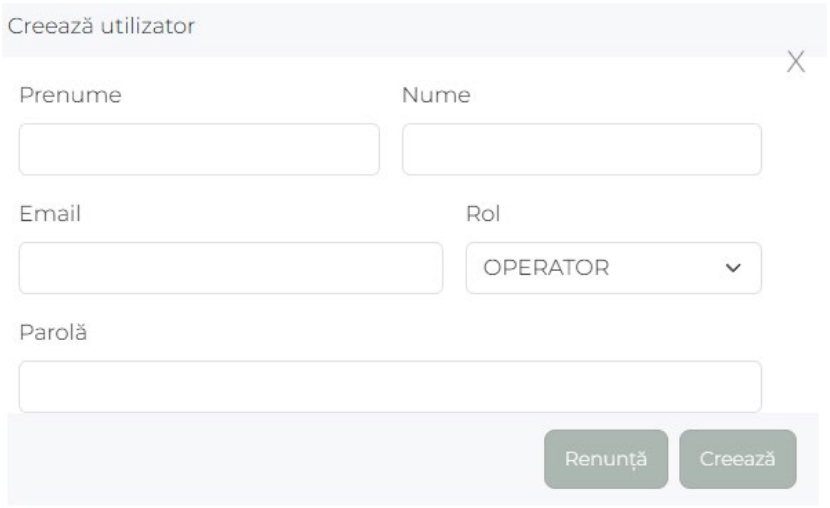


Fig.5.21 Meniul de creare a unui utilizator nou

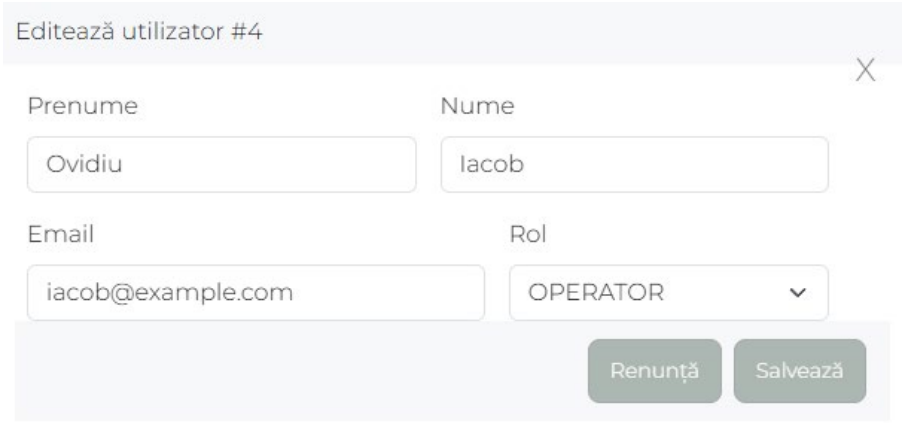


Fig.5.22 Meniul de editare a unui utilizator

6. PROIECTAREA ȘI FABRICAREA REPERULUI „ARBORE PINION” DIN CADRUL PRODUSULUI „POMPĂ CU ROȚI DINȚATE”

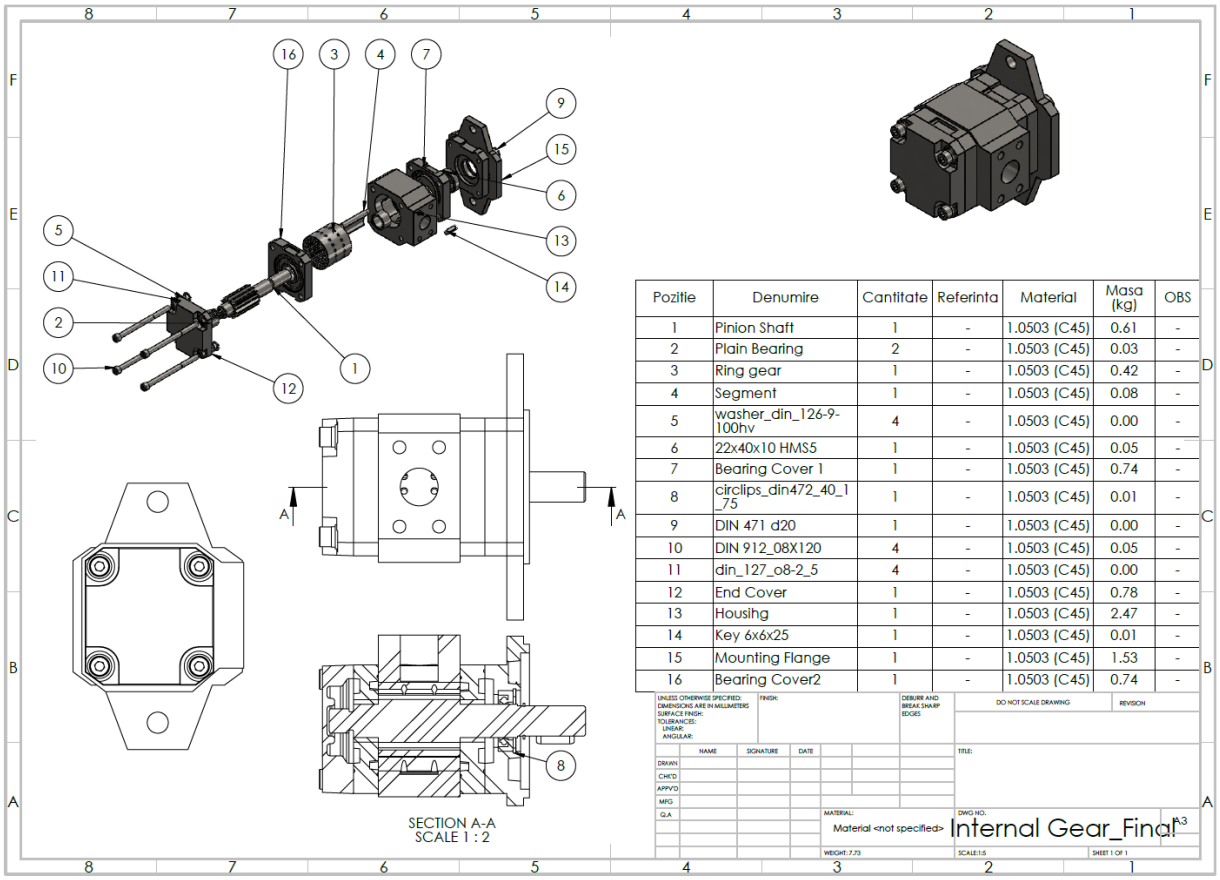
6.1 Date generale

- Produs: **Pinion Shaft.**
- Cerința economică: cost minim de producție.

6.2 Date constructive - funcționale

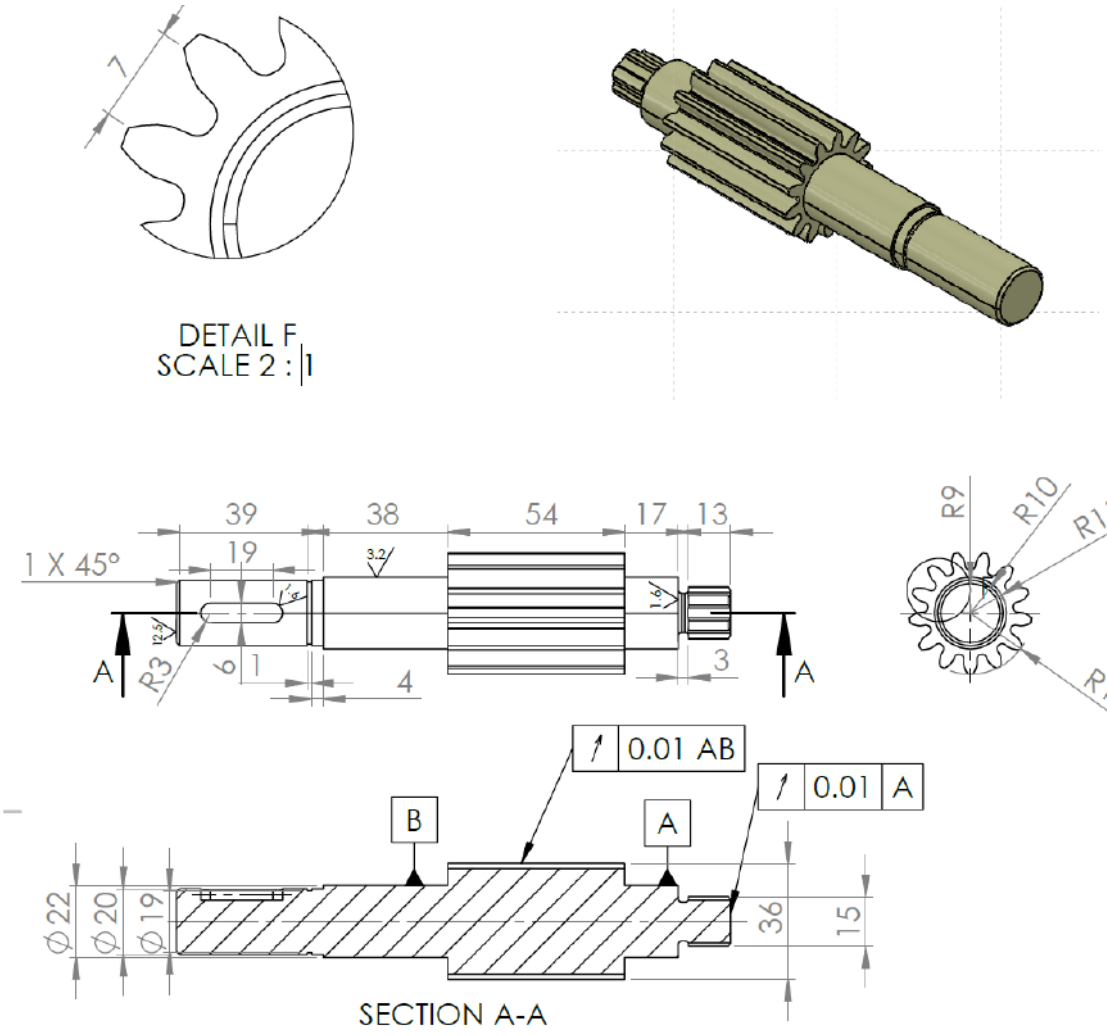
6.2.1 Schița ansamblului

O schiță a ansamblului „Pompă cu roți dințate” se prezintă în Fig.6.1, în care produsul considerat „Pinion Shaft” (Arbore pinion) are poziția 1.



Pozitie	Denumire	Cantitate	Referinta	Material	Masa (kg)	OBS
1	Pinion Shaft	1	-	1.0503 (C45)	0.61	-
2	Plain Bearing	2	-	1.0503 (C45)	0.03	-
3	Ring gear	1	-	1.0503 (C45)	0.42	-
4	Segment	1	-	1.0503 (C45)	0.08	-
5	washer_din_126-9-100hv	4	-	1.0503 (C45)	0.00	-
6	22x40x10 HMS5	1	-	1.0503 (C45)	0.05	-
7	Bearing Cover 1	1	-	1.0503 (C45)	0.74	-
8	circlips_din472_40_1_75	1	-	1.0503 (C45)	0.01	-
9	DIN 471 d20	1	-	1.0503 (C45)	0.00	-
10	DIN 912_08X120	4	-	1.0503 (C45)	0.05	-
11	din_127_o8-2_5	4	-	1.0503 (C45)	0.00	-
12	End Cover	1	-	1.0503 (C45)	0.78	-
13	Housihg	1	-	1.0503 (C45)	2.47	-
14	Key 6x6x25	1	-	1.0503 (C45)	0.01	-
15	Mounting Flange	1	-	1.0503 (C45)	1.53	-
16	Bearing Cover2	1	-	1.0503 (C45)	0.74	-

Fig.6.1 Tabel de componență al ansamblului „Arbore Pinion”



CONDIȚII TEHNICE

- Muchii necotate: 45°
- Teșituri la alezaje: 45°
- Toleranțe generale ISO 2768 - mK
- Marcare: AD01.05

3.2

Proiectat și Desen original v1	Desenat_v2	Verificat	Control standarde	Aprobat	Data/ *Aprobare*
XY01		.....	.....	.....	.....
Universitatea POLITEHNICA din București Facultatea IIR Departament TCM/ 641 AD		*Ansamblul,* Arbore Pinion.	*Material produs,* C45 SR EN 10083-2:2007 (OLC45)		*Revizuire,* 02.11.2024
		Reper, **Corp XY01 AD01.05**	*Masă produs,* 0.61 kg	*Scară,* 1 : 2	*Proiecție:*
		Cod proiect, PFP_XY01 AD01.05	*Format,* A4	*Desen nr.,* PFP_XY01 AD01.05_2	

Fig.6.2 Schița reperului „Arbore Pinion”

6.2.2 Caracteristici constructive prescrise produsului

După analiza caracteristicilor atribuite ansamblului și produsului în raport cu cerințele dispozitiv-tehnologice, am constatat necesitatea implementării unor modificări pentru a îmbunătăți conformitatea și performanța acestora.

Schița produsului care include suprafețele definitorii Sk, k = 15, este prezentată în Fig.6.3 și în Fig.6.4.

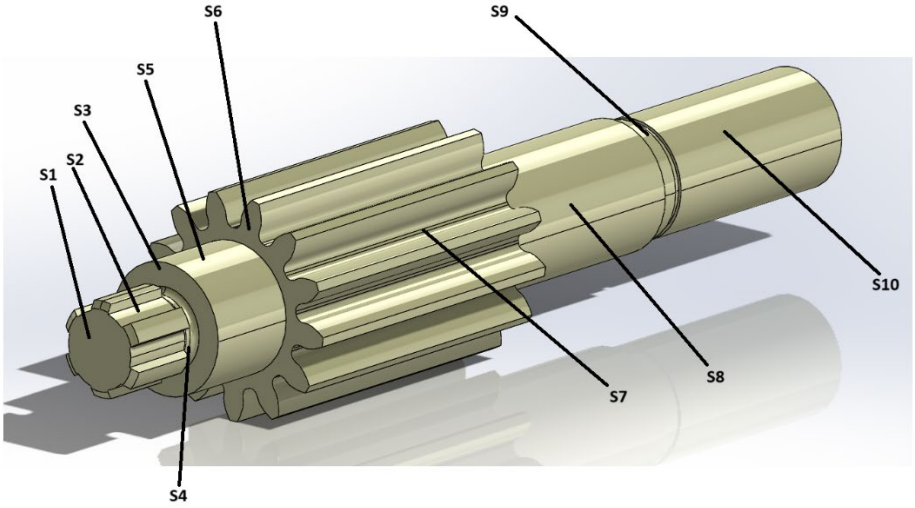


Fig.6.3 Suprafețe piesă „Arbore Pinion” (partea 1)

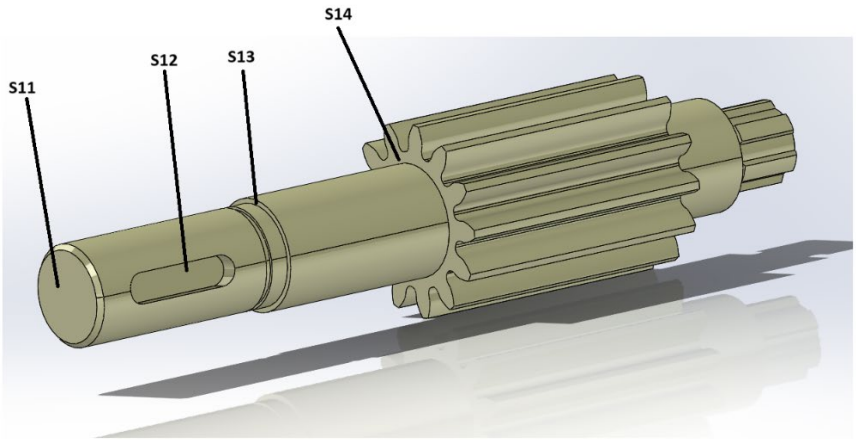


Fig.6.4 Suprafețe piesă „Arbore Pinion” (partea 2)

Caracteristicile principale ale suprafețelor Sk se prezintă în Tabelul 6.1.

Tabelul 6.1 Caracteristicile principale alea suprafetelor

S_k	**Forma nominală**	**Dimensiuni**	**Rugozitate, Ra**	**Poziţia reciprocă**	**Alte condiții/** Toleranțe generale, muchii, duritate, acoperiri de protecție etc.
S_1	Plană	Ø13 / Ø16	3.2	-	· Muchii 1x45° · Racordări R2 · Brunare chimică · Toleranțe generale ISO 2768 - mK
S_2	Canelată	13 * Ø13 * Ø16	1.6	⌖ 0,01 A	
S_3	Plană	Ø14 / Ø22	3.2	-	
S_4	Toroidală	R1 x Ø12	1.6	-	
S_5	Cilindrică	Ø22/17	3.2	Bază de referință A	
S_6	Plan-canelată	Ø22 / Ø38	3.2	-	
S_7	Canelată	54 * Ø25 * Ø38	1.6	⌖ 0,01 A B	
S_8	Cilindrică	Ø22/38	3.2	Bază de referință B	
S_9	Cilindrică	Ø19/4	1.6	-	
S_{10}	Cilindrică	Ø20/39	3.2	-	
S_{11}	Plană	Ø18 / Ø20	3.2	-	
S_{12}	Canal pană	19 * Ø6 * Ø6	1.6	-	
S_{13}	Plană	Ø20 / Ø22	3.2	-	
S_{14}	Plan-canelată	Ø22 / Ø38	3.2	-	
S_{15}	Conică	1x 45°	12.5	-	

Funcțiile suprafețelor sunt redate în următorul tabel:

Tabelul 6.2 – Caracteristicile principale alea suprafețelor

Suprafața Sk	**Funcția/-ile suprafeței**
S1	Funcție de asamblare - ajută la asamblarea elementelor în cadrul unui sistem
S2	Funcție de asamblare - transmite momentul și turația
S3	Transferă mișcarea și face legătura cu mecanismul conjugat
S4	Funcție tehnologică - face trecerea între două suprafețe cu precizie ridicată
S5	Transferă mișcarea și face legătura cu mecanismul conjugat
S6	Transferă mișcarea și face legătura cu mecanismul conjugat
S7	creează mișcare
S8	Transferă mișcarea și face legătura cu mecanismul conjugat
S9	Transferă mișcarea și face legătura cu mecanismul conjugat
S10	Transferă mișcarea și face legătura cu mecanismul conjugat
S11	Funcție de asamblare - ajută la asamblarea elementelor în cadrul unui sistem
S12	
S13	Funcție auxiliară - delimitare a conturului exterior al piesei
S14	Funcție de asamblare - transmite momentul şi turaţia
S15	Funcție tehnologică - împiedică degradarea muchiilor piesei şi rănirea manipulatorului

Abaterile limită pentru dimensiuni liniare conform ISO 2768 - mK sunt prezentate în Tabelul 6.3.

Tabelul 6.3

Clasa de toleranță		**Abateri limită pentru dimensiuni nominale,** mm			
Simbol	Descriere	0,5 - 6	6 - 30	30 - 120	120 - 400
m	mijlocie	± 0,1	± 0,2	± 0,3	± 0,5

6.2.3 Caracteristici de material

Materialul prescris este C45 (SB8673 Steel). Elementele chimice principale componente prescrise sunt prezentate în Tabelul 6.4.

Tabelul 6.4

C, %	**Cr, %**	**Ni, %**	**P, %**	**Mn, %**	**Si, %**
0,46% - 0,49%	0,25% - 0,30%	0,10% - 0,20%	≤ 0.020%	0,70% - 0,80%	0,15% - 0,35%

Tratamentul termic prescris pentru oțelul C45 este călirea, care implică încălzirea materialului la o temperatură de 820-860 °C, urmată de răcirea rapidă în ulei sau apă. După călire, este necesară revenirea, ce presupune încălzirea materialului la o temperatură de 400-650 °C și răcirea lentă, pentru a reduce fragilitatea și a ajusta duritatea și tenacitatea oțelului.

Proprietățile fizico – mecanice principale prescrise sunt prezentate în Tabelul 6.5.

Tabelul 6.5

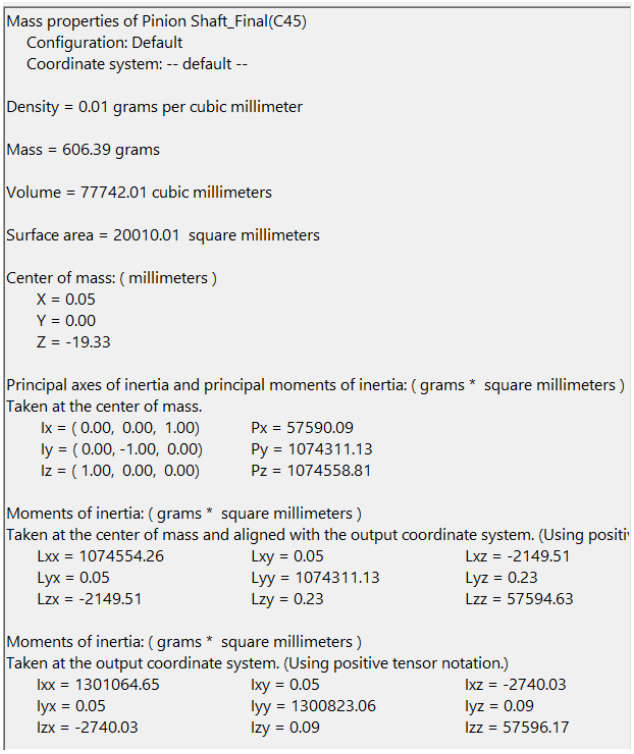
Marca fontei	**Rezistența minimă de rupere la tracțiune (MPa)**	**Rezistența minimă de rupere la încovoiere (MPa)**	**Rezistenţa minimă de rupere la compresiune (MPa)**	**Duritatea Brinell minimă (HB)**
C45	590 - 740 MPa	590 - 740 MPa	590 - 740 MPa	165 - 220 HB

6.2.4 Masa produsului

Masa produsului = > [0.61kg]

Masa produsului s-a determinat prin opțiunea Mass_Properties din SolidWorks.

Astfel, masa produsului, m, este: m = 0.61 kg.



Mass properties of Pinion Shaft_Final(C45)
Configuration: Default
Coordinate system: -- default --

Density = 0.01 grams per cubic millimeter

Mass = 606.39 grams

Volume = 77742.01 cubic millimeters

Surface area = 20010.01 square millimeters

Center of mass: (millimeters)
X = 0.05
Y = 0.00
Z = -19.33

Principal axes of inertia and principal moments of inertia: (grams * square millimeters)
Taken at the center of mass.
Ix = (0.00, 0.00, 1.00) Px = 57590.09
Iy = (0.00, -1.00, 0.00) Py = 1074311.13
Iz = (1.00, 0.00, 0.00) Pz = 1074558.81

Moments of inertia: (grams * square millimeters)
Taken at the center of mass and aligned with the output coordinate system. (Using positi
Lxx = 1074554.26 Lxy = 0.05 Lxz = -2149.51
Lyx = 0.05 Lyy = 1074311.13 Lyz = 0.23
Lzx = -2149.51 Lzy = 0.23 Lzz = 57594.63

Moments of inertia: (grams * square millimeters)
Taken at the output coordinate system. (Using positive tensor notation.)
Ixx = 1301064.65 Ixy = 0.05 Ixz = -2740.03
Iyx = 0.05 Iyy = 1300823.06 Iyz = 0.09
Izx = -2740.03 Izy = 0.09 Izz = 57596.17

Fig.6.5 Masa produsul „Arbore Pinion”

Având în vedere forma de gabarit, forma și poziția suprafețelor componente, se apreciază că produsul face parte din clasa Arbori.

6.2.5 Tehnologicitatea produsului

Gradul de unificare a elementelor constructive, λe, este

$$\lambda_e = \frac{e_d}{e_t}, \lambda_e \in (0,1]$$

unde: e_d este numărul dimensiunilor diferite, iar e_t numărul total al elementelor.

Se analizează o serie de caracteristici geometrice, după cum urmează.

$$\lambda_{e1} = \frac{e_d}{e_t} = 0$$

$$e_2 = \frac{e_d}{e_t} = \frac{1}{2} = 0.5$$

$$\lambda_{e3} = \frac{e_d}{e_t} = \frac{1}{2} = 0.5$$

Astfel, gradul mediu de unificare constructivă λ_e este

$$\lambda_e = \frac{1}{q}\sum_{i=1}^{q}\lambda_{ei} = \frac{1}{3}(0.5 + 0.5 + 0) = 1$$

Concordanța dintre caracteristicile constructive prescrise și cele impuse de rolul funcțional/ tehnologic

Gradul de concordanță dintre caracteristicile constructive (CC) prescrise și cele impuse de rolul funcțional/ tehnologic, λ_c, este

$$\lambda_c = \frac{c_c}{c_t}, \lambda_c \in (0,1]$$

unde: c_c este numărul CC prescrise care sunt în concordanţă cu cele impuse de cerinţele funcţional- tehnologice, iar c_t numărul total al CC prescrise.

După operarea modificărilor propuse:

$$\lambda_c = 1$$

A. Condiții de tehnologicitate impuse de unificarea constructivă:

Din punct de vedere al condiţiei principale impuse de *unificarea constructivă,* respectiv număr minim posibil al dimensiunilor elementelor constructive (v. λ_e), se apreciază că aceasta este acceptabilă.

B .Condiții de tehnologicitate impuse de procedeele tehnologice

Condiţiile de tehnologicitate impuse de procedeele tehnologice de fabricare sunt prezentate în Tabelul 6.6.

Tabelul 6.6

Nr. crt.	**Condiţie**	**Grad de satisfacere :** Da sau Nu
1	Forma de gabarit să prezinte axe sau plane de simetrie care, după caz, să fie utile pentru definirea planului(-elor) de separaţie a semimatriţelor	Nu
2	Elementele constructive de tip înclinare, racordare, grosime de perete, de dimensiune l, să îndeplinească condiţia $l \geq l_{min}$ sau/şi $l \leq l_{max}$, în care l_{min} şi l_{max} sunt valori limită impuse de procedeul tehnologic	Da
3	Forma sau poziţia unor suprafeţe să fie astfel încât să prezinte „înclinări” în raport cu planul de separaţie a semimatriţelor	Nu
4	Trecerile dintre suprafeţe să fie astfel încât să eVite[14] deteriorarea acestora în timpul manevrării semifabricatului	Da

5	Trecerile dintre anumite suprafeţe să fie sau nu sub formă de degajare/ canal sau teşitură în funcţie de cerinţele funcţionale şi/sau de cerinţele impuse de procedeul de prelucrare	Da
6	Forma şi poziţia bosajelor, găurilor sau locaşurilor sa fie astfel încât să conducă la un număr minim de poziţii ale piesei/sculei în timpul operaţiei, acces uşor al sculelor în zona de lucru	Da
7	Suprafeţele frontale ale găurilor să fie plane şi perpendiculare pe axele găurilor	Nu
8	Forma şi poziţia suprafeţelor să fie astfel încât să permită prinderi simple şi sigure în cadrul operaţiilor	Da

Material	Rezistenta la rupere (MPa)	Modul Young (GPa)	Rezistenta la coroziune (nota)	Cost (nota)
AISI 1019	455	205	1	4
AISI 1012	370	205	1	5
C45	740	210	1	3
AISI 304	517	190	5	2
AISI 1021	470	205	1	4
Max	740	210	5	5

Valori normalizate	Rezistenta la rupere	Modul Young	Rezistenta la coroziune	Cost
AISI 1019	0.615	0.976	0.200	0.800
AISI 1012	0.500	0.976	0.200	1.000
C45	1.000	1.000	0.200	0.600
AISI 304	0.699	0.905	1.000	0.400
AISI 1021	0.635	0.976	0.200	0.800
Factor de importanta	35	30	10	25

Valori normalizate	Rezistenta la rupere	Modul Young	Rezistenta la coroziune	Cost	Indice de performanta
AISI 1019	21.520	29.286	2.000	20.000	**72.806**
AISI 1012	17.500	29.286	2.000	25.000	**73.786**
C45	35.000	30.000	2.000	15.000	**82.000**
AISI 304	24.453	27.143	10.000	10.000	**71.596**
AISI 1021	22.230	29.286	2.000	20.000	**73.515**

Material	Indice de performanta
C45	**82.000**

Fig.6.6 Analiza indicelui de performanță

6.2.6 Interdependența toleranțelor dimensionale

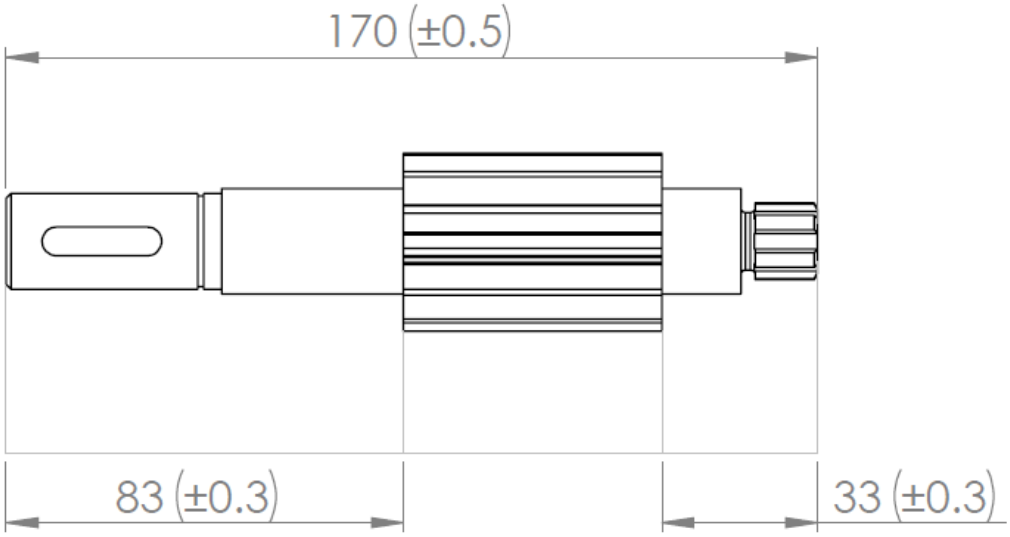


Fig.6.7 Analiza toleranțelor dimensionale

$$J_{MAX} = \sum(Dimensiuni\ măritoare)_{MAX} - \sum(Dimensiuni\ reducătoare)_{min} = \\ = B_{MAX} - A_{min} - C_{min}$$

$$J_{MAX} = 170{,}5 - (82{,}7 + 32{,}7)$$

$$J_{MAX} = 55{,}1\ mm$$

$$J_{min} = \sum(Dimensiuni\ măritoare)_{min} - \sum(Dimensiuni\ reducătoare)_{MAX} = \\ = B_{min} - A_{MAX} - C_{MAX}$$

$$J_{min} = 169{,}5 - (83{,}3 + 33{,}3)$$

$$J_{min} = 52{,}9\ mm$$

$$TOL_J = (A_{MAX} - A_{min}) + (B_{MAX} - B_{min}) + (C_{MAX} - C_{min})$$

$$TOL_J = (83{,}3 - 82{,}7) + (170{,}5 - 169{,}5) + (33{,}3 - 32{,}7)$$

$$TOL_J = 2{,}2\ \text{mm}$$

Analiza procesului de fabricație pentru reperul reprezentat prin desenul de execuție din figura următoare.

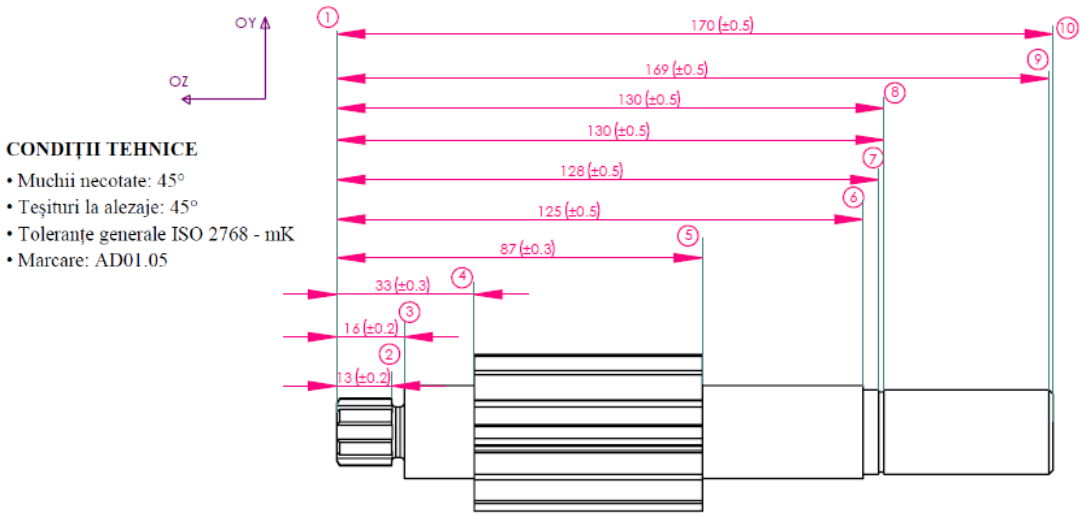
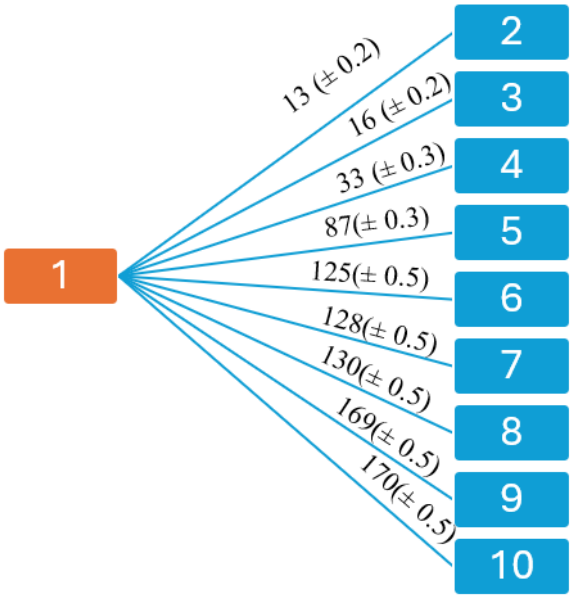


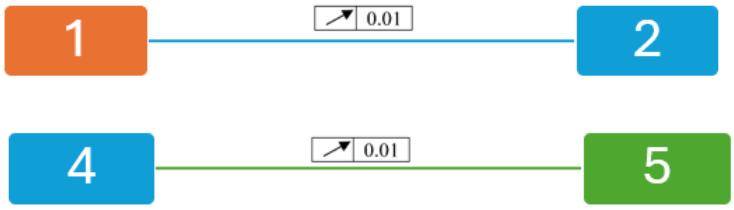
Fig.6.8 Analiza procesului de fabricație

Analiza constrângerilor de anterioritate:

Axa Oz:



Constrângeri geometrice



Constrângeri tehnologice

- Găurire înainte de lărgire
- Faza de degroșare înainte de cea de finisare
- Faza de degroșare înainte de cea de semi-finisare și de finisare

Toate celelalte suprafețe au o rugozitate Ra indicată de 3.2 µm. Așadar, aceste suprafețe se realizează direct prin finisare.

6.3 Stabilirea etapelor de prelucrare

6.3.1 Procese tehnologice

Semifabricate: Laminare

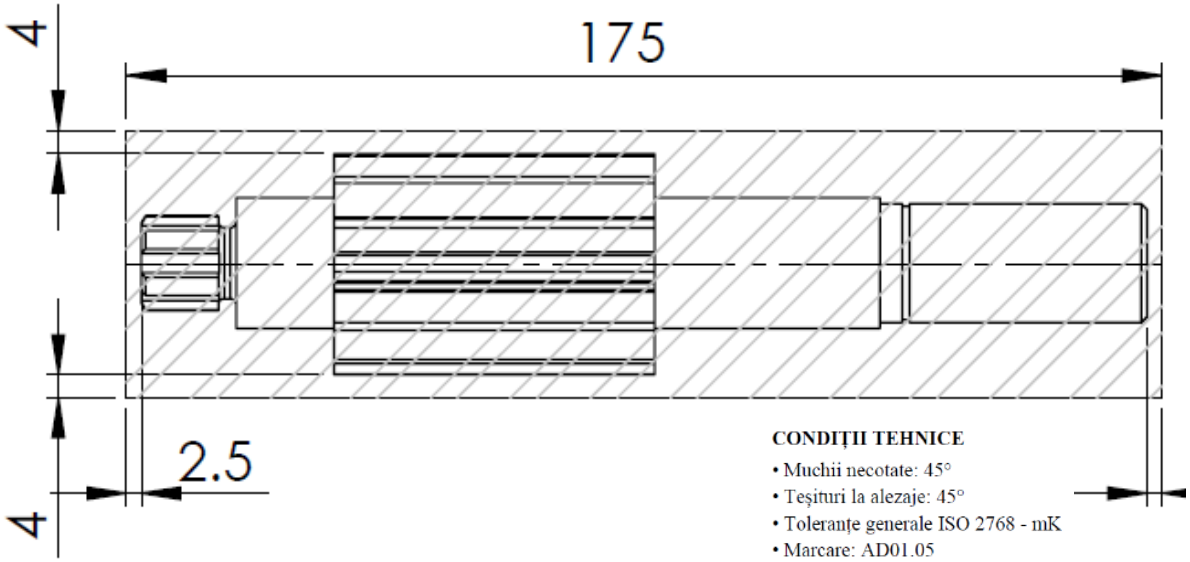


Fig.6.9 Piesă fabricată prin laminare

- Masa: 0.61 kg
- Volum: 77742,01 mm^3
- $V_{semifabricat}$= π*$r^2 * h$ = 3.14*19^2*175=198545.5 mm^3
- M = ρ * V = 0.01g/mm^3 * 198545.5 mm^3= 1985,455g = 1.985kg

Gradul de utilizare a materialului : η = $\frac{M}{M_m}$ unde: M este masa produsului și M_m este masa materialului sau materialelor consumate.

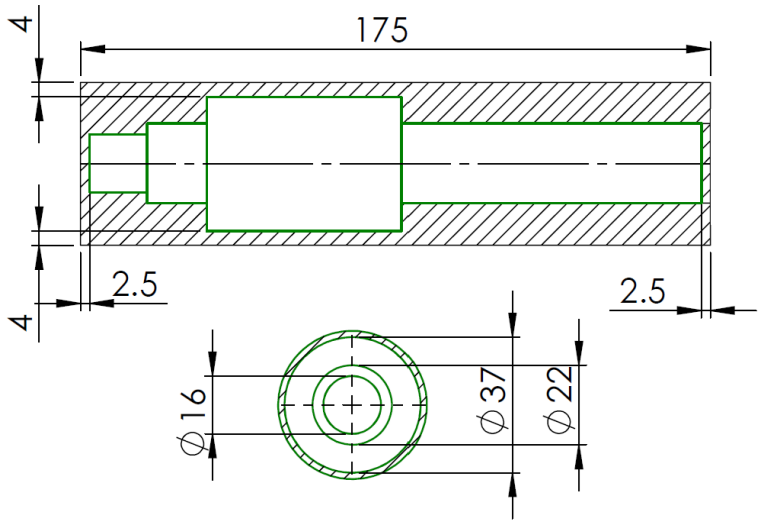
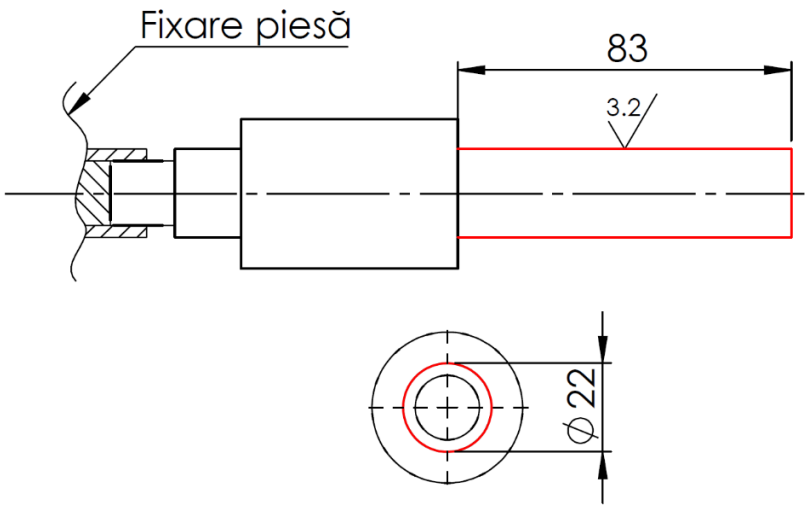
η = $\frac{M}{M_m}$ = $\frac{0.61}{1.985}$ = 0.3073 ⇒ Procentual: η =30,73%

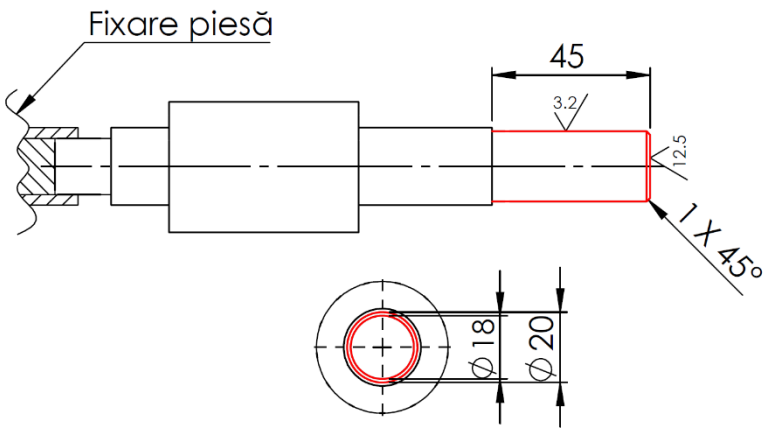
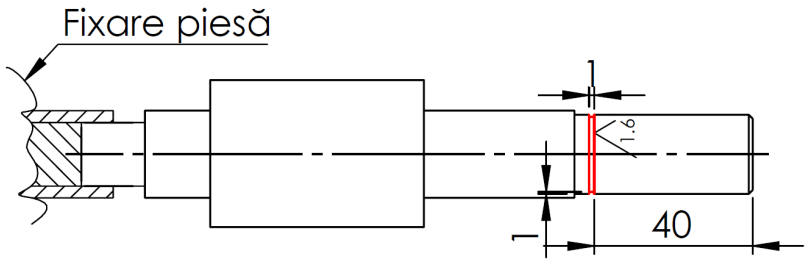
Având în vedere considerentele de mai sus, pentru fiecare suprafață sau grup de suprafețe similare, s-au determinat variante tehnic-acceptabile privind succesiunea de prelucrări necesare, care se prezintă în tabelul următor:

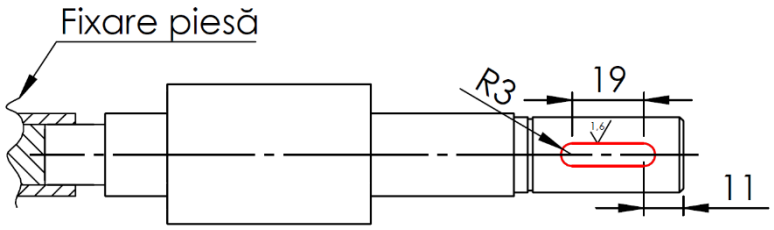
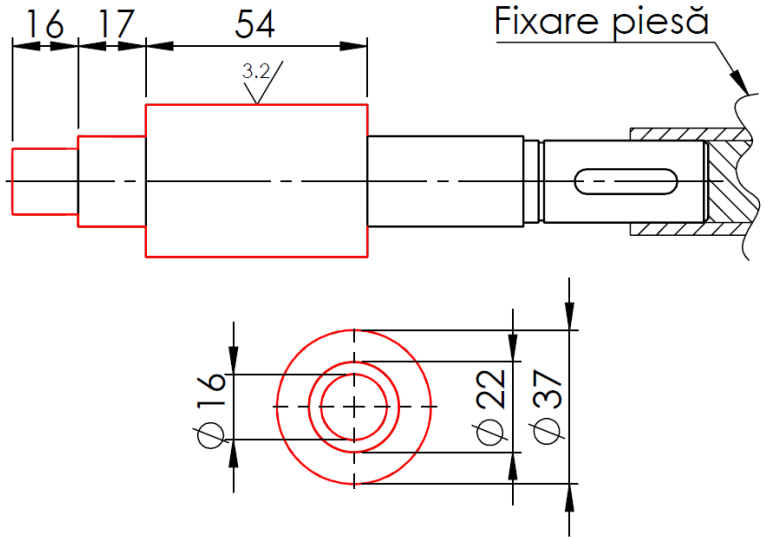
Tabelul 6.7 – Prelucrări tehnice acceptabile

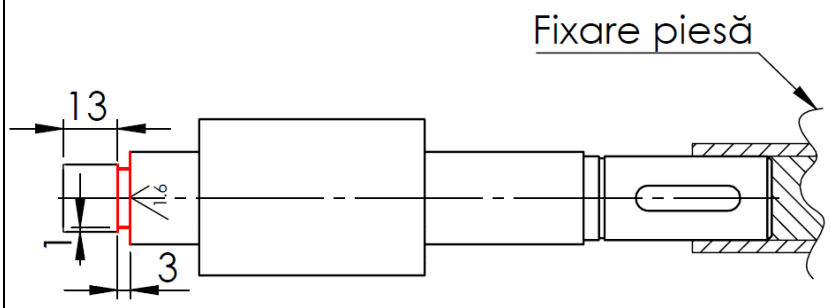
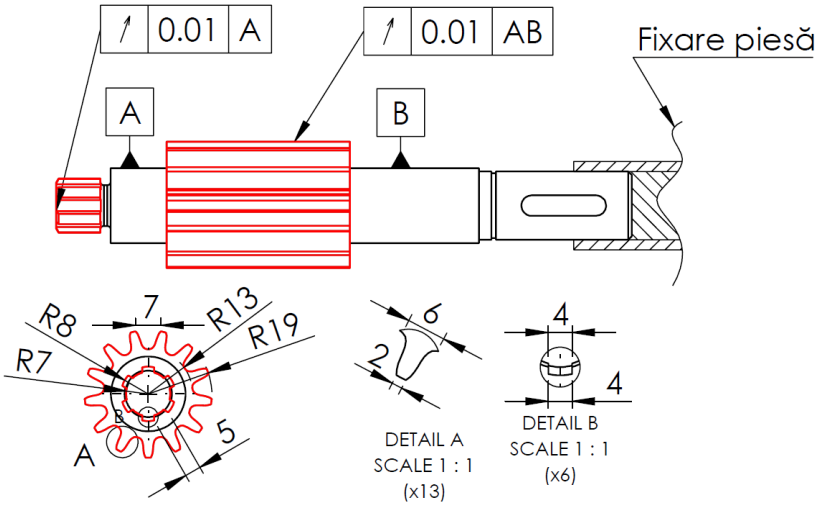
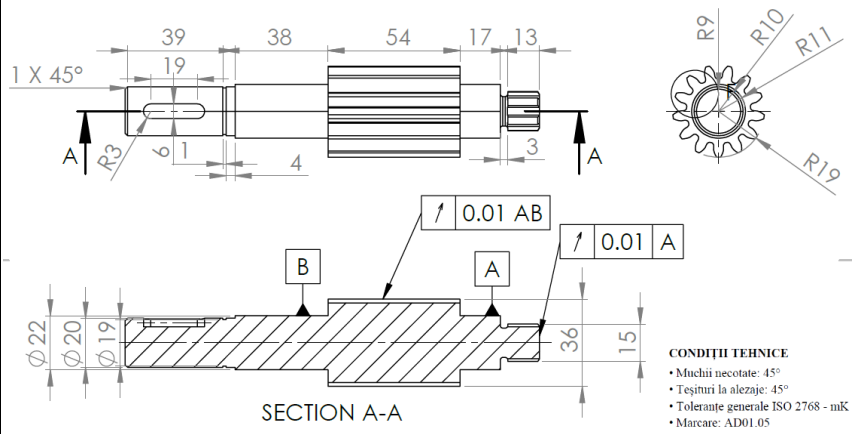
Nr. supr. Sk	Forma suprafeței	Var.	Prelucrări/Ra [μm]		
S1	Plană exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S2	Canelată exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S3	Plană exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S4	Toroidală exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de semifinisare/ IT9 Ra 3,2	Strunjire de finisare/ IT7 Ra 1,6
S5	Cilindrică exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S6	Plan-canelată exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S7	Canelată exterioară	I	Prelucrarea danturii prin frezare/ IT6 Ra1,6	Prelucrarea danturii prin rectificare/ IT5 Ra0,8	-
S8	Cilindrică exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S9	Cilindrică exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de semifinisare/ IT9 Ra 3,2	Strunjire de finisare/ IT7 Ra 1,6
S10	Cilindrică exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S11	Plană exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S12	Canal pană exterioară	I	Frezare de degroșare/ IT9 Ra12,5	Frezare de finisare/ IT8 Ra 1,6	-
S13	Plană exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S14	Plan-canelată exterioară	I	Strunjire de degroșare/ IT10 Ra12,5	Strunjire de finisare/ IT7 Ra 3,2	-
S15	Conică exterioară	I	Strunjire de finisare/ IT10 Ra12,5	-	-

6.3.2 Structura preliminară a proceselor și sistemelor de producție

Nr. de ordine şi denumirea operaţiei	Schița preliminară a operației	Utilaj şi SDVuri (tip)
Laminare		U: Instalație de laminare D/S: Formă de laminar V: Șubler
Strunjire		U: Strung CNC D: Dispozitiv special S:Cuțit pentru strunjire frontală, Cuțit pentru strunjire exterior V: Șubler, raportor, etalon de rugozitate

Strunjire		U: Strung CNC D: Dispozitiv special S:Cuțit pentru strunjire frontală, Cuțit pentru strunjire exterior V: Șubler, raportor, etalon de rugozitate
Strunjire		U: Strung CNC D: Dispozitiv special S:Cuțit pentru strunjire frontală, Cuțit pentru strunjire exterior V: Șubler, raportor, etalon de rugozitate

Canal pană		U: Strung CNC D: Dispozitiv special S:Cuțit pentru strunjire frontală, Cuțit pentru strunjire exterior V: Șubler, raportor, etalon de rugozitate
Strunjire		U: Strung CNC D: Dispozitiv special S: Cuțit pentru strunjire frontală, Cuțit pentru strunjire exterior V: Șubler, raportor, etalon de rugozitate

Strunjire		U: Strung CNC D: Dispozitiv special S:Cuțit pentru strunjire frontală, Cuțit pentru strunjire exterior V: Șubler, raportor, etalon de rugozitate
Frezare dantură		U: Mașină de frezat D: Dispozitiv special S: Burghiu elicoidal; Freză cilindro-frontală V: Șubler
Inspecție finală		U: Banc De inspecție D: Dispozitiv de control S: - V: Șubler, etalon rugozitate ,calibru filete

6.3.3 Calcul adaos prelucrare

Se stabilește numărul de prelucrări pentru suprafața S9. Numărul de prelucrări pentru suprafața S9 este 3.

Pentru fiecare prelucrare se stabilește adaosul de prelucrare minim.

În tabelul 6.8 sunt prezentate prelucrările necesare suprafeței S9.

Tabelul 6.8 – prelucrarile necesare pentru S9

Prelucrarile necesare pentru suprafata S9		
1	Strunjire exterioară de degroșare IT10 Ra 12,5	A_{1min} = 1664 µm
2	Strunjire de semifinisare IT 9 Ra 3.2	A_{2min} = 170 µm
3	Strunjire de finisare IT 7 Ra 1,6	A_{3min} = 15 µm

Calculul adaosului de prelucrare se face invers, de la finisare la degroșare:

$$2A_{3min} = 2(Rz_2 + S_2) + 2\sqrt{p_2^2 + \mathcal{E}_3^2} = 2 * 6.3 + 2\sqrt{1.26^2 + 0^2} = 15\ \mu m$$

$$2A_{2min} = 2(Rz_1 + S_1) + 2\sqrt{p_1^2 + \mathcal{E}_1^2} = 2(30 + 30) + 2\sqrt{25^2 + 0^2} = 170\ \mu m$$

$$2A_{1min} = 2(Rz_{SF} + S_{SF}) + 2\sqrt{p_o^2 + \mathcal{E}_1^2} = 2 * 410 + 2\sqrt{422^2 + 0^2} = 1664\ \mu m$$

6.3.4 Calcul dimensiuni intermediare

Metoda de reglare / Prelucrare	Reglare individuala
Strunjire de finisare (i=3, T3 = IT7 (0.025mm), $2A_3$=15 µm)	• Dimensiune prescrisa pentru alezarea de finisare **Ø18H7**$\left(^{+0,01}_{0}\right)$ • $2A_{3nom}=2A_{3min}+T2=0,015+0,062=0,077mm$ ≈ 0,08mm • $2A_{3max}=2A_{3min}+T2+T3=0,08+0,025=0,105mm$ $a_{p3} = \frac{2A_{3max}}{2} = \frac{0.105}{2} = 0.055mm$

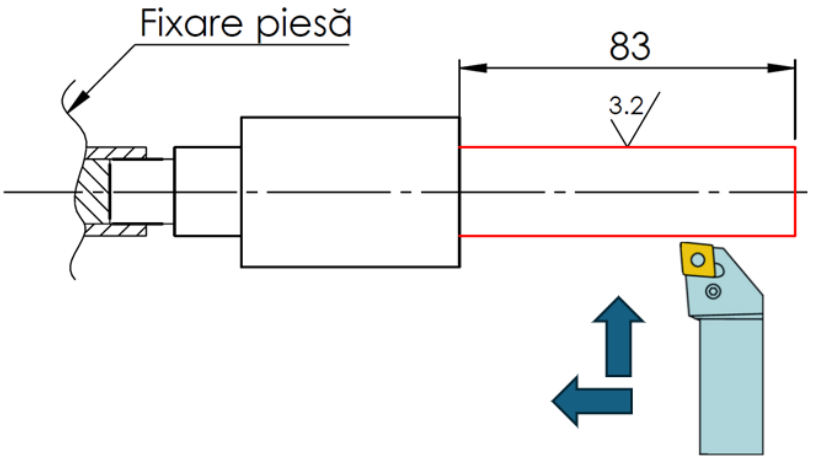
	• dimensiunile intermediare pentru prelucrarea precedenta: $D_{2min}=D_{3min}-2A_{3nom}=18-0,08=17,92$ mm $D_{2min\ r}=17.95$ mm $D_{2max} = D_{2min\ r} + T2=18+0,062=17.938$ mm
Strunjire de semifinisare (i=2, T2 = IT9 (0.062mm), $2A_2$=170 μm)	• Dimensiune prescrisa pentru alezarea de degroșare **Ø17,95 H9**$\binom{+0,04}{0}$ • $2A_{2nom}=2A_{2min}+T1=0,170+0,1=0,27$ mm ≈ 0,3mm • $2A_{2max}=2A_{2min}+T1+T2=0,3+0,062=0,362$mm ≈0,36 mm $a_{p2} = \frac{2A_{2max}}{2} = \frac{0.36}{2} = 0.18\ mm$ • dimensiunile intermediare pentru prelucrarea precedenta: $D_{1min}=D_{2min+}2A_{2nom}=17.95+0.3=18.25$ mm $d_{1max\ r}=18,25$ mm $D_{1max} = D_{1min\ r\ -}\ T1=18.25-0,1= 18,15$ mm
Strunjire exterioară de degroșare (i=1, T1 = IT10 (0.1mm), $2A_1$=1664 μm)	• Dimensiune prescrisa pentru gaurirea cu burghiul **Ø18,15 H12**$\binom{+0,15}{0}$ • $2A_{1nom}=2A_{1min}+\lvert Asup_{SF}\rvert=1.664+0,3=1.964$ mm • $2A_{1max}=2A_{1nom}+\lvert Ainf_{SF}\rvert+T_1=1.964+\lvert-0,4\rvert+0,1=$ 2,464 mm • $D_{SF\ nom} = D_{1\ min} -2A_{1\ nom} = 18.25-1.964=16,286$ mm • $D_{SF\ max} = D_{SF\ nom} + Asup_{SF}=16.286+0,3=16.583$ mm • $D_{SF\ min} = D_{SF\ nom} + Ainf_{SF}=16.286-0,4=15.886$ mm Abateri semifabricat: +0,3/-0,4

6.4 Structura proceselor tehnologice cu sculele în poziție de lucru

Scule și dispozitive de prindere scule:

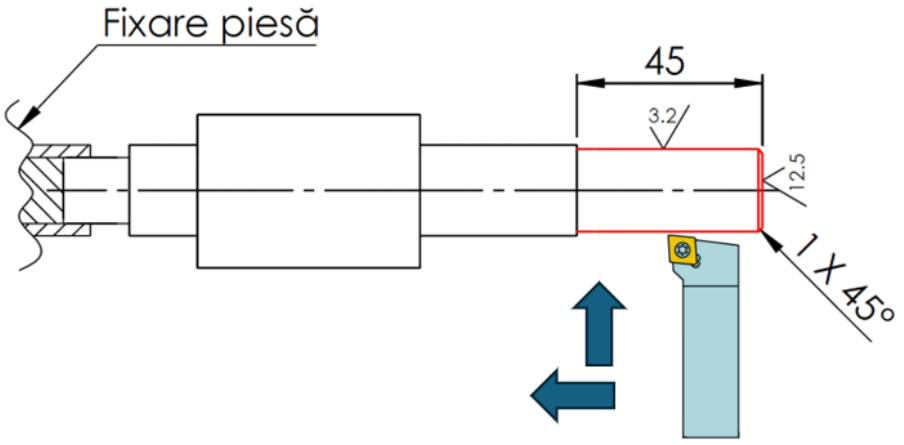
Tabelul 6.9 – Structura proceselor tehnologice

Operația 10 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
10.1	SC1	Unealtă cu tijă pentru strunjire (ext)	PCLNR 2525M 16	10157906



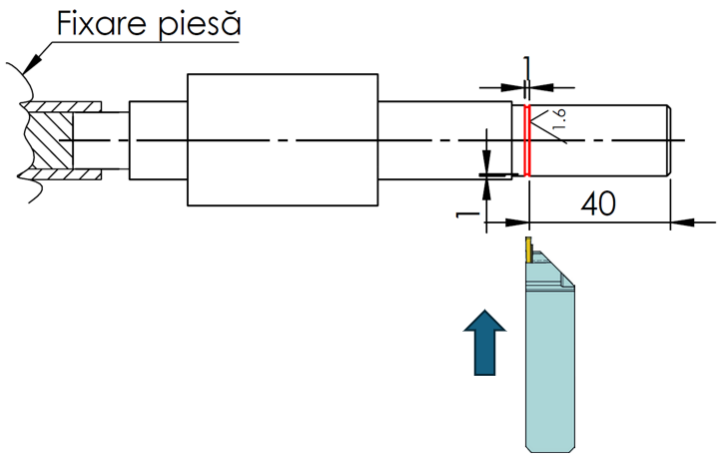
Tabelul 6.10

Operația 20 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
20.1	SC2	Unealtă cu tijă pentru strunjire (ext)	SCLCR 2020K 12	10135602



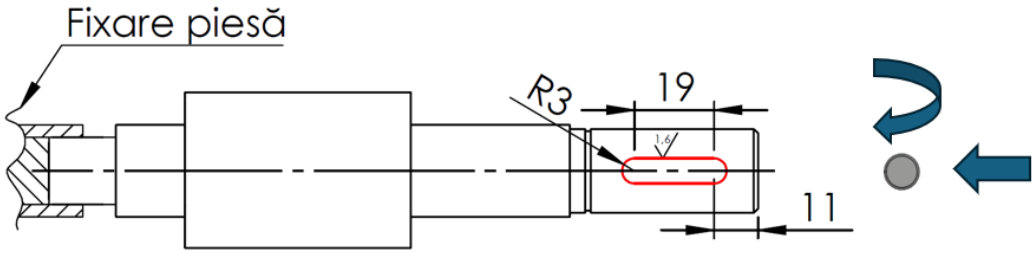
Tabelul 6.11

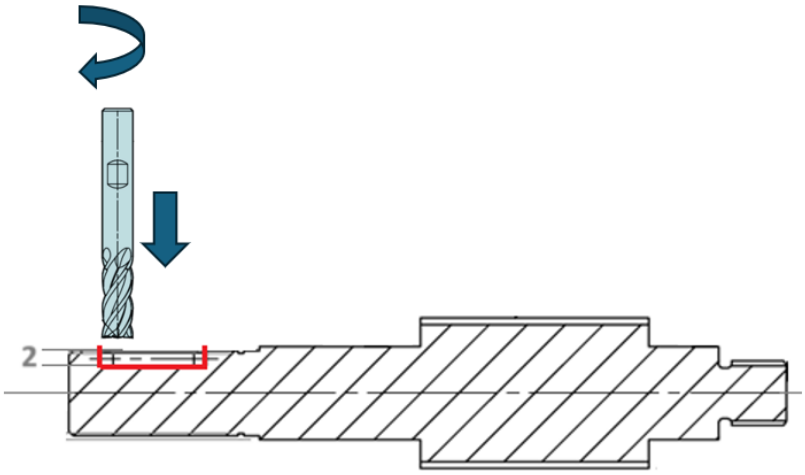
Operația 20 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
20.1	SC3	Unealtă cu tijă pentru despărțire și canelare (ext)	QS-SMALR2020X3HP	7323221058463



Tabelul 6.12

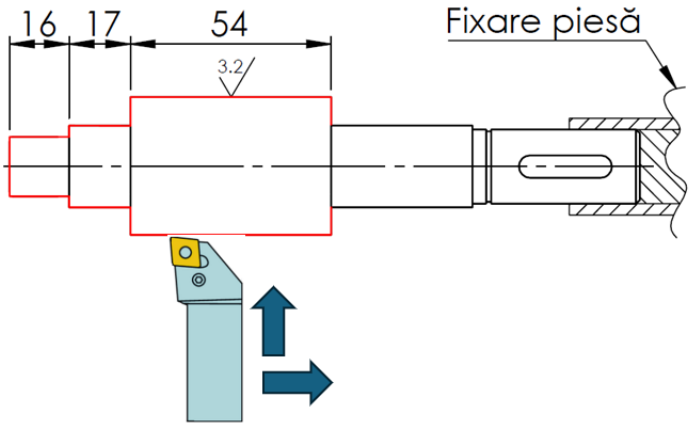
Operația 20 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
20.1	SC4	Freză din carbură solidă pentru frezare grea (ext)	2P342-0600-PB P2BM	7323227193441





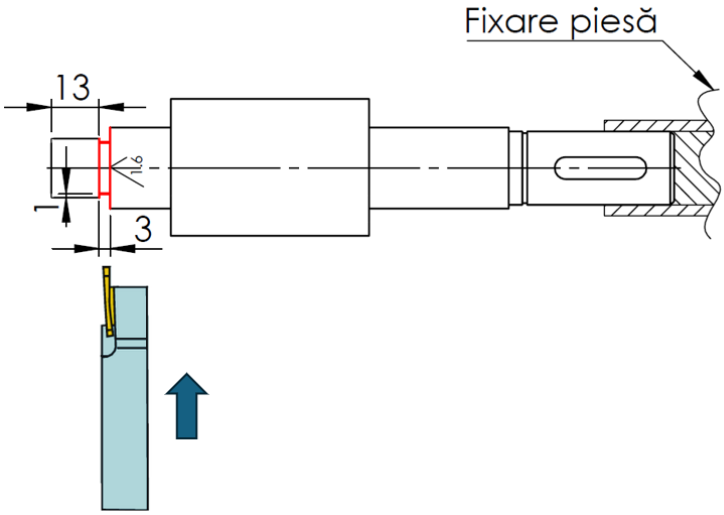
Tabelul 6.13

Operația 20 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
20.1	SC1	Unealtă cu tijă pentru strunjire (ext)	PCLNR 2525M 16	10157906



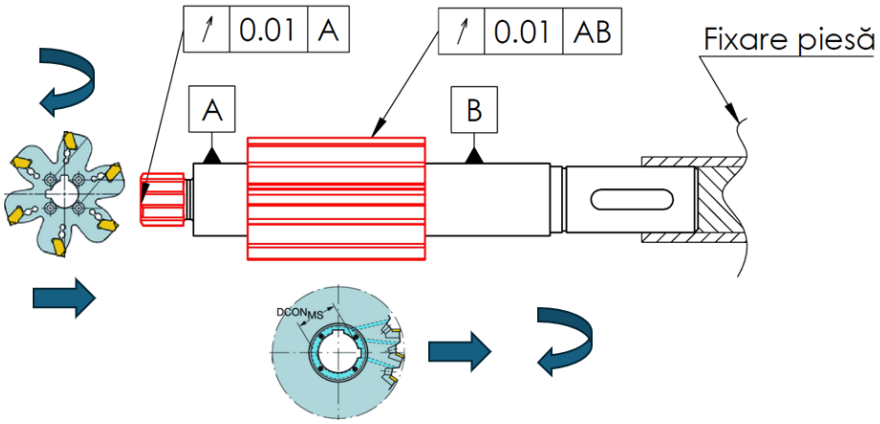
Tabelul 6.14

Operația 20 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
20.1	SC5	Unealtă cu tijă pentru despărțire și canelare(ext)	RF123T06-2525BM	12090956



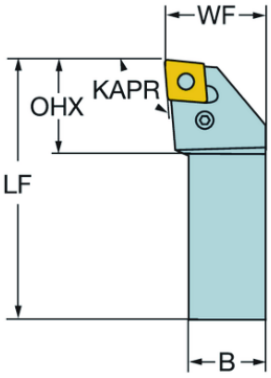
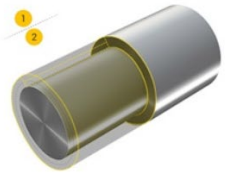
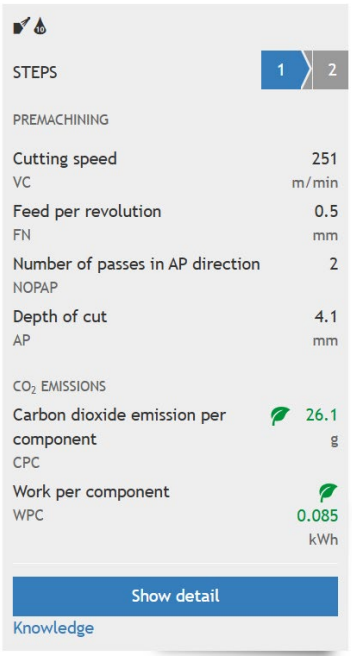
Tabelul 6.15

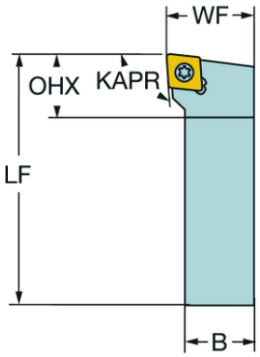
Operația 20 Prelucrare complexă 1				
Faza de prelucrare	**Simbol**	**Tip**	**Cod sculă**	**Cod plăcuță**
20.1	SC6	Freză de canelare și despărțire indexabilă (ext)	QD-H-100X22-M	26657450
	SC7	Freză reglabilă cu disc lateral și frontal (ext)	N331.32C-100S32CM	7323221987039

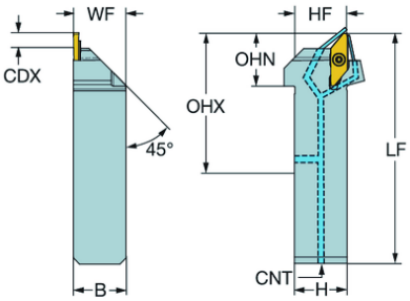


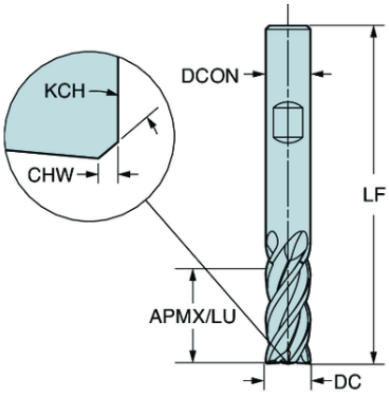
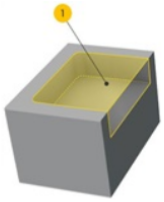
În tabelul de mai jos sunt prezentate specificațiile sculelor folosite

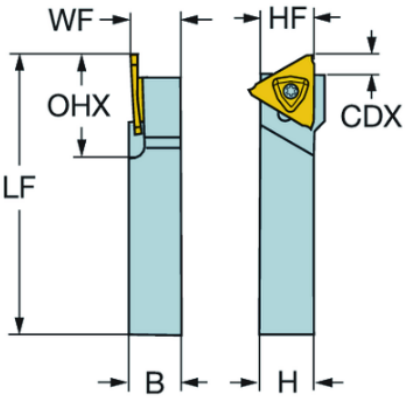
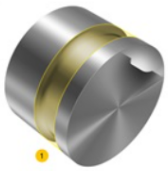
Tabelul 6.16– Specificații scule

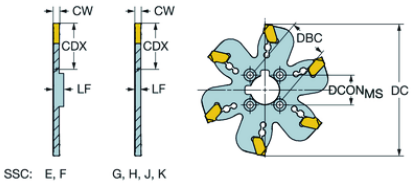
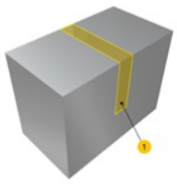
Detalii scule				
Scula	**Reprezentare**	**Cote**	**Dpsc**	**Caracteristici**
SC1	  	LF =150mm OHX =33.9 mm WF = 32 mm KAPR = 95° B =2 5 mm	Unealtă cu tijă pentru strunjire (ext) PCLNR 2525M 16	Greutate:0.7261kg Vite[14]za de taiere: 251 m/min Material Id: 5738417

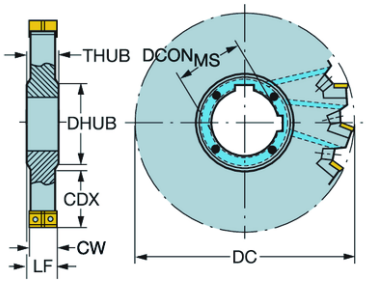
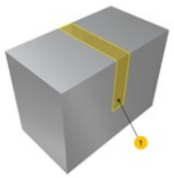
SC2	  STEPS 1 FINISHING Cutting speed VC 239 m/min Feed per revolution FN 0.288 mm Number of passes in AP direction NOPAP 1 Depth of cut AP 1 mm CO_2 EMISSIONS Carbon dioxide emission per component CPC 2.1 g Work per component WPC 0.00741 kWh Show detail Knowledge	WF = 25 mm KAPR = 95° OHX = 21.7mm LF = 125 mm B = 20 mm	Unealtă cu tijă pentru strunjire (ext) SCLCR 2020K 12	Greutate: 0.3856kg Vite[14]za de taiere: 239 m/min Material Id: 5748714
SC2				

SC3	  STEPS 1 GROOVING Cutting speed VC 143 m/min Feed per revolution FN 0.08 mm Rotational speed maximum RPMX 2520 1/min CO_2 EMISSIONS Carbon dioxide emission per component CPC 0.201 g Work per component WPC 0.000711 kWh Show detail Knowledge	WF = 22 mm CDX = 7.2542 mm HF = 20 mm OHN = 25 mm CNT = G 1/8-28 H = 20 mm OHX = 45 mm LF = 94.1 mm B = 20 mm	Unealtă cu tijă pentru despărțire și canelare (ext) QS-SMALR2020X3HP	Greutate: 0.24kg Vite[14]za de taiere: 143 m/min Material Id: 7048267

SC4	  STEPS 1 RAMPING Cutting speed VC 278 m/min Feed per tooth FZ 0.0296 mm CO_2 EMISSIONS Carbon dioxide emission per component CPC 0.56 g Work per component WPC 0.00211 kWh Show detail Knowledge	APMX = 13 mm LF = 57 mm DC = 6 mm CHW = 0.1 mm KCH = 45°	Freză din carbură solidă pentru frezare grea (ext) 2P342-0600-PB P2BM	Greutate: 0.0203kg Vite[14]za de taiere: 278 m/min Material Id: 8351832

SC5	  STEPS 1 GROOVING Cutting speed VC 178 m/min Feed per revolution FN 0.1 mm Rotational speed maximum RPMX 4000 1/min CO_2 EMISSIONS Carbon dioxide emission per component CPC 0.643 g Work per component WPC 0.00242 kWh Show detail Knowledge	WF = 25 mm OHX = 23 mm LF = 150 mm B = 25 mm HF = 25 mm CDX = 6.4 mm H = 25 mm	Unealtă cu tijă pentru despărțire și canelare(ext) RF123T06-2525BM	Greutate: 0.673kg Vite[14]za de taiere: 178 m/min Material Id: 5740938

SC6	  STEPS 1 PREMACHINING Cutting speed VC 419 m/min Feed per tooth FZ 0.32 mm CO_2 EMISSIONS Carbon dioxide emission per component CPC 0.787 g Work per component WPC 0.00296 kWh Show detail Knowledge	DC = 100 mm DBC = 32 mm DCON = 22 mm CW = 4 mm CDX = 30 mm LF = 3.65 mm	Freză de canelare și despărțire indexabilă (ext) QD-H-100X22-M	Greutate: 0.113kg Vite[14]za de taiere: 419 m/min Material Id: 6657450

SC7 SC7	  STEPS 1 PREMACHINING Cutting speed VC 364 m/min Feed per tooth FZ 0.198 mm CO_2 EMISSIONS Carbon dioxide emission per component CPC 6.65 g Work per component WPC 0.025 kWh Show detail Knowledge	THUB = 14 mm DHUB = 47 mm CDX = 25.5 mm CW = 6 mm LF = 10 mm DCON = 32 mm DC = 100 mm	Freză reglabilă cu disc lateral și frontal (ext) N331.32C-100S32CM	Greutate: 0.3574kg Vite[14]za de taiere: 364 m/min Material Id: 7295361

În tabelul de mai jos sunt prezentate specificațiile mașinii de frezat și strunjire folosite:

Tabelul 6.17 – Specificații mașină de frezat și strunjire

Nr. și denumirea operației	**Tip utilaj**	**Imagine utilaj**	**Caracteristici tehnice**
Realizarea operațiilor de strunjire	Strung CNC Haas ST-20		**Amprenta la sol** (lungime, adâncime, înălțime) **fără transportor**: 3302 x 1626 x 1956 mm **Oscilație maximă a părții**: 533 mm **Bar**: 63.5 mm **Vite[14]za maximă**: 4000-rpm **Putere nominală maximă**: 14.9 kW **Sistem de acționare**: Belt-Drive **Lungimea maximă de tăiere** (diferă în funcție de suportul de lucru): 572 mm **Deplasări ale axei (X, Z):** 213 mm, 572 mm **Cursa axei Y:** 100 mm

Realizarea operațiilor de frezare și a canalului pană	Freză CNC Haas VF-2		**Lungime și înălțime**: 914 mm x 356 mm **Cuplu maxim**: 122.0 Nm @ 2000 rpm **Vite[14]za maximă**: 8100-rpm **Putere nominală maximă**: 22.4 kW **Sistem de acționare**: Inline Direct-Drive **Tăiere maximă:** 16.5 m/min **Deplasări ale axei (X, Z):** 762 mm, 508 mm **Cursa axei Y:** 406 mm **Greutate maximă:** 1361 kg **Capacitate:** 20

6.5 Realizarea piesei prin fabricare aditivă folosind programul Z-suite

Pentru realizarea piesei prin fabricare aditiva s-a folosit programul Z-Suite, materialul folosit fiind PLA

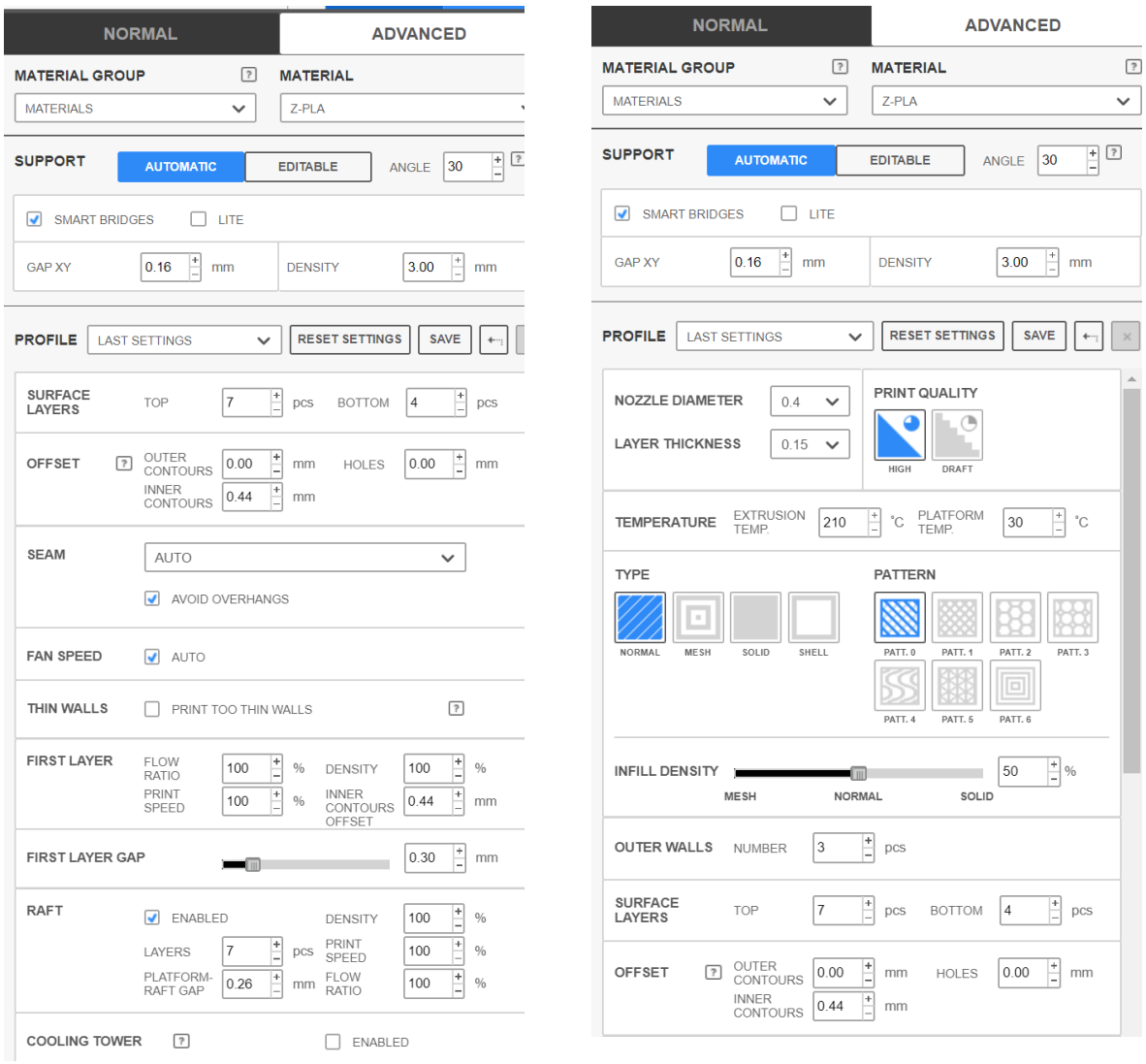


Fig.6.10 Parametrii de printare 3D

Piesă realizată prin fabricare aditivă

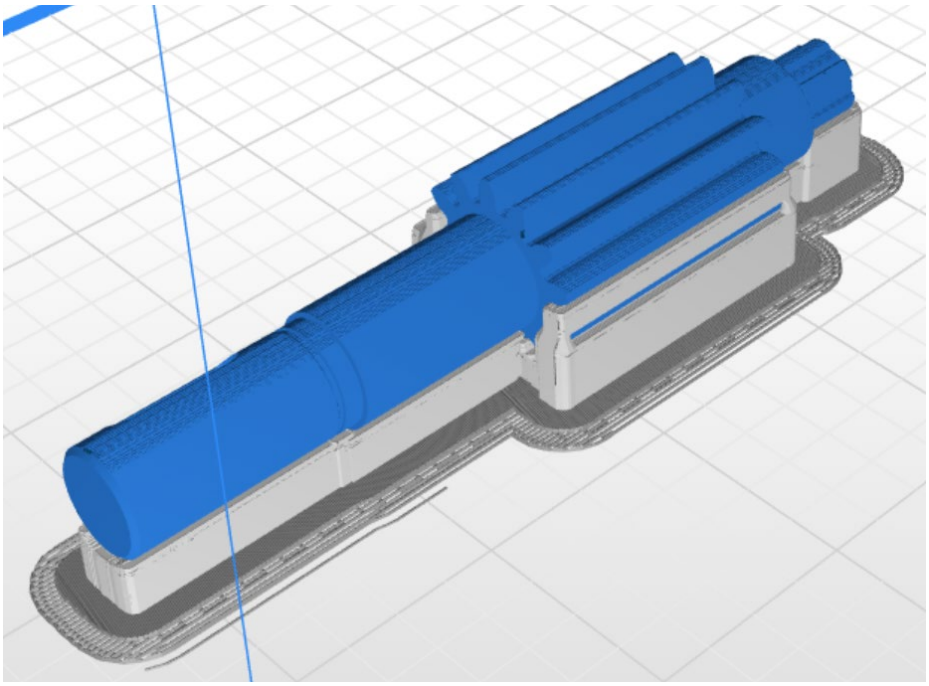


Fig.6.11 Piesa printată 3D

Raportul print-ului este prezentat în următoarea figură, durata fiind 8 ore și 12 de minute, iar consumul de material fiind 76 grame (26.70m de filament).

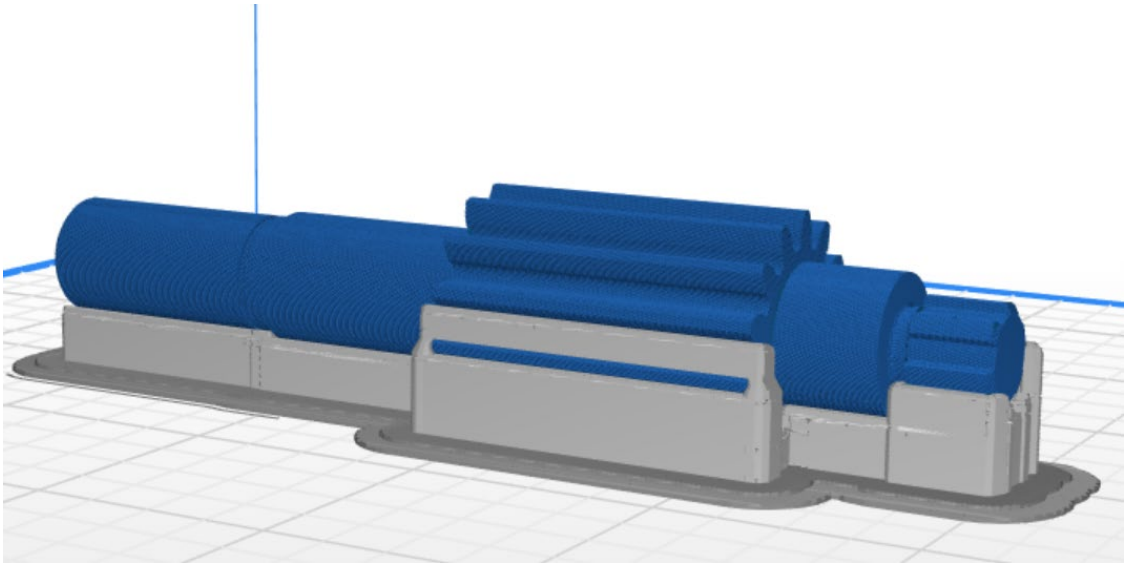


Fig.6.12 Raportul print-ului 3D (partea 1)

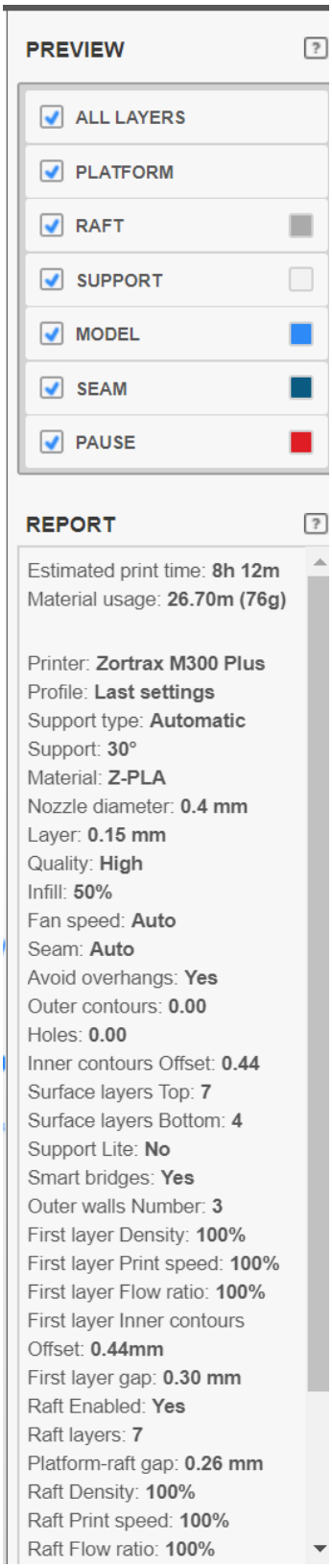


Fig.6.13 Raportul print-ului 3D (partea 2)

7. CONCLUZII ȘI DEZVOLTĂRI ULTERIOARE

7.1 Concluzii

Coroborând informațiile prezentate pe parcursul lucrării, aplicația RAWVISTA reprezintă o soluție software destinată gestionării materiilor prime din cadrul unui depozit, prin evidențierea produselor, administrarea utilizatorilor și înregistrarea tranzacțiilor de tip intrare și ieșire din stoc, toate acestea fiind realizate într-un mediu digital. Platforma permite monitorizarea în timp real a cantităților disponibile, urmărirea istoricului mișcărilor de stoc și menținerea unui control clar asupra responsabilităților fiecărui utilizator.

Aplicația a fost realizată utilizând tehnologii moderne și compatibile între ele, precum Node.js[4] și Express[5] pentru server-ul de backend, PostgreSQL[6] pentru baza de date relațională și React[12],[13] pentru partea de frontend. Gestionarea stării globale a fost realizată prin Redux Toolkit[20], iar comunicarea dintre client și server prin biblioteca Axios[15], cu suport pentru sesiuni gestionate prin express-session[17]. De asemenea, funcționalitatea de export al datelor în format Excel a fost implementată prin integrarea bibliotecii SheetJS[16] (XLSX), oferind utilizatorilor posibilitatea de a prelucra datele și în afara platformei.

Proiectul a avut ca obiectiv realizarea unei aplicații „User-Friendly” pentru utilizatori, care să diferențieze clar nivelurile de acces ale acestora prin roluri distincte. Operatorul poate gestiona produsele și tranzacțiile proprii, în timp ce administratorul dispune de control absolut în raport cu utilizatorii și cu întregului istoric de operațiuni. Acest mecanism contribuie la prevenirea erorilor critice și la menținerea integrității datelor din sistem.

În concluzie, obiectivele propuse la începutul lucrării au fost îndeplinite, rezultând o aplicație funcțională, coerent structurată și pregătită pentru utilizare într-un mediu real. Platforma demonstrează modul în care tehnologiile web moderne pot fi utilizate pentru eficientizarea și automatizarea proceselor administrative din domeniul gestionării resurselor.

7.2 Dezvoltări ulterioare

Aplicația poate fi dezvoltată în continuare atât din punct de vedere funcțional, cât și tehnic. O direcție importantă de extindere ar fi implementarea unui sistem de notificări automate atunci când stocul unui produs scade sub nivelul minim setat, oferind astfel o reacție rapidă din partea personalului responsabil. De asemenea, se poate integra o funcționalitate care să permită generarea de grafice, care permit vizualizarea evoluției stocurilor pe perioade de timp, tipuri de produse sau utilizatori.

Pe partea de extindere și publicare, o direcție importantă ar fi realizarea unei aplicații mobile dedicate platformei. În același timp, o astfel de versiune ar crește semnificativ accesibilitatea sistemului, permițând utilizatorilor să gestioneze informațiile direct de pe dispozitivele mobile. Totodată, o altă dezvoltare utilă ar fi integrarea unei modalități de feedback direct din aplicație, prin intermediul căreia utilizatorii ar putea semnala problemele întâmpinate pe tot parcursul folosirii platformei.

Toate aceste lucruri vor avea un rol esențial în evoluția viitoarei aplicații. RAWVISTA este construită pe o arhitectură solidă, ceea ce permite adăugarea de funcționalități noi fără modificări majore asupra structurii existente. Astfel, proiectul poate constitui baza unei soluții software scalabile, adaptabile și pregătite pentru integrare într-un sistem mai complex de administrare a resurselor.

8. BIBLIOGRAFIE

[1] Site oficial InvenTree: https://inventree.org, accesat 06.09.2025

[2] Site oficial Odoo: https://www.odoo.com, accesat 06.09.2025

[3] Site oficial documentație JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript, accesat 07.09.2025

[4] Site oficial documentație Node.js: https://nodejs.org/en/learn/getting-started/introduction-to-nodejs , accesat 07.09.2025

[5] Site oficial documentație Express: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction, accesat 08.09.2025

[6] Site oficial documentație PostgreSQL: https://www.postgresql.org/docs/current/index.html, accesat 08.09.2025

[7] Site oficial documentație pg: https://node-postgres.com, accesat 08.09.2025

[8] Site oficial documentație HTML: https://developer.mozilla.org/en-US/docs/Web/HTML, accesat 08.09.2025

[9] Introducere HTML: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content, accesat 08.09.2025

[10] Site oficial documentație CSS: developer.mozilla.org/en-US/docs/Learn/css/First_steps/What_is_css, accesat 08.09.2025

[11] Site oficial documentație Bootstrap: https://getbootstrap.com/docs/5.3/getting-started/introduction, accesat 08.09.2025

[12] Site oficial documentație React: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction , accesat 08.09.2025

[13] Introducere React: https://react.dev/learn/describing-the-ui#, accesat 08.09.2025

[14] Site oficial Vite: https://vite.dev/guide/why, accesat 08.09.2025

[15] Site introducere în Axios: https://www.geeksforgeeks.org/html/what-is-axios, accesat în 08.09.2025

[16] Site introducere în SheetJS: https://www.berg-software.com/en/get-browser-data-into-excel-with-sheetjs, accesat 08.09.2025

[17]Site oficial documentație express-session: https://www.npmjs.com/package/express-session, accesat 08.09.2025
[18] Site oficial documentație bcrypt: https://www.npmjs.com/package/bcrypt, accesat 08.09.2025

[19] Site oficial documentație CORS: https://www.npmjs.com/package/cors, accesat 08.09.2025

[20] Site oficial documentație Redux Toolkit: https://redux-toolkit.js.org/introduction/getting-started, accesat 08.09.2025

[21] Site oficial documentație React Router: https://reactrouter.com/home, accesat 08.09.2025

[22] Site oficial documentație React Hot Toast: https://react-hot-toast.com/docs, accesat 08.09.2025

Plagiarism Detector v. 2945 - Originality Report 17/02/2026 16:20:19

Analyzed document: INTRODUCERE.pdf Licensed to: Vlad Gheorghita

Comparison Preset: Rewrite Detected language: Ro

Check type: Internet Check

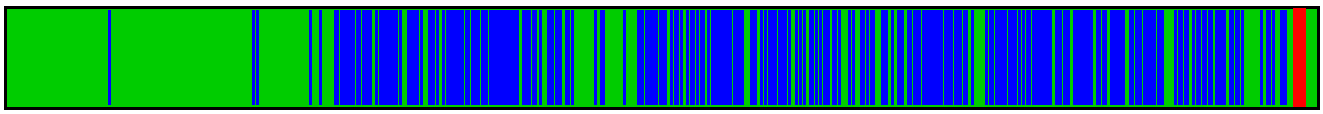
TEE and encoding: PdfPig

Detailed document body analysis:

Relation chart:



Distribution graph:



Top sources of plagiarism: 5

2%	204	1.	https://getbootstrap.com/docs/5.3/getting-started/introduction/
2%	161	2.	https://react.dev/learn/writing-markup-with-jsx
1%	104	3.	https://styleguide.measured.com/docs/5.3/getting-started/introduction/

Processed resources details: 115 - Ok / 6 - Failed

Important notes:

Wikipedia:	Google Books:	Ghostwriting services:	Anti-cheating:
[not detected]	[not detected]	[not detected]	[not detected]

UACE: UniCode Anti-Cheat Engine report:

1. Status: Analyzer **On** Normalizer **On** character similarity set to **100%**
2. Detected UniCode contamination percent:**0%** with limit of: 4%
3. Document not normalized: percent not reached 5%
4. All suspicious symbols will be marked in purple color:Abcd...
5. Invisible symbols found: 0

Assessment recommendation:

No special action is required. Document is Ok.

Alphabet stats and symbol analyzes:

Active References (Urls Extracted from the Document):

No URLs detected

Excluded Urls:

No URLs detected

Included Urls:

No URLs detected

Detailed document analysis:

Proiect de Diplomă 1. INTRODUCERE – PREZENTARE GENERALĂ 1.1 Introducere Managementul materiei prime are o importanță deosebită în operațiunile multor companii, fie că se referă la eficientizarea lanțurilor de producție sau la gestionarea întregului proces de aprovizionare. Totuși, administrarea manuală a acestor activități nu constituie, de cele mai multe ori, soluția optimă în ceea ce privește monitorizarea erorilor, stocurilor și a pierderilor de timp. În prezent, opțiunea digitalizării proceselor de lucru dintr-un depozit a devenit o necesitate, fiind esențială îndeplinirii standardelor actuale de organizare și protecție a datelor. Adaptarea instrumentelor la spațiul electronic oferă o evidență precisă, limitează dezvoltarea erorilor și permit un control sporit asupra activităților efectuate. Principiul de a extinde utilizarea acestor mijloace stă la baza realizării unui proiect menit să fluidizeze, atât procurarea de bunuri necesare, cât și coordonarea resurselor umane. 1.2 Prezentarea proiectului Lucrarea urmărește realizarea unei aplicații informatice care să ofere sprijin în gestionarea stocurilor și vizualizarea clară a produselor, tranzacțiilor și utilizatorilor, păstrând totodată o interfață prietenoasă și eficientă. Platforma integrează un sistem de autentificare bazat pe mai multe roluri, având două categorii de utilizatori: operatori și administratori. Operatorul, având un rol limitat, are posibilitatea de a examina produsele înregistrate în depozit, urmărind detaliile acestora, dar și calitatea de introducere a noilor tranzacții. În cazul editării mișcărilor de stoc, operatorul are dreptul doar de a modifica descrierea, fără a fi posibilă ștergerea sau a schimbarea lor semnificativă. Administratorul, pe de altă parte, are un control complet asupra aplicației, fiindu-i atribuită producerea sau prefacerea oricărui element, procedeu sau client al platformei. Administrarea utilizatorilor este vizibilă exclusiv celor care au acest rol, subliniind distincția clară dintre cele două niveluri de acces. 1.3 Beneficiile informatizării procesului de gestionare a materiilor prime Metoda clasică de management a materiilor prime constituie un efort semnificativ în ceea ce privește urmărirea inventarului și actualizarea tranzacțiilor, existând posibilitatea expunerii lor la erori și lipsă de ordine. Datorită integrării fluxurilor de lucru într-un mediu digital, ne putem bucura de o multitudine de beneficii însemnate, care ne permit să monitorizăm eficient și organizat fiecare proces în parte. Indiferent dacă ne referim la actualizarea constantă a tuturor produselor, notelor de intrare și respectiv ieșire sau la personalul înregistrat în sistem, performanța se resimte atât în materie de timp cât și în materie de securitate și integritate a datelor. 1 Vasile Giulia-Georgiana Prin urmare, dirijarea complexă a procedurilor fundamentale oricărui depozit de materie primă, răspunde unei nevoi concrete din câmpul economic, dar și a cerinței dezvoltărilor viitoare. 1.4 Stadiul actual În prezent există o varietate de aplicații web care permit îndeplinirea numeroaselor nevoi specifice, ce includ urmărirea loturilor, vizualizarea activităților, gestionarea mai multor depozite și generarea de rapoarte detaliate. Câteva exemple relevante sunt prezentate în cele ce urmează. 1.4.1 InvenTree InvenTree[1] este o platformă de planificare și control a stocurilor cu o multitudine de caracteristici diferite. Concentrată atât pe furnizori și producători, cât și pe vânzările către clienți, reușind astfel să urmărească tot ce ține de lanțul de aprovizionare. Ceea ce este unic în această interfață este reprezentat de integrarea API-urilor care facilitează legătura cu alte aplicații externe. Pagina de start include și o variantă demo concepută pentru evidențierea modului în care sistemul de roluri și permisiuni funcționează. Nivelurile diferite de acces, conferă o experiență unică datorită perspectivei variate a fiecărui punct de autorizare în parte, facilitând înțelegerea avantajelor unui sistem flexibil în administrarea unui depozit digitalizat. Fig.1.1 Pagina de start InvenTree[1] 2 Proiect de Diplomă Fig.1.2 Pagina demo InvenTree[1] Fig.1.3 Vizualizarea stocului în InvenTree[1] 3 Vasile Giulia-Georgiana Fig.1.4 Vizualizarea producției în InvenTree[1] Fig.1.5 Vizualizarea ordinelor de asamblare în InvenTree[1] 4 Proiect de Diplomă Fig.1.6 Vizualizarea componentelor necesare asamblării în InvenTree[1] Fig.1.7 Vizualizarea clienților în InvenTree[1] 5 Vasile Giulia-Georgiana 1.4.2 Odoo Odoo[2] este o tehnologie ce permite utilizatorilor să își monitorizeze activitatea fără a folosi programe separate pentru contabilitate, vânzări, gestionare de produse sau resurse umane, toate acestea aflându-se într-un singur loc și folosind aceeași bază de date. Cu un design intuitiv, utilizatorul își poate alege doar programele de care are nevoie, iar în funcție de ele poate asigura un mediu de lucru simplu și eficient. Fig.1.8 Pagina de start Odoo[2] 6 Proiect de Diplomă Fig.1.9 Prezentarea generală a inventarului în Odoo[2] Fig.1.10 Afișarea produselor în Odoo[2] 7 Vasile Giulia-Georgiana Fig.1.11 Afișarea datelor personalului în Odoo[2] Fig.1.12 Dashboards în Odoo[2] 2. PRINCIPII ȘI METODE UTILIZATE PENTRU REZOLVAREA TEMEI DE PROIECT 2.1 Tehnologii utilizate 2.1.1 JavaScript – Limbaj de programare JavaScript[3] este un limbaj de programare, interpretat, utilizat special pentru dezvoltarea aplicațiilor web. Fiind un limbaj ușor de învățat și flexibil, are o dinamică care permite implementarea unor funcții interactive în

paginile noastre web. Programarea bazată pe evenimente permite manipularea DOM-urilor (Modelul de obiecte al documentului), oferind dezvoltatorilor capacitatea de a modifica în timp real structura și stilul unei pagini. Facilitatea în realizarea aplicațiilor complexe este evidențiată de numeroase biblioteci și framework-uri care permit folosirea JavaScript atât pe partea de backend, cât și pe cea a clientului. În plus, JavaScript este unul dintre cele mai folosite limbaje de programare, fiind esențial progresului digital. Totodată dispune de actualizări în mod regulat, ajutând astfel la îmbunătățirea funcționalității și performanței. 2.1.2 Node.js – Mediu de execuție pentru JavaScript pe partea de server Node.js[4] este un mediu de execuție pentru JavaScript care rulează pe motorul JavaScript V8, în afara browser-ului, ceea ce îi permite să fie foarte performant în dezvoltarea aplicațiilor server-side. Cu un model asincron orientat pe evenimente, reușește să administreze mii de conexiuni simultane cu un singur server, optimizând astfel utilizarea resurselor. De asemenea, Node.js include unul dintre cele mai mari registre de pachete și module reutilizabile care permit extinderea accelerată a oricărui proces în dezvoltare. În prezent, Node.js este una dintre cele mai folosite tehnologii pentru realizarea oricărei platforme moderne, justificarea rămânând flexibilitatea și comunitatea care prioritizează performanța. 8 Proiect de Diplomă 2.1.3 Express – Framework backend Express[5] este cel mai utilizat framework bazat pe Node.js și servește drept bibliotecă principală pentru diverse alte framework-uri populare de Node.js. Conceput pentru o gestionare flexibilă a rutelor și cererilor HTTP unde arhitectura sa modulară simplifică procesul de creare a serverelor. De asemenea, introducerea de

Quotes detected: **0.01%** id: **1**

„middleware”

tratează cererile în orice etapă a procesului de manevrare a acestora, ele fiind extinse rapid cu funcționalități precum autentificare, logare, validarea datelor sau tratarea erorilor. Succesul de care dă dovadă Express este reprezentat de reducerea timpului necesar implementării soluțiilor complexe, atât pentru dezvoltatorii începători, cât și pe cei avansați. Așadar, Express rămâne una dintre cele mai utilizate soluții backend, construind baza multor aplicații web moderne. 2.1.4 PostgreSQL – Sistem de gestiune a bazelor de date PostgreSQL este un sistem important de baze de date relaționale, open source, care utilizează și extinde limbajul SQL oferind astfel siguranță în rezolvarea celor mai complicate sarcini de lucru. Reputația solidă este câștigată prin arhitectura sa fiabilă și integritatea datelor care oferă în mod constant soluții performante și inovatoare. Unul dintre principalele avantaje ale PostgreSQL constă în compatibilitatea cu tranzacțiile ACID (Atomicitate, Consistență, Izolare, Durabilitate) definite de utilizator. Adăugarea de funcții personalizate, integritatea datelor și mecanismele eficiente de optimizare a interogărilor, justifică preferința dezvoltatorilor și a marilor companii de a alege această tehnologie. Datorită arhitecturii robuste, a comunității open-source și a extensibilității de care dă dovadă, PostgreSQL constituie alegerea frecventă în diverse proiecte indiferent de cât de mare sau mic este setul de date. 2.1.5 pg – Driver PostgreSQL pentru Node.js pg[6] este un modul oficial Node.js care permite conectarea cu baza de date din PostgreSQL. Acesta include mecanisme de administrare a erorilor datorită suportului pentru interogări parametrizate, devenind astfel standardul principal de conectare dintre PostgreSQL și Node.js. 2.1.6 HTML – Hyper Text Markup Language HTML (Hyper Text Markup Language)[7], [8] este un limbaj marker standard pentru crearea paginilor Web. Prin intermediul acestuia, se pot crea și organiza elemente de tip text, titlu, paragrafe, imagini, link, listă, tabel cu ajutorul etichetelor de tip tags, proces care se numește marcare. Etichetele au nume sugestiv și sunt încadrate între simbolurile ” ” ” ”. Acestea nu sunt case-sensitive, deci pot fi scrise cu litere mici sau majuscule, fără a fi afectată funcționalitatea. Etichetele sunt poziționate la începutul și sfârșitul unei secțiuni, cum ar fi title Title /title . 9 Vasile Giulia-Georgiana Secțiunea head a unui document HTML nu apare pe site-ul propriu-zis atunci când pagina este încărcată, conținând informații precum: titlul (title), link-uri către CSS, sau alte elemente utilizate pentru stilizarea paginii web și date despre documentul HTML. Secțiunea body a unui document HTML include elemente de tip heading (heading), paragraf (p), liste (li), formulare (form), care formează conținutul principal al paginii web. Acestea pot fi așezate în pagină cu ajutorul containerelor de tiv div , elemente block non- semantice care se ocupa cu organizarea informațiilor din pagina în secțiuni separate. 2.1.7 CSS – Cascading Style Sheets CSS (Cascading Style Sheets) [9] oferă posibilitatea de înfrumusețare a paginilor web, oferind control asupra modului în care acestea sunt prezentate către utilizator. În general, paginile web sunt scrise în limbaje de marcare precum HTML sau XML care apar în browser ca niște documente text, fără o structură, culori sau stiluri de afișare specifice. CSS este un limbaj bazat pe reguli, ceea ce înseamnă că dezvoltatorul poate crea reguli specificând grupuri de stiluri care vor fi aplicate către anumite elemente ale paginii Web. Acestea

sunt definite prin intermediul unui sistem de tipul proprietate-valoare. Se pot aplica mai multe proprietăți în cadrul unui element, care însumate, vor rezulta în stilul de afișare dorit. 2.1.8 Bootstrap – Framework pentru frontend Bootstrap[10] este un toolkit pentru frontend robust și extensibil care permite personalizarea ușoară și rapidă a interfețelor Web folosind SASS (Syntactically Awesome Style Sheets), un sistem de așezare în pagină bazat pe o grilă de plugin-uri JavaScript ce pot fi incluse cu ușurință în proiecte pentru a oferi un aspect plăcut și pentru a genera elemente interactive în cadrul paginii Web, într-un timp scurt. Printre clasele și elementele disponibile, se numără următoarele: butoane, formulare, elemente de tip carusel, dropdown-uri, bară de navigare, card-uri, modale, alerte și elemente ce țin de așezarea în pagină, reprezentate de sistemul grilă, împărțit în linii și coloane. Bootstrap poate fi instalat prin intermediul npm (Node Package Manager), folosind comenzile ”npm install bootstrap@5.3.3”. 2.1.9 React – Bibliotecă frontend React[11],[12] este o bibliotecă JavaScript pentru randarea interfețelor utilizatorilor (UI) care este construită din unități mici, cum ar fi butoane, text și imagini. Indiferent de aplicația la care dezvoltatul lucrează, React îi permite combinarea lor în componente reutilizabile. Documentația extinsă și biblioteca de resurse contribuie la evoluția și popularitatea sa continuă deoarece poate fi integrat cu alte pachete sau framework-uri pentru gestionarea și dezvoltarea de platforme moderne. De asemenea, React și ReactDOM sunt atât de des utilizate împreună pentru că au roluri complementare care creează și optimizează sarcinile proiectelor. Abordarea fiecărui framework JavaScript oferă o modalitate de a scrie interfețe într-un mod mai declarativ, descrierea lor fiind posibilă în DOM care face ca acest lucru să fie posibil. 10 Proiect de Diplomă 2.1.10 Vite – Instrument în dezvoltarea frontend Vite[13] este un instrument de dezvoltare mai rapidă și mai eficientă pentru proiectele web moderne. A fost dezvoltat de Evan You, cunoscut și ca autor al framework-ului Vue.js, și s- a răspândit rapid datorită simplității și performanței sale. Principalul avantaj al Vite este folosirea modulelor ESM (ECMAScript), pe care browser- ul le poate încărca rapid, unde transformă și servește codul sursă așa cum este solicitat. Codul din spatele importurilor dinamice condiționate este procesat numai dacă este utilizat efectiv pe ecranul curent. Datorită comunității în creștere și a documentației accesibile, Vite a devenit una dintre cele mai populare soluții moderne pentru construirea interfețelor web, fiind totodată și foarte flexibil deoarece permite integrarea cu mai multe framework-uri populare precum React. 2.1.11 Axios – Bibliotecă pentru cereri HTTP Axios[14] este o bibliotică JavaScript open-source populară, utilizată pentru a face cereri HTTP din browsere web sau medii Node.js. Este o modalitate de a simplifica trimiterea cererilor asincrone către endpoint-uri REST, gestionarea răspunsurilor și efectuarea diverselor sarcini legate de rețea. Construit pe o bază de XMLHttpRequest din JavaScript și a API-ului fetch, Axios oferă o API mai ușor de utilizat, cu funcții precum transformarea automată a datelor JSON, gestionarea erorilor și suport pentru browserele mai vechi. Datorită sintaxei simplificate, consistenței datelor, compatibilității și a securității îmbunătățite, Axios a devenit una dintre cele mai utilizate biblioteci pentru cereri HTTP în proiectele bazate pe utilizarea framework-urilor precum React sau a programelor realizate în JavaScript. 2.1.12 SheetJS (xlsx) – Bibliotecă pentru manipularea fișierelor Excel SheetJS [15] este biblioteca JavaScript care permite analizarea și scrierea în diverse formate de foi de calcul, inclusiv xlsx din Excel. Devine foarte utilă în proiectele unde datele trebuie să fie exportate într-un format tabelar. De asemenea, în loc să fie nevoie să citești sau să scrii fiecare celulă, SheetJS te ajută sa mapezi foaia de calcul într-un obiect JavaScript. Concentrarea pe transferarea datelor, dar și posibilitatea de a converti datele într-o structură JSON îi permite să fie potrivit pentru o gamă largă de aplicații. 2.2 Structura Aplicația este construită pentru a gestiona eficient materiile prime dintr-un depozit prin definirea produselor, urmărirea tranzacțiilor de intrare sau de ieșire și administrarea utilizatorilor pe roluri. Scopul principal al acesteia este de a reduce producerea de erori operaționale prin definirea clară a obiectivului fiecărui angajat. Proiectul este bazat pe interacțiunea tuturor părților, administratorii fiind cei care vor adăuga, șterge sau edita produse, tranzacții sau personal. Aceștia se vor autentifica în platformă 11 Vasile Giulia-Georgiana cu ajutorul email-ului și parolei, care după verificarea permisiunii rolului, îi vor permite utilizatorului să acceseze pagina principală. Rolul unui administrator este de a defini catalogul de produse și de utilizatori, de a supraveghea stocul și de a corecta situațiile excepționale, dacă este cazul. Operatorul în schimb, este persoana care primește catalogul de produse, iar cu acesta pregătește mișcările de stoc, reprezentând practic tranzacțiile de intrare și de ieșire. Nu îi este permis să adauge produse noi, iar în ceea ce privește recepționarea de tranzacții acesta are câteva limitări, pagina utilizatorilor nefiind activă pentru acest rol. Toate acțiunile aplicației sunt transparente, fiind mereu vizibilă persoana care a produs schimbări bazei de date a proiectului, iar claritatea interfeței impune reguli care previn scăderea stocului sub zero sau introducerea de date invalide. 2.2.1 Interfața cu operatorul Interfața destinată operatorului este gândită ca fiind

utilizată pentru treburi mai uzuale sau care nu necesită o responsabilitate foarte mare. Imediat după autentificare, operatorul ajunge pe pagina de start (Dashboard) unde îi este afișat numele și rolul. Examinarea produselor se face dintr-un tabel care are detalii precum denumirea, SKU-ul, unitatea de măsură, stocul curent, stocul minim, raftul (locația fizică), prețul și autorul ultimei modificări. În bara de căutare poți să găsești produsele după nume, SKU, raft, autor și descriere, iar produsele care au un stoc sub nivelul stocului minim sunt evidențiate vizual pentru a permite utilizatorului să ia măsuri în privința aprovizionării. În meniul de acțiuni există secțiunea

Quotes detected: **0.02%** id: **2**

„Detalii produs”

care permite vizualizarea tuturor componentelor produsului, inclusiv și poza acestuia. Înregistrarea tranzacțiilor reprezintă scopul zilnic al operatorilor. Responsabilitatea lor este de a crea tranzacții de tip

Quotes detected: **0.01%** id: **3**

„IN”

(intrare, recepție) sau

Quotes detected: **0.01%** id: **4**

„OUT”

(ieșire/consum/livrare) pentru un produs, selectându-i tipul, cantitatea și descrierea (care este opțională). Sistemul verifică dacă tipul este IN/OUT, cantitatea să fie un număr întreg mai mare decât zero și dacă stocul este suficient, asta în condițiile tranzacțiilor de ieșire. Pentru a fi păstrată diferența dintre cele două roluri, operatorul are posibilități limitate de editare asupra mișcărilor de stoc deja înregistrate, modificările majore rămân astfel responsabilitatea administratorului. Atât lista de produse, cât și istoricul tranzacțiilor pot fi exportate în format Excel, respectând orice filtru introdus în tabelul fiecăruia de valori, astfel orice înregistrare sau materie primă, să poate fi accesibilă oricui. 2.2.2 Interfața cu administratorul Administratorul lucrează pe aceeași interfață ca operatorul, dar are la dispoziție pagini și acțiuni suplimentare acolo unde este nevoie de responsabilitate și decizie. Autentificare pentru ambele roluri rămâne aceeași, sesiunea fiind păstrată prin cookie-uri de sesiune. Paginile aferente acestui rol sunt pagina de start (Dashboard) cu numele și rolul său, paginile pentru produse și respectiv tranzacții și cea a utilizatorilor, special creată acestui rol. Definirea catalogului de produse se realizează prin pagina corespunzătoare acestei acțiuni unde, administratorul poate crea, edita sau șterge articolele din stoc. Pentru fiecare produs trebuie completată denumirea, SKU-ul unic, unitatea de măsură, stocul și stocul minim, raftul locației fizice, descrierea, prețul și dacă se dorește și link-ul către o imagine care ajută la identificarea mai clară a acestuia. Administratorul poate să creeze, editeze sau să șteargă orice tranzacție inclusiv schimbarea tipului acesteia, subliniind importanța unui mediu controlat asupra stocului. La salvarea sau editarea fiecărei mișcări de stoc, recalcularea materiilor prime realizându-se instant 12 Proiect de Diplomă și corespunzător. Când ștergerea unui stoc sub zero are loc, operația este blocată pentru a preveni dezechilibrările. Gestionarea utilizatorilor este atribuită doar acestui rol, dar este desfășurată cu aceeași logică și simplitate. Administratorul poate să creeze conturi noi unde trebuie introduse date precum nume, prenume, adresa de email, parola și rolul pentru cei care au nevoie de acces în aplicație, dar poate să editeze sau să elimine utilizatori dacă este cazul, fiind singura persoana care poate edita rolul altui client în aplicație. Eficiența aplicației se bazează pe informațiile care sunt vizibile pentru fiecare dintre elementele din aceasta, fiind clar și rapid cine a făcut ultima modificare și ce produse sunt sub stocul minim. Exportul în formatul Excel permite păstrarea filtrelor active pe ecran și facilitează funcționarea optimă a platformei. Fig.2.1 Modalitate de lucru a interfețelor 2.3 Logica aplicației Aplicația se bazează pe cele două nivele de acces, Operator și Administrator, păstrând pentru fiecare o interfață simplă pentru gestiunea de materii prime dintr-un depozit. Acțiunile din cadrul său se realizează cu ajutorul paginilor și butoanelor aferente fiecărei nevoi ale utilizatorului. După autentificarea acestuia, în pagina principală (Dashboard) se observă numele și rolul său, iar sesiunea este păstrată cu ajutorul cookie-urilor, astfel încât reconectarea frecventă să nu fie necesară. 13 Vasile Giulia-Georgiana Principalul scop al aplicației este reprezentat de activitatea dintre produse și tranzacții. În lista de produse sunt oferite detalii precum denumirea, SKU-ul unic, unitatea de măsură, stocul curent, stocul minim, raftul (locația fizică), prețul, descrierea și autorul ultimei modificări. Detalierea oricărui produs este afișată printr-un buton

Quotes detected: **0.02%** id: **5**

„Detalii produs”

unde se poate vizualiza și imaginea acestuia, dacă aceasta a fost introdusă la crearea articolului. Dacă se dorește exportarea unui anumit produs într-un tabel Excel, acest lucru este posibil cu ajutorul meniului de căutare care funcționează multi-câmp pentru numele, SKU-ul, raftul, autorul și descrierea produsului. Tranzacțiile reprezintă nucleul operării zilnice, iar principalul său utilizator este operatorul care selectează produsul, tipul operațiunii (IN/OUT – intrare/ieșire) și cantitatea. Dacă apar nereguli cum ar fi în cazul cantității unde trebuie introdus un număr întreg strict pozitiv sau stoc inexistent în cazul tranzacțiilor de ieșire, se vor afișa mesaje clare precum

Quotes detected: **0.02%** id: **6**

„Stoc insuficient”

sau

Quotes detected: **0.02%** id: **7**

„Cantitate invalidă”.

Pentru o bună funcționare a programului, fiecare tranzacție reține stocul (înainte și după fiecare operațiune), autorul și momentul exact al creării sau editării. Gestionarea utilizatorilor este reprezentată de administrator care poate crea conturi cu nume, prenume, email, parolă și rol, totodată să modifice rolurile sau de să elimine accesul în aplicație. Supravegherea produselor care au intrat sub minimul acceptat îi revin tot administratorului care constituie responsabilitatea platformei de lucru, care trebuie să urmărească și sa cerceteze orice neregularitate care se arată pe tot parcursul desfășurării organizatorice a depozitului. Fig.2.2 Diagrama flux a aplicației 3. PROIECTAREA DETALIATĂ A APLICAȚIEI INFORMATICE 3.1 Crearea unui proiect back-end și configurarea aplicației Pentru realizarea acestei aplicații, a fost necesară instalarea Node.js și PostgreSQL. Serviciul back-end a fost construit cu Express și folosește pachete precum

Quotes detected: **0.01%** id: **8**

„pg”

pentru accesul la baza de date,

Quotes detected: **0.01%** id: **9**

„dotenv”

pentru variabilele ascunse,

Quotes detected: **0.01%** id: **10**

„bcrypt”

pentru hașurarea parolelor,

Quotes detected: **0.02%** id: **11**

„express- session”

pentru gestionarea sesiunii și

Quotes detected: **0.01%** id: **12**

„cors”

pentru verificarea accesului dintre back-end și front- end. Toate liniile de cod necesare pentru funcționarea platformei au fost scrise în Visual Studio Code, fiind un editor gratuit, foarte popular printre programatori. Configurarea scheletului de bază al aplicației este definit de rularea comenzii

Quotes detected: **0.02%** id: **13**

„npm init -y”

care creează un fișier

Quotes detected: **0.02%** id: **14**

„package.json”

cu setări implicite, de adăugarea pachetelor necesare prin comanda

Quotes detected: **0.06%** id: **15**

„npm i express pg dotenv bcrypt express-session cors”

și de folosirea

Quotes detected: **0.06%** id: **16**

„npm install -g 14 Proiect de Diplomă nodemon”

care repornește automat aplicația prin

Quotes detected: **0.02%** id: **17**

„nodemon index.js”

de fiecare dată când detectează schimbări la codul sursă. Fig.3.1 Instalarea pachetelor necesare Conectarea la baza de date se realizează în interiorul folder-ului și fișierului

Quotes detected: **0.02%** id: **18**

„config/connectDB.js”

unde, se citesc din

Quotes detected: **0.02%** id: **19**

„.env”

variabilele ascunse necesare relației dintre baza de date și aplicație. După realizarea conexiunii, în consola VS Code apare mesajul

Quotes detected: **0.03%** id: **20**

„Conectat la BD!”

15 Vasile Giulia-Georgiana Fig.3.2 Codul necesar conectării cu PostgreSQL Back-end-ul pornește în

Quotes detected: **0.02%** id: **21**

„index.js”

prin încărcarea variabilelor ascunse din

Quotes detected: **0.02%** id: **22**

„.env”,

pregătirea API-urilor cu middleware-ul

Quotes detected: **0.02%** id: **23**

„express.json()”,

configurarea CORS pe originea front-end- ului și prin permiterea cookie-urilor de sesiune între domenii în timpul autentificării. Gestiunea sesiunii se face cu ajutorul pachetului

Quotes detected: **0.01%** id: **24**

„express-session”

unde cookie-ul cu numele

Quotes detected: **0.01%** id: **25**

„warehouse_session”

are o durată de viață de 24 de ore și nu este re-salvat inutil. După confirmarea conexiunii, serverul montează rutele necesare, domeniile de funcționalitate fiind separate și ușor de testat. Pentru verificarea endpoint-ului se face o testare minimală unde acesta răspunde cu

Quotes detected: **0.02%** id: **26**

„API funcționează!”.

Fig.3.2 Codul necesar conectării la baza de date în index.js 3.2 Crearea bazei de date PostgreSQL și conectarea la server Baza de date a fost creată în programul PostgreSQL și conține tabelele

Quotes detected: **0.01%** id: **27**

„users”,

Quotes detected: **0.01%** id: **28**

„products”

și

Quotes detected: **0.01%** id: **29**

„transactions”

acoperind astfel, personalul depozitului, catalogul de produse și istoricul mișcărilor de stoc.

Conectarea aplicației la aceasta se realizează printr-un pool care permite reutilizarea datelor, iar verificarea este suportată de funcția

Quotes detected: **0.02%** id: **30**

„dbConnect()”

care confirmă practic conexiunea. (de continuat aici) 16 Proiect de Diplomă Fig.3.3 Afișarea tabelelor în PostgreSQL Tabelul

Quotes detected: **0.01%** id: **31**

„users”

a fost conceput pentru administrarea utilizatorilor aplicației, cuprinzând diferite câmpuri de identificare (nume, prenume), autentificare (email, parolă) și autorizare (rol). În cadrul acestuia, email-ul unic utilizat la crearea sau accesarea contului, parola criptată și rolul, constituie împreună un element logic în ceea ce privește securitatea și diferențierea drepturilor în sistem. Fig.3.4 Structura tabelului

Quotes detected: **0.01%** id: **32**

„users”

Tabelul

Quotes detected: **0.01%** id: **33**

„products”

a fost creat pentru gestionarea produselor și a stocurilor, având atribute precum: denumire, unitate, poziție pe raft (rack), descriere și link imagine. Un element important utilizat pentru prevenirea dublurilor produselor, folosit de asemenea și ca identificator, este câmpul

Quotes detected: **0.01%** id: **34**

„sku”

care are o valoare unică. Pentru controlul stocurilor s-au definit coloanele

Quotes detected: **0.01%** id: **35**

„stock”

și

Quotes detected: **0.01%** id: **36**

„stock_min”,

folosite pentru semnalarea oricărei situații în care cantitatea poate scădea sub pragul minim acceptat. Tabelul conține de asemenea și câmpul

Quotes detected: **0.01%** id: **37**

„user_id”,

prin care se permite vizualizarea utilizatorului care a creat produsul. 17 Vasile Giulia-Georgiana Fig.3.5 Structura tabelului

Quotes detected: **0.01%** id: **38**

„products”

Tabelul

Quotes detected: **0.01%** id: **39**

„transactions”

a fost proiectat pentru evidențierea mișcărilor de stoc din depozit (intrări/ieșiri). Structura acestuia include câmpuri cu rol specific precum

Quotes detected: **0.01%** id: **40**

„delta”,

care indică modificarea efectivă a stocului (

Quotes detected: **0.01%** id: **41**

„+”

pentru

Quotes detected: **0.01%** id: **42**

„IN”,

Quotes detected: **0.01%** id: **43**

„-”

pentru

Quotes detected: **0.01%** id: **44**

„OUT”

), dar și câmpuri care oferă trasabilitate prin salvarea stocul înainte și după operație, precum

Quotes detected: **0.01%** id: **45**

„before_stock”

și

Quotes detected: **0.01%** id: **46**

„after_stock”.

Aceste operații ajută la verificarea ulterioară a coerenței stocului fără a depinde exclusiv de valoarea curentă din tabel. În plus, acesta conține relații între tabelele

Quotes detected: **0.01%** id: **47**

„users”

și

Quotes detected: **0.01%** id: **48**

„products”

prin câmpurile

Quotes detected: **0.01%** id: **49**

„product_id”

și

Quotes detected: **0.01%** id: **50**

„user_id”,

asociind fiecare tranzacție cu utilizatorul și produsul aferent. Fig.3.6 Structura tabelului

Quotes detected: **0.01%** id: **51**

„transactions”

Baza de date astfel definită asigură integritatea datelor, relațiile fiind testate prin înserarea de înregistrări care permit legături între utilizatori, produse și tranzacții. Fig.3.4 IMAGINE RELATII INTRE TABELE (transcris coloane tabele și funcțiile lor) 18 Proiect de Diplomă 3.3 Realizarea modelelor pentru baza de date din PostgreSQL Accesul la baza de date din PostgreSQL se regăsește în folder-ul

Quotes detected: **0.01%** id: **52**

„models/”

care reprezintă centralizarea rutelor de acces și transformarea SQL-ului într-o interfață clară pe care rutele o folosesc. Practic, cu ajutorul modelelor create pentru fiecare tabel, codul devine mai lizibil și ușor de întreținut, protejând astfel regulile codului sursă. Modelele au rolul de a oferi reutilizarea funcțiilor pentru manipularea datelor păstrând o logică de interogare atribuită fiecărei funcții în parte, asigurându-se astfel o separare clară între componentele aplicației și logica accesării datelor. 3.3.1 Fișierul

Quotes detected: **0.02%** id: **53**

„userModel.js”

În

Quotes detected: **0.02%** id: **54**

„userModel.js”

sunt implementate funcțiile

Quotes detected: **0.01%** id: **55**

„createUser”,

Quotes detected: **0.01%** id: **56**

„getUserEmail”,

Quotes detected: **0.01%** id: **57**

„getUserId”,

Quotes detected: **0.01%** id: **58**

„getAllUsers”,

Quotes detected: **0.01%** id: **59**

„updateUser”,

Quotes detected: **0.01%** id: **60**

„deleteUser”

care ajută la transferarea informațiilor din tabele pentru prelucrarea lor ulterioară în cadrul platformei. Funcția

Quotes detected: **0.01%** id: **61**

„createUser”

este utilizată pentru introducerea unui utilizator nou în tabelul

Quotes detected: **0.01%** id: **62**

„users”.

Datele necesare creării unui cont reprezintă principalii parametrii care construiesc o interogare SQL de tip

Quotes detected: **0.01%** id: **63**

„INSERT”,

rezultatul fiind returnat cu datele utilizatorului creat, dar fără ca parola acestuia să fie vizibilă pentru a evita posibilele expuneri de informații cu caracter personal. Utilizarea acestei funcții este îndeplinită atât în procesul de înregistrare a utilizatorilor, cât și în administrarea utilizatorilor de către administrator. Fig.3.7 Implementarea funcției

Quotes detected: **0.01%** id: **64**

„createUser”

Funcția

Quotes detected: **0.01%** id: **65**

„getUserEmail”

are rolul de a căuta în baza de date un utilizator căruia i se atribuie o anumită adresă de email. Aceasta este folosită în special pentru a verifica existența unui utilizator înainte de înregistrarea sa în aplicație, precum și la procesul de autentificare. Evitarea duplicării adresei de email contribuie totodată în procesul de securitate al autentificării în cont. Această funcție returnează primul rezultat găsit sau nedefinit (în cazul în care utilizatorul nu există) după ce se execută o interogare SQL de tip

Quotes detected: **0.01%** id: **66**

„SELECT”.

19 Vasile Giulia-Georgiana Fig.3.8 Implementarea funcției

Quotes detected: **0.01%** id: **67**

„getUserEmail”

Funcția

Quotes detected: **0.01%** id: **68**

„getUserId”

este utilizată pentru a obține informații referitoare la un utilizator pe baza identificatorului său unic numit

Quotes detected: **0.01%** id: **69**

„id”.

Diferența dintre această funcție și cea de căutare după email este că, la returnarea datelor, parola nu este expusă, afișarea fiind una stric necesară utilizării în aplicație. În acest fel, se poate verifica existența unui utilizator în cadrul operației administrative. Fig.3.9 Implementarea funcției

Quotes detected: **0.01%** id: **70**

„getUserId”

Funcția

Quotes detected: **0.01%** id: **71**

„getAllUsers”

este responsabilă de prelucrarea tuturor utilizatorilor din baza de date, vizualizând astfel lista completă a celor existenți. Utilizatorul care are rolul de administrator poate să proceseze toate aceste informații esențiale atribuite fiecărui membru din cadrul aplicației. La finalul interogării SQL, se returnează un set de rezultate care conține informații esențiale despre fiecare participant din cadrul depozitului. Fig.3.9 Implementarea funcției

Quotes detected: **0.01%** id: **72**

„getAllUsers”

Funcția

Quotes detected: **0.01%** id: **73**

„updateUser”

este folosită pentru modificarea datelor unui utilizator deja existent. Aceasta abordare oferă o flexibilitate in gestiunea datelor, evitând astfel o posibilă suprascriere inutilă a datelor existente. 20 Proiect de Diplomă Fig.3.10 Implementarea funcției

Quotes detected: **0.01%** id: **74**

„updateUser”

Conform Fig.3.10

Quotes detected: **0.01%** id: **75**

„updateUser”

este una dintre cele mai dezvoltate funcții din

Quotes detected: **0.02%** id: **76**

„usersModel.js”,

rolul său fiind de a actualiza un utilizator deja existent în tabelul

Quotes detected: **0.01%** id: **77**

„users”

și de a întoarce rândul parțial actualizat. Valoarea

Quotes detected: **0.01%** id: **78**

„COALESCE”

este cea care aplică o modificare parțială pentru fiecare coloană unde, dacă datele primite sunt

Quotes detected: **0.01%** id: **79**

„NULL”,

se păstrează valoarea veche, orice altă valoare trimisă devenind valoarea respectivă, coloana actualizându-se. Funcția

Quotes detected: **0.01%** id: **80**

„deleteUser”

este aplicată doar în cazul în care se dorește ștergerea unui utilizator din baza de date. Aceasta interogare SQL de tip

Quotes detected: **0.01%** id: **81**

„DELETE”

identifică utilizatorul pe baza id-ului unic și execută cerința de ștergere a acestuia, utilizarea funcției fiind exclusiv în zona de administrare a aplicației. Fig.3.11 Implementarea funcției

Quotes detected: **0.01%** id: **82**

„deleteUser”

3.3.2 Fișierul

Quotes detected: **0.02%** id: **83**

„productsModel.js”

În

Quotes detected: **0.02%** id: **84**

„productsModel.js”

sunt implementate funcțiile

Quotes detected: **0.01%** id: **85**

„createProduct”,

Quotes detected: **0.01%** id: **86**

„getIdProduct”,

Quotes detected: **0.01%** id: **87**

„getAllProducts”,

Quotes detected: **0.01%** id: **88**

„updateProduct”,

Quotes detected: **0.01%** id: **89**

„deleteProduct”

care ajută la gestionarea câmpurilor principale ale produsului. Funcția

Quotes detected: **0.01%** id: **90**

„createProduct”

ajută la inserarea unui produs nou în tabelul de produse. În cadrul acesteia se regăsește o interogare SQL de tip

Quotes detected: **0.01%** id: **91**

„INSERT”,

care adaugă valorile produsului creat în tabel. 21 Vasile Giulia-Georgiana Un aspect important al acestei funcții este că permite obținerea imediată a informațiilor despre utilizatorul care a introdus datele noului articol, printr-o subinterogare asupra tabelului

Quotes detected: **0.01%** id: **92**

„users”.

Fig.3.12 Implementarea funcției

Quotes detected: **0.01%** id: **93**

„createProduct”

Funcția

Quotes detected: **0.01%** id: **94**

„getIdProduct”

are rolul de a recunoaște un produs pe baza identificatorului unic (id) unde, prin interogarea SQL de tip

Quotes detected: **0.01%** id: **95**

„SELECT”,

se recunoaște existența sau nu a acestuia înainte de efectuarea operațiilor de modificare sau ștergere din tabel. Totodată, dacă articolul există, funcția va returna un singur rezultat, dacă acesta nu există, returnarea se va face sub formă de

Quotes detected: **0.01%** id: **96**

„undefined”.

Fig.3.13 Implementarea funcției

Quotes detected: **0.01%** id: **97**

„getIdProduct”

Funcția

Quotes detected: **0.01%** id: **98**

„getAllProducts”

este utilizată pentru prelucrarea tuturor produselor din baza de date. Interogarea SQL folosește

Quotes detected: **0.02%** id: **99**

„LEFT JOIN”

care combină datele din tabelele

Quotes detected: **0.01%** id: **100**

„products”

și

Quotes detected: **0.01%** id: **101**

„users”,

pentru a obține informații despre fiecare produs creat de un anumit utilizator. Rezultatele returnate la executarea funcției permit ordonarea descrescătoare după identificator, ceea ce facilitează afișarea celor mai recente produse din cadrul bazei de date. 22 Proiect de Diplomă Fig.3.13 Implementarea funcției

Quotes detected: **0.01%** id: **102**

„getAllProducts”

Funcția

Quotes detected: **0.01%** id: **103**

„updateProduct”

este principala modalitate de modificare a datelor unui produs existent. Actualizarea selectivă a campurilor este posibilă folosind

Quotes detected: **0.01%** id: **104**

„COALESCE”,

ceea ce permite furnizarea valorilor modificate și păstrarea celor neschimbate. În plus, câmpul

Quotes detected: **0.01%** id: **105**

„updated_at”

ajută la actualizarea automată a momentului în care a fost înregistrată o modificare a unui articol, fiind returnată versiunea recentă a produsului și informațiile despre utilizatorul implicat. Fig.3.14 Implementarea funcției

Quotes detected: **0.01%** id: **106**

„updateProduct”

(partea 1) 23 Vasile Giulia-Georgiana Fig.3.13 Implementarea funcției

Quotes detected: **0.01%** id: **107**

„updateProduct”

(partea 2) Funcția

Quotes detected: **0.01%** id: **108**

„deleteProduct”

este responsabilă cu ștergerea unui produs din baza de date, executând o interogare SQL de tip

Quotes detected: **0.01%** id: **109**

„DELETE”.

Cu ajutorul identificatorului produsului, se confirmă eliminarea acestuia din baza de date. Fig.3.14 Implementarea funcției

Quotes detected: **0.01%** id: **110**

„deleteProduct”

3.3.3 Fișierul

Quotes detected: **0.02%** id: **111**

„transactionsModel.js”

În

Quotes detected: **0.02%** id: **112**

„transactionsModel.js”

fiecare mișcare de stoc păstrează o trasabilitate completă prin descrierea tipului și calcularea valorii

Quotes detected: **0.01%** id: **113**

„delta”

care ne ajută să actualizăm sau nu stocul produsului. Produsul este blocat cu ajutorul funcției

Quotes detected: **0.02%** id: **114**

„SELECT ... FOR UPDATE”

ceea ce ne permite verificarea

Quotes detected: **0.02%** id: **115**

„after_stock = 0”,

unde acesta este actualizat și inserat în tranzacție. La orice eșec al funcției, are loc

Quotes detected: **0.01%** id: **116**

„ROLLBACK”

care practic anulează întreaga operație începută cu

Quotes detected: **0.01%** id: **117**

„BEGIN”

astfel încât baza de date să revină la starea de dinainte de începerea operațiunilor, iar dacă aceasta este un succes,

Quotes detected: **0.01%** id: **118**

„END”

confirmă modificările. Funcția

Quotes detected: **0.01%** id: **119**

„createTransaction”

are rolul de a înregistra o nouă tranzacție de tip

Quotes detected: **0.01%** id: **120**

„IN”

sau

Quotes detected: **0.01%** id: **121**

„OUT”.

Pentru evitarea modificărilor concurente, produsul este selectat cu

Quotes detected: **0.02%** id: **122**

„SELECT ... FOR UPDATE”,

ceea ce blochează rândul corespunzător acestuia până la finalizarea tranzacției. În cadrul acestei funcții se calculează o valoare notată cu

Quotes detected: **0.01%** id: **123**

„delta”,

prin care se poate observa variația efectivă a stocului precum și valorile

Quotes detected: **0.01%** id: **124**

„before_stock”

și

Quotes detected: **0.01%** id: **125**

„after_stock”.

După ce este înregistrată actualizarea stocului, înserarea se face automat în tabelul de tranzacții, iar în cazul unei erori apărute, modificările sunt anulate prin comanda

Quotes detected: **0.01%** id: **126**

„ROLLBACK”.

24 Proiect de Diplomă Fig.3.15 Implementarea funcției

Quotes detected: **0.01%** id: **127**

„createTransaction”

(partea 1) Fig.3.16 Implementarea funcției

Quotes detected: **0.01%** id: **128**

„createTransaction”

(partea 2) Funcția

Quotes detected: **0.01%** id: **129**

„getAllTransactions”

este utilizată pentru prelucrarea tuturor tranzacțiilor din baza de date. Cu ajutorul operațiunii

Quotes detected: **0.01%** id: **130**

„JOIN”

din cadrul interogării SQL, se permite obținerea informațiilor despre tranzacționarea produsului și a utilizatorului care a realizat operația. Rezultatele sunt afișate în ordine descrescătoare după data creării, facilitând afișarea celor mai recente tranzacții din cadrul aplicației. 25 Vasile Giulia-Georgiana Fig.3.17 Implementarea funcției

Quotes detected: **0.01%** id: **131**

„getAllTransactions”

Funcția

Quotes detected: **0.01%** id: **132**

„getAllTransactionsID”

permite obținerea și afișarea detaliilor complete ale unei tranzacții pe baza id-ului unic al acesteia. Fig.3.18 Implementarea funcției

Quotes detected: **0.01%** id: **133**

„getAllTransactionsID”

Funcția

Quotes detected: **0.01%** id: **134**

„getAllProductTransactions”

se folosește pentru gestionarea tuturor tranzacțiilor aferente unui anumit produs prin filtrarea sa după

Quotes detected: **0.01%** id: **135**

„product_id”

și vizualizarea istoricului complet al acestuia. Fig.3.18 Implementarea funcției

Quotes detected: **0.01%** id: **136**

„getAllProductTransactions”

26 Proiect de Diplomă Funcția

Quotes detected: **0.01%** id: **137**

„getUserTransactions”

ajută la returnarea tuturor tranzacțiilor efectuate de un anumit utilizator, datele fiind vizibile numai pentru utilizatorul care a realizat propriile înregistrări. Cu toate acestea, accesul la afișarea întregului istoric este posibil doar prin autentificarea într- un cont care are rolul de administrator.

Fig.3.18 Implementarea funcției

Quotes detected: **0.01%** id: **138**

„getAllProductTransactions”

Funcția

Quotes detected: **0.01%** id: **139**

„updateTransactionDescription”

modifică descrierea unei tranzacții existente, fără a permite afectarea stocului unui produs. În plus, funcția este destinată în special utilizatorilor care au rolul de operator și care pot aduce modificări minore în editarea tranzacțiilor. Fig.3.19 Implementarea funcției

Quotes detected: **0.01%** id: **140**

„updateTransactionDescription”

Funcția

Quotes detected: **0.01%** id: **141**

„updateTransactionFULL”

este modalitatea prin care utilizatorul cu rolul de administrator poate modifica total o tranzacție, reprezentând una dintre cele mai complexe funcții din aplicație. Aceasta folosește o tranzacție SQL care îi permite să revină asupra informațiilor tranzacției inițiale și să modifice valoarea stocului unde, după aplicarea noilor valori și revizuirea datelor, recalculează stocul final. Principalele comenzi prin care ajustarea datelor tranzacțiilor este posibilă sunt comenzile

Quotes detected: **0.01%** id: **142**

„BEGIN”

și

Quotes detected: **0.01%** id: **143**

„ROLLBACK”

care asigură prevenirea apariției datelor incorecte sau parțial actualizate din cadrul aplicației. 27 Vasile Giulia-Georgiana Fig.3.20 Implementarea funcției

Quotes detected: **0.01%** id: **144**

„updateTransactionFULL”

(partea 1) Fig.3.21 Implementarea funcției

Quotes detected: **0.01%** id: **145**

„updateTransactionFULL”

(partea 2) 28 Proiect de Diplomă Fig.3.22 Implementarea funcției

Quotes detected: **0.01%** id: **146**

„updateTransactionFULL”

(partea 3) Funcția

Quotes detected: **0.01%** id: **147**

„deleteTransaction”

are ca scop principal ștergerea unei tranzacții existente din baza de date. Totuși, valoarea stocului este ajustată prin inversarea valorii

Quotes detected: **0.01%** id: **148**

„delta”

care permite păstrarea unei consistențe a inventarul chiar dacă valoarea sa a fost înlăturată. Fig.3.23 Implementarea funcției

Quotes detected: **0.01%** id: **149**

„deleteTransaction”

(partea 1) 29 Vasile Giulia-Georgiana Fig.3.23 Implementarea funcției

Quotes detected: **0.01%** id: **150**

„deleteTransaction”

(partea 2) În urma analizei fiecărei funcții din fișierele

Quotes detected: **0.02%** id: **151**

„usersModel.js”,

Quotes detected: **0.02%** id: **152**

„productsModel.js”

și

Quotes detected: **0.02%** id: **153**

„transactionsModels.js”,

se observă că definirea modelelor aplicației construiește structura de funcționare a tabelelor. Acestea sunt scrise în limbajul de programare JavaScript și rulate cu ajutorul mediului de execuție Node.js care, prin intermediul unei biblioteci specializate, realizează operații esențiale accesului la baza de date din PostgreSQL, precum crearea, citirea, actualizarea și ștergerea înregistrărilor din tabele. Gestionarea operațiilor mai complexe de modificare totală sau de ștergere a tranzacțiilor, a fost posibilă cu ajutorul comenzilor

Quotes detected: **0.01%** id: **154**

„BEGIN”,

Quotes detected: **0.01%** id: **155**

„COMMIT”

și

Quotes detected: **0.01%** id: **156**

„ROLLBACK”

care mențin executarea logică a mai multor comenzi SQL. Datorită funcțiilor JavaScript care manipulează aceleași entități, evidențierea relațiilor dintre tabelele din baza de date reiese din fig. 3.24, unde tabelul

Quotes detected: **0.01%** id: **157**

„transactions”

face legătura între

Quotes detected: **0.01%** id: **158**

„users”

și

Quotes detected: **0.01%** id: **159**

„products”.

De asemenea, fiecare tranzacție corespunde unui singur produs și unui singur utilizator prin atributul

Quotes detected: **0.01%** id: **160**

„product_id”

și, respectiv,

Quotes detected: **0.01%** id: **161**

„user_id”,

ceea ce permite produsului sau utilizatorului să poată avea multiple tranzacții asociate, formând astfel relații One-to-Many. Fig.3.24 Diagrama relațională a bazei de date din PostgreSQL 30 Proiect de Diplomă Tabelele din baza de date, prezentate în schema anterioară, sunt detaliate în cele ce urmează: users: tabel realizat cu scopul înregistrării și autentificării în aplicație, dar și pentru gestionarea utilizatorilor din sistem. Coloane: ID = tip: număr întreg, autoincrementabil, cheie primară. first_name = tip: text (varchar), maxim 150 de caractere, nu permite valori nule. last_name = tip: text (varchar), maxim 150 de caractere, nu permite valori nule. email = tip: text (varchar), maxim 254 caractere, nu permite valori nule, valoare unică. password = tip: text, nu permite valori nule. role = tip: text (varchar), maxim 50 de caractere, nu permite valori nule. created_at = tip: timestamp fără fus orar, nu permite valori nule. products: tabel realizat pentru stocarea informațiilor despre produsele din cadrul depozitului. Coloane: ID = tip: număr întreg,

autoincrementabil, cheie primară. name = tip: text, nu permite valori nule. sku = tip: text (varchar), maxim 50 de caractere, nu permite valori nule, valoare unică. unit = tip: text (varchar), maxim 50 de caractere. stock = tip: număr întreg. stock_min = tip: număr întreg. user_id: tip: număr întreg, cheie străină, relație de tip Many-to-One cu tabelul

Quotes detected: **0.01%** id: **162**

„users”.

rack = tip: text (varchar), maxim 50 de caractere. description = tip: text. price = tip: numeric (12,2). image_url = tip: text. created_at = tip: timestamp fără fus orar. updated_at = tip: timestamp fără fus orar. transactions: tabel destinat înregistrării istoricului mișcărilor de stoc, având rolul de a monitoriza toate operațiile de intrare și de ieșire ale produselor. Coloane: ID = tip: număr întreg, autoincrementabil, cheie primară. product_id = tip: număr întreg, cheie străină, relație de tip Many-to-One cu tabelul

Quotes detected: **0.01%** id: **163**

„products”.

user_id = tip: număr întreg, cheie străină, relație de tip Many-to-One cu tabelul

Quotes detected: **0.01%** id: **164**

„users”,

permite valoare nulă. type = tip: text (varchar), maxim 3 caractere, nu permite valori nule. quantity = tip: număr întreg, nu permite valori nule. delta = tip: număr întreg. before_stock = tip: număr întreg. after_stock = tip: număr întreg. description = tip: text. created_at = tip: timestamp fără fus orar, nu permite valori nule. 31 Vasile Giulia-Georgiana 3.4 Arhitectura și funcționarea back-end-ului aplicației 3.4.1 Structura componentelor back-end În structura back-end-ului aplicației, sistemul este organizat în mai multe directoare, fiecare având un rol clar în procesarea cererilor HTTP. Conform fig. 3.25, stratul de acces la baza de date este realizat cu ajutorul fișierelor următoare: Controller, acest document conține funcții care gestionează datele primite de la cererile HTTP, unde aplică validările necesare, accesează baza de date și construiește răspunsurile returnate clientului. Cu alte cuvinte, controller-ele conțin logica principală a aplicației, fiind punctul central unde rutele definite și stratul de acces la baza de date se coordonează. Models, un fișier care definește funcții JavaScript și execută interogări SQL asupra bazei de date din PostgreSQL, fiind astfel modalitatea de acces la baza de date. Routes, componenta responsabilă de definirea punctelor de acces ale aplicației. Astfel, fișierele stocate în aceasta declară rute pentru autentificare, utilizatori, produse sau tranzacții, utilizând metode HTTP precum

Quotes detected: **0.01%** id: **165**

„GET”,

Quotes detected: **0.01%** id: **166**

„POST”,

Quotes detected: **0.01%** id: **167**

„PUT”,

Quotes detected: **0.01%** id: **168**

„PATCH”

și

Quotes detected: **0.01%** id: **169**

„DELETE”.

În schimb, componentele Config și Middleware nu reprezintă o relație directă între componentele aplicației, de aceea poziționarea lor în diagramă are rolul de relaționare opțională. Cu toate acestea, componenta Config stabilește conexiunea la baza de date, asigurându-se ca informațiile sensibile sunt separate de logica de bază a aplicației, iar Middleware-ul este responsabil cu implementarea mecanismelor de acces și securitate, verificându-se astfel autentificarea utilizatorilor, rolurile și permisiunile pentru accesarea anumitor rute din cadrul platformei. Fig.3.25 Arhitectura back-end-ului aplicației Node.js 3.4.2 Logarea, înregistrarea și autentificarea utilizatorilor Mecanismele oferite de protocolul HTTP, mediul Node.js și framework-ul Express permit funcționalităților de înregistrare, autentificare și gestionare a sesiunilor utilizatorului să se

grupeze în cadrul rutelor dedicate autentificării, acestea fiind regăsite în fișierul

Quotes detected: **0.02%** id: **170**

„authRoutes.js”.

Accesarea generală a serviciului de autentificare a fost realizată prin intermediul adresei locale

Quotes detected: **0.01%** id: **171**

„http://localhost:3000/api/auth”,

portul utilizat fiind obținut prin citirea fișierului

Quotes detected: **0.02%** id: **172**

„.env”

care permite modificarea acestuia fără a putea fi afectat codul sursă al aplicației. 32 Proiect de Diplomă Fig.3.26 Preluarea portului din fișierul

Quotes detected: **0.02%** id: **173**

„.env”

și pornirea serverului Întregul proces de verificare a sesiunii, înregistrare, autentificare, delogare a fost realizat prin intermediul cererilor HTTP diferite: logarea și înregistrarea au fost realizate prin folosirea cererilor de tip

Quotes detected: **0.01%** id: **174**

„POST”,

verificarea sesiunii cu ajutorul cererii

Quotes detected: **0.01%** id: **175**

„GET”,

iar delogarea utilizând comanda

Quotes detected: **0.01%** id: **176**

„DELETE”.

Fig.3.27 Definirea rutelor de autentificare din fișierul

Quotes detected: **0.02%** id: **177**

„authRoutes.js”

Conform fig. 3.27, se poate observa că înregistrarea utilizatorilor folosește ruta

Quotes detected: **0.01%** id: **178**

„/register”,

unde această cerere transmite datele utilizatorului către server înainte de a fi salvate în baza de date. Totuși, în cadrul înregistrării, parola este procesată folosind biblioteca

Quotes detected: **0.01%** id: **179**

„bcrypt”,

care permite aplicarea algoritmului de criptare, asigurându-se astfel o stocare securizată a parolei în baza de date. Autentificarea utilizatorului folosește ruta

Quotes detected: **0.01%** id: **180**

„/login”

prin care serverul verifică existența sa și validează accesarea în sesiune. De asemenea, verificarea sesiunii prin accesarea rutei

Quotes detected: **0.01%** id: **181**

„/me”

oferă datele persoanei conectate fără retrimiterea informațiilor necesare identificării contului. Mecanismul care stă la baza menținerii stării de autentificare este oferit de middleware-ul

Quotes detected: **0.01%** id: **182**

„express-session”,

care configurează cookie-ul de sesiune cu o durată de viață de 24 de ore și îl transmite automat între client și server. În momentul autentificării, cookie-ul denumit

Quotes detected: **0.01%** id: **183**

„warehouse_session”

stochează informații esențiale despre utilizatorul conectat la server. Proprietățile de securitate pe care acesta le are sunt

Quotes detected: **0.01%** id: **184**

„httpOnly”

și

Quotes detected: **0.01%** id: **185**

„sameSite”,

care, în primă fază, împiedică codul JavaScript rulat în browser să acceseze valoarea cookie-ului de sesiune atât pentru citire cât și pentru modificare, ca mai apoi să nu fie trimis în cereri care provin din alte site-uri, prevenindu-se utilizarea acestuia în contexte cu potențial malițios. Fig.3.28 Configurarea cookie-ului de sesiune din

Quotes detected: **0.02%** id: **186**

„index.js”

33 Vasile Giulia-Georgiana Pentru transmiterea corectă a sesiunii între client și server, sunt activate opțiunile

Quotes detected: **0.02%** id: **187**

„credentials: true”

și

Quotes detected: **0.02%** id: **188**

„withCredentials: true”,

regăsite în configurația CORS pe server, respectiv în configurația Axios pe client, permițând transferul cookie-ului de sesiune în cererile HTTP. Fig.3.29 Configurarea CORS din Fig.3.30 Configurarea Axios din

Quotes detected: **0.02%** id: **189**

„axios.js”

Quotes detected: **0.02%** id: **190**

„index.js”

Delogarea utilizatorului are ca efect încheierea sesiunii curente și eliminarea cookie-ului asociat contului prin accesarea rutei

Quotes detected: **0.01%** id: **191**

„/logout”.

În urma acestei operații, utilizatorului nu i se mai permite accesarea rutelor protejate din cadrul aplicației. 3.4.3 Implementarea controllerelor și a rutelor HTTP ale aplicației Logica de procesare a cererilor HTTP se regăsește în fișierul

Quotes detected: **0.01%** id: **192**

„controllers”,

care preia cererile din interfața front-end-ului și apelează funcțiile din documentul

Quotes detected: **0.01%** id: **193**

„models”,

unde se înregistrează operațiile asupra bazei de date. Totodată, pe lângă mecanismele de securitate aplicate prin cadrul unui middleware de autentificare, anumite controllere includ măsuri de protecție sporite datorită verificărilor suplimentare ale rolului utilizatorului, prevenind orice urmă de acces neautorizat. 3.4.3.1 Controller-ul de autentificare din

Quotes detected: **0.02%** id: **194**

„authController.js”

Controller-ul de autentificare administrează numeroase funcții prin care este realizată înregistrarea, logarea, delogarea și identificarea utilizatorului conectat în aplicație. Implementarea acestora este îndeplinită prin obținerea datelor din modelul

Quotes detected: **0.02%** id: **195**

„userModel.js”,

iar biblioteca

Quotes detected: **0.01%** id: **196**

„bcrypt”

verifică și criptează parolele asociate conturilor. Funcția

Quotes detected: **0.01%** id: **197**

„register”

primește informațiile asociate profilului utilizatorului prin

Quotes detected: **0.02%** id: **198**

„req.body”,

unde în primă fază se validează existența datelor, iar ulterior se verifică dacă emailul utilizat există deja în sistem. Dacă înregistrarea emailului a fost deja realizată, serverul returnează un status care indică acest lucru printr-un mesaj corespunzător. Pentru securizarea și criptarea parolei se utilizează

Quotes detected: **0.02%** id: **199**

„bcrypt.hash”,

care, printr-un număr de iterații de salt, permite algoritmului să aplice funcția de hash de mai multe ori, ceea ce face imposibilă ghicirea rapidă a parolei. După procesul de criptare, datele sunt salvate în baza de date prin funcția

Quotes detected: **0.01%** id: **200**

„createUser”,

iar înregistrarea utilizatorului este confirmată prin returnarea unui status pozitiv la finalul operației. 34 Proiect de Diplomă Fig.3.30 Implementarea funcției

Quotes detected: **0.01%** id: **201**

„register”

Funcția

Quotes detected: **0.01%** id: **202**

„login”

permite autentificarea utilizatorului prin primirea emailului și parolei din cadrul bazei de date. În primă fază, se face o verificare a emailului, iar dacă acesta este atribuit unui cont se trece la verificarea parolei cu ajutorul

Quotes detected: **0.02%** id: **203**

„bcrypt.compare”.

În cazul în care autentificarea a reușit, sesiunea activă din care face parte utilizatorul reține identificatorul și rolul său printr-un cookie de sesiune păstrat în browser. Fig.3.31 Implementarea funcției

Quotes detected: **0.01%** id: **204**

„login”

35 Vasile Giulia-Georgiana Funcția

Quotes detected: **0.01%** id: **205**

„me”

verifică dacă utilizatorul este sau nu autentificat în cont. Dacă sesiunea există, datele utilizatorului sunt returnate către front-end, iar dacă nu există, proprietatea

Quotes detected: **0.02%** id: **206**

„req.session.user”

este nulă, returnându-se astfel un răspuns negativ. Fig.3.32 Implementarea funcției

Quotes detected: **0.01%** id: **207**

„me”

Funcția

Quotes detected: **0.01%** id: **208**

„logout”

elimină sesiunea activă de pe server utilizând proprietatea

Quotes detected: **0.02%** id: **209**

„req.session.destroy”,

iar cookie-ul

Quotes detected: **0.01%** id: **210**

„warehouse_session”

este șters din browser utilizând

Quotes detected: **0.02%** id: **211**

„res.clearCookie”,

ceea ce permite delogarea completă a utilizatorului. Fig.3.33 Implementarea funcției

Quotes detected: **0.01%** id: **212**

„logout”

3.4.3.2 Controller-ul pentru produse din

Quotes detected: **0.02%** id: **213**

„productsController.js”

Controller-ul pentru produse permite gestionarea corectă a catalogului de materii prime cu ajutorul operațiilor de tip CRUD. Utilizarea metodelor din

Quotes detected: **0.02%** id: **214**

„productsModel.js”

și restricționarea rutelor de accesare a datelor reprezintă principalele funcții ale acestuia. Funcția

Quotes detected: **0.01%** id: **215**

„createProductController”

ajută la crearea unui produs prin înserarea datelor din

Quotes detected: **0.02%** id: **216**

„req.body”.

Înainte de stocarea lor în sistem este necesară verificarea rolului de administrator al utilizatorului pentru o protecție sporită, dar și a câmpurilor obligatorii necesare în finalizarea procesului de creare a produsului. În cazul în care există posibilitatea de

Quotes detected: **0.01%** id: **217**

„SKU”

duplicat, codul

Quotes detected: **0.01%** id: **218**

„23505”

confirmă o încălcare a constrângerii din baza de date PostgreSQL și indică acest lucru prin mesajul

Quotes detected: **0.05%** id: **219**

„Acest SKU a fost deja folosit!”,

totul fără a opri funcționarea serverului. 36 Proiect de Diplomă Fig.3.34 Implementarea funcției Fig.3.35 Implementarea funcției

Quotes detected: **0.01%** id: **220**

„createProductController”

(partea 1)

Quotes detected: **0.01%** id: **221**

„createProductController”

(partea 2) Funcția

Quotes detected: **0.01%** id: **222**

„getAllProductsController”

accesează toate produsele din

Quotes detected: **0.01%** id: **223**

„getAllProducts”

pe care, mai apoi, le returnează către front-end într-un format JSON pentru o utilizare eficientă a datelor. Fig.3.34 Implementarea funcției

Quotes detected: **0.01%** id: **224**

„getAllProductsController”

Funcția

Quotes detected: **0.01%** id: **225**

„getIdProductController”

preia identificatorul produsului din

Quotes detected: **0.02%** id: **226**

„req.params”,

verifică existența acestuia în

Quotes detected: **0.01%** id: **227**

„getIdProduct”,

iar apoi îl returnează în format JSON către front-end. Fig.3.35 Implementarea funcției

Quotes detected: **0.01%** id: **228**

„getIdProductController”

37 Vasile Giulia-Georgiana Funcția

Quotes detected: **0.01%** id: **229**

„updateProductController”

permite actualizarea unui produs care deja există în baza de date, verificând mai întâi dacă utilizatorul este conectat la sesiune și dacă acesta are rolul necesar editării informațiilor produsului dorit. De asemenea, după identificarea articolului stocat în baza de date, se realizează o verificare suplimentară care, în cazul în care rolul nu este cel de administrator, să se poată edita doar produsul pe care utilizatorul autentificat în sesiune l- a creat. Fig.3.36 Implementarea funcției

Quotes detected: **0.01%** id: **230**

„updateProductController”

Funcția

Quotes detected: **0.01%** id: **231**

„deleteProductController”

permite ștergerea unui produs din baza de date prin verificarea autentificării utilizatorului și a rolului acestuia, iar în cazul utilizatorilor care au rolul de operator, ștergerea este posibilă doar pentru produsele create de aceștia. Fig.3.37 Implementarea funcției

Quotes detected: **0.01%** id: **232**

„deleteProductController”

38 Proiect de Diplomă 3.4.3.3 Controller-ul pentru tranzații din

Quotes detected: **0.02%** id: **233**

„transactionsController.js”

Controller-ul pentru tranzacții gestionează toate operațiile de intrare și ieșire din stocul aplicației. Un element important din cadrul acestei funcții este că tranzacțiile nu afectează doar tabelul din care fac parte, ci modifică și stocul unui produs asociat procesului. Funcția

Quotes detected: **0.01%** id: **234**

„createTransactionController”

permite crearea de tranzacții noi și identifică utilizatorul asociat cu întregul proces. În primă fază, funcția extrage elementele necesare din

Quotes detected: **0.02%** id: **235**

„req.body”

pentru a efectua verificări asupra datelor de intrare, precum și a tipului și cantității tranzacției. Totodată, orice eroare apărută în timpul realizării operației de mișcare a stocului din depozit este semnalată cu ajutorul funcției

Quotes detected: **0.01%** id: **236**

„mapTxError”

care are rolul de a transforma erorile tehnice în mesaje HTTP ușor de înțeles pentru utilizator. Fig.3.38 Implementarea funcției

Quotes detected: **0.01%** id: **237**

„createTransactionController”

Funcția

Quotes detected: **0.01%** id: **238**

„getAllTransactionsController”

reprezintă un filtru al rolurilor din sistem, unde administratorului îi este permis prin

Quotes detected: **0.01%** id: **239**

„getAllTransactions”

să vizualizeze toate tranzacțiile înregistrate vreodată în cadrul aplicației, iar operatorului îi este permisă vizualizarea prin

Quotes detected: **0.01%** id: **240**

„getUserTransactions”

a propriilor tranzacții. 39 Vasile Giulia-Georgiana Fig.3.39 Implementarea funcției

Quotes detected: **0.01%** id: **241**

„getAllTransactionsController”

Funcția

Quotes detected: **0.01%** id: **242**

„getAllTransactionsIDController”

permite vizualizarea unei tranzacții pe baza identificatorului propriu al acesteia, fiind astfel și o modalitate prin care se verifică drepturile de acces din sistem. În cazul în care utilizatorul autentificat dorește să vizualizeze o tranzacție care nu îi aparține, dar are rolul de operator, funcția îi va returna mesaje de eroare indicând acest lucru. Fig.3.40 Implementarea funcției

Quotes detected: **0.01%** id: **243**

„getAllTransactionsIDController”

Funcția

Quotes detected: **0.01%** id: **244**

„getAllTransactionsProductController”

reprezintă o modalitate prin care se pot afișa toate tranzacțiile asociate unui singur produs din cadrul aplicației. 40 Proiect de Diplomă Fig.3.41 Implementarea funcției

Quotes detected: **0.01%** id: **245**

„getAllTransactionsProductController”

Funcția

Quotes detected: **0.01%** id: **246**

„updateTransactionDescriptionController”

permite modificarea descrierii unei tranzacții deja existente în sistem. Totodată, reprezintă o metodă prin care utilizatorii cu rolul de operator pot edita, adăuga sau corecta diferite informații

referitoare la statusul unei mișcări de stoc, fără a fi necesară intervenția unui administrator. Fig.3.42 Implementarea funcției

Quotes detected: **0.01%** id: **247**

„updateTransactionDescriptionController”

Funcția

Quotes detected: **0.01%** id: **248**

„updateTransactionFullController”

este modalitatea prin care administratorul poate să editeze și să actualizeze toate datele unei tranzacții din sistem. În timpul modificării, tipul mișcărilor de stoc este convertit în majuscule, iar inventarul asociat operației se ajustează automat, asigurând-se astfel o consistență a datelor. 41 Vasile Giulia-Georgiana Fig.3.43 Implementarea funcției

Quotes detected: **0.01%** id: **249**

„updateTransactionFullController”

Funcția

Quotes detected: **0.01%** id: **250**

„deleteTransactionController”

permite ștergerea unei tranzacții din aplicație, recalculând stocul acesteia după terminarea operației. Fig.3.44 Implementarea funcției

Quotes detected: **0.01%** id: **251**

„deleteTransactionController”

3.4.3.3 Controller-ul administrării utilizatorilor din

Quotes detected: **0.02%** id: **252**

„usersController.js”

Controller-ul permite gestionarea operațiilor de creare, afișare, actualizare sau ștergere a utilizatorilor din aplicație. De asemenea, accesarea acestor funcționalități necesită rolul de administrator pentru a preveni posibilele modificări neautorizate ale informațiilor regăsite în sistem. 42 Proiect de Diplomă Funcția

Quotes detected: **0.01%** id: **253**

„createUserController”

permite inserarea unui nou utilizator în platformă, preluând datele necesare și verificând dacă adresa introdusă la înregistrarea în cont există deja în baza de date. În situația în care nu a fost furnizat un rol la crearea contului, utilizatorul va fi salvat în mod automat cu rolul de operator. Fig.3.45 Implementarea funcției

Quotes detected: **0.01%** id: **254**

„createUserController”

Funcția

Quotes detected: **0.01%** id: **255**

„getUserIdController”

verifică, pe baza identificatorului propriu al utilizatorului, dacă acesta este sau nu înregistrat în platformă. De asemenea, funcția

Quotes detected: **0.01%** id: **256**

„getAllUsersController”

permite listarea completă a conturilor pentru o gestionare mai ușoară de către administrator direct din interfața aplicației. Fig.3.46 Implementarea funcției Fig.3.47 Implementarea funcției

Quotes detected: **0.01%** id: **257**

„getUserIdController”

Quotes detected: **0.01%** id: **258**

„getAllUsersController”

43 Vasile Giulia-Georgiana Funcția

Quotes detected: **0.01%** id: **259**

„updateUserController”

facilitează procesul de actualizare a informațiilor conturilor existente, asigurând o înserare corectă a câmpurilor necesare pe parcursul procesului. În cazul în care un utilizator este inactiv, nu mai dorește să facă parte din sistem sau contul acestuia nu mai este necesar, se poate folosi funcția

Quotes detected: **0.01%** id: **260**

„deleteUserController”,

care reprezintă modalitatea de eliminare în siguranță a datelor contului din baza de date. Fig.3.49 Implementarea funcției Fig.3.48 Implementarea funcției

Quotes detected: **0.01%** id: **261**

„updateUserController”

(partea 2)

Quotes detected: **0.01%** id: **262**

„updateUserController”

(partea 1) Fig.3.50 Implementarea funcției

Quotes detected: **0.01%** id: **263**

„deleteUserController”

44 Proiect de Diplomă 3.4.4 Autorizarea și securizarea rutelor Resursele regăsite în server și securitatea aplicației utilizează mecanisme de autorizare a sesiunii, implementate cu ajutorul platformei software Node.js și al framework-ului Express, prin intermediul fișierului

Quotes detected: **0.02%** id: **264**

„authMiddleware.js”.

Acesta interceptează cererile de tip HTTP înainte de procesarea lor de către controllere, verificând utilizatorii și drepturile lor de acces. Middleware-ul

Quotes detected: **0.01%** id: **265**

„isAuth”

permite validarea unei sesiuni existente a unei cereri HTTP, analizând dacă contul autentificat conține informațiile stocate în obiectul de sesiune

Quotes detected: **0.02%** id: **266**

„req.session”.

Dacă sesiunea curentă nu există sau a expirat, accesul este refuzat printr-un mesaj care indică lipsa posibilității de conectare pentru utilizarea elementului dorit. Pentru protejarea și diferențierea autorizărilor bazate pe rolul utilizatorului, a fost necesară dezvoltarea unui middleware, denumit

Quotes detected: **0.01%** id: **267**

„reqRole”,

care verifică corespondența dintre un cont autentificat și rolul asociat cu acesta. Operația de validare se realizează printr-o comparație dintre statutul profilului și lista cu roluri acceptate pe ruta respectivă. În cazul în care persoana care este conectată pe platformă nu deține permisiunile necesare pentru accesarea rutei solicitate, serverul va transmite un mesaj prin care se înțelege că accesul este interzis, ruta fiind restricționată. Astfel, se face o distincție clară dintre mecanismul de autentificare și cel de gestionare al permisiunilor indiferent de rolul prin care este accesată aplicația. Gestionarea sesiunilor și a cookie-urilor se bazează pe modulul

Quotes detected: **0.01%** id: **268**

„express-session”,

prezentat anterior în diferite funcții din cadrul menținerii autentificării (fig.3.28), prin care un cookie rămâne stocat în browser printr-o sesiune unică creată pentru a-i permite utilizatorului menținerea activă în sistem fără a fi necesară reautentificarea. Fig.3.51 Implementarea middleware-ului de autentificare

Quotes detected: **0.02%** id: **269**

„authMiddleware.js”

45 Vasile Giulia-Georgiana 3.5 Arhitectura și funcționarea front-end-ului aplicației 3.5.1 Structura aplicației front-end În procesul de realizare a front-end-ului a fost utilizată biblioteca React, care permite folosirea componentelor într-o modalitate independentă, fiecare fișier având un rol bine definit în fluxul aplicației. Conform Fig.3.52, arhitectura software este împărțită în mai multe directoare principale. Componentele reprezentative pentru autentificare, administrare produse, tranzacții, utilizatori și a punctelor principale de acces îi revin directorului

Quotes detected: **0.01%** id: **270**

„pages”,

care îi permite utilizatorului să coordoneze și să afișeze elementele vizuale cu care acesta interacționează. Directorul

Quotes detected: **0.01%** id: **271**

„components”

ajută la evitarea duplicării codului prin reutilizarea unor componente, precum bara de navigare, ferestrele de dialog sau elemente de feedback vizual, aplicarea lor fiind posibilă în mai multe pagini din cadrul platformei. De asemenea, mecanismul prin care front-end- ul comunică cu back-end-ul se regăsește în directorul

Quotes detected: **0.01%** id: **272**

„libs”,

unde sunt definite funcții care ajută la realizarea cererilor HTTP, precum și configurarea Axios, care stabilește modul în care acestea sunt transmise către server. În paralel, directorul

Quotes detected: **0.01%** id: **273**

„features”

reprezintă gestionarea principalelor informații legate de utilizatorul autentificat, dar și despre sesiunea activă, folosind Redux Toolkit. Totodată, fișierele

Quotes detected: **0.02%** id: **274**

„App.jsx”

și

Quotes detected: **0.02%** id: **275**

„index.jsx”

permit separarea clară între interfață, logica de funcționare și comunicarea cu serverul. Fig.3.52 Arhitectura aplicației front-end 3.5.2 Implementarea rutării în aplicația React Pentru implementarea rutării a fost folosită biblioteca React Router, care îi permite aplicației să fie construită ca SPA (Single Page Application), oferind posibilitatea navigării rapide între pagini și reprezentând modalitatea prin care se poate realiza o integrare ușoară cu rolurile și autentificarea din interiorul platformei. 46 Proiect de Diplomă În cadrul Fig.3.52, este prezentată principala metodă de rutare din fișierul

Quotes detected: **0.02%** id: **276**

„App.jsx”,

unde componenta

Quotes detected: **0.01%** id: **277**

„BrowserRouter”

conține întreaga aplicație, ceea ce permite gestionarea oricărei modificări aflate în bara de adrese și afișarea elementului corespunzător rutei accesate. Totodată, în interiorul Fig.3.52 Definirea rutelor publice în

Quotes detected: **0.02%** id: **278**

„App.jsx”

fișierului se regăsește și componenta

Quotes detected: **0.01%** id: **279**

„Routes”,

prin care sunt definite toate rutele aplicației. Astfel, fiecare rută este asociată cu o componentă React prin intermediul unui anumit URL, fiind posibilă crearea căilor publice de navigare care pot fi accesate fără o autentificare ulterioară în aplicație. Rutele publice care se regăsesc în această situație sunt următoarele: pagina de bun venit (

Quotes detected: **0.01%** id: **280**

„/”

), pagina de înregistrare (

Quotes detected: **0.01%** id: **281**

„/register”

) și pagina de autentificare (

Quotes detected: **0.01%** id: **282**

„/login”

). Rutele protejate prin

Quotes detected: **0.01%** id: **283**

„RequireAuth”

reprezintă funcționalitățile principale ale aplicației și sunt accesibile doar după realizarea autentificării. La introducerea comenzii, se verifică existența unui utilizator în centrul informațional al aplicației, iar dacă acesta nu este autentificat, componenta

Quotes detected: **0.01%** id: **284**

„Navigate”

îl va redirecționa automat către pagina Fig.3.53 Mecanismului de redirecționare automată de logare. De asemenea, mecanismul prin

Quotes detected: **0.01%** id: **285**

„Navigate”

care asigură gestionarea corectă a datelor este susținut de Redux Toolkit, facilitând întregul proces de autentificare. Pentru cazul în care accesarea anumitor secțiuni nu trebuie să fie valabilă utilizatorilor cu rol de operator, a fost utilizată componenta

Quotes detected: **0.01%** id: **286**

„RequireRole”

pentru a restricționa conectarea celor care nu se încadrează în lista rolurilor permise. Dacă utilizatorul nu are atribuit un rol corespunzător cererii, va fi redirecționat către o rută publică sau către o rută pentru care deține drepturi de acces. Fig.3.54 Protejarea rutelor utilizând

Quotes detected: **0.01%** id: **287**

„RequireAuth”

și

Quotes detected: **0.01%** id: **288**

„RequireRole”

3.5.3 Gestionarea stării globale cu Redux Toolkit 47 Vasile Giulia-Georgiana Datele disponibile la nivel global sunt configurate în fișierul

Quotes detected: **0.02%** id: **289**

„store.js”,

unde mecanismul central de gestionare al acestora este creat prin funcția

Quotes detected: **0.01%** id: **290**

„configureStore”,

permițând logicii de actualizare a autentificării să asigure accesul uniform la informațiile despre cont în orice componentă a sistemului. În Fig.3.55 Configurarea magazinului Redux în același timp, esența întregului proces de

Quotes detected: **0.02%** id: **291**

„store.js”

autorizare a contului se regăsește în fișierul

Quotes detected: **0.02%** id: **292**

„authSlice.js”,

unde este definită o porțiune Redux care permite administrarea datelor utilizatorului, dar și a unor acțiuni asincrone implementate prin

Quotes detected: **0.01%** id: **293**

„createAsyncThrunk”

și destinate colaborării cu serviciile back-end. De asemenea, în timpul tratării stărilor de

Quotes detected: **0.01%** id: **294**

„pending”,

Quotes detected: **0.01%** id: **295**

„fulfilled”

și

Quotes detected: **0.01%** id: **296**

„rejected”

se setează statusul de

Quotes detected: **0.01%** id: **297**

„loading”

pentru așteptarea actualizării corecte a contului și pentru verificarea sau transmiterea erorilor care pot apărea în interfață, gestionarea acestor rezultate fiind realizată în secțiunea

Quotes detected: **0.01%** id: **298**

„extraReducers”.

Fig.3.56 Gestionarea stărilor asincrone din Fig.3.57 Implementarea operației

Quotes detected: **0.02%** id: **299**

„authSlice.js”

asincrone de autentificare În Fig.3.57 este prezentat un exemplu de operație prin care sunt transferate datele de autentificare către server cu ajutorul funcției

Quotes detected: **0.01%** id: **300**

„apiLogin”,

obținând starea de autentificare a utilizatorului prin

Quotes detected: **0.01%** id: **301**

„apiMe”.

La finalizarea acestor pași, starea gestionată prin cadrul constantei

Quotes detected: **0.01%** id: **302**

„store”

este actualizată cu noul statut al utilizatorului în platformă. Prin abordarea statusurilor de tip

Quotes detected: **0.01%** id: **303**

„pending”,

Quotes detected: **0.01%** id: **304**

„fulfilled”

și

Quotes detected: **0.01%** id: **305**

„rejected”,

se pot afișa mesaje relevante în funcție de fiecare rezultat al operațiilor asincrone datorită constantei

Quotes detected: **0.01%** id: **306**

„STATUS”.

Această constantă este definită prin Fig.3.58 Configurarea statusurilor și a stării inițiale în

Quotes detected: **0.02%** id: **307**

„Object.freeze”

și stabilește valori modulul de autentificare standard pentru stările din sistem, cele trei etape fiind modificate astfel: în etapa de

Quotes detected: **0.01%** id: **308**

„pending”,

statusul devine

Quotes detected: **0.01%** id: **309**

„loading”,

indicând o procesare în curs a cererii; în

Quotes detected: **0.01%** id: **310**

„fulfilled”,

statusul devine

Quotes detected: **0.01%** id: **311**

„success”,

iar datele se actualizează după primirea lor de la server prin obiectul

Quotes detected: **0.01%** id: **312**

„user”

; iar în etapa

Quotes detected: **0.01%** id: **313**

„rejected”,

statusul devine

Quotes detected: **0.01%** id: **314**

„error”,

proprietatea

Quotes detected: **0.01%** id: **315**

„error”

salvând astfel mesajul stării de eșec. Un alt element important în procesul de gestionare al modulului de autentificare este

Quotes detected: **0.01%** id: **316**

„initialState”,

care contribuie la evitarea afișării unor informații incorecte înainte de finalizarea 48 Proiect de Diplomă cererilor către server. Structura constantei este formată din trei termeni, fiecare având un rol specific. Primul termen este

Quotes detected: **0.02%** id: **317**

„user: null”

care clarifică faptul că nu există în prezent un alt utilizator autentificat, al doilea este

Quotes detected: **0.02%** id: **318**

„status: STATUS.idle”

care păstrează starea neutră înaintea efectuării oricărei operații în sistem, iar al treilea este

Quotes detected: **0.02%** id: **319**

„error: null”

care previne afișarea unor mesaje eronate la inițializarea procesului. Pentru integrarea magazinului Redux în aplicație se folosește componenta

Quotes detected: **0.01%** id: **320**

„Provider”

din fișierul

Quotes detected: **0.02%** id: **321**

„index.jsx”,

fiind o modalitate prin care se pot accesa ușor informațiile stocate în

Quotes detected: **0.01%** id: **322**

„store”

din orice componentă a sistemului. Prin aceasta se realizează o legătură strânsă între interfața cu utilizatorul și Fig.3.59 Integrarea store-ului Redux prin componenta nivelul de procesare a datelor.

Quotes detected: **0.01%** id: **323**

„Provider”

3.5.4 Comunicarea cu back-end-ul prin Axios Conform datelor prezentate anterior în Fig.3.30, configurarea Axios este realizată în fișierul

Quotes detected: **0.02%** id: **324**

„libs/axios.js”,

unde aceasta definește URL-ul prin variabila

Quotes detected: **0.01%** id: **325**

„VITE_API_URL”,

dar și opțiunea

Quotes detected: **0.02%** id: **326**

„withCredentials: true”,

necesară transmiterii cookie-urilor de sesiune între client și server. În același fișier însă, se regăsește și funcția

Quotes detected: **0.03%** id: **327**

„api.interceptors.response.use”

care procesează răspunsurile și definește modalitatea prin care erorile sunt tratate, construind astfel un mesaj pe baza codului HTTP sau pe baza informațiilor transmise de back-end prin

Quotes detected: **0.02%** id: **328**

„resp.data.message”

sau prin

Quotes detected: **0.02%** id: **329**

„resp.data.error”.

Fig.3.60 Configurarea completă a clientului HTTP Axios în aplicația front-end 49 Vasile Giulia-Georgiana După configurarea Axios, centralizarea apelurilor către server se realizează în fișierul

Quotes detected: **0.02%** id: **330**

„api.js”

prin implementarea fiecărei funcții corespunzătoare unei rute. Folosirea operațiilor de tip autentificare, produse, tranzacții și utilizatori permite reutilizarea lor într-o manieră controlată, fără a fi necesară duplicarea codului. Totodată, aceste funcții corespund deja rutelor definite pe partea de back-end, întorcând astfel răspunsul prin

Quotes detected: **0.02%** id: **331**

„response.data”

și simplificând întregul proces de gestionare al erorilor și transmitere al informațiilor. Fig.3.61 Funcții pentru apelurile API de autentificare și gestionare a produselor 3.5.5 Implementarea funcționalității de export în format Excel Pentru implementarea funcționalității de export în format Excel a fost necesară integrarea bibliotecii SheetJS (XLSX)[nr], ceea ce permite generarea directă în browser a aplicațiilor de calcul tabelar. Datorită capacității sale de a procesa informațiile regăsite în tabelele din cadrul platformei, precum și flexibilității în afișarea structurii fișierului rezultat, biblioteca reprezintă modalitatea principală prin care este îmbunătățită experiența cu utilizatorul. Realizarea întregii funcționalități a fost posibilă prin organizarea sa pe

module, ceea ce îi permite logicii de export să fie reutilizată cu ușurință, facilitând astfel menținerea unei arhitecturi clare și ușor de extins. Astfel, biblioteca este importată prin

Quotes detected: **0.03%** id: **332**

„import * as XLSX”

în fișierul

Quotes detected: **0.02%** id: **333**

„utils.js”,

oferind posibilitatea accesării elementelor necesare pentru crearea unui registru de lucru (workbook) și a foilor de lucru (worksheets) din acesta. Primul pas pentru implementarea propriu-zisă a funcționalității este realizat prin funcția

Quotes detected: **0.04%** id: **334**

„exportXLSX(filename, rows, columns)”,

care primește informații referitoare la datele ce urmează a fie exportate, dar și despre întreaga configurație a coloanelor. În plus, structura câmpurilor este definită sub forma unui array de obiecte ce include proprietăți precum

Quotes detected: **0.01%** id: **335**

„key”,

Quotes detected: **0.01%** id: **336**

„label”

sau

Quotes detected: **0.01%** id: **337**

„format”,

necesare în corelarea dintre termenii introduși în sistem și denumirile vizibile în Excel. 50 Proiect de Diplomă Fig.3.62 Implementarea funcției

Quotes detected: **0.01%** id: **338**

„exportXLSX”

Așa cum se ilustrează în Fig.3.62, metoda

Quotes detected: **0.02%** id: **339**

„XLSX.utils.aoa_to_sheet”

convertește tabelul creat în JavaScript într-un obiect pe care Excel îl înțelege ca foaie de calcul , ca mai apoi să creeze registru de lucru nou prin

Quotes detected: **0.02%** id: **340**

„XLSX.utils.book_new”

și o foaie de lucru prin

Quotes detected: **0.02%** id: **341**

„XLSX.utils.book_append_sheet”

atașat de acesta. Generarea fișierului final se realizează prin

Quotes detected: **0.02%** id: **342**

„XLSX.writeFile”,

care descarcă automat documentul de tip

Quotes detected: **0.02%** id: **343**

„.xlsx”

în browser. Pentru a fi posibilă integrarea în diferitele secțiuni ale aplicației, a fost creat un eveniment

Quotes detected: **0.01%** id: **344**

„onClick”

care declanșează funcția de export la apăsarea butoanelor regăsite la nivelul platformei. Cu alte cuvinte, tranzacțiile, produsele și utilizatorii păstrează aceeași structură prin care se realizează

extractul de date din sistem, chiar dacă se dorește filtrarea acestora. De exemplu, în cazul produselor, funcția este invocată cu parametrii

Quotes detected: **0.01%** id: **345**

„PRODUCT_XLSX_FILE”,

Quotes detected: **0.01%** id: **346**

„filtered”

și

Quotes detected: **0.01%** id: **347**

„PRODUCT_COLUMNS”,

unde variabila de filtrare reprezintă lista de valori obținută după procesul de selecție prin

Quotes detected: **0.01%** id: **348**

„filterByQuery”,

proces care exportă exact datele afișate în interfață. Fig.3.63 Integrarea funcției

Quotes detected: **0.01%** id: **349**

„exportXLSX”

prin intermediul unui buton 51 Vasile Giulia-Georgiana Fig.3.64 Definirea structurii coloanelor și a fișierului Excel în

Quotes detected: **0.02%** id: **350**

„Products.jsx”

3.5.6 Componenta Bara de navigare Bara de navigare reprezintă modalitatea principală prin care utilizatorul poate interacționa în mod direct cu aplicația. De asemenea, permite o navigare sigură între pagini prin adaptarea interfeței la funcțiile de autentificare și rol. Definirea componentei fiind astfel regăsită în fișierul

Quotes detected: **0.02%** id: **351**

„Navnar.jsx”,

unde este integrată la nivel global în

Quotes detected: **0.02%** id: **352**

„App.jsx”,

ca mai apoi să fie afișată pe tot parcursul interacțiunii din platformă. În prezentările anterioare din Fig.3.52, se putea observa implementarea componentei

Quotes detected: **0.03%** id: **353**

„ Navbar / ”

în interiorul structurii

Quotes detected: **0.01%** id: **354**

„BrowserRouter”,

poziționare care permite afișarea permanentă a meniului, indiferent de existența unei rute active. Totuși, componenta folosește mecanismele oferite de React și Redux pentru identificarea oricărui utilizator conectat în aplicație, dar și a rolului pe care acesta îl deține. Având în vedere că, structura este definită în interiorul

Quotes detected: **0.02%** id: **355**

„Navbar.js”,

accesul la informațiile despre utilizator se face prin mecanismul

Quotes detected: **0.01%** id: **356**

„useSelector”,

unde se permite extragerea stării globale gestionate prin Redux Toolkit. Prin intermediul acestei instrucțiuni, interfața va afișa diferitele adaptări ale barei de navigare, elemente disponibile atât pentru persoanele autentificate în cont, cât și pentru rolul acestora. În situația în care variabila

Quotes detected: **0.01%** id: **357**

„user”

este

Quotes detected: **0.01%** id: **358**

„null”,

aplicația va considera că nu există un cont activ în sistem și va afișa doar opțiunile de

Quotes detected: **0.01%** id: **359**

„Loghează-te”

și

Quotes detected: **0.01%** id: **360**

„Înregistrează-te”.

Dacă utilizatorul este autentificat, se vor afișa opțiuni suplimentare de tipul

Quotes detected: **0.01%** id: **361**

„Dashboard”,

Quotes detected: **0.01%** id: **362**

„Produse”

și

Quotes detected: **0.01%** id: **363**

„Tranzacții”,

iar dacă acesta are rolul de administrator, se va afișa și meniul

Quotes detected: **0.01%** id: **364**

„Utilizatori”.

52 Proiect de Diplomă Fig.3.65 Implementarea componentei

Quotes detected: **0.01%** id: **365**

„Navbar”

în

Quotes detected: **0.02%** id: **366**

„Navbar.jsx”

În Fig.3.65 sunt prezentate elemente precum

Quotes detected: **0.01%** id: **367**

„Link”

sau

Quotes detected: **0.01%** id: **368**

„NavLink”

importate din biblioteca react-router-dom[nr], care mențin caracterul dinamic al aplicației de tip SPA (Single Page Application). Totodată, sunt menționate și clasele Bootstrap () precum

Quotes detected: **0.01%** id: **369**

„nav”,

Quotes detected: **0.01%** id: **370**

„nav__row”,

Quotes detected: **0.01%** id: **371**

„nav__brand”

și

Quotes detected: **0.01%** id: **372**

„nav_link”,

personalizând astfel aspectul întregului meniu prin setarea unor stiluri diferite ale elementelor barei de navigare în fișierul

Quotes detected: **0.02%** id: **373**

„index.css”.

Un element important al componentei este reprezentat de capacitatea sa de a se deconecta din sesiunea activă, fiind implementată apelând o acțiune Redux asincronă din

Quotes detected: **0.02%** id: **374**

„authSlice.js”.

În momentul în care utilizatorul dorește să se deconecteze, acesta apasă un buton de

Quotes detected: **0.01%** id: **375**

„Logout”

prin care se declanșează o cerere HTTP către ruta

Quotes detected: **0.01%** id: **376**

„/api/auth/logout”,

starea

Quotes detected: **0.01%** id: **377**

„user”

fiind resetată ulterior înapoi la valoare

Quotes detected: **0.01%** id: **378**

„null”,

indicând faptul ca niciun al utilizator nu mai este conectat în sesiune. Datorită acestui proces, se realizează o sincronizare cu sesiunea de pe server, meniul actualizându-se automat. 53 Vasile Giulia-Georgiana 3.5.7 Implementarea componentei de autentificare și înregistrare Autentificarea și înregistrarea reprezintă principala modalitate prin care se pot accesa funcționalitățile aplicației, fiind implementate cu ajutorul componentelor

Quotes detected: **0.02%** id: **379**

„Login.jsx”

și

Quotes detected: **0.02%** id: **380**

„Register.jsx”

și apoi integrate în

Quotes detected: **0.02%** id: **381**

„App.jsx”.

Prin intermediul acestora se realizează atât validarea datelor introduse de către utilizator, cât și accesarea diferitelor rute disponibile la începerea sesiunii. Componenta

Quotes detected: **0.02%** id: **382**

„Login.jsx”

gestionează întregul proces de autentificare prin utilizarea diferitelor mecanisme de tipul React și Redux Toolkit, stocând astfel, prin intermediul funcției

Quotes detected: **0.01%** id: **383**

„useState”,

variabilele

Quotes detected: **0.01%** id: **384**

„email”

și

Quotes detected: **0.01%** id: **385**

„password”.

În momentul în care au fost transmise datele necesare autentificării, se declanșează

Quotes detected: **0.01%** id: **386**

„onSubmit”,

verificând câmpurile introduse de utilizator și prevenind transmiterea unor date incomplete către back-end. De asemenea, este apelată și acțiunea

Quotes detected: **0.01%** id: **387**

„LoginUser”,

definită în

Quotes detected: **0.02%** id: **388**

„authSlice.js”,

prin intermediul

Quotes detected: **0.07%** id: **389**

„dispatch(LoginUser({ email: email.trim(), password }))”,

care trimite o cerere către server pentru actualizarea stării globale în funcție de răspunsul primit de la acesta. Conform prezentării anterioare din Fig.3.57, se remarcă transmiterea, verificarea și finalizarea pașilor de actualizare a statutului utilizatorului prin modificarea automată a interfeței la care acesta este conectat. După realizarea autentificării, componenta

Quotes detected: **0.02%** id: **390**

„Login.jsx”

permite redirecționarea utilizatorului către pagina dorită prin

Quotes detected: **0.01%** id: **391**

„useNavigate”,

utilizând informația stocată in

Quotes detected: **0.01%** id: **392**

„useLocation”,

ceea ce menține o coerență pe tot parcursul fluxului de navigare în sistem. Fig.3.66 Implementarea funcției

Quotes detected: **0.01%** id: **393**

„Login”

din

Quotes detected: **0.02%** id: **394**

„Login.jsx”

(partea 1) 54 Proiect de Diplomă Fig.3.67 Implementarea funcției

Quotes detected: **0.01%** id: **395**

„Login”

din

Quotes detected: **0.02%** id: **396**

„Login.jsx”

(partea 2) Componenta

Quotes detected: **0.02%** id: **397**

„Register.jsx”

păstrează un principiu similar cu cel de autentificare, însă în interiorul acesteia se află și procesul de creare a unui cont nou. Dezvoltarea câmpurilor

Quotes detected: **0.01%** id: **398**

„first_name”,

Quotes detected: **0.01%** id: **399**

„last_name”,

Quotes detected: **0.01%** id: **400**

„email”,

Quotes detected: **0.01%** id: **401**

„password”

și

Quotes detected: **0.01%** id: **402**

„role”

a fost realizată printr-un obiect

Quotes detected: **0.01%** id: **403**

„form”,

care apelează acțiunea

Quotes detected: **0.01%** id: **404**

„RegisterUser”

din

Quotes detected: **0.02%** id: **405**

„authSlice.js”

pentru a permite conectarea automată în platformă a utilizatorului imediat după ce acesta a fost înregistrat. În Fig.3.68 se poate observa că în dreptul câmpului

Quotes detected: **0.01%** id: **406**

„role”

este deja introdusă o valoare implicită

Quotes detected: **0.01%** id: **407**

„OPERATOR”,

necesară în cazul în care la momentul creării contului, rolul nu a fost selectat. Un element important al întregului mecanism este utilizarea sesiunilor gestionate de back-end prin express-session. Datorită configurării

Quotes detected: **0.02%** id: **408**

„withCredentials: true”

în instanța Axios, cookie-ul de sesiune este transmis automat între client și server. Astfel, după autentificare, toate cererile ulterioare includ informațiile necesare identificării utilizatorului, fără a fi nevoie de token- uri suplimentare sau manipulări manuale la nivelul front-end-ului. Protejarea paginilor interne ale aplicației este realizată prin componenta

Quotes detected: **0.01%** id: **409**

„RequireAuth”,

care verifică existența obiectului

Quotes detected: **0.01%** id: **410**

„user”

în starea globală. Dacă utilizatorul nu este autentificat, acesta este redirecționat automat către ruta

Quotes detected: **0.01%** id: **411**

„/login”.

În mod similar, componenta

Quotes detected: **0.01%** id: **412**

„RequireRole”

permite restricționarea accesului în funcție de rol, asigurând separarea clară între funcționalitățile disponibile administratorului și cele disponibile operatorului. 55 Vasile Giulia-Georgiana Fig.3.68 Implementarea funcției

Quotes detected: **0.01%** id: **413**

„Register”

din

Quotes detected: **0.02%** id: **414**

„Register.jsx”

(partea 1) Fig.3.69 Implementarea funcției

Quotes detected: **0.01%** id: **415**

„Register”

din

Quotes detected: **0.02%** id: **416**

„Register.jsx”

(partea 2) 56 Proiect de Diplomă Fig.3.70 Implementarea funcției

Quotes detected: **0.01%** id: **417**

„Register”

din

Quotes detected: **0.02%** id: **418**

„Register.jsx”

(partea 3) 3.5.8 Implementarea modulului de gestionare a produselor Modulul de gestionare a produselor reprezintă una dintre componentele centrale ale aplicației, fiind responsabil de afișarea, filtrarea, crearea, editarea și ștergerea produselor existente în baza de date. Implementarea acestui modul este realizată prin mai multe componente React interconectate, dintre care cea principală este

Quotes detected: **0.02%** id: **419**

„Products.jsx”,

completată de

Quotes detected: **0.02%** id: **420**

„AddProducts.jsx”,

Quotes detected: **0.02%** id: **421**

„EditProducts.jsx”

și

Quotes detected: **0.02%** id: **422**

„ProductDetailsModal.jsx”.

Împreună, acestea Fig.3.71 Încărcarea listei de produse din formează un flux coerent de interacțiune între server în

Quotes detected: **0.02%** id: **423**

„Products.jsx”

utilizator și sistem. Componenta

Quotes detected: **0.02%** id: **424**

„Products.jsx”

este responsabilă de afișarea listei de produse într-un tabel dinamic. La încărcarea paginii, prin intermediul

Quotes detected: **0.01%** id: **425**

„useEffect”,

este apelată funcția

Quotes detected: **0.01%** id: **426**

„apiAllProducts”,

definită în fișierul

Quotes detected: **0.02%** id: **427**

„api.js”.

Această funcție execută o cerere HTTP de tip GET către ruta

Quotes detected: **0.01%** id: **428**

„/api/products”,

iar răspunsul primit este stocat în variabila locală

Quotes detected: **0.01%** id: **429**

„rows”

prin

Quotes detected: **0.01%** id: **430**

„useState”.

Astfel, datele sunt menținute la nivelul componentei și pot fi manipulate ulterior pentru filtrare sau actualizare locală. În interiorul Fig.3.64, a fost implementată o funcționalitate de căutare locală prin utilizarea funcției

Quotes detected: **0.01%** id: **431**

„filterByQuery”,

care permite filtrarea dinamică a produselor pe baza unui șir de caractere introdus într-un câmp de tip input. Variabila

Quotes detected: **0.01%** id: **432**

„filtered”

reprezintă rezultatul acestei filtrări și este utilizată direct în generarea rândurilor tabelului. Astfel, utilizatorul poate vizualiza doar produsele relevante fără a genera cereri suplimentare către server. Un element important al acestei componente este gestionarea permisiunilor. Variabila

Quotes detected: **0.01%** id: **433**

„isAdmin”

este determinată pe baza rolului utilizatorului autentificat, preluat din Redux prin

Quotes detected: **0.01%** id: **434**

„useSelector”.

În funcție de această valoare, sunt 57 Vasile Giulia-Georgiana afișate sau ascunse butoanele pentru creare, editare și ștergere. În plus, funcția

Quotes detected: **0.02%** id: **435**

„canManage(p)”

verifică dacă utilizatorul este administrator sau autorul produsului respectiv, asigurând astfel un control suplimentar la nivelul interfeței. Totoodată, crearea unui produs nou este realizată prin componenta

Quotes detected: **0.02%** id: **436**

„AddProducts.jsx”,

unde datele formularului sunt gestionate prin

Quotes detected: **0.01%** id: **437**

„useState”.

La trimiterea formularului, este apelată funcția

Quotes detected: **0.01%** id: **438**

„apiCreateProducts”,

care transmite un obiect JSON către server printr-o cerere POST. Valorile numerice sunt convertite explicit prin

Quotes detected: **0.02%** id: **439**

„Number()”,

iar câmpurile opționale sunt tratate astfel încât valorile goale să fie transformate în

Quotes detected: **0.01%** id: **440**

„null”.

După crearea cu succes a produsului, utilizatorul este redirecționat către pagina principală a produselor. Editarea însă, este implementată în

Quotes detected: **0.02%** id: **441**

„EditProducts.jsx”.

În această componentă, produsul este preluat inițial prin

Quotes detected: **0.02%** id: **442**

„apiProductsId(id)”,

utilizând parametrul extras din URL prin

Quotes detected: **0.01%** id: **443**

„useParams”.

După validarea drepturilor de modificare, datele sunt populate în formular și pot fi actualizate prin apelarea funcției

Quotes detected: **0.01%** id: **444**

„apiUpdateProducts”.

Această funcție transmite modificările printr-o cerere HTTP de tip PATCH, actualizând doar câmpurile necesare. Fig.3.72 Funcția

Quotes detected: **0.01%** id: **445**

„apiProductsId”

din Fig.3.73 Funcția

Quotes detected: **0.01%** id: **446**

„apiUpdateProducts”

din

Quotes detected: **0.02%** id: **447**

„EditProducts.jsx”

Quotes detected: **0.02%** id: **448**

„EditProducts.jsx”

Pentru vizualizarea detaliată a unui produs, este utilizată componenta

Quotes detected: **0.02%** id: **449**

„ProductDetailsModal.jsx”.

Aceasta este afișată condiționat prin intermediul variabilei

Quotes detected: **0.01%** id: **450**

„open”

și primește ca parametru obiectul

Quotes detected: **0.01%** id: **451**

„item”.

Modalul afișează informații complete despre produs, inclusiv imaginea, prețul formatat cu două zecimale și data ultimei actualizări convertită într-un format lizibil prin

Quotes detected: **0.02%** id: **452**

„toLocaleString()”.

Implementarea unui modal dedicat contribuie la păstrarea unei interfețe curate, evitând supraîncărcarea paginii principale. În cele din urmă, ștergerea unui produs este realizată prin apelarea funcției

Quotes detected: **0.01%** id: **453**

„apiDeleteProducts”,

asociată unei cereri HTTP

Quotes detected: **0.01%** id: **454**

„DELETE”.

Înainte de executarea operației, este afișată o fereastră de confirmare prin

Quotes detected: **0.02%** id: **455**

„confirm()”,

prevenind ștergerea accidentală. Fig.3.74 Afișarea detaliilor produsului în

Quotes detected: **0.02%** id: **456**

„ProductDetailsModal.jsx”

58 Proiect de Diplomă Structura tabelului este organizată astfel încât să evidențieze informațiile relevante, iar în cazul în care valoarea stocului este mai mică decât pragul minim (stock stock_min), rândul respectiv este stilizat diferit, utilizând clase CSS pentru a evidenția vizual situațiile critice. Această abordare oferă un mecanism rapid de identificare a produselor ce necesită reaprovizionare. Integrarea funcționalității de export în format Excel, implementată prin biblioteca SheetJS, completează acest modul. Fig.3.75 Evidențierea produselor cu stocul sub pragul minim 3.5.9 Implementarea componentei care gestionează tranzacțiile Componenta destinată gestionării tranzacțiilor reprezintă unul dintre cele mai importante module funcționale ale aplicației, întrucât reflectă în timp real modificările de stoc produse prin operații de tip

Quotes detected: **0.01%** id: **457**

„IN”

și

Quotes detected: **0.01%** id: **458**

„OUT”.

Implementarea acestei componente este realizată în fișierul

Quotes detected: **0.02%** id: **459**

„Transactions.jsx”,

unde sunt integrate mecanismele de afișare, filtrare, creare, editare și ștergere a tranzacțiilor. La încărcarea paginii, datele sunt preluate de la server prin apelul funcției

Quotes detected: **0.02%** id: **460**

„apiAllTransactions()”,

în cadrul unui

Quotes detected: **0.01%** id: **461**

„useEffect”.

Rezultatul obținut este salvat în starea locală

Quotes detected: **0.01%** id: **462**

„rows”,

iar înainte de afișare fiecare obiect este trecut prin funcția auxiliară

Quotes detected: **0.01%** id: **463**

„decorate”,

care completează anumite câmpuri lipsă, precum

Quotes detected: **0.01%** id: **464**

„product_name”

sau

Quotes detected: **0.01%** id: **465**

„created_by”.

Această etapă contribuie la menținerea unei afișări coerente în interfață, chiar și în situațiile în care anumite informații nu sunt transmise explicit de back-end. Fig.3.76 Încărcarea și decorarea listei de tranzacții în

Quotes detected: **0.02%** id: **466**

„Transactions.jsx”

Datele sunt afișate într-un tabel Bootstrap, cu evidențiere vizuală a tipului tranzacției prin utilizarea claselor

Quotes detected: **0.01%** id: **467**

„text-bg-success”

pentru

Quotes detected: **0.01%** id: **468**

„IN”

și

Quotes detected: **0.01%** id: **469**

„text-bg-danger”

pentru

Quotes detected: **0.01%** id: **470**

„OUT”.

Această diferențiere cromatică facilitează interpretarea rapidă a datelor și contribuie la o experiență vizuală intuitivă. 59 Vasile Giulia-Georgiana În partea superioară a paginii este implementat un mecanism de căutare locală, similar celui din componenta Produse. Variabila

Quotes detected: **0.01%** id: **471**

„q”

stochează textul introdus de utilizator, iar lista filtrată este calculată prin apelul funcției

Quotes detected: **0.04%** id: **472**

„filterByQuery(rows, TRX_SEARCH_FIELDS, q)”.

Astfel, utilizatorul poate căuta tranzacții după produs, SKU, tip, cantitate sau descriere, fără a genera apeluri suplimentare către server. Un element esențial al componentei îl reprezintă gestionarea permisiunilor. Variabila

Quotes detected: **0.01%** id: **473**

„isAdmin”

este calculată pe baza rolului utilizatorului, iar funcția

Quotes detected: **0.02%** id: **474**

„canManage(t)”

determină dacă utilizatorul poate modifica sau șterge tranzacția respectivă. În interfață, butoanele

Quotes detected: **0.01%** id: **475**

„Editează”

și

Quotes detected: **0.01%** id: **476**

„Șterge”

sunt afișate doar dacă această condiție este îndeplinită. Fig.3.78 Filtrarea tranzacțiilor, configurarea exportului și afișarea funcției

Quotes detected: **0.01%** id: **477**

„Transactions”

Crearea unei tranzacții este realizată prin intermediul componentei

Quotes detected: **0.01%** id: **478**

„CreateTransactionModal”,

care este deschisă la apăsarea butonului

Quotes detected: **0.02%** id: **479**

„+ Creează tranzacție”.

Formularul colectează informații despre produs, tip, cantitate și descriere, iar la validare trimite datele către server prin funcția

Quotes detected: **0.01%** id: **480**

„apiCreateTransaction”.

După confirmarea operației, lista locală este actualizată prin inserarea noii tranzacții la începutul array-ului

Quotes detected: **0.01%** id: **481**

„rows”.

Editarea tranzacției este realizată prin componenta

Quotes detected: **0.01%** id: **482**

„EditTransactionModal”,

care permite două tipuri de modificări: administratorul poate actualiza toate câmpurile, în timp ce operatorul poate modifica doar descrierea. În cazul ștergerii însă, funcția

Quotes detected: **0.01%** id: **483**

„onDelete”

solicită confirmarea utilizatorului și, în urma apelului

Quotes detected: **0.01%** id: **484**

„apiDeleteTransaction”,

actualizează lista locală prin eliminarea elementului corespunzător. 3.5.10 Implementarea componentei care gestionează utilizatorii Componenta destinată gestionării utilizatorilor este implementată în fișierul

Quotes detected: **0.02%** id: **485**

„Users.jsx”

și este accesibilă exclusiv utilizatorilor cu rolul de

Quotes detected: **0.01%** id: **486**

„ADMINISTRATOR”,

conform mecanismului de autorizare implementat la nivelul aplicației. Această restricționare este realizată atât prin componenta

Quotes detected: **0.01%** id: **487**

„RequireRole”

din sistemul de rutare, cât și prin verificarea directă a rolului utilizatorului în interiorul componentei, prin variabila

Quotes detected: **0.01%** id: **488**

„isAdmin”.

60 Proiect de Diplomă La încărcarea paginii, lista utilizatorilor este preluată de la server prin apelul funcției

Quotes detected: **0.02%** id: **489**

„apiAllUsers()”,

în cadrul unui

Quotes detected: **0.01%** id: **490**

„useEffect”.

Rezultatul este salvat în starea locală

Quotes detected: **0.01%** id: **491**

„rows”,

iar în cazul unei erori este afișat un mesaj corespunzător prin intermediul bibliotecii react-hot-toast. Această abordare permite sincronizarea interfeței cu baza de date fără reîncărcarea manuală a paginii. La fel ca în cazul tranzacțiilor și produselor, utilizatorul poate filtra rezultatele după prenume, nume, email sau rol. Crearea unui utilizator este realizată prin intermediul componentei

Quotes detected: **0.01%** id: **492**

„CreateUserModal”,

deschisă la apăsarea butonului

Quotes detected: **0.02%** id: **493**

„+ Creează utilizator”.

Formularul colectează informații precum prenume, nume, email, parolă și rol. La trimiterea datelor, funcția

Quotes detected: **0.01%** id: **494**

„apiCreateUser”

transmite informațiile către server, iar în cazul unui răspuns favorabil, utilizatorul nou creat este adăugat în partea superioară a listei. Fig.3.79 Formularul de creare a utilizatorului în

Quotes detected: **0.02%** id: **495**

„CreateUserModal.jsx”

Un aspect important al acestei componente îl reprezintă controlul accesului. Înainte de afișarea tabelului, este verificat rolul utilizatorului prin condiția

Quotes detected: **0.02%** id: **496**

„if (!isAdmin)”,

iar în cazul în care acesta nu are drepturi administrative, este afișat un mesaj de restricționare a accesului. Această verificare oferă un nivel suplimentar de protecție la nivel de interfață, completând mecanismul de autorizare implementat în back-end. Fig.3.80 Condiția

Quotes detected: **0.02%** id: **497**

„if (!isAdmin)”

în

Quotes detected: **0.02%** id: **498**

„Users.jsx”

61 Vasile Giulia-Georgiana 4. FUNCȚIONALITĂȚII APLICAȚIEI Testarea aplicației a fost realizată în două etape distincte. În prima etapă a fost testat server-ul de back-end dezvoltat în mediul Node.js, utilizând framework-ul Express și conectat la baza de date PostgreSQL, unde a fost necesară verificarea funcționării corecte a tuturor rutelor HTTP implementate anterior, precum cele de autentificare, gestionare a produselor, tranzacțiilor și utilizatorilor. Această etapă a fost esențială pentru validarea logicii aplicației înainte de integrarea completă cu interfața front-end realizată în React. A doua etapă a constat în testarea aplicației front-end, prin verificarea apelurilor către server, a funcționării rutării, a gestionării stării globale prin Redux Toolkit, precum și a mecanismului de autentificare bazat pe sesiuni și cookie-uri. În această fază au fost analizate atât răspunsurile HTTP primite de la server, cât și comportamentul interfeței în diferite scenarii de utilizare. 4.1 Testarea back-end-ului Testarea server-ului de back-end a fost realizată utilizând aplicația Postman, un instrument dedicat testării și simulării cererilor HTTP. Prin intermediul acestuia au fost create colecții de request-uri corespunzătoare fiecărei rute implementate în aplicație. Conform Fig.4.1, se prezintă partea stângă a folder-ului numit

Quotes detected: **0.01%** id: **499**

„RAWVISTA”,

unde se află toate cererile către server-ul de back-end, încadrate într-un director specific rutei. Fiecare dintre acestea are câte un nume specific și operația HTTP atribuită în urma apelului. Din figură se poate observa un exemplu de testare a rutei de înregistrare a unui utilizator, utilizând metoda HTTP POST și adresa

Quotes detected: **0.01%** id: **500**

„http://localhost:3000/api/auth/register”.

În corpul cererii este transmis un obiect de tip JSON care conține câmpurile necesare creării contului, precum

Quotes detected: **0.01%** id: **501**

„first_name”,

Quotes detected: **0.01%** id: **502**

„last_name”,

Quotes detected: **0.01%** id: **503**

„email”,

Quotes detected: **0.01%** id: **504**

„password”

și

Quotes detected: **0.01%** id: **505**

„role”.

Fig.4.1 Aplicația

Quotes detected: **0.01%** id: **506**

„Postman”

62 Proiect de Diplomă În cazul în care se încearcă înregistrarea unui utilizator cu o adresă de email deja existentă în baza de date, server-ul returnează un cod de eroare (de exemplu 400 sau 409), împreună cu un mesaj explicit de tipul

Quotes detected: **0.02%** id: **507**

„Email deja utilizat”.

Acest comportament confirmă existența validărilor implementate la nivelul modelului și controller-ului. Fig.4.2 Exemplu de răspuns de eroare la înregistrare În situația în care datele sunt valide și adresa de email nu există în sistem, server-ul returnează un cod de succes (201 Created sau 200 OK), confirmând inserarea corectă în tabela users din baza de date PostgreSQL. Fig.4.3 Răspuns corect la operația de înregistrare 4.2 Testarea front-end-ului Testarea aplicației front-end a fost realizată utilizând instrumentele integrate în browser, în special consola JavaScript și tab-ul

Quotes detected: **0.01%** id: **508**

„Network”

din DevTools. În timpul dezvoltării, au fost utilizate mesaje de tip

Quotes detected: **0.02%** id: **509**

„console.log()”

pentru a verifica datele primite din apelurile API, precum și actualizarea corectă a stării globale prin Redux Toolkit. Această metodă a permis identificarea rapidă a eventualelor discrepanțe între datele primite de la server și cele afișate în interfață. Fig.4.4 Tab-ul

Quotes detected: **0.01%** id: **510**

„Network”

din browser 63 Vasile Giulia-Georgiana 5. INTERFAȚA CU UTILIZATORUL 5.1 Interfața utilizatorului cu rolul de administrator 5.2 Interfața utilizatorului cu rolul de operator BIBLIOGRAFIE [1] Site oficial InvenTree: https://inventree.org, accesat 06.09.2025 [2] Site oficial Odoo: https://www.odoo.com, accesat 06.09.2025 [3] Site oficial documentație JavaScript:

Plagiarism detected: **0.77%** https://react.dev/learn/writing-markup-with-jsx + 4 resources! id: **511**

https://developer.mozilla.org/en-US/docs/Web/JavaScript, accesat 07.09.2025 [4] Site oficial documentație Node.js: https://nodejs.org/en/learn/getting-started/introduction-to- nodejs , accesat 07.09.2025 [5] Site oficial documentație Express: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server- side/Express_Nodejs/Introduction, accesat 08.09.2025 [6] Site oficial documentație pg: https://node-postgres.com, accesat 08.09.2025 [7] Site oficial documentație HTML: https://developer.mozilla.org/en-US/docs/Web/HTML, accesat 08.09.2025 [8] Introducere HTML: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content accesat 08.09.2025 [9] Site oficial documentație CSS: developer.mozilla.org/en-US/docs/Learn/CSS/First_steps/What_is_CSS accesat 08.09.2025 [10] Site oficial documentație Bootstrap: https://getbootstrap.com/docs/5.3/getting-started/introduction accesat 08.09.2025 [11] Site oficial documentație React: https://developer.mozilla.org/en- US/docs

/Learn_web_development/Core/Frameworks_libraries/Introduction , accesat 08.09.2025 [12] Introducere React: https://react.dev/learn/describing-the-ui#, accesat 08.09.2025 [13] Site oficial Vite: https://vite.dev/guide/why, accesat 08.09.2025 [14] Site introducere în Axios: https://www.geeksforgeeks.org/html/what-is-axios, accesat în 08.09.2025 [15] Site introducere în SheetJS: https://www.berg-software.com/en/get-browser-data-into-excel- with-sheetjs, 08.09.2025 [16] Site Redux Toolkit de prezentat 64 Proiect de Diplomă [17] Site react router de prezentat [18] Site react hot toast de prezentat [19] Site express session de prezentat [20] Site bcrypt de prezentat [21] Site cors de prezentat 65

Disclaimer:

This report must be correctly interpreted and analyzed by a qualified person who bears the evaluation responsibility!

Any information provided in this report is not final and is a subject for manual review and analysis. Please follow the guidelines:
Assessment recommendations

Plagiarism Detector - Your right to know the authenticity! □ SkyLine LLC